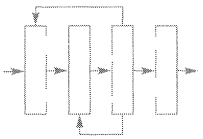


CHAPTER
4

Superscalar Organization

CHAPTER OUTLINE

While pipelining has proved to be an extremely effective microarchitecture technique, the type of *scalar pipelines* presented in Chapter 2 have a number of shortcomings or limitations. Given the never-ending push for higher performance, these limitations must be overcome in order to continue to provide further speedup for existing programs. The solution is *superscalar pipelines* that are able to achieve performance levels beyond those possible with just scalar pipelines.

Superscalar machines go beyond just a single-instruction pipeline by being able to simultaneously advance multiple instructions through the pipeline stages. They incorporate multiple functional units to achieve greater concurrent processing of multiple instructions and higher instruction execution throughput. Another foundational attribute of superscalar processors is the ability to execute instructions in an order different from that specified by the original program. The sequential ordering of instructions in standard programs implies some unnecessary precedences between the instructions. The capability of executing instructions out of program order relieves this sequential imposition and allows more parallel processing of instructions without requiring modification of the original program. This and the following chapters attempt to codify the body of knowledge on superscalar processor design in a systematic fashion. This chapter focuses on issues related to the pipeline organization of superscalar machines. The techniques that address the dynamic

interaction between the superscalar machine and the instructions being processed are presented in Chapter 5. Case studies of two commercial superscalar processors are presented in Chapters 6 and 7, while Chapter 8 provides a broad survey of historical and current designs.

4.1 Limitations of Scalar Pipelines

Scalar pipelines are characterized by a single-instruction pipeline of k stages. All instructions, regardless of type, traverse through the same set of pipeline stages. At most, one instruction can be resident in each pipeline stage at any one time, and the instructions advance through the pipeline stages in a lockstep fashion. Except for the pipeline stages that are stalled, each instruction stays in each pipeline stage for exactly one cycle and advances to the next stage in the next cycle. Such rigid scalar pipelines have three fundamental limitations:

1. The maximum throughput for a scalar pipeline is bounded by one instruction per cycle.
2. The unification of all instruction types into one pipeline can yield an inefficient design.
3. The stalling of a lockstep or rigid scalar pipeline induces unnecessary pipeline bubbles.

We elaborate on these limitations in Sections 4.1.1 to 4.1.3.

4.1.1 Upper Bound on Scalar Pipeline Throughput

As stated in Chapter 1 and as shown in Equation (4.1), processor performance can be increased either by increasing instructions per cycle (IPC) and/or frequency or by decreasing the total instruction count.

$$\text{Performance} = \frac{1}{\text{instruction count}} \times \frac{\text{instructions}}{\text{cycle}} \times \frac{1}{\text{cycle time}} = \frac{\text{IPC} \times \text{frequency}}{\text{instruction count}} \tag{4.1}$$

Frequency can be increased by employing a deeper pipeline. A deeper pipeline has fewer logic gate levels in each pipeline stage, which leads to a shorter cycle time and a higher frequency. However, there is a point of diminishing return due to the hardware overhead of pipelining. Furthermore, a deeper pipeline can potentially incur higher penalties, in terms of the number of penalty cycles, for dealing with inter-instruction dependences. The additional average cycles per instruction (CPI) overhead due to this higher penalty can possibly eradicate the benefit due to the reduction of cycle time.

Regardless of the pipeline depth, a scalar pipeline can only initiate the processing of at most one instruction in every machine cycle. Essentially, the average IPC for a scalar pipeline is fundamentally bounded by one. To get more instruction throughput, especially when deeper pipelining is no longer the most cost-effective

way to get performance, the ability to initiate more than one instruction in every machine cycle is necessary. To achieve an IPC greater than one, a pipelined processor must be able to initiate the processing of more than one instruction in every machine cycle. This will require increasing the width of the pipeline to facilitate having more than one instruction resident in each pipeline stage at any one time. We identify such pipelines as *parallel pipelines*.

4.1.2 Inefficient Unification into a Single Pipeline

Recall that the second idealized assumption of pipelining is that all the repeated computations to be processed by the pipeline are identical. For instruction pipelines, this is clearly not the case. There are different instruction types that require different sets of subcomputations. In unifying these different requirements into one pipeline, difficulties and/or inefficiencies can result. Looking at the unification of different instruction types into the TYP pipeline in Chapter 2, we can observe that in the earlier pipeline stages (such as IF, ID, and RD stages) there is significant uniformity. However, in the execution stages (such as ALU and MEM stages) there is substantial diversity. In fact, in the TYP example, we have ignored floating-point instructions on purpose due to the difficulty of unifying them with the other instruction types. It is for this reason that at one point in time during the "RISC revolution," floating-point instructions were categorized as inherently CISC and considered to be violating RISC principles.

Certain instruction types make their unification into a single pipeline quite difficult. These include floating-point instructions and certain fixed-point instructions (such as multiply and divide instructions) that require multiple execution cycles. Instructions that require long and possibly variable latencies are difficult to unify with simple instructions that require only a single cycle latency. As the disparity between CPU and memory speeds continues to widen, the latency (in terms of number of machine cycles) of memory instructions will continue to increase. Other than latency differences, the hardware resources required to support the execution of these different instruction types are also quite different. With the continued push for faster hardware, more specialized execution units customized for specific instruction types will be required. This will also contribute to the need for greater diversity in the execution stages of the instruction pipeline.

Consequently, the forced unification of all the instruction types into a single pipeline becomes either impossible or extremely inefficient for future high-performance processors. For parallel pipelines there is a strong motivation not to unify all the execution hardware into one pipeline, but instead to implement multiple different execution units or subpipelines in the execution portion of parallel pipelines. We call such parallel pipelines *diversified pipelines*.

4.1.3 Performance Lost due to a Rigid Pipeline

Scalar pipelines are rigid in the sense that instructions advance through the pipeline stages in a lockstep fashion. Instructions enter a scalar pipeline according to program order, i.e., in order. When there are no stalls in the pipeline, all the instructions in the pipeline stages advance synchronously and the program order of instructions is

maintained. When an instruction is stalled in a pipeline stage due to its dependence on a leading instruction, that instruction is held in the stalled pipeline stage while all leading instructions are allowed to proceed down the pipeline stages. Because of the rigid nature of a scalar pipeline, if a dependent instruction is stalled in pipeline stage i, then all earlier stages, i.e., stages $1, 2, \ldots, i-1$, containing trailing instructions are also stalled. All i stages of the pipeline are stalled until the instruction in stage i is forwarded its dependent operand. After the inter-instruction dependence is satisfied, then all i stalled instructions can again advance synchronously down the pipeline. For a rigid scalar pipeline, a stalled stage in the middle of the pipeline affects all earlier stages of the pipeline; essentially the stalling of stage i is propagated backward through all the preceding stages of the pipeline.

The backward propagation of stalling from a stalled stage in a scalar pipeline induces unnecessary pipeline bubbles or idling pipeline stages. While an instruction is stalled in stage i due to its dependence on a leading instruction, there may be another instruction trailing the stalled instruction which does not have a dependence on any leading instruction that would require its stalling. For example, this independent trailing instruction could be in stage $i-1$ and would be unnecessarily stalled due to the stalling of the instruction in stage i. According to program semantics, it is not necessary for this instruction to wait in stage $i-1$. If this instruction is allowed to bypass the stalled instruction and continue down the pipeline stages, an idling cycle of the pipeline can be eliminated, which effectively reduces the penalty due to the stalled instruction by one cycle; see Figure 4.1. If multiple instructions are able and allowed to bypass the stalled instruction, then multiple penalty cycles can be eliminated or "covered" in the sense that idling pipeline stages are given useful instructions to process. Potentially all the penalty cycles due to the stalled instruction can be covered. Allowing the bypassing of a stalled leading instruction by trailing instructions is referred to as an *out-of-order execution* of instructions. A rigid scalar pipeline does not allow out-of-order execution and hence can incur unnecessary penalty cycles in enforcing inter-instruction dependences. Parallel pipelines that support out-of-order execution are called *dynamic pipelines*.

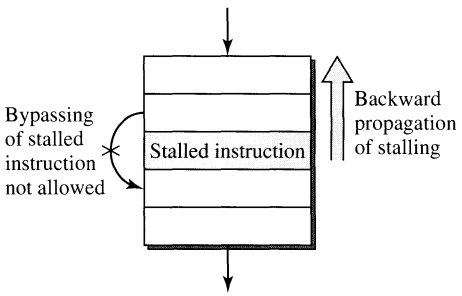


Figure 4.1
Unnecessary Stall Cycles Induced by Backward Propagation of Stalling in a Rigid Pipeline.

4.2 From Scalar to Superscalar Pipelines

Superscalar pipelines can be viewed as natural descendants of the scalar pipelines and involve extensions to alleviate the three limitations (see Section 4.1) with scalar pipelines. Superscalar pipelines are parallel pipelines, instead of scalar pipelines, in that they are able to initiate the processing of multiple instructions in every machine cycle. In addition, superscalar pipelines are diversified pipelines in employing multiple and heterogeneous functional units in their execution stage(s). Finally, superscalar pipelines can be implemented as dynamic pipelines in order to achieve the best possible performance without requiring reordering of instructions by the compiler. These three characterizing attributes of superscalar pipelines will be further elaborated in this section.

4.2.1 Parallel Pipelines

The degree of parallelism of a machine can be measured by the maximum number of instructions that can be concurrently in progress at any one time. A k-stage scalar pipeline can have k instructions concurrently resident in the machine and can potentially achieve a factor-of-k speedup over a nonpipelined machine. Alternatively, the same speedup can be achieved by employing k copies of the nonpipelined machine to process k instructions in parallel. These two forms of machine parallelism are illustrated in Figure 4.2(b) and (c), and they can be denoted *temporal machine parallelism* and *spatial machine parallelism,* respectively. Temporal and spatial parallelism of the same degree can yield about the same factor of potential speedup. Clearly, temporal parallelism via pipelining requires less hardware than spatial parallelism, which requires replication of the entire processing unit. Parallel pipelines can be viewed as employing both temporal and spatial machine parallelism, as illustrated in Figure 4.2(d), to achieve higher instruction processing throughput in an efficient manner.

The speedup of a scalar pipeline is measured with respect to a nonpipelined design and is primarily determined by the depth of the scalar pipeline. For parallel pipelines, or superscalar pipelines, the speedup now is usually measured with respect to a scalar pipeline and is primarily determined by the *width* of the parallel pipeline. A parallel pipeline with width s can concurrently process up to s instructions in each of its pipeline stages, which can lead to a potential speedup of s over a scalar pipeline. Figure 4.3 illustrates a parallel pipeline of width $s = 3$.

Significant additional hardware resources are required for implementing parallel pipelines. Each pipeline stage can potentially process and advance up to s instructions in every machine cycle. Hence, the logic complexity of each pipeline stage can increase by a factor of s. In the worst case, the circuitry for interstage interconnection can increase by a factor of s^2 if an $s \times s$ crossbar is used to connect all s instruction buffers from one stage to all s instruction buffers of the next stage. In order to support concurrent register file accesses by s instructions, the number of read and write ports of the register file must be increased by a factor of s. Similarly, additional I-cache and D-cache access ports must be provided.

As shown in Chapter 2, the Intel i486 is a five-stage scalar pipeline [Crawford, 1990]. The sequel to the i486 was the Pentium microprocessor from

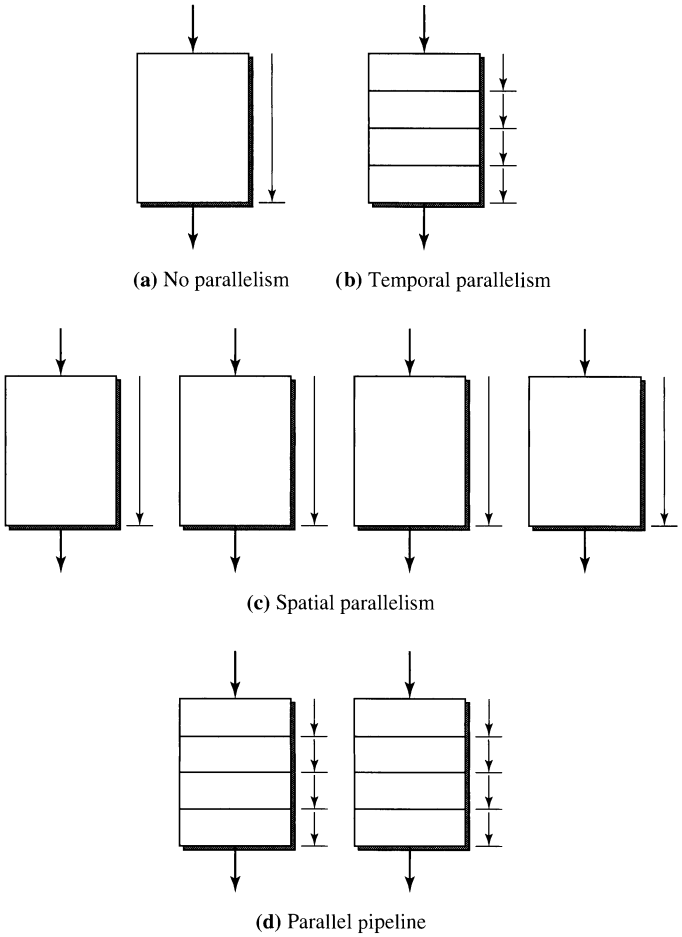


Figure 4.2
Machine Parallelism: (a) No Parallelism (Nonpipelined); (b) Temporal Parallelism (Pipelined); (c) Spatial Parallelism (Multiple Units); (d) Combined Temporal and Spatial Parallelism.

Intel [Intel Corp., 1993]. The Pentium microprocessor is a superscalar machine implementing a parallel pipeline of width $s = 2$. It essentially implements two i486 pipelines; see Figure 4.4. Multiple instructions can be fetched and decoded by the first two stages of the parallel pipeline in every machine cycle. In each cycle, potentially two instructions can be issued into the two execution pipelines, i.e., the U pipe and the V pipe. The goal is to maximize the number of dual-issue cycles. The superscalar Pentium microprocessor can achieve a peak execution rate of two instructions per machine cycle.

As compared to the scalar pipeline of i486, the Pentium parallel pipeline requires significant additional hardware resources. First, the five pipeline stages

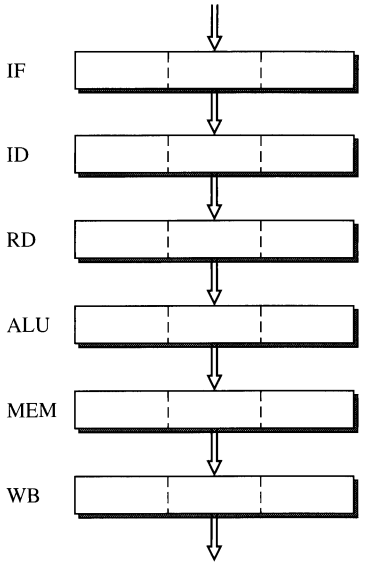


Figure 4.3
A Parallel Pipeline of Width $s = 3$.

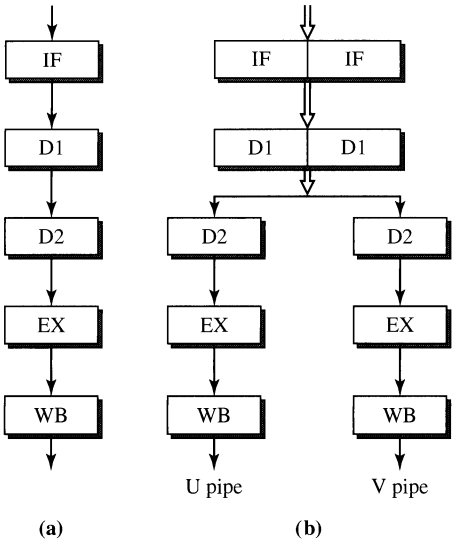
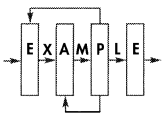


Figure 4.4
(a) The Five-Stage i486 Scalar Pipeline;
(b) The Five-Stage Pentium Parallel Pipeline of Width $s = 2$.



have doubled in width. The two execution pipes can accommodate up to two instructions in each of the last three stages of the pipeline. The execute stage can perform an ALU operation or access the D-cache. Hence, additional ports to the register file must be provided to support the concurrent execution of two ALU operations in every cycle. If the two instructions in the execute stage are both load/store instructions, then the D-cache must provide dual access. A true dual-ported D-cache is expensive to implement. Instead, the Pentium D-cache is implemented as a single-ported D-cache with eight-way interleaving. Simultaneous accesses to two different banks by the two load/store instructions in the U and V pipes can be supported. If there is a bank conflict, i.e., both load/store instructions must access the same bank, then the two D-cache accesses are serialized.

4.2.2 Diversified Pipelines

The hardware resources required to support the execution of different instruction types can vary significantly. For a scalar pipeline, all the diverse requirements for the execution of all instruction types must be unified into a single pipeline. The resultant pipeline can be highly inefficient. Each instruction type only requires a subset of the execution stages, but it must traverse all the execution stages. Every instruction is idling as it traverses the unnecessary stages and incurs significant dynamic external fragmentation. The execution latency for all instruction types is equal to the total number of execution stages. This can result in unnecessary stalling of trailing instructions and/or require additional forwarding paths.

This inefficiency due to unification into one single pipeline is naturally addressed in parallel pipelines by employing multiple different functional units in the execution stage(s). Instead of implementing s identical pipes in an s-wide parallel pipeline, in the execution portion of the parallel pipeline, diversified execution pipes can be implemented; see Figure 4.5. In this example, four execution pipes, or functional units, of differing pipe depths are implemented. The RD stage dispatches instructions to the four execution pipes based on the instruction types.

There are a number of advantages in implementing diversified execution pipes. Each pipe can be customized for a particular instruction type, resulting in efficient hardware design. Each instruction type incurs only the necessary latency and makes use of all the stages of an execution pipe. This is certainly more efficient than implementing s identical copies of a universal execution pipe each of which can execute all instruction types. If all inter-instruction dependences between different instruction types are resolved prior to dispatching, then once instructions are issued into the individual execution pipes, no further stalling can occur due to instructions in other pipes. This allows the distributed and independent control of each execution pipe.

The design of a diversified parallel pipeline does require special considerations. One important consideration is the number and mix of functional units. Ideally the number of functional units should match the available instruction-level parallelism of the program, and the mix of functional units should match the dynamic mix of instruction types of the program. Most first-generation superscalar processors

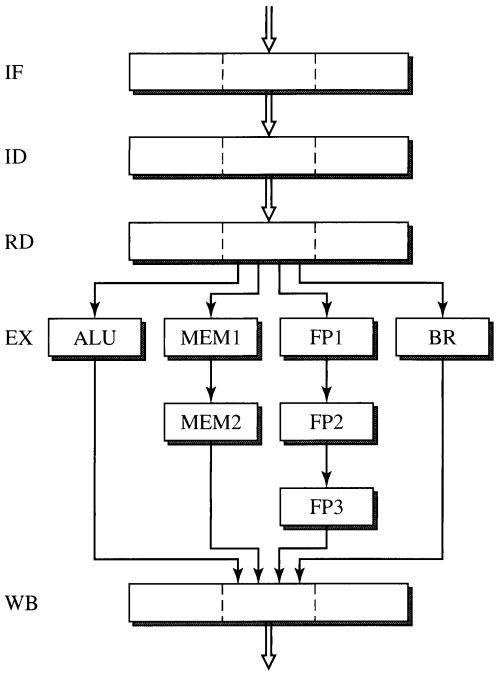
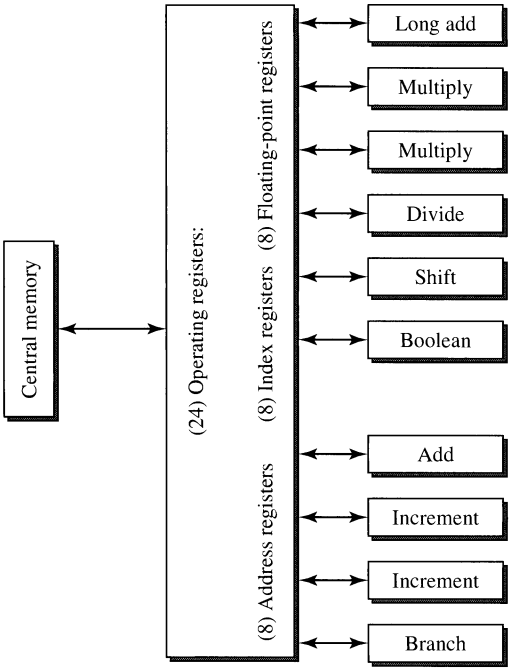


Figure 4.5
A Diversified Parallel Pipeline with Four Execution Pipes.

simply integrated a second execution pipe for processing floating-point instructions with the existing scalar pipe for processing non-floating-point instructions. As superscalar designs evolved from two-issue machines to four-issue machines, typically four functional units are implemented for executing integer, floating-point, load/store, and branch instructions. Some recent designs incorporate multiple integer units, some of which are dedicated to long-latency integer operations such as multiply and divide, and others are dedicated to the processing of special operations for image, graphics, and signal processing applications.

Similar to pipelining, the employment of a multiplicity of diversified functional units in the design of a high-performance CPU is not a recent invention. The CDC 6600 incorporates both pipelining and the use of multiple functional units [Thornton, 1964]. The CPU of the CDC 6600 employs 10 diversified functional units, as shown in Figure 4.6. The 10 functional units operate on data stored in 24 operating registers, which consist of 8 address registers (18 bits), 8 index registers (18 bits), and 8 floating-point registers (60 bits). The 10 functional units operate independently and consist of a fixed-point adder (18 bits), a floating-point adder (60 bits), two multiply units (60 bits), a divide unit (60 bits), a shift unit (60 bits), a boolean unit (60 bits), two increment units, and a branch unit. The CDC 6600 CPU is a pipelined processor with two decoding stages preceding the execution portion;



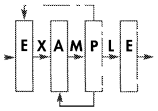


Figure 4.6
The CDC 6600 with 10 Diversified Functional Units in Its CPU.

however, the 10 functional units are not pipelined and have variable execution latencies. For example, a fixed-point add requires 3 cycles, and a floating-point multiply (divide) requires 10 (29) cycles. The goal of the CDC 6600 CPU is to sustain an issue rate of one instruction per machine cycle.

Another superscalar microprocessor employed a similar mix of functional units as the CDC 6600. Just prior to the formation of the PowerPC alliance with IBM and Apple, Motorola had developed a very clean design of a wide superscalar microprocessor called the 88110 [Diefendorf and Allen, 1992]. Interestingly, the 88110 also employs 10 functional units; see Figure 4.7. The 10 functional units consist of two integer units, a bit field unit, a floating-point add unit, a multiply unit, a divide unit, two graphic units, a load/store unit, and an instruction sequencing/branch unit. Most of the units have single-cycle latency. With the exception of the divide unit, the other units with multicycle latencies are all pipelined. In terms of the total number of functional units, the 88110 represents one of the wider superscalar designs.

4.2.3 Dynamic Pipelines

In any pipelined design, buffers are required between pipeline stages. In a scalar rigid pipeline, a single-entry buffer is placed between two consecutive pipeline stages

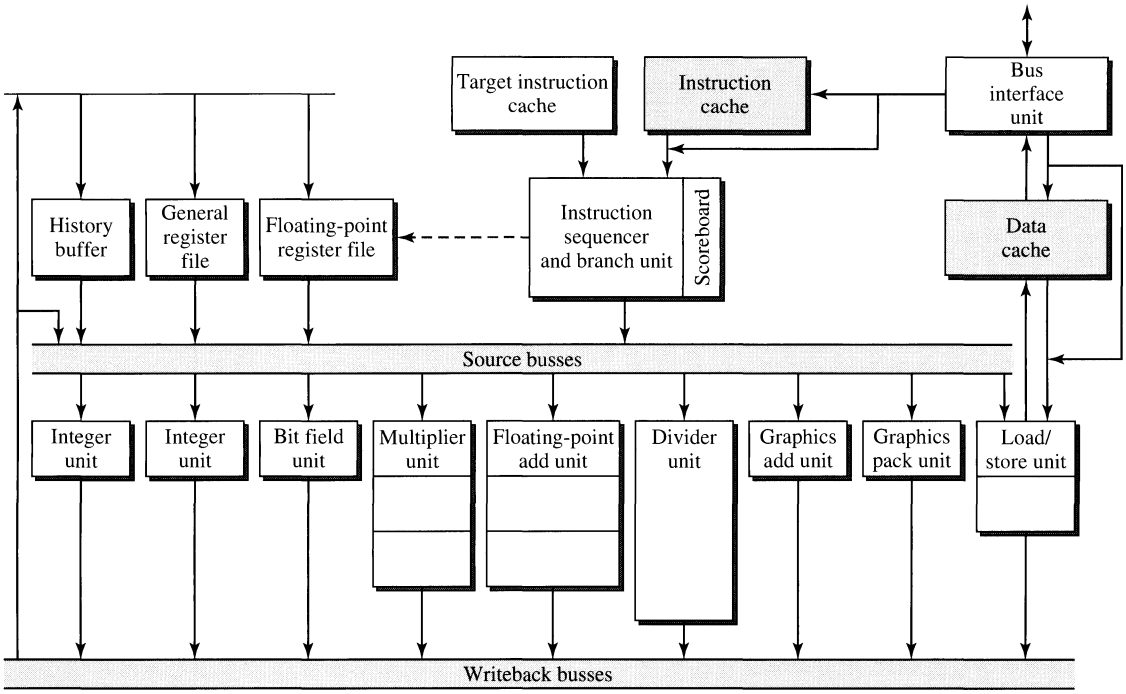
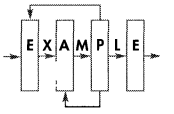


Figure 4.7
The Motorola 88110 Superscalar Microprocessor.
Source: Diefendorf and Allen, 1992.



(stages i and $i + 1$), as shown in Figure 4.8(a). The buffer holds all essential control and data bits for the instruction that has just traversed stage i of the pipeline and is ready to traverse stage $i + 1$ in the next machine cycle. Single-entry buffers are quite easy to control. In every machine cycle, the buffer's current content is used as input to stage $i + 1$; and at the end of the cycle, the buffer latches in the result produced by stage i. Essentially the buffer is clocked in every machine cycle. The exception occurs when the instruction in the buffer must be held back and prevented from traversing stage $i + 1$. In that case, the clocking of the buffer is disabled, and the instruction is stalled in the buffer. Clearly if this buffer is stalled in a scalar rigid pipeline, all stages preceding stage i must also be stalled. Hence, in a scalar rigid pipeline, if there is no stalling, then every instruction remains in each buffer for exactly one machine cycle and then advances to the next buffer. All the instructions enter and leave each buffer in exactly the same order as specified in the original sequential code.

In a parallel pipeline, multientry buffers are needed between two consecutive pipeline stages as shown in Figure 4.8(b). Multientry buffers can be viewed as simple extensions of the single-entry buffers. Multiple instructions can be latched into each multientry buffer in every machine cycle. In the next cycle, these instructions can then traverse the next pipeline stage. If all the instructions in a multientry buffer are required to advance simultaneously in a lockstep fashion, then the control of the

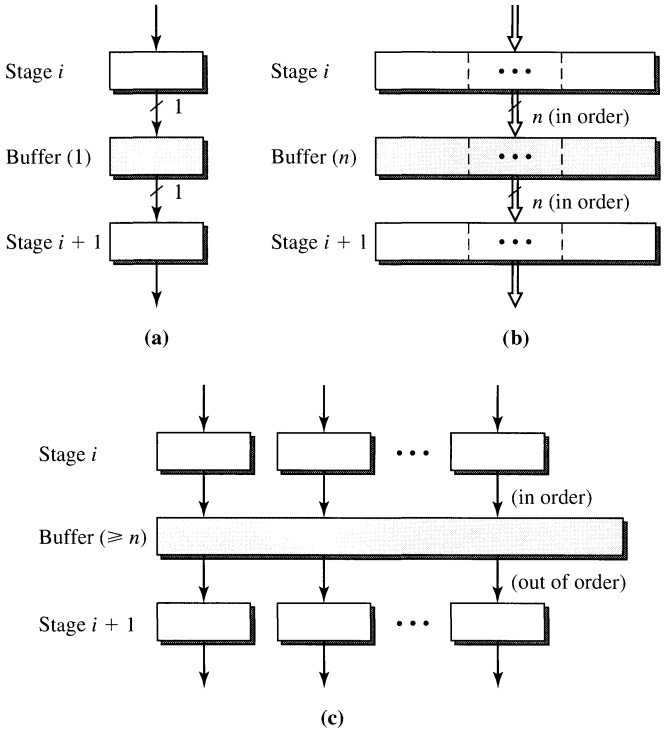


Figure 4.8
Interpipeline-Stage Buffers: (a) Single-Entry Buffer; (b) Multientry Buffer; (c) Multientry Buffer with Reordering.

multientry buffer is similar to that of the single-entry buffer. The entire multientry buffer is either clocked or stalled in each machine cycle. However, such operation of the parallel pipeline may induce unnecessary stalling of some of the instructions in a multientry buffer. For more efficient operation of a parallel pipeline, much more sophisticated multientry buffers are needed.

Each entry of the simple multientry buffer of Figure 4.8(b) is hardwired to one write port and one read port, and there is no interaction between the multiple entries. One enhancement to this simple buffer is to add connectivity between the entries to facilitate movement of data between entries. For example, the entries can be connected into a linear chain like a shift register and function as a FIFO queue. Another enhancement is to provide a mechanism for independent accessing of each entry in the buffer. This will require the ability to explicitly address each individual entry in the buffer and independently control the reading and writing of each entry. If each input/output port of the buffer is given the ability to access any entry in the buffer, then such a multientry buffer will effectively resemble a small multiported RAM. With such a buffer an instruction can remain in an entry of the buffer for many machine cycles and can be updated or modified while resident in that buffer. A further enhancement can incorporate associative accessing of the entries in the buffer. Instead of using conventional addressing to index into an entry in the buffer, the content of an

entry can be used as an associative tag to index into that entry. With such accessing mechanism, the multientry buffer becomes a small associative cache memory.

Superscalar pipelines differ from (rigid) scalar pipelines in one key aspect, which is the use of complex multientry buffers for buffering instructions in flight. In order to minimize unnecessary stalling of instructions in a parallel pipeline, trailing instructions must be allowed to bypass a stalled leading instruction. Such bypassing can change the order of execution of instructions from the original sequential order of the static code. With out-of-order execution of instructions, there is the potential of approaching the data flow limit of instruction execution; i.e., instructions are executed as soon as their operands are available. A parallel pipeline that supports out-of-order execution of instructions is called a *dynamic pipeline*. A dynamic pipeline achieves out-of-order execution via the use of complex multientry buffers that allow instructions to enter and leave the buffers in different orders. Such a reordering multientry buffer is shown in Figure 4.8(c).

Figure 4.9 illustrates a parallel diversified pipeline of width $s = 3$ that is a dynamic pipeline. The execution portion of the pipeline, consisting of the four pipelined

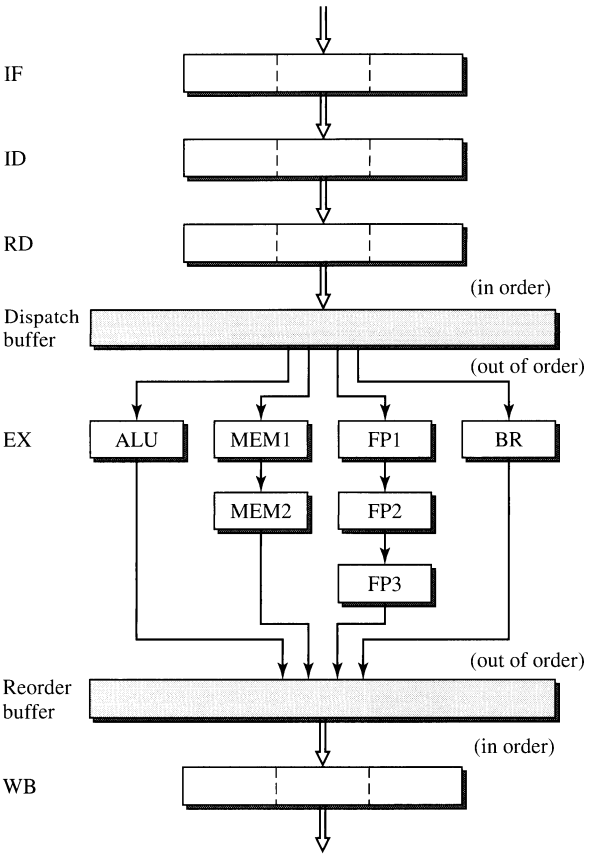


Figure 4.9
A Dynamic Pipeline of Width $s = 3$.

functional units, is bracketed by two reordering multientry buffers. The first buffer, called the *dispatch buffer,* is loaded with decoded instructions according to program order and then dispatches instructions to the functional units potentially in an order different from the program order. Hence instructions can leave the dispatch buffer in a different order than the order in which they enter the dispatch buffer. This pipeline also implements a set of diverse functional units with different latencies.

With potential out-of-order issuing into the functional units and/or the variable latencies of the functional units, instructions can finish execution out of order. To ensure that exceptions can be handled according to the original program order, the instructions must be completed (i.e., the machine state must be updated), in program order. When instructions finish execution out of order, another reordering multientry buffer is needed at the back end of the execution portion of the pipeline to ensure in-order completion. This buffer, called the *completion buffer,* buffers the instructions that may have finished execution out of order and retires the instructions in order by outputting instructions to the final writeback stage in program order. Such a dynamic pipeline facilitates the out-of-order execution of instructions in order to achieve the shortest possible execution time, and yet is able to provide precise exception by retiring the instructions and updating the machine state according to the program order.

4.3 Superscalar Pipeline Overview

This section presents an overview of the critical issues involved in the design of superscalar pipelines. The focus is on the organization, or structural design, of superscalar pipelines. Issues and techniques related to the dynamic interaction of machine organization and instruction semantics and the optimization of the resultant machine performance are covered in Chapter 5. Essentially this chapter focuses on the design of the machine organization, while Chapter 5 takes into account the interaction between the machine and the program.

Similar to the use of the six-stage TYP pipeline in Chapter 2 as a vehicle for presenting scalar pipeline design, we use the six-stage TEM superscalar pipeline shown in Figure 4.10 as a "template" for discussion on the organization of superscalar pipelines. Compared to scalar pipelines, there is far more variety and greater diversity in the implementation of superscalar pipelines. The TEM superscalar pipeline should not be viewed as an actual implementation of a typical or representative superscalar pipeline. The six stages of the TEM superscalar pipeline should be viewed as *logical* pipeline stages which may or may not correspond to six physical pipeline stages. The six stages of the TEM superscalar pipeline provide a nice framework or outline for discussing the six major portions of, or six major tasks performed by, most superscalar pipeline organizations.

The six stages of the TEM superscalar pipeline are *fetch, decode, dispatch, execute, complete,* and *retire*. The execute stage can include multiple (pipelined) functional units of different types with different execution latencies. This necessitates the dispatch stage to distribute instructions of different types to their corresponding functional units. With out-of-order execution of instructions in the execute stage, the complete stage is needed to reorder the instructions and ensure the in-order

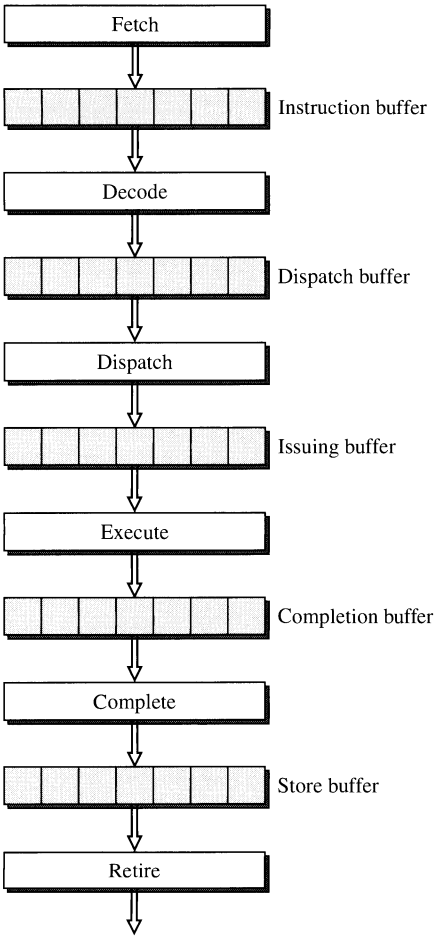


Figure 4.10
The Six-Stage Template (TEM) Superscalar Pipeline.

updating of the machine state. Note also that there are multientry buffers separating these six stages. The complexity of these buffers can vary depending on their functionality and location in the superscalar pipeline. These six stages and design issues related to them are now addressed in turn.

4.3.1 Instruction Fetching

Unlike a scalar pipeline, a superscalar pipeline, being a parallel pipeline, is capable of fetching more than one instruction from the I-cache in every machine cycle. Given a superscalar pipeline of width s, its fetch stage should be able to fetch s instructions from the I-cache in every machine cycle. This implies that the physical organization of the I-cache must be wide enough that each row of the I-cache array can store

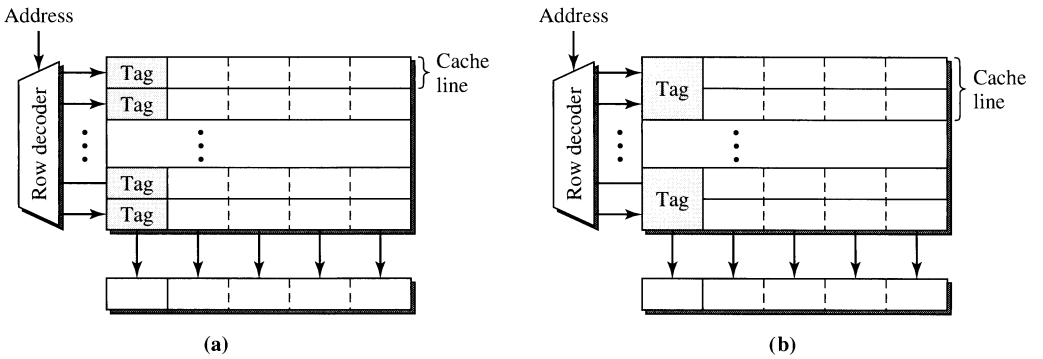


Figure 4.11
Organization of a Wide I-Cache: (a) One Cache Line is Equal to One Physical Row; (b) One Cache Line is Equal to Two Physical Rows.

s instructions and that an entire row can be accessed at one time. In our current discussion, we assume that the access latency of the I-cache is one cycle and that the fetch width is equal to the row width. Typically in such a wide cache organization, a cache line corresponds to a physical row in the cache array; it is also possible that a cache line can span several physical rows of the cache array, as illustrated in Figure 4.11.

The primary objective of the fetch stage is to maximize the instruction-fetching bandwidth. The sustained throughput achieved by the fetch stage will impact the overall throughput of the superscalar pipeline, because the throughput of all subsequent stages depends on and cannot possibly exceed the throughput of the fetch stage. Two primary impediments to achieving the maximum throughput of s instructions fetched per cycle are (1) the misalignment of the s instructions being fetched, called the *fetch group,* with respect to the row organization of the I-cache array; and (2) the presence of control-flow changing instructions in the fetch group.

In every machine cycle, the fetch stage uses the program counter (PC) to index into the I-cache to fetch the instruction pointed to by the PC along with the next $s - 1$ instructions, i.e., the s instructions of the fetch group. If the entire fetch group is stored in the same row of the cache array, then all s instructions can be fetched. On the other hand, if the fetch group crosses a row boundary, then not all s instructions can be fetched in that cycle (assuming that only one row of the I-cache can be accessed in each cycle). Hence, only those instructions in the first row can be fetched; the remaining instructions will require another cycle for their fetching. The fetch bandwidth is effectively reduced by one-half, for it now requires two cycles to fetch s instructions. This is due to the misalignment of the fetch group with respect to the row boundaries of the I-cache array, as illustrated in Figure 4.12. Such misalignments reduce the effective fetch bandwidth. In the case where each cache line corresponds to a physical row, as shown in Figure 4.11(a), then the crossing of a row boundary also corresponds to the crossing of a cache line boundary, which can incur additional problems. If a fetch group spans two cache lines, then it can induce an I-cache miss involving the second line even though the

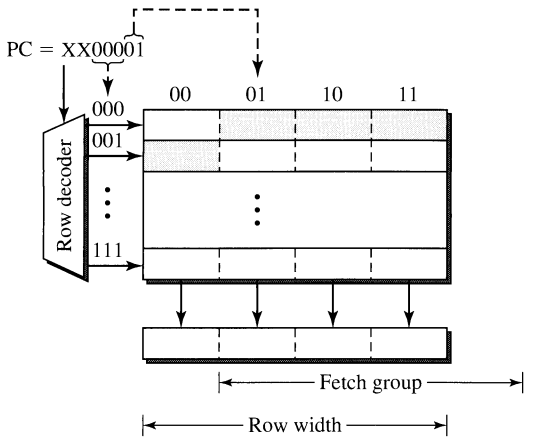


Figure 4.12
Misalignment of the Fetch Group Relative to the Row Boundaries of the I-Cache Array.

first line is resident. Even if both lines are resident in the I-cache, the physical accessing of multiple cache lines in one cycle is problematic.

There are two possible solutions to the misalignment problem. The first solution is a static technique employed at compile time. The compiler can be given information on the organization of the I-cache, e.g., its indexing scheme and row size. Based on this information, instructions can be appropriately placed in memory locations so as to ensure the aligning of fetch groups with physical rows. For example, every instruction that is the target of a branch can be placed in a memory location that is mapped to the first instruction of a row. This will increase the probability of fetching s instructions from the beginning of a row. Such techniques have been implemented and are reasonably effective. A problem with this solution is that the object code is tuned to a particular I-cache organization and may not be properly aligned for other I-cache organizations. Another problem is that the static code now can occupy a larger address range, which can potentially lead to a higher I-cache miss rate.

The second solution to the misalignment problem involves using hardware at run time. Alignment hardware can be incorporated to ensure that s instructions are fetched in every cycle even if the fetch group crosses a row boundary (but not a cache line boundary). Such alignment hardware is incorporated in the IBM RS/6000 design; we now briefly describe this design [Grohoski, 1990; Oehler and Groves, 1990].

The RS/6000 employs a two-way set-associative I-cache with a line size of 16 instructions (64 bytes). Each row of the I-cache array stores four associative sets (two per set) of instructions. Hence, each line of the I-cache spans four physical rows, as shown in Figure 4.13. The physical I-cache array is actually composed of four independent subarrays (denoted 0, 1, 2, and 3), which can be accessed in parallel. One instruction can be fetched from each subarray in every I-cache access. Which

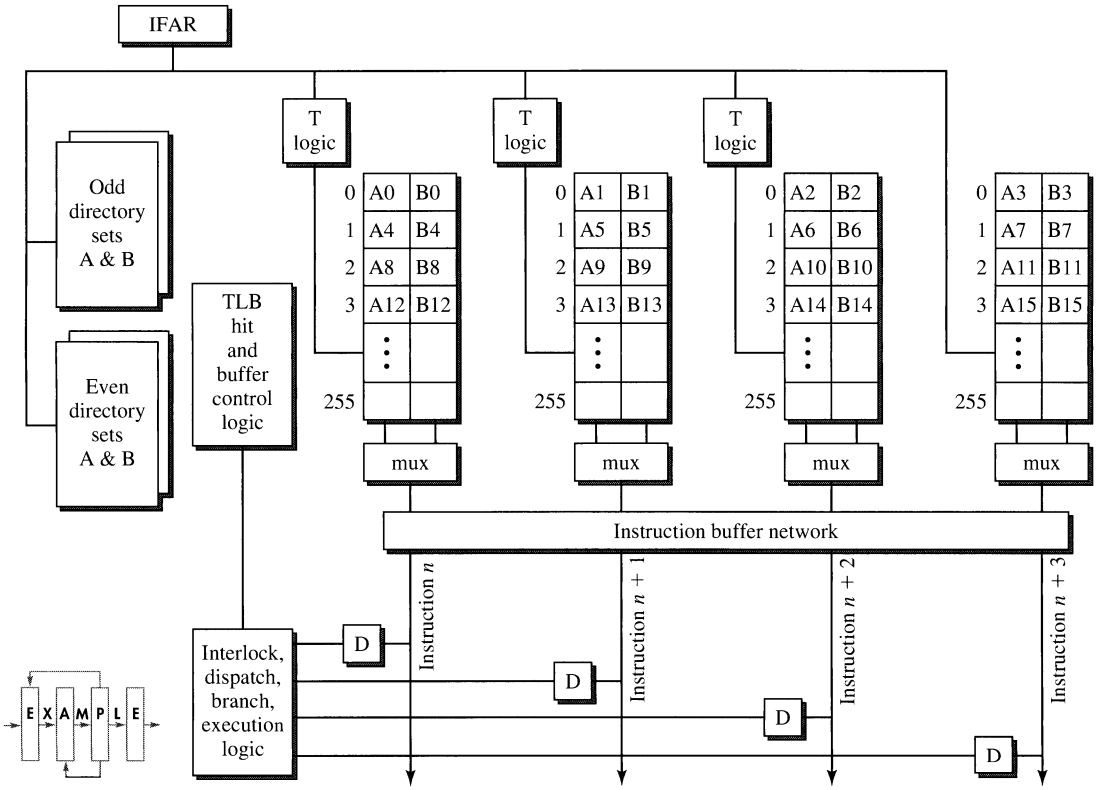


Figure 4.13
Organization of the RS/6000 Two-Way Set-Associative I-Cache with Auto-Realignment.

of the two instructions (either A or B) in the associative set is accessed depends on which of the two has a tag match with the address. The instruction addresses are allocated in an interleaved fashion across the four subarrays.

If the PC happens to point to the first subarray, i.e., subarray 0, then four consecutive instructions can be simultaneously fetched from the four subarrays. All four of these instructions reside in the same physical row of the I-cache, and all four subarrays are accessed using the same row address. On the other hand, if the PC indexes into the middle of the row, e.g., the first instruction of the fetch group resides in subarray 2, then the four consecutive instructions in the fetch group will span across two rows. The RS/6000 deals with this problem by detecting when the starting address points to a subarray other than subarray 0 and automatically incrementing the row address of the nonconsecutive subarrays. This is done by the "T-logic" hardware associated with each subarray. For example, if the PC indexes into subarray 2, then subarrays 2 and 3 will be accessed with the same row address presented to them. However the T-logic of subarrays 0 and 1 will detect this condition and automatically increment the row address presented to subarrays 0 and 1.

Consequently the two instructions fetched from subarrays 0 and 1 will actually be from the next physical row of the I-cache.

Therefore, regardless of the starting address and where that address points in an I-cache row, four consecutive instructions can always be fetched in every cycle as long as the fetch group does not cross a cache line boundary. When a fetch group crosses a cache line boundary, only instructions in the first cache line can be fetched in that cycle. Given the fact that the cache line of the RS/6000 consists of 16 instructions, and that there are 16 possible starting addresses of a word in a cache line, on the average the fetch bandwidth of this I-cache organization is $(13/16) \times 4 + (1/16) \times 3 + (1/16) \times 2 + (1/16) \times 1 = 3.625$ instructions per cycle.

Although the fetch group can begin in any one of the four subarrays, only subarrays 0, 1, and 2 require the T-logic hardware. The row address of subarray 3 never needs to be incremented regardless of the starting subarray of a fetch group. The *instruction buffer network* in the RS/6000 contains a rotating network which can rotate the four fetched instructions so as to present the four instructions, at its output, in original program order. This design of the I-cache is quite sophisticated and can ensure high fetch bandwidth even if the fetch group is misaligned with respect to the row organization of the I-cache. However, it is quite hardware intensive and was made feasible because the RS/6000 was implemented on multiple chips.

Other than the misalignment problem, the second impediment to sustaining the maximum fetch bandwidth of s instructions per cycle is the presence of control-flow changing instructions within the fetch group. If one of the instructions in the middle of the fetch group is a conditional branch, then the subsequent instructions in the fetch group will be discarded if the branch is taken. Consequently, when this happens, the fetch bandwidth is effectively reduced. This problem is fundamentally due to the presence of control dependences between instructions and is related to the handling of conditional branches. This topic, viewed as more related to the dynamic interaction between the machine and the program, is addressed in greater detail in Chapter 5, which covers techniques for dealing with control dependences and branch instructions.

4.3.2 Instruction Decoding

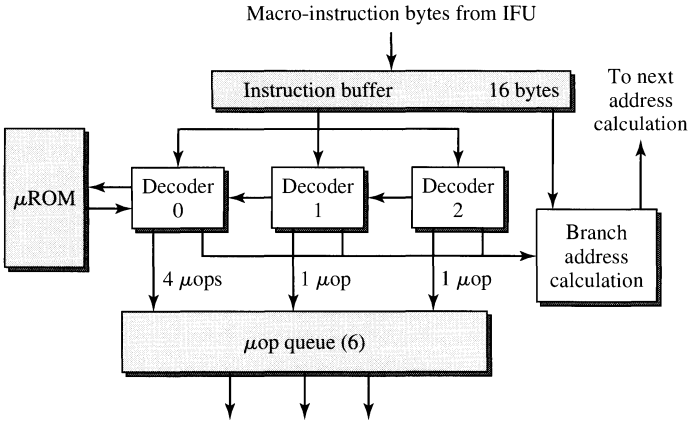
Instruction decoding involves the identification of the individual instructions, determination of the instruction types, and detection of inter-instruction dependences among the group of instructions that have been fetched but not yet dispatched. The complexity of the instruction decoding task is strongly influenced by two factors, namely, the ISA and the width of the parallel pipeline. For a typical RISC instruction set with fixed-length instructions and simple instruction formats, the decoding task is quite straightforward. No explicit effort is needed to determine the beginning and ending of each instruction. The relatively few different instruction formats and addressing modes make the distinguishing of instruction types reasonably easy. By simply decoding a small portion, e.g., one op code byte, of an instruction, the instruction type and the format used can be determined and the

remaining fields of the instruction and their interpretation can be quickly determined. A RISC instruction set simplifies the instruction decoding task.

For a RISC scalar pipeline, instruction decoding is quite trivial. Frequently the decode stage is used for accessing the register operands and is merged with the register read stage. However, for a RISC parallel pipeline with multiple instructions being simultaneously decoded, the decode stage must identify dependences between these instructions and determine the independent instructions that can be dispatched in parallel. Furthermore, to support efficient instruction fetching, the decode stage must quickly identify control-flow changing branch instructions among the instructions being decoded in order to provide quick feedback to the fetch stage. These two tasks in conjunction with accessing many register operands can make the logic for the decode stage of a RISC parallel pipeline somewhat complex. A large number of comparators are needed for determining register dependences between instructions. The register files must be multiported and able to support many simultaneous accesses. Multiple busses are also needed to route the accessed operands to their appropriate destination buffers. It is possible that the decode stage can become the critical stage in the overall superscalar pipeline.

For a CISC parallel pipeline, the instruction decoding task can become even more complex and usually requires multiple pipeline stages. For such a parallel pipeline, the identification of individual instructions and their types is no longer trivial. Both the Intel Pentium and the AMD K5 employ two pipeline stages for decoding IA32 instructions. On the more deeply pipelined Intel Pentium Pro, a total of five machine cycles are required to access the I-cache and decode the IA32 instructions. The use of variable instruction lengths imposes an undesirable sequentiality to the instruction decoding task; the leading instruction must be decoded and have its length determined before the beginning of the next instruction can be identified. Consequently, the simultaneous parallel decoding of multiple instructions can become quite challenging. In the worst case, it must be assumed that a new instruction can begin anywhere within the fetch group, and a large number of decoders are used to simultaneously and "speculatively" decode instructions, starting at every byte boundary. This is extremely complex and can be quite inefficient.

There is an additional burden on the instruction decoder of a CISC parallel pipeline. The decoder must translate the architected instructions into internal low-level operations that can be directly executed by the hardware. Such a translation process was first described by Patt, Hwu, and Shebanow in their seminal paper on the high-performance substrate (HPS), which decomposed complex VAX CISC instructions into RISC-like primitives [Patt et al., 1985]. These internal operations resemble RISC instructions and can be viewed as vertical micro-instructions. In the AMD K5 these operations are called *RISC operations* or *ROPs* (pronounced "ar-ops"). In the Intel P6 these internal operations are identified as *micro-operations* or *μops* (pronounced "you-ops"). Each IA32 instruction is translated into one or more ROPs or μops. According to Intel, on average, one IA32 instruction is translated into 1.5 to 2.0 μops. In these CISC parallel pipelines, between the instruction decoding and instruction completion stages, all instructions in flight within the



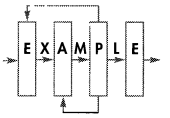


Figure 4.14
The Fetch/Decode Unit of the Intel P6 Superscalar Pipeline.

machine are these internal operations. In this book, for convenience we will adopt the Intel terminology and refer to these internal operations as μops.

The instruction decoder for the Intel Pentium Pro is presented as an illustrative example of instruction decoding for a CISC parallel pipeline. A diagram of the fetch/decode unit of the P6 is shown in Figure 4.14. In each machine cycle, the I-cache can deliver 16 aligned bytes to the instruction queue. Three parallel decoders simultaneously decode instruction bytes from the instruction queue. The first decoder at the front of the queue is capable of decoding all IA32 instructions, while the other two decoders have more limited capability and can only decode simple IA32 instructions such as register-to-register instructions.

The decoders translate IA32 instructions into the internal three-address μops. The μops employ the load/store model. Each IA32 instruction with complex addressing modes is translated into multiple μops. The first (generalized) decoder can generate up to four μops per cycle in response to the decoding of an IA32 instruction. Each of the other two (restricted) decoders can generate only one μop per cycle in response to the decoding of a simple IA32 instruction. In each machine cycle at least one IA32 instruction is decoded by the generalized decoder, leading to the generation of one or more μops. The goal is to go beyond this and have the other two restricted decoders also decode two simple IA32 instructions that trail the leading IA32 instruction in the same machine cycle. In the most ideal case the three parallel decoders can generate a total of six μops in one machine cycle. For those complex IA32 instructions that require more than four μops to translate, when they reach the front of the instruction queue, the generalized decoder will invoke a μops sequencer to emit microcode, which is simply a pre-programmed sequence of normal μops. These μops will require two or more machine cycles to generate. All the μops generated by the three parallel decoders are loaded into the reorder buffer (ROB), which has 40 entries to hold up to 40 μops, to await dispatching to the functional units.

For many superscalar processors, especially those that implement wide and/or CISC parallel pipelines, the instruction decoding hardware can be extremely complex and require partitioning into multiple pipeline stages. When the number of decoding stages is increased, the branch penalty, in terms of number of machine cycles, is also increased. Hence, it is not desirable to just keep increasing the depth of the decoding portion of the parallel pipeline. To help alleviate this complexity, a technique called *predecoding* has been proposed and implemented.

Predecoding moves a part of the decoding task to the other side, i.e., the input side, of the I-cache. When an I-cache miss occurs and a new cache line is being brought in from the memory, the instructions in that cache line are partially decoded by decoding hardware placed between the memory and the I-cache. The instructions and some additional decoded information are then stored in the I-cache. The decoded information, in the form of predecode bits, simplifies the instruction decoding task when the instructions are fetched from the I-cache. Hence, part of the decoding is performed only once when instructions are loaded into the I-cache, instead of every time when these instructions are fetched from the I-cache. With some of the decoding hardware having been moved to the input side of the I-cache, the instruction decoding complexity of the parallel pipeline can be simplified.

The AMD K5 is an example of a CISC superscalar pipeline that employs aggressive predecoding of IA32 instructions as they are fetched from memory and prior to their being loaded into the I-cache. In a single bus transaction a total of eight instruction bytes are fetched from memory. These bytes are predecoded, and five additional bits are generated by the predecoder for each of the instruction bytes. These five predecode bits contain information about the location of the start and end of an IA32 instruction, the number of μops (or ROPs) needed to translate that IA32 instruction, and the location of op codes and prefixes. These additional predecode bits are stored in the I-cache along with the original instruction's bytes. Consequently, the original I-cache line size of 128 bits (16 bytes) is increased by an additional 80 bits; see Figure 4.15. In each I-cache access, the 16 instruction bytes are fetched along with the 80 predecode bits. The predecode bits significantly simplify instruction decoding and allow the simultaneous decoding of multiple IA32 instructions by four identical decoders/translators that can generate up to four μops in each cycle.

There are two forms of overhead associated with predecoding. The I-cache miss penalty can be increased due to the necessity of predecoding the instruction bytes fetched from memory. This is not a serious problem if the I-cache miss rate is very low. The other overhead involves the storing of the predecode bits in the I-cache and the consequent increase of the I-cache size. For the K5 the size of the I-cache is increased by about 50%. There is clearly a tradeoff between the aggressiveness of predecoding and the I-cache size increase.

Predecoding is not just limited to alleviating the sequential bottleneck in parallel decoding of multiple CISC instructions in a CISC parallel pipeline. It can also be used to support RISC parallel pipelines. RISC instructions can be predecoded when they are being loaded into the I-cache. The predecode bits can be used to identify control-flow changing branch instructions within the fetch group and to explicitly

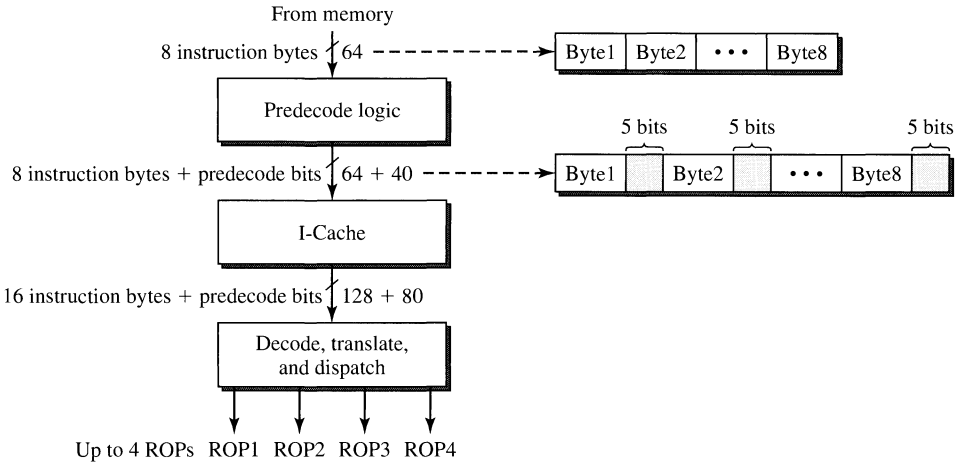
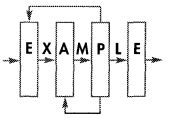
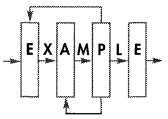


Figure 4.15
The Predecoding Mechanism of the AMD K5.



identify subgroups of independent instructions within the fetch group. For example, the PowerPC 620 employs 7 predecode bits for each instruction word in the I-cache. The UltraSPARC, MIPS R10000, and HP PA-8000 also employ either 4 or 5 predecode bits for each instruction.



As superscalar pipelines become wider and the number of instructions that must be simultaneously decoded increases, the instruction decoding task will become more of a bottleneck and more aggressive use of predecoding can be expected. The predecoder partially decodes the instructions, and effectively transforms the original undecoded instructions into a format that makes the final decoding task easier. One can view the predecoder as translating the instructions fetched from memory into different instructions that are then loaded into the I-cache. To expand this view, the possibility of enhancing the predecoder to do run-time object code translation between ISAs could be interesting.

4.3.3 Instruction Dispatching

Instruction dispatching is necessary for superscalar pipelines. In a scalar pipeline, all instructions regardless of their type flow through the same single pipeline. Superscalar pipelines are diversified pipelines that employ a multiplicity of heterogeneous functional units in their execution portion. Different types of instructions are executed by different functional units. Once the type of an instruction is identified in the decode stage, it must be routed to the appropriate functional unit for execution; this is the task of instruction dispatching.

Although superscalar pipelines are parallel pipelines, both the instruction fetching and instruction decoding tasks are usually carried out in a centralized fashion; i.e., all the instructions are managed by the same controller. Although multiple instructions are fetched in a cycle, all instructions must be fetched from the same I-cache. Hence all the instructions in the fetch group are accessed from the

I-cache at the same time, and they are all deposited into the same buffer. Instruction decoding is done in a centralized fashion because in the case of CISC instructions, all the bytes in the fetch group must be decoded collectively by a centralized decoder in order to identify the individual instructions. Even with RISC instructions, the decoder must identify inter-instruction dependences, which also requires centralized instruction decoding.

On the other hand, in a diversified pipeline all the functional units can operate independently in a distributed fashion in executing their own types of instructions once the inter-instruction dependences are resolved. Consequently, going from instruction decoding to instruction execution, there is a change from centralized processing of instructions to distributed processing of instructions. This change is carried out by, and is the reason for, the instruction dispatching stage in a superscalar pipeline. This is illustrated in Figure 4.16.

Another mechanism that is necessary between instruction decoding and instruction execution is the temporary buffering of instructions. Prior to its execution, an instruction must have all its operands. During decoding, register operands are fetched from the register files. In a superscalar pipeline it is possible that some of these operands are not yet ready because earlier instructions that update these registers have not finished their execution. When this situation occurs, an obvious solution is to stall the decoding stage until all register operands are ready. This solution seriously restricts the decoding throughput and is not desirable. A better solution is to fetch those register operands that are ready and go ahead and advance these

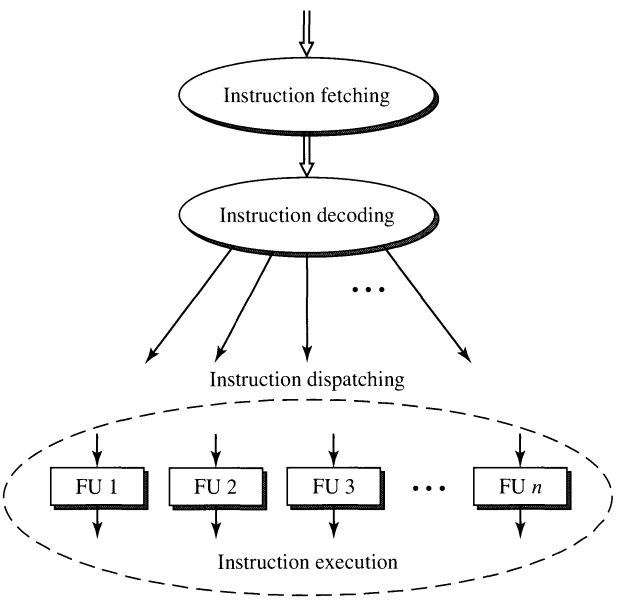


Figure 4.16
The Necessity of Instruction Dispatching in a Superscalar Pipeline.

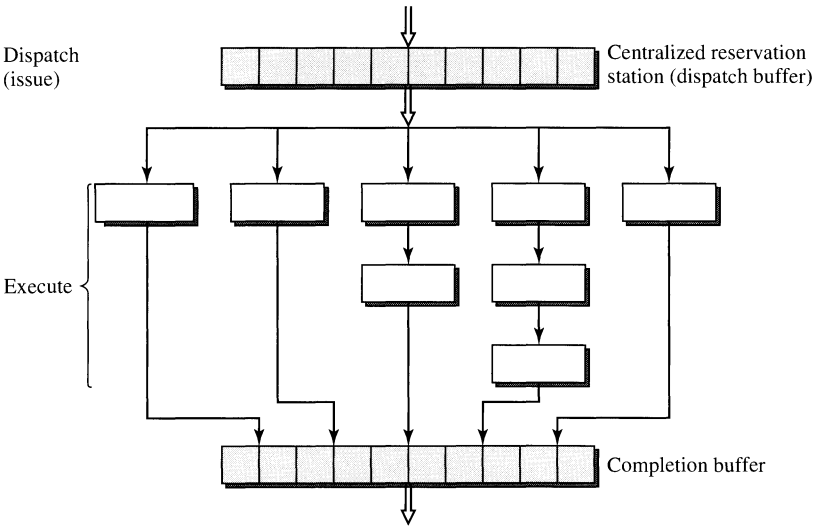


Figure 4.17
Centralized Reservation Station.

instructions into a separate buffer to await those register operands that are not ready. When all register operands are ready, those instructions can then exit this buffer and be issued into the functional units for execution. Borrowing the term used in the Tomasulo's algorithm employed in the IBM 360/91 [Tomasulo, 1967], we denote such a temporary instruction buffer as a *reservation station*. The use of a reservation station decouples instruction decoding and instruction execution and provides a buffer to take up the slack between decoding and execution stages due to the temporal variation of throughput rates in the two stages. This eliminates unnecessary stalling of the decoding stage and prevents unnecessary starvation of the execution stage.

Based on the placement of the reservation station relative to instruction dispatching, two types of reservation station implementations are possible. If a single buffer is used at the source side of dispatching, we identify this as a *centralized reservation station*. On the other hand, if multiple buffers are placed at the destination side of dispatching, they are identified as *distributed reservation stations*. Figures 4.17 and 4.18 illustrate the two ways of implementing reservation stations.

The Intel Pentium Pro implements a centralized reservation station. In such an implementation, one reservation station with many entries feeds all the functional units. Instructions are dispatched from this centralized reservation station directly to all the functional units to begin execution. On the other hand, the PowerPC 620 employs distributed reservation stations. In this implementation, each functional unit has its own reservation station on the input side of the unit. Instructions are dispatched to the individual reservation stations based on instruction type. These instructions remain in these reservation stations until they are ready to be issued into the functional units for execution. Of course, these two implementations of

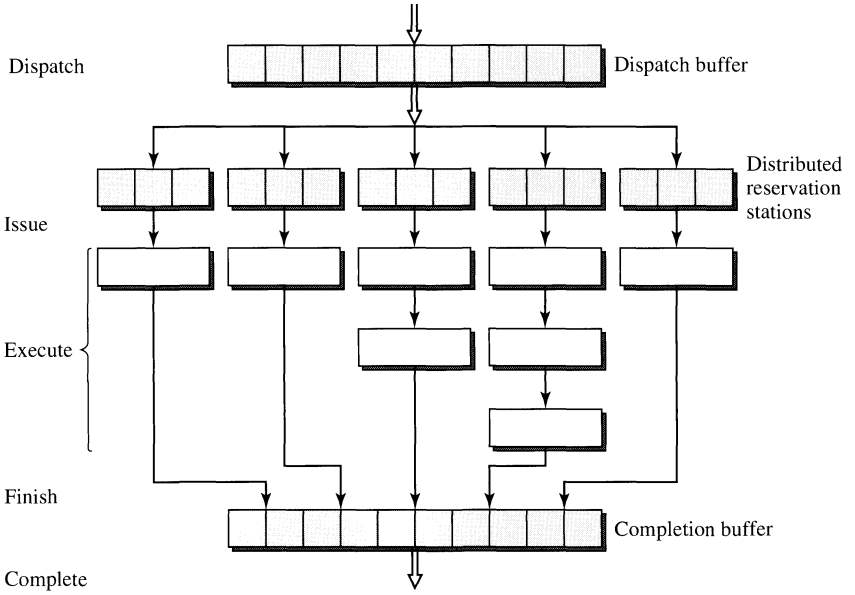
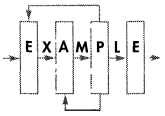


Figure 4.18
Distributed Reservation Stations.



reservation stations represent only the two extreme alternatives. Hybrids of these two approaches are also possible. For example, the MIPS R10000 employs one such hybrid implementation. We identified such hybrid implementations as *clustered reservation stations*. With clustered reservation stations, instructions are dispatched to multiple reservation stations, and each reservation station can feed or be shared by more than one functional unit. Typically the reservation stations and functional units are clustered based on instruction or data types.

Reservation station design involves certain tradeoffs. A centralized reservation station allows all instruction types to share the same reservation station and will likely achieve the best overall utilization of all the reservation station entries. However, a centralized implementation can incur the greatest complexity in its hardware design. It requires centralized control and a buffer that is highly multiported to allow multiple concurrent accesses. Distributed reservation stations can be single-ported buffers, each with only a small number of entries. However, each reservation station's idling entries cannot be used by instructions destined for execution in other functional units. The overall utilization of all the reservation station entries will be lower. It is also likely that one reservation station can saturate when all its entries are occupied and hence induce stalls in instruction dispatching.

With the different alternatives for implementing reservation stations, we need to clarify our use of certain terms. In this book the term *dispatching* implies the associating of instruction types with functional unit types after instructions have been decoded. On the other hand, the term *issuing* always means the initiation of execution in functional units. In a distributed reservation station design, these two events occur

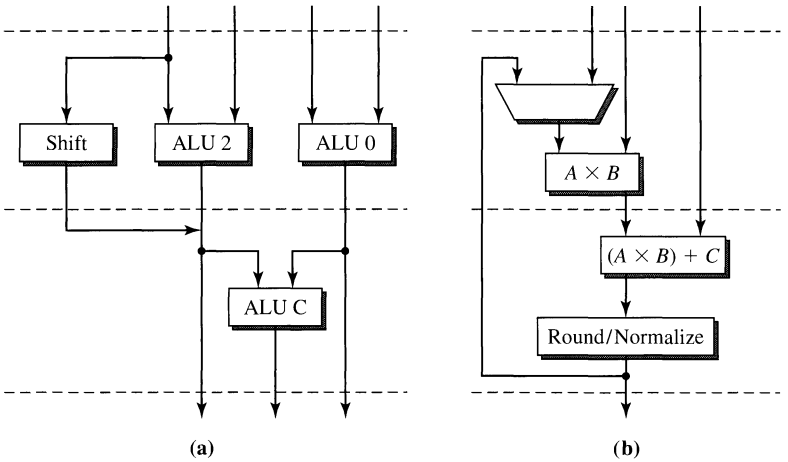
separately. Instructions are *dispatched* from the centralized decode/dispatch buffer to the individual reservation stations first, and when all their operands are available, then they are *issued* into the individual functional units for execution. With a centralized reservation station, the dispatching of instructions from the centralized reservation station does not occur until all their operands are ready. All instructions, regardless of type, are held in the centralized reservation station until they are ready to execute, at which time instructions are *dispatched* directly into the individual functional units to begin execution. Hence, in a machine with a centralized reservation station, the associating of instructions to individual functional units occurs at the same time as their execution is initiated. Therefore, with a centralized reservation station, instruction *dispatching* and instruction *issuing* occur at the same time, and these two terms become interchangeable. This is illustrated in Figure 4.17.

4.3.4 Instruction Execution

The instruction execution stage is the heart of a superscalar machine. The current trend in superscalar pipeline design is toward more parallel and more diversified pipelines. This translates into having more functional units and having these functional units be more specialized. By specializing them for executing specific instruction types, these functional units can be more performance efficient. Early scalar pipelined processors have essentially one functional unit. All instruction types (excluding floating-point instructions that are executed by a separate floating-point coprocessor chip) are executed by the same functional unit. In the TYP pipeline example, this functional unit is a two-stage pipelined unit consisting of the ALU and MEM stages of the TYP pipeline. Most first-generation superscalar processors are parallel pipelines with two diversified functional units, one executing integer instructions and the other executing floating-point instructions. These early superscalar processors simply integrated floating-point execution in the same instruction pipeline instead of employing a separate coprocessor unit.

Current superscalar processors can employ multiple integer units, and some have multiple floating-point units. These are the two most fundamental functional unit types. Some of these units are becoming quite sophisticated and are capable of executing more than one operation involving more than two source operands in each cycle. Figure 4.19(a) illustrates the integer execution unit of the TI SuperSPARC which contains a cascaded ALU configuration [Blanck and Krueger, 1992]. Three ALUs are included in this two-stage pipelined unit, and up to two integer operations can be issued into this unit in one cycle. If they are independent, then both operations are executed in the first stage using ALU0 and ALU2. If the second operation depends on the first, then the first one is executed in ALU2 during the first stage with the second one executed in ALUC in the second stage. Implementing such a functional unit allows more cycles in which two instructions are simultaneously issued.

The floating-point unit in the IBM RS/6000 is implemented as a two-stage pipelined multiply-add-fused (MAF) unit that takes three inputs (A, B, C) and performs $(A \times B) + C$. This is illustrated in Figure 4.19(b). The MAF unit is motivated by the most common use of floating-point multiplication to carry out the dot-product operation $D = (A \times B) + C$. If the compiler is able to merge many multiply-add



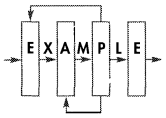
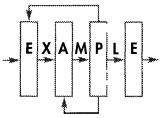
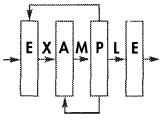


Figure 4.19
(a) Integer Functional Unit in the TI SuperSPARC; (b) Floating-Point Unit in the IBM RS/6000.

pairs of instructions into single MAF instructions, and the MAF unit can sustain the issuing of one MAF instruction in every cycle, then an effective throughput of two floating-point instructions per cycle can be achieved using only one MAF unit. The normal floating-point multiply instruction is actually executed by the MAF unit as $(A \times B) + 0$, while the floating-point add instruction is performed by the MAF unit as $(A \times 1) + C$. Since the MAF unit is pipelined, even without executing MAF instructions, it can still sustain an execution rate of one floating-point instruction per cycle.

In addition to executing integer ALU instructions, an integer unit can be used for generating memory addresses and executing branch and load/store instructions. However, in most recent designs separate branch and load/store units have been incorporated. The branch unit is responsible for updating the PC, while the load/store unit is directly connected to the D-cache. Other specialized functional units have emerged for supporting graphics and image processing applications. For example, in the Motorola 88110 there is a dedicated functional unit for bit manipulation and two functional units for supporting pixel processing. For many of the signal processing and multimedia applications, the common data type is a byte. Frequently 4 bytes are packed into a 32-bit word for simultaneous processing by specialized 32-bit functional units for increased throughput. In the TriMedia VLIW processor intended for such applications, such functional units are employed [Slavenburg et al., 1996]. For example, the TriMedia-1 processor can execute the quadavg instruction in one cycle. The quadavg instruction sums four rounded averages and is quite useful in MPEG decoding for decompressing compressed video images; it carries out the following computation.



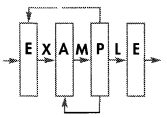


$$\text{quadavg} = \frac{(a+e+1)}{2} + \frac{(b+f+1)}{2} + \frac{(c+g+1)}{2} + \frac{(d+h+1)}{2} \qquad (4.2)$$

The eight variables denote 8-byte operands with a, b, c, and d stored as one 32-bit quantity and e, f, g, and h stored as another 32-bit quantity. The functional unit takes as input these two 32-bit operands and produces the quadavg result in one cycle. This single-cycle operation replaces numerous add and divide instructions that would have been required if the eight single-byte operands were manipulated individually. With the widespread deployment of multimedia applications, such specialized functional units that operate on special data types have emerged.

What is the best mix of functional units for a superscalar pipeline is an interesting question. The answer is dependent on the application domain. If we use the statistics from Chapter 2 of typical programs having 40% ALU instructions, 20% branches, and 40% load/store instructions, then we can have a 4-2-4 rule of thumb. For every four ALU units, we should have two branch units and four load/store units. Many of the current leading superscalar processors have four or more ALU-type functional units (including both integer and floating-point units). Most of them have only one branch unit but are able to speculate beyond one conditional branch instruction. However, most of these processors have only one load/store unit; some are able to process two load/store instructions in every cycle with some constraints. Clearly there seems be an imbalance in having too few load/store units. The reason is that implementing multiple load/store units that operate in parallel in accessing the same D-cache is a difficult task. It requires the D-cache to be multiported. Multiported memory modules involve very complex circuit design and can significantly slow down the memory speed.

In many designs multiple memory banks are used to simulate a truly multiported memory. A memory is partitioned into multiple banks. Each bank can perform a read/write operation in a machine cycle. If the effective addresses of two load/store instructions happen to reside on different banks, then both instructions can be carried out by the two different banks at the same time. However, if there is a bank conflict, then the two instructions must be serialized. Multibanked D-caches have been used to simulate multiported D-caches. For example, the Intel Pentium processor uses an eight-banked D-cache to simulate a two-ported D-cache [Intel Corp., 1993]. A truly multiported memory can guarantee conflict-free simultaneous accesses. Typically, more read ports than write ports are needed. Multiple read ports can be implemented by having multiple copies of the memory. All memory writes are broadcast to all the copies, with all the copies having identical content. Each copy can provide a small number of read ports with the total number of read ports being the sum of all the read ports on all the copies. For example, a memory with four read ports and two write ports can be implemented as two copies of simpler memory modules, each with only two write ports and two read ports. Implementing multiple, especially more than two, load/store units to operate in parallel can be a challenge in designing wide superscalar pipelines.



The amount of resource parallelism in the instruction execution portion is determined by the combination of *spatial* and *temporal* parallelism. Having multiple functional units is a form of spatial parallelism. Alternatively, parallelism can be obtained via pipelining of these functional units, which is a form of temporal

parallelism. For example, instead of implementing a dual-ported D-cache, in some current designs D-cache access is pipelined into two pipeline stages so that two load/store instructions can be concurrently serviced by the D-cache. Currently, there is a general trend toward implementing deeper pipelines in order to reduce the cycle time and increase the clock speed. Spatial parallelism also tends to require greater hardware complexity and silicon real estate. Temporal parallelism makes more efficient use of hardware but does increase the overall instruction processing latency and potentially pipeline stall penalties due to inter-instruction dependences.

In real superscalar pipeline designs, we often see that the total number of functional units exceeds the actual width of the parallel pipeline. Typically the width of a superscalar pipeline is determined by the number of instructions that can be fetched, decoded, or completed in every machine cycle. However, because of the dynamic variation of instruction mix and the resultant nonuniform distribution of instruction mix during program execution on a cycle-by-cycle basis, there is a potential dynamic mismatch of instruction mix and functional unit mix. The former varies in time and the latter stays fixed. Because of the specialization and heterogeneity of the functional units the total number of functional units must exceed the width of the superscalar pipeline to avoid having the instruction execution portion become the bottleneck due to excessive structural dependences related to the unavailability of certain functional unit types. Some of the aggressive compiler back ends actually try to smooth out this dynamic variation of instruction mix to ensure a better sustained match with the functional unit mix. Of course, different application programs can exhibit a different inherent overall mix of instruction types. The compiler can only make localized adjustments to achieve some performance gain. Studies have been done in assessing the best number and mix of functional units based on SPEC benchmarks [Jourdan et al., 1995].

With a large number of functional units, there is additional hardware complexity other than the functional units themselves. Results from the outputs of functional units need to be forwarded to inputs of the functional units. A multiplicity of busses are required, and potentially logic for bus control and arbitration is needed. Usually a full crossbar interconnection network is too costly and not absolutely necessary. The mechanism for routing operands between functional units introduces another form of structural dependence. The interconnect mechanism also contributes to the latency of the execution stage(s) of the pipeline. In order to support data forwarding the reservation station(s) must monitor the busses for tag matches, indicating the availability of needed operands, and latch in the operands when they are broadcasted on the busses. Potentially the complexity of the instruction execution stage can grow at the rate of n^2, where n is the total number of functional units.

4.3.5 Instruction Completion and Retiring

An instruction is considered *completed* when it finishes execution and updates the machine state. An instruction finishes execution when it exits the functional unit and enters the completion buffer. Subsequently it exits the completion buffer and becomes completed. When an instruction finishes execution, its result may only

reside in nonarchitected buffers. However, when it is completed, its result is written into an architecture register. With instructions that actually update memory locations, there can be a time period between when they are architecturally completed and when the memory locations are updated. For example, a store instruction can be architecturally completed when it exits the completion buffer and enters the store buffer to wait for the availability of a bus cycle in order to write to the D-cache. This store instruction is considered *retired* when it exits the store buffer and updates the D-cache. Hence, in this book instruction *completion* involves the updating of the machine state, whereas instruction *retiring* involves the updating of the memory state. For instructions that do not update the memory, retiring occurs at the same time as completion. So, in a distributed reservation station machine, an instruction can go through the following phases: *fetch, decode, dispatch, issue, execute, finish, complete,* and *retire*. Issuing and finishing simply refer to starting execution and ending execution, respectively. Some of the superscalar processor vendors use these terms in slightly different ways. Frequently, dispatching and issuing are used almost interchangeably, similar to completion and retiring. Sometimes completion is used to mean finishing execution, and retiring is used to mean updating the machine's architectural state. There is yet no standardization on the use of these terms.

During the execution of a program, interrupts and exceptions can occur that will disrupt the execution flow of a program. Superscalar processors employing dynamic pipelines that facilitate out-of-order execution must be able to deal with such disruptions of program execution. *Interrupts* are usually induced by the external environment such as I/O devices or the operating system. These occur in an asynchronous fashion with respect to the program execution. When an interrupt occurs, the program execution must be suspended to allow the operating system to service the interrupt. One way to do this is to stop fetching new instructions and allow the instructions that are already in the pipeline to finish execution, at which time the state of the machine can be saved. Once the interrupt has been serviced by the operating system, the saved machine state can be restored and the original program can resume execution.

Exceptions are induced by the execution of the instructions of the program. An instruction can induce an exception due to arithmetic operations, such as dividing by zero and floating-point overflow or underflow. When such exceptions occur, the results of the computation may no longer be valid and the operating system may need to intervene to log such exceptions. Exceptions can also occur due to the occurrence of page faults in a paging-based virtual memory system. Such exceptions can occur when instructions reference the memory. When such exceptions occur, a new page must be brought in from secondary storage, which can require on the order of thousands of machine cycles. Consequently, the execution of the program that induced the page fault is usually suspended, and the execution of a new program is initiated in the multiprogramming environment. After the page fault has been serviced, the original program can then resume execution.

It is important that the architectural state of the machine present at the time the excepting instruction is executed be saved so that the program can resume execution after the exception is serviced. Machines that are capable of supporting this

suspension and resumption of execution of a program at the granularity of each individual instruction are said to have *precise exception*. Precise exception involves being able to checkpoint the state of the machine just prior to the execution of the excepting instruction and then resume execution by restoring the checkpointed state and restarting execution at the excepting instruction. In order to support precise exception, the superscalar processor must maintain its architectural state and evolve this machine state in such a way as if the instructions in the program are executed one at a time according to the original program order. The reason is that when an exception occurs, the state the machine is in at that time must reflect the condition that all instructions preceding the excepting instruction have completed while no instructions following the excepting instruction have completed. For a dynamic pipeline to have precise exception, this sequential evolving of the architectural state must be maintained even though instructions are actually executed out of program order.

In a dynamic pipeline, instructions are fetched and decoded in program order but are executed out of program order. Essentially, instructions can enter the reservation station(s) in order but exit the reservation station(s) out of order. They also finish execution out of order. To support precise exception, instruction completion must occur in program order so as to update the architectural state of the machine in program order. In order to accommodate out-of-order finishing of execution and in-order completion of instructions, a *reorder buffer* is needed in the instruction completion stage of the parallel pipeline. As instructions finish execution, they enter the reorder buffer out of order, but they exit the reorder buffer in program order. As they exit the reorder buffer, they are considered architecturally completed. This is illustrated in Figure 4.20 with the reservation station and the reorder buffer bounding the out-of-order region of the pipeline or essentially the instruction execution portion of the pipeline. The terms adopted in this book, referring to the various phases of instruction processing, are illustrated in Figure 4.20.

Precise exception is handled by the instruction completion stage using the reorder buffer. When an exception occurs, the excepting instruction is tagged in the reorder buffer. The completion stage checks each instruction before that instruction is completed. When a tagged instruction is detected, it is not allowed to be completed. All the instructions prior to the tagged instructions are allowed to be completed. The machine state is then checkpointed or saved. The machine state includes all the architected registers and the program counter. The remaining instructions in the pipeline, some of which may have already finished execution, are discarded. After the exception has been serviced, the checkpointed machine state is restored and execution resumes with the fetching of the instruction that triggered the original exception.

Early work on support for providing precise exceptions in a processor that supports out-of-order execution was conducted by Acosta et al. [1986], Sohi and Vajapeyam [1987], and Smith and Pleszkun [1988]. An early proposal describing the Metaflow processor, which was never completed, also provides interesting insights for the curious reader [Popescu et al., 1991].

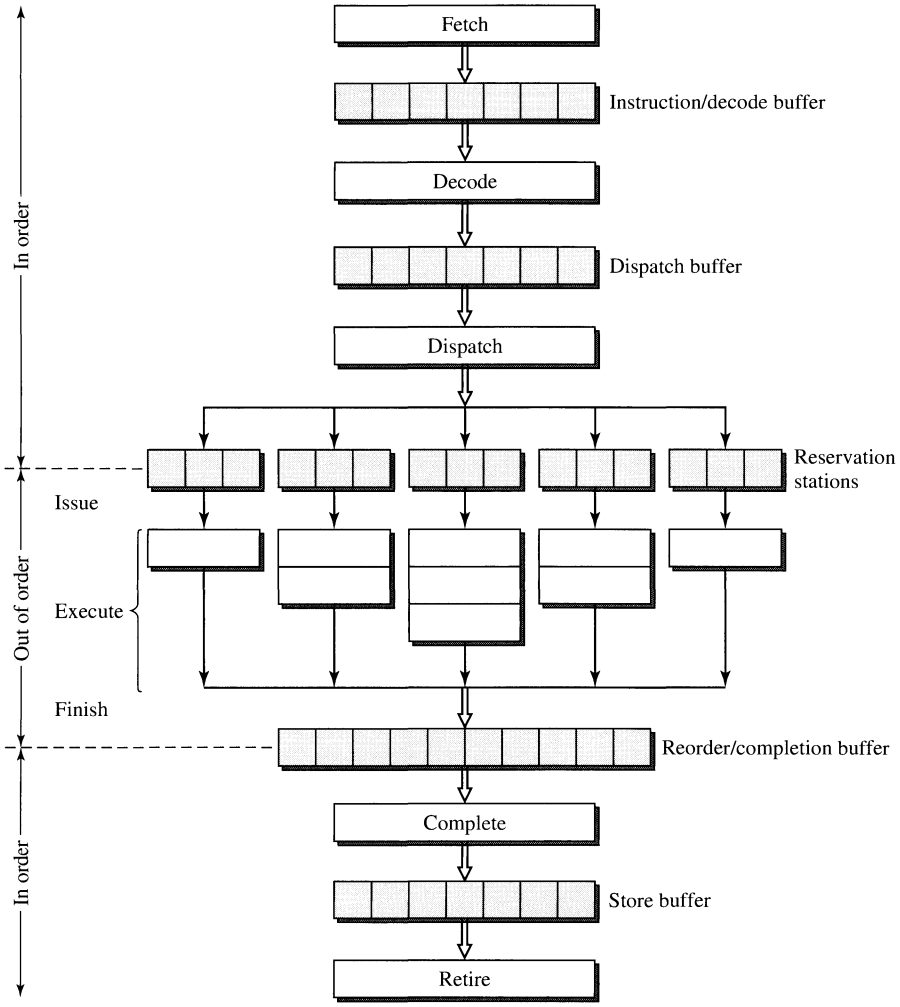


Figure 4.20
A Dynamic Pipeline with Reservation Station and Reorder Buffer.

4.4 Summary

Figure 4.20 represents an archetype of a contemporary out-of-order superscalar pipeline. It has an in-order front end, an out-of-order execution core, and an in-order back end. Both the front-end and back-end pipeline stages can advance multiple instructions per machine cycle. Instructions can remain in the reservation stations for one or more cycles while waiting for their source operands. Once the source operands are available, an instruction is issued from the reservation station into the execution unit. After execution, the instruction enters the reorder buffer (or completion buffer). Instructions in the reorder buffer are completed according to program

order. In fact the reorder buffer is managed as a circular queue with instructions arranged according to the program order.

This chapter focuses on the superscalar pipeline organization and highlights the issues associated with the various pipeline stages. So far, we have addressed mostly the static structures of superscalar pipelines. Chapter 5 will get into the dynamic behavior of superscalar processors. We have chosen to present superscalar pipeline organization at a fairly high level, avoiding the implementation details. The main purpose of this chapter is to provide a bridge from scalar to superscalar pipelines and to convey a high-level framework for superscalar pipelines that will be a useful navigational aid when we get into the plethora of superscalar processor techniques in Chapter 5.

REFERENCES

Acosta, R., J. Kilestrup, and H. Torng: "An instruction issuing approach to enhancing performance in multiple functional unit processors," *IEEE Trans. on Computers,* C35, 9, 1986, pp. 815–828.

Blanck, G., and S. Krueger: "The SuperSPARC microprocessor," *Proc. IEEE COMPCON,* 1992, pp. 136–141.

Crawford, J.: "The execution pipeline of the Intel i486 CPU," *Proc. COMPCON Spring'90,* 1990, pp. 254–258.

Diefendorf, K., and M. Allen: "Organization of the Motorola 88110 superscalar RISC microprocessor," *IEEE MICRO,* 12, 2, 1992, pp. 40–63.

Grohoski, G.: "Machine organization of the IBM RISC System/6000 processor," *IBM Journal of Research and Development,* 34, 1, 1990, pp. 37–58.

Intel Corp.: *Pentium Processor User's Manual,* Vol. 3: *Architecture and Programming Manual.* Santa Clara, CA: Intel Corp., 1993.

Jourdan, S., P. Sainrat, and D. Litaize: "Exploring configurations of functional units in an out-of-order superscalar processor," *Proc. 22nd Annual Int. Symposium on Computer Architecture,* 1995, pp. 117–125.

Oehler, R. R., and R. D. Groves: "IBM RISC System/6000 processor architecture," *IBM Journal of Research and Development,* 34, 1, 1990, pp. 23–36.

Patt, Y., W. Hwu, and M. Shebanow: "HPS, a new microarchitecture: Introduction and rationale," *Proc. 18th Annual Workshop on Microprogramming (MICRO-18),* 1985, pp. 103–108.

Popescu, V., M. Schulz, J. Spracklen, G. Gibson, B. Lightner, and D. Isaman: "The Metaflow architecture," *IEEE Micro.,* June 1991, pp. 10–13, 63–73.

Slavenburg, G., S. Rathnam, and H. Dijkstra: "The TriMedia TM-1 PCI VLIW media processor," *Proc. Hot Chips 8,* 1996, pp. 171–178.

Smith, J., and A. Pleszkun: "Implementing precise interrupts in pipelined processors," *IEEE Trans. on Computers,* 37, 5, 1988, pp. 562–573.

Sohi, G., and S. Vajapeyam: "Instruction issue logic for high-performance, interruptible pipelined processors," *Proc. 14th Annual Int. Symposium on Computer Architecture,* 1987, pp. 27–34.

Thornton, J. E.: "Parallel operation in the Control Data 6600," *AFIPS Proc. FJCC* part 2, vol. 26, 1964, pp. 33–40.

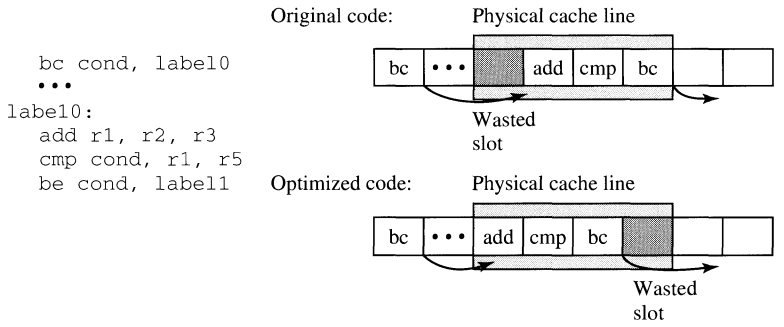
Tomasulo, R.: "An efficient algorithm for exploiting multiple arithmetic units," *IBM Journal of Research and Development,* 11, 1967, pp. 25–33.

HOMEWORK PROBLEMS

P4.1 Is it reasonable to build a scalar pipeline that supports out-of-order execution? If so, describe a code execution scenario where such a pipeline would perform better than a conventional in-order scalar pipeline.

P4.2 Superscalar pipelines require replication of pipeline resources across each parallel pipeline, naively including the replication of cache ports. In practice, however, a two-wide superscalar pipeline may have two data cache ports but only a single instruction cache port. Explain why this is possible, but also discuss why a single instruction cache port can perform worse than two (replicated) instruction cache ports.

P4.3 Section 4.3.1 suggests that a compiler can generate object code where branch targets are aligned at the beginning of physical cache lines to increase the likelihood of fetching multiple instructions from the branch target in a single cycle. However, given a fixed number of instructions between taken branches, this approach may simply shift the unused fetch slots from before the branch target to after the branch that terminates sequential fetch at the target. For example, moving the code at `label0` so it aligns with a physical cache line will not improve fetch efficiency, since the wasted fetch slot shifts from the beginning of the physical line to the end.



Discuss the relationship between fetch block size and the dynamic distance between taken branches. Describe how one affects the other, describe how important is branch target alignment for small vs. large fetch blocks and short vs. long dynamic distance, and describe how well static compiler-based target alignment might work in all cases.

P4.4 The auto-realigning instruction fetch hardware shown in Figure 4.13 still fails to achieve full-width fetch bandwidth (i.e., four instructions per cycle). Describe a more aggressive organization that is always able to fetch four instructions per cycle. Comment on the additional hardware such an organization implies.

P4.5 One idea to eliminate the branch misprediction penalty is to build a machine that executes both paths of a branch. In a two to three paragraph essay, explain why this may or may not be a good idea.

P4.6 Section 4.3.2 discusses adding predecode bits to the instruction cache to simplify the task of decoding instructions after they have been fetched. A logical extension of predecode bits is to simply store the instructions in decoded form in a decoded instruction cache; this is particularly attractive for processors like the Pentium Pro that dynamically translate fetched instructions into a sequence of simpler RISC-like instructions for the core to execute. Identify and describe at least one factor that complicates the building of decoded instruction caches for processors that translate from a complex instruction set to a simpler RISC-like instruction set.

P4.7 What is the most important advantage of a centralized reservation station over distributed reservation stations?

P4.8 In an in-order pipelined processor, pipeline latches are used to hold result operands from the time an execution unit computes them until they are written back to the register file during the writeback stage. In an out-of-order processor, rename registers are used for the same purpose. Given a four-wide out-of-order processor TYP pipeline, compute the minimum number of rename registers needed to prevent rename register starvation from limiting concurrency. What happens to this number if frequency demands force a designer to add five extra pipeline stages between dispatch and execute, and five more stages between execute and retire/writeback?

P4.9 A banked or interleaved cache can be an effective approach for allowing multiple loads or stores to be performed in one cycle. Sketch out the data flow for a two-way interleaved data cache attached to two load/store units. Now sketch the data flow for an eight-way interleaved data cache attached to four load/store units. Comment on how well interleaving scales or does not scale.

P4.10 The Pentium 4 processor operates its integer arithmetic units at double the nominal clock frequency of the rest of the processor. This is accomplished by pipelining the integer adder into two stages, computing the low-order 16 bits in the first cycle and the high-order 16 bits in the second cycle. Naively, this appears to increase ALU latency from one cycle to two cycles. However, assuming that two dependent

instructions are both arithmetic instructions, it is possible to issue the second instruction in the cycle immediately following issue of the first instruction, since the low-order bits of the second instruction are dependent only on the low-order bits of the first instruction. Sketch out a pipeline diagram of such an ALU along with the additional bypass paths needed to handle this optimized case.

P4.11 Given the ALU configuration described in Problem 4.10, specify how many cycles a trailing dependent instruction of each of the following types must delay, following the issue of a leading arithmetic instruction: arithmetic, logical (and/or/xor), shift left, shift right.

P4.12 Explain why the kind of bit-slice pipelining described in Problem 4.10 cannot be usefully employed to pipeline dependent floating-point arithmetic instructions.

P4.13 Assume a four-wide superscalar processor that attempts to retire four instructions per cycle from the reorder buffer. Explain which data dependences need to be checked at this time, and sketch the dependence-checking hardware.

P4.14 Four-wide superscalar processors rarely sustain throughput much greater than one instruction per cycle (IPC). Despite this fact, explain why four-wide retirement is still useful in such a processor.

P4.15 Most general-purpose instruction sets have recently added multimedia extensions to support vector-like operations over arrays of small data types. For example, Intel IA32 has added the MMX and SSE instruction set extensions for this purpose. A single multimedia instruction will load, say, eight 8-bit operands into a 64-bit register in parallel, while arithmetic instructions will perform the same operation on all eight operands in single-instruction, multiple data (SIMD) fashion. Describe the changes you would have to make to the fetch, decode, dispatch, issue, execute, and retire logic of a typical superscalar processor to accommodate these instructions.

P4.16 The PowerPC instruction set provides support for a fused floating-point multiply-add operation that multiplies two of its input registers and adds the product to the third input register. Explain how the addition of such an instruction complicates the decode, dispatch, issue, and execute stages of a typical superscalar processor. What effect do you think these changes will have on the processor's cycle time?

P4.17 The semantics of the fused multiply-add instruction described in Problem 4.16 can be mimicked by issuing a separate floating-point add and floating-point multiply whenever such an instruction is decoded. In fact, the MIPS R10000 does just that; rather than supporting this instruction (which also exists in the MIPS instruction set) directly, the

decoder simply inserts the add/multiply instruction pair into the execution window. Identify and discuss at least two reasons why this approach could reduce performance as measured in instructions per cycle.

P4.18 Does the ALU mix for the Motorola 88110 processor shown in Figure 4.7 agree with the IBM instruction mix provided in Section 2.2.4.3? If not, how would you change the ALU mix?

Terms and Buzzwords

These problems are similar to the "Jeopardy Game" on TV. The "answers" are given and you are to provide the best correct "questions." For each "answer" there may be more than one appropriate "question"; you need to provide the best one.

P4.19 A: A mechanism that tracks out-of-order execution and maintains speculative machine state.

Q: What is _____ ?

P4.20 A: It will significantly reduce the machine cycle time, but can increase the branch penalty.

Q: What is _____ ?

P4.21 A: Additional I-cache bits generated at cache refill time to ease the decoding/dispatching task.

Q: What are _____ ?

P4.22 A: A program attribute that causes inefficiencies in a superscalar fetch unit.

Q: What is _____ ?

P4.23 A: The internal RISC-like instruction executed by the Pentium Pro (P6) microarchitecture.

Q: What is _____ ?

P4.24 A: The logical pipeline stage that assigns an instruction to the appropriate execution unit.

Q: What is _____ ?

P4.25 A: An early processor design that incorporated 10 diverse functional units.

Q: What is _____?

P4.26 A: A new instruction that allows a scalar pipeline to achieve more than one floating-point operation per cycle.

Q: What is _____ ?

P4.27 A: An effective technique for allowing more than one memory operation to be performed per cycle.

Q: What is _____ ?

P4.28 A: A useful architectural property that simplifies the task of writing low-level operating system code.

Q: What is _____ ?

P4.29 A: The first research paper to describe run-time, hardware translation of one instruction set to another, simpler one.

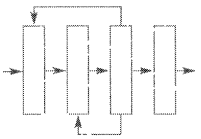
Q: What was _____ ?

P4.30 A: The first real processor to implement run-time, hardware translation of one instruction set to another, simpler one.

Q: What was _____ ?

P4.31 A: This attribute of most RISC instruction sets substantially simplifies the task of decoding multiple instructions in parallel.

Q: What was _____ ?



CHAPTER

5

Superscalar Techniques

CHAPTER OUTLINE

In Chapter 4 we focused on the structural, or organizational, design of the superscalar pipeline and dealt with issues that were somewhat independent of the specific types of instructions being processed. In this chapter we focus more on the dynamic behavior of a superscalar processor and consider techniques that deal with specific types of instructions. The ultimate performance goal of a superscalar pipeline is to achieve maximum throughput of instruction processing. It is convenient to view instruction processing as involving three component flows of instructions and/or data, namely, *instruction flow, register data flow,* and *memory data flow*. This partitioning into three flow paths is similar to that used in Mike Johnson's 1991 textbook entitled *Superscalar Microprocessor Design* [Johnson, 1991]. The overall performance objective is to maximize the volumes in all three of these flow paths. Of course, what makes this task interesting is that the three flow paths are not independent and their interactions are quite complex. This chapter classifies and presents superscalar microarchitecture techniques based on their association with the three flow paths.

The three flow paths correspond roughly to the processing of the three major types of instructions, namely, branch, ALU, and load/store instructions. Consequently, maximizing the throughput of the three flow paths corresponds to the minimizing of the branch, ALU, and load penalties.

1. *Instruction flow*. Branch instruction processing.
2. *Register data flow*. ALU instruction processing.
3. *Memory data flow*. Load/store instruction processing.

This chapter uses these three flow paths as a convenient framework for presenting the plethora of microarchitecture techniques for optimizing the performance of modern superscalar processors.

5.1 Instruction Flow Techniques

We present instruction flow techniques first because these deal with the early stages, e.g., the fetch and decode stages, of a superscalar pipeline. The throughput of the early pipeline stages will impose an upper bound on the throughput of all subsequent stages. For contemporary pipelined processors, the traditional partitioning of a processor into control path and data path is no longer clear or effective. Nevertheless, the early pipeline stages along with the branch execution unit can be viewed as corresponding to the traditional control path whose primary function is to enforce the control flow semantics of a program. The primary goal for all instruction flow techniques is to maximize the supply of instructions to the superscalar pipeline subject to the requirements of the control flow semantics of a program.

5.1.1 Program Control Flow and Control Dependences

The control flow semantics of a program are specified in the form of the control flow graph (CFG), in which the nodes represent basic blocks and the edges represent the transfer of control flow between basic blocks. Figure 5.1(a) illustrates a CFG with four basic blocks (dashed-line rectangles), each containing a number of instructions (ovals). The directed edges represent control flows between basic blocks. These edges are induced by conditional branch instructions (diamonds). The run-time execution of a program entails the dynamic traversal of the nodes and edges of its CFG. The actual path of traversal is dictated by the branch instructions and their branch conditions which can be dependent on run-time data.

The basic blocks, and their constituent instructions, of a CFG must be stored in sequential locations in the program memory. Hence the partial ordered basic blocks in a CFG must be arranged in a total order in the program memory. In mapping a CFG to linear consecutive memory locations, additional unconditional branch instructions must be added, as illustrated in Figure 5.1(b). The mapping of the CFG to a linear program memory facilitates an implied sequential flow of control along the sequential memory locations during program execution. However, the encounter of both conditional and unconditional branches at run time induces deviations from this implied sequential control flow and the consequent disruptions to the sequential fetching of instructions. Such disruptions cause stalls in the instruction fetch stage of the pipeline and reduce the overall instruction fetching bandwidth. Subroutine jump and return instructions also induce similar disruptions to the sequential fetching of instructions.

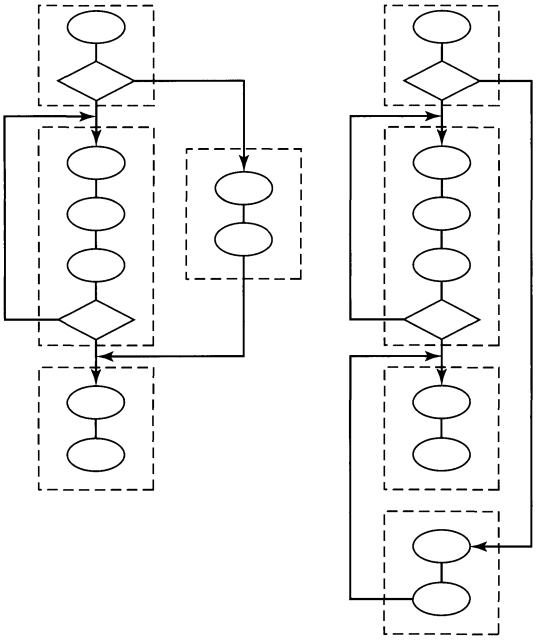


Figure 5.1
Program Control Flow: (a) The Control Flow Graph (CFG); (b) Mapping the CFG to Sequential Memory Locations.

5.1.2 Performance Degradation Due to Branches

A pipelined machine achieves its maximum throughput when it is in the streaming mode. For the fetch stage, streaming mode implies the continuous fetching of instructions from sequential locations in the program memory. Whenever the control flow of the program deviates from the sequential path, potential disruption to the streaming mode can occur. For unconditional branches, subsequent instructions cannot be fetched until the target address of the branch is determined. For conditional branches, the machine must wait for the resolution of the branch condition, and if the branch is to be taken, it must further wait until the target address is available. Figure 5.2 illustrates the disruption of the streaming mode by branch instructions. Branch instructions are executed by the branch functional unit. For a conditional branch, it is not until it exits the branch unit and when both the branch condition and the branch target address are known that the fetch stage can correctly fetch the next instruction.

As Figure 5.2 illustrates, this delay in processing conditional branches incurs a penalty of three cycles in fetching the next instruction, corresponding to the traversal of the decode, dispatch, and execute stages by the conditional branch. The actual lost-opportunity cost of three stalled cycles is not just three empty instruction

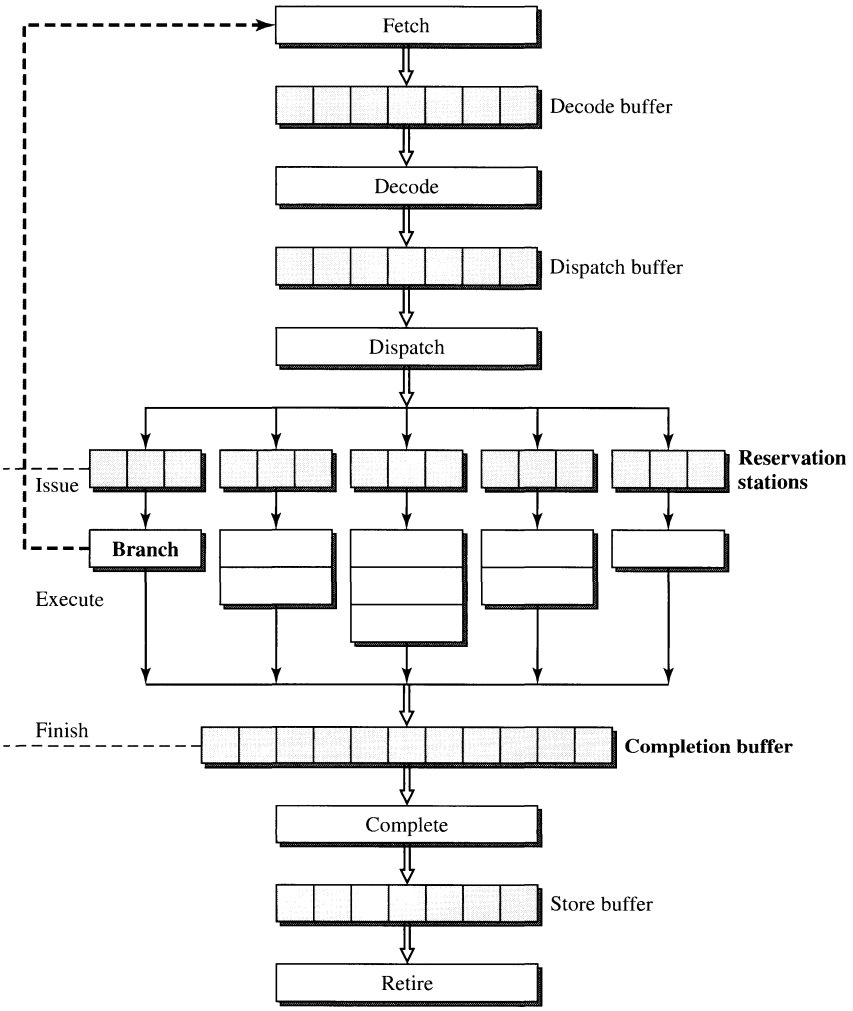


Figure 5.2
Disruption of Sequential Control Flow by Branch Instructions.

slots as in the scalar pipeline, but the number of empty instruction slots must be multiplied by the width of the machine. For example, for a four-wide machine the total penalty is 12 instruction "bubbles" in the superscalar pipeline. Also recall from Chapter 1, that such pipeline stall cycles effectively correspond to the *sequential bottleneck* of Amdahl's law and rapidly and significantly reduce the actual performance from the potential peak performance.

For conditional branches, the actual number of stalled or penalty cycles can be dictated by either target address generation or condition resolution. Figure 5.3 illustrates the potential cycles that can be incurred by target address generation. The

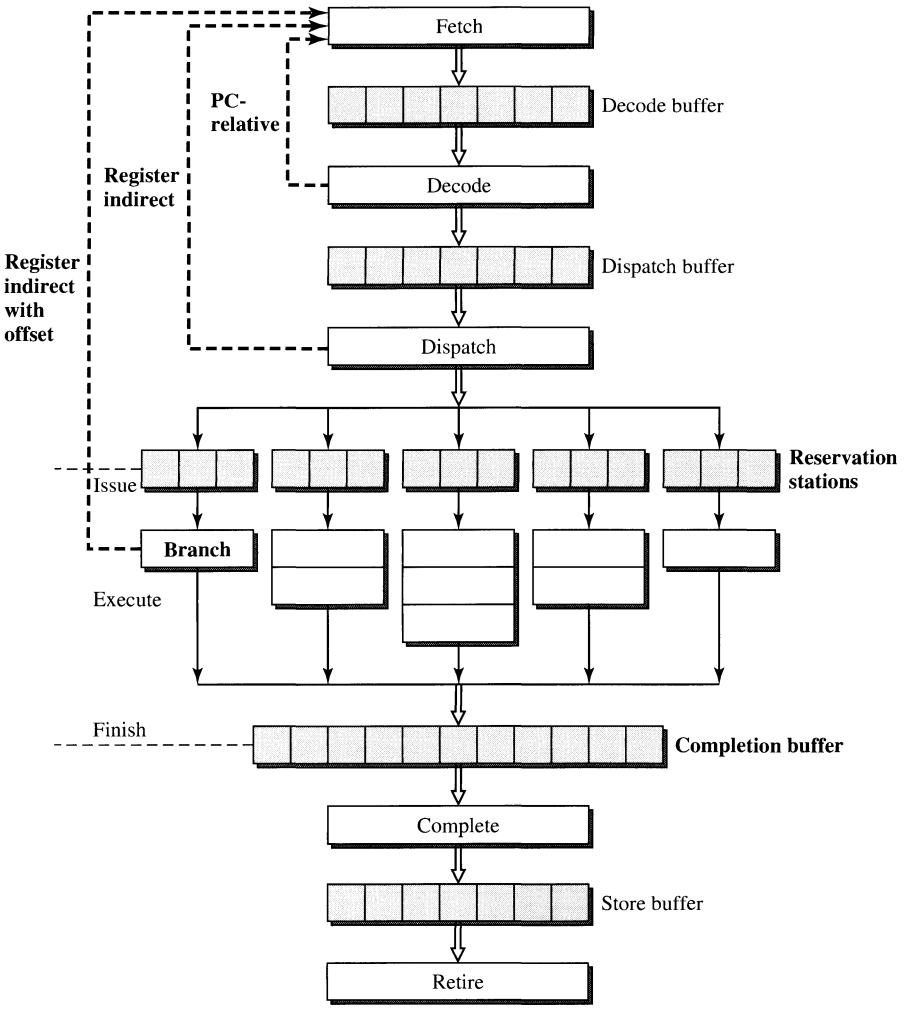


Figure 5.3
Branch Target Address Generation Penalties.

actual number of penalty cycles is determined by the addressing modes of the branch instructions. For the PC-relative addressing mode, the branch target address can be generated during the fetch stage, resulting in a penalty of one cycle. If the register indirect addressing mode is used, the branch instruction must traverse the decode stage to access the register. In this case a two-cycle penalty is incurred. For register indirect with an offset addressing mode, the offset must be added after register access and a total three-cycle penalty can result. For unconditional branches, only the penalty due to target address generation is of concern. For conditional branches, branch condition resolution latency must also be considered.

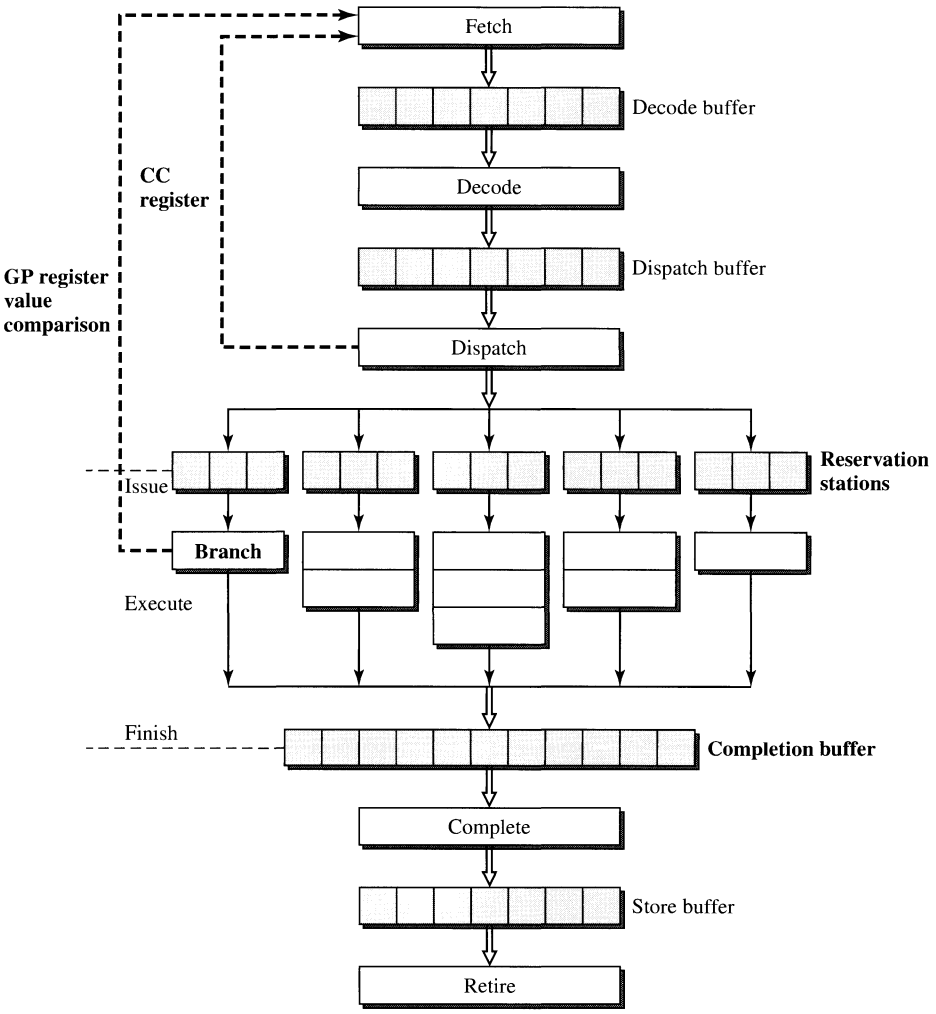


Figure 5.4
Branch Condition Resolution Penalties.

Different methods for performing condition resolution can also lead to different penalties. Figure 5.4 illustrates two possible penalties. If condition code registers are used, and assuming that the relevant condition code register is accessed during the dispatch stage, then a penalty of two cycles will result. If the ISA permits the comparison of two general-purpose registers to generate the branch condition, then one more cycle is needed to perform an ALU operation on the contents of the two registers. This will result in a penalty of three cycles. For a conditional branch, depending on the addressing mode and condition resolution method used, either one of the penalties may be the critical one. For example, even if the PC-relative

addressing mode is used, a conditional branch that must access a condition code register will still incur a two-cycle penalty instead of the one-cycle penalty for target address generation.

Maximizing the volume of the instruction flow path is equivalent to maximizing the sustained instruction fetch bandwidth. To do this, the number of stall cycles in the fetch stage must be minimized. Recall that the total lost-opportunity cost is equal to the product of the number of penalty cycles and the width of a machine. For an n-wide machine each stalled cycle is equal to fetching n no-op instructions. The primary aim of instruction flow techniques is to minimize the number of such fetch stall cycles and/or to make use of these cycles to do potentially useful work. The current dominant approach to accomplishing this is via branch prediction which is the subject of Section 5.1.3.

5.1.3 Branch Prediction Techniques

Experimental studies have shown that the behavior of branch instructions is highly predictable. A key approach to minimizing branch penalty and maximizing instruction flow throughput is to speculate on both branch target addresses and branch conditions of branch instructions. As a static branch instruction is repeatedly executed at run time, its dynamic behavior can be tracked. Based on its past behavior, its future behavior can be effectively predicted. Two fundamental components of branch prediction are *branch target speculation* and *branch condition speculation*. With any speculative technique, there must be mechanisms to validate the prediction and to safely recover from any mispredictions. Branch misprediction recovery will be covered in Section 5.1.4.

Branch target speculation involves the use of a branch target buffer (BTB) to store previous branch target addresses. BTB is a small cache memory accessed during the instruction fetch stage using the instruction fetch address (PC). Each entry of the BTB contains two fields: the branch instruction address (BIA) and the branch target address (BTA). When a static branch instruction is executed for the first time, an entry in the BTB is allocated for it. Its instruction address is stored in the BIA field, and its target address is stored in the BTA field. Assuming the BTB is a fully associative cache, the BIA field is used for the associative access of the BTB. The BTB is accessed concurrently with the accessing of the I-cache. When the current PC matches the BIA of an entry in the BTB, a hit in the BTB results. This implies that the current instruction being fetched from the I-cache has been executed before and is a branch instruction. When a hit in the BTB occurs, the BTA field of the hit entry is accessed and can be used as the next instruction fetch address if that particular branch instruction is predicted to be taken; see Figure 5.5.

By accessing the BTB using the branch instruction address and retrieving the branch target address from the BTB all during the fetch stage, the speculative branch target address will be ready to be used in the next machine cycle as the new instruction fetch address if the branch instruction is predicted to be taken. If the branch instruction is predicted to be taken and this prediction turns out to be correct, then the branch instruction is effectively executed in the fetch stage, incurring no branch penalty. The nonspeculative execution of the branch instruction is still

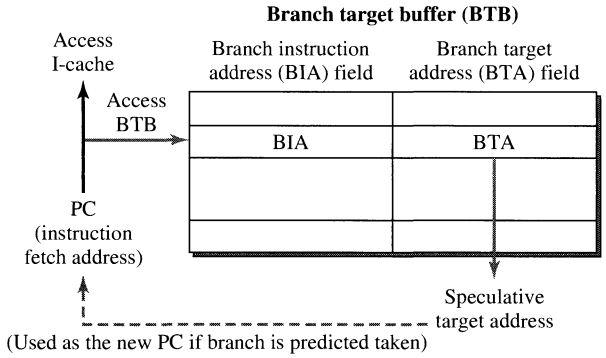


Figure 5.5
Branch Target Speculation Using a Branch Target Buffer.

performed for the purpose of validating the speculative execution. The branch instruction is still fetched from the I-cache and executed. The resultant target address and branch condition are compared with the speculative version. If they agree, then correct prediction was made; otherwise, misprediction has occurred and recovery must be initiated. The result from the nonspeculative execution is also used to update the content, i.e., the BTA field, of the BTB.

There are a number of ways to do branch condition speculation. The simplest form is to design the fetch hardware to be biased for not taken, i.e., to always predict not taken. When a branch instruction is encountered, prior to its resolution, the fetch stage continues fetching down the fall-through path without stalling. This form of minimal branch prediction is easy to implement but is not very effective. For example, many branches are used as loop closing instructions, which are mostly taken during execution except when exiting loops. Another form of prediction employs software support and can require ISA changes. For example, an extra bit can be allocated in the branch instruction format that is set by the compiler. This bit is used as a hint to the hardware to perform either predict not taken or predict taken depending on the value of this bit. The compiler can use branch instruction type and profiling information to determine the most appropriate value for this bit. This allows each static branch instruction to have its own specified prediction. However, this prediction is static in the sense that the same prediction is used for all dynamic executions of the branch. Such static software prediction technique is used in the Motorola 88110 [Diefendorf and Allen, 1992]. A more aggressive and dynamic form of prediction makes prediction based on the branch target address offset. This form of prediction first determines the relative offset between the address of the branch instruction and the address of the target instruction. A positive offset will trigger the hardware to predict not taken, whereas a negative offset, most likely indicating a loop closing branch, will trigger the hardware to predict taken. This branch offset-based technique is used in the original IBM RS/6000 design and has been adopted by other machines as well [Grohoski, 1990; Oehler and Groves, 1990]. The most common branch condition speculation technique

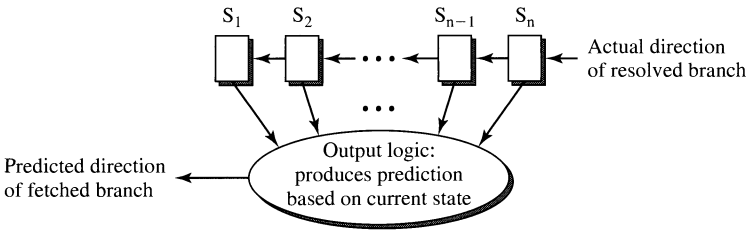


Figure 5.6
FSM Model for History-Based Branch Direction Predictors.

employed in contemporary superscalar machines is based on the history of previous branch executions.

History-based branch prediction makes a prediction of the branch direction, whether taken (T) or not taken (N), based on previously observed branch directions. This approach was first proposed by Jim Smith, who patented the technique on behalf of his employer, Control Data, and later published an important early study [Smith, 1981]. The assumption is that historical information on the direction that a static branch takes in previous executions can give helpful hints on the direction that it is likely to take in future executions. Design decisions for such type of branch prediction include how much history should be tracked and for each observed history pattern what prediction should be made. The specific algorithm for history-based branch direction prediction can be characterized by a finite state machine (FSM); see Figure 5.6. The n state variables encode the directions taken by the last n executions of that branch. Hence each state represents a particular history pattern in terms of a sequence of takens and not takens. The output logic generates a prediction based on the current state of the FSM. Essentially, a prediction is made based on the outcome of the previous n executions of that branch. When a predicted branch is finally executed, the actual outcome is used as an input to the FSM to trigger a state transition. The next state logic is trivial; it simply involves chaining the state variables into a shift register, which records the branch directions of the previous n executions of that branch instruction.

Figure 5.7(a) illustrates the FSM diagram of a typical 2-bit branch predictor that employs two history bits to track the outcome of two previous executions of the branch. The two history bits constitute the state variables of the FSM. The predictor can be in one of four states: NN, NT, TT, or TN, representing the directions taken in the previous two executions of the branch. The NN state can be designated as the initial state. An output value of either T or N is associated with each of the four states representing the prediction that would be made when a predictor is in that state. When a branch is executed, the actual direction taken is used as an input to the FSM, and a state transition occurs to update the branch history which will be used to do the next prediction.

The particular algorithm implemented in the predictor of Figure 5.7(a) is biased toward predicting branches to be taken; note that three of the four states

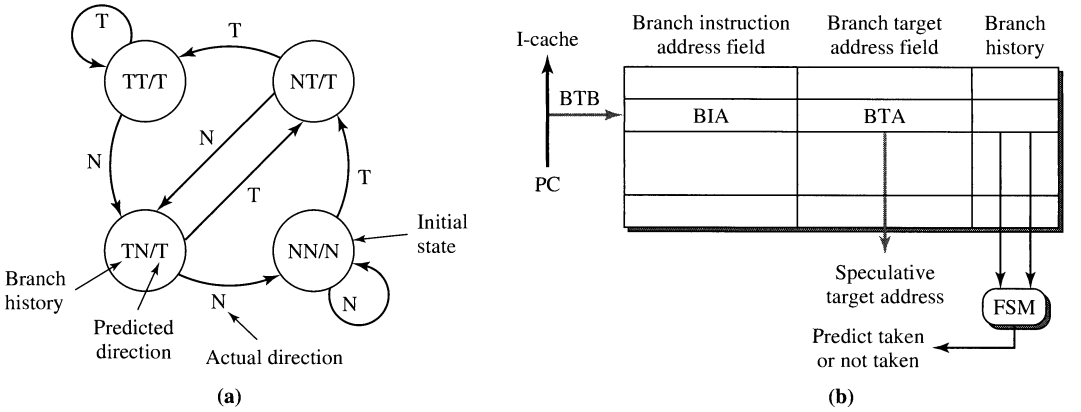


Figure 5.7

History-Based Branch Prediction: (a) A 2-Bit Branch Predictor Algorithm; (b) Branch Target Buffer with an Additional Field for Storing Branch History Bits.

predict the branch to be taken. It anticipates either long runs of N's (in the NN state) or long runs of T's (in the TT state). As long as at least one of the two previous executions was a taken branch, it will predict the next execution to be taken. The prediction will only be switched to not taken when it has encountered two consecutive N's in a row. This represents one particular branch direction prediction algorithm; clearly there are many possible designs for such history-based predictors, and many designs have been evaluated by researchers.

To support history-based branch direction predictors, the BTB can be augmented to include a history field for each of its entries. The width, in number of bits, of this field is determined by the number of history bits being tracked. When a PC address hits in the BTB, in addition to the speculative target address, the history bits are retrieved. These history bits are fed to the logic that implements the next-state and output functions of the branch predictor FSM. The retrieved history bits are used as the state variables of the FSM. Based on these history bits, the output logic produces the 1-bit output that indicates the predicted direction. If the prediction is a taken branch, then this output is used to steer the speculative target address to the PC to be used as the new instruction fetch address in the next machine cycle. If the prediction turns out to be correct, then effectively the branch instruction has been executed in the fetch stage without incurring any penalty or stalled cycle.

A classic experimental study on branch prediction was done by Lee and Smith [1984]. In this study, 26 programs from six different types of workloads for three different machines (IBM 370, DEC PDP-11, and CDC 6400) were used. Averaged across all the benchmarks, 67.6% of the branches were taken while 32.4% were not taken. Branches tend to be taken more than not taken by a ratio of 2 to 1. With static branch prediction based on the op-code type, the prediction accuracy ranged from 55% to 80% for the six workloads. Using only 1 bit of history, history-based dynamic branch prediction achieved prediction accuracies ranging from 79.7% to

96.5%. With 2 history bits, the accuracies for the six workloads ranged from 83.4% to 97.5%. Continued increase of the number of history bits brought additional incremental accuracy. However, beyond four history bits there is a very minimal increase in the prediction accuracy. They implemented a four-way set associative BTB that had 128 sets. The averaged BTB hit rate was 86.5%. Combining prediction accuracy with the BTB hit rate, the resultant average prediction effectiveness was approximately 80%.

Another experimental study was done in 1992 at IBM by Ravi Nair using the RS/6000 architecture and Systems Performance Evaluation Cooperative (SPEC) benchmarks [Nair, 1992]. This was a very comprehensive study of possible branch prediction algorithms. The goal for branch prediction is to overlap the execution of branch instructions with that of other instructions so as to achieve zero-cycle branches or accomplish *branch folding;* i.e., branches are folded out of the critical latency path of instruction execution. This study performed an exhaustive search for optimal 2-bit predictors. There are 2^{20} possible FSMs of 2-bit predictors. Nair determined that many of these machines are uninteresting and pruned the entire design space down to 5248 machines. Extensive simulations are performed to determine the optimal (achieves the best prediction accuracy) 2-bit predictor for each of the benchmarks. The list of SPEC benchmarks, their best prediction accuracies, and the associated optimal predictors are shown in Figure 5.8.

In Figure 5.8, the states denoted with bold circles represent states in which the branch is predicted taken; the nonbold circles represent states that predict not taken. Similarly the bold edges represent state transitions when the branch is actually taken; the nonbold edges represent transitions corresponding to the branch

Benchmark	Optimal	"Counter"
spice2g6	97.2	97.0
doduc	94.3	94.3
gcc	89.1	89.1
espresso	89.1	89.1
li	87.1	86.8
eqntott	87.9	87.2

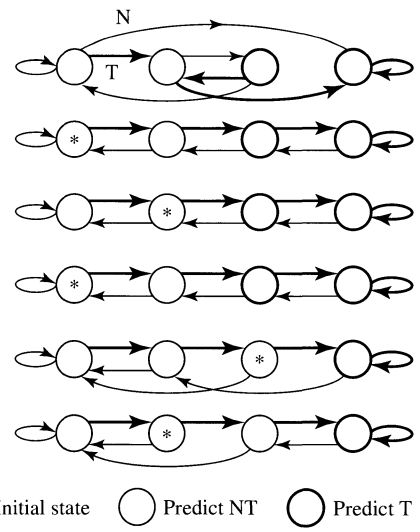


Figure 5.8
Optimal 2-Bit Branch Predictors for Six SPEC Benchmarks from the Nair Study.

actually not taken. The state denoted with an asterisk indicates the initial state. The prediction accuracies for the optimal predictors of these six benchmarks range from 87.1% to 97.2%. Notice that the optimal predictors for *doduc, gcc,* and *espresso* are identical (disregarding the different initial state of the *gcc* predictor) and exhibit the behavior of a 2-bit up/down saturating counter. We can label the four states from left to right as 0, 1, 2, and 3, representing the four count values of a 2-bit counter. Whenever a branch is resolved taken, the count is incremented; and it is decremented otherwise. The two lower-count states predict a branch to be not taken, while the two higher-count states predict a branch to be taken. Figure 5.8 also provides the prediction accuracies for the six benchmarks if the 2-bit saturating counter predictor is used for all six benchmarks. The prediction accuracies for *spice2g6, li,* and *eqntott* only decrease minimally from their optimal values, indicating that the 2-bit saturating counter is a good candidate for general use on all benchmarks. In fact, the 2-bit saturating counter, originally invented by Jim Smith, has become a popular prediction algorithm in real and experimental designs.

The same study by Nair also investigated the effectiveness of counter-based predictors. With a 1-bit counter as the predictor, i.e., remembering the direction taken last time and predicting the same direction for the next time, the prediction accuracies ranged from 82.5% to 96.2%. As we have seen in Figure 5.8, a 2-bit counter yields an accuracy range of 86.8% to 97.0%. If a 3-bit counter is used, the increase in accuracy is minimal; accuracies range from 88.3% to 97.0%. Based on this study, the 2-bit saturating counter appears to be a very good choice for a history-based predictor. Direct-mapped branch history tables are assumed in this study. While some programs, such as *gcc,* have more than 7000 conditional branches, for most programs, the branch penalty due to aliasing in finite-sized branch history tables levels out at about 1024 entries for the table size.

5.1.4 Branch Misprediction Recovery

Branch prediction is a speculative technique. Any speculative technique requires mechanisms for validating the speculation. Dynamic branch prediction can be viewed as consisting of two interacting engines. The leading engine performs speculation in the front-end stages of the pipeline, while a trailing engine performs validation in the later stages of the pipeline. In the case of misprediction the trailing engine also performs recovery. These two aspects of branch prediction are illustrated in Figure 5.9.

Branch speculation involves predicting the direction of a branch and then proceeding to fetch along the predicted path of control flow. While fetching from the predicted path, additional branch instructions may be encountered. Prediction of these additional branches can be similarly performed, potentially resulting in speculating past multiple conditional branches before the first speculated branch is resolved. Figure 5.9(a) illustrates speculating past three branches with the first and the third branches being predicted taken and the second one predicted not taken. When this occurs, instructions from three speculative basic blocks are now resident in the machine and must be appropriately identified. Instructions from each speculative basic block are given the same identifying tag. In the example of

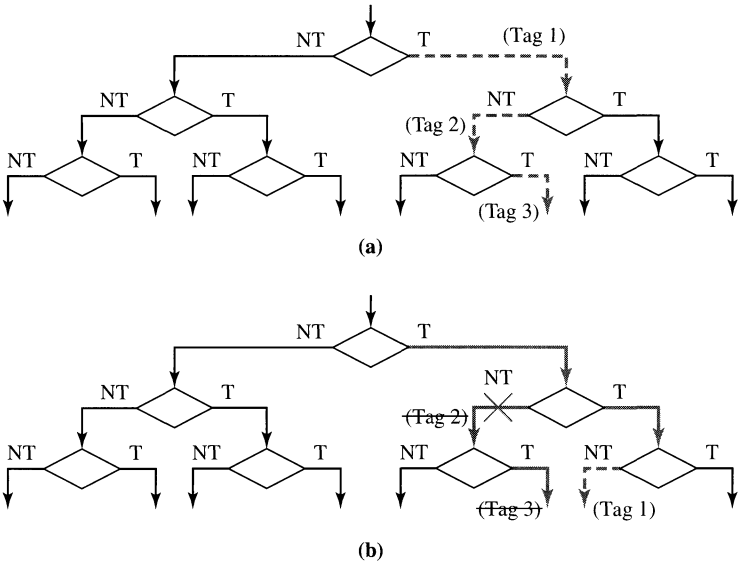


Figure 5.9
Two Aspects of Branch Prediction: (a) Branch Speculation; (b) Branch Validation/Recovery.

Figure 5.9(a), three distinct tags are used to identify the instructions from the three speculative basic blocks. A tagged instruction indicates that it is a speculative instruction, and the value of the tag identifies which basic block it belongs to. As a speculative instruction advances down the pipeline stages, the tag is also carried along. When speculating, the instruction addresses of all the speculated branch instructions (or the next sequential instructions) must be buffered in the event that recovery is required.

Branch validation occurs when the branch is executed and the actual direction of a branch is resolved. The correctness of the earlier prediction can then be determined. If the prediction turns out to be correct, the speculation tag is deallocated and all the instructions associated with that tag become nonspeculative and are allowed to complete. If a misprediction is detected, two actions are required; namely, the incorrect path must be terminated, and fetching from a new correct path must be initiated. To initiate a new path, the PC must be updated with a new instruction fetch address. If the incorrect prediction was a not-taken prediction, then the PC is updated with the computed branch target address. If the incorrect prediction was a taken prediction, then the PC is updated with the sequential (fall-through) instruction address, which is obtained from the previously buffered instruction address when the branch was predicted taken. Once the PC has been updated, fetching of instructions resumes along the new path, and branch prediction begins anew. To terminate the incorrect path, speculation tags are used. All the tags that are associated with the mispredicted branch are used to identify the

instructions that must be eliminated. All such instructions that are still in the decode and dispatch buffers as well as those in reservation station entries are invalidated. Reorder buffer entries occupied by these instructions are deallocated. Figure 5.9(b) illustrates this validation/recovery task when the second of the three predictions is incorrect. The first branch is correctly predicted, and therefore instructions with Tag 1 become nonspeculative and are allowed to complete. The second prediction is incorrect, and all the instructions with Tag 2 and Tag 3 must be invalidated and their reorder buffer entries must be deallocated. After fetching down the correct path, branch prediction can begin once again, and Tag 1 is used again to denote the instructions in the first speculative basic block. During branch validation, the associated BTB entry is also updated.

We now use the PowerPC 604 superscalar microprocessor to illustrate the implementation of dynamic branch prediction in a real superscalar processor. The PowerPC 604 is a four-wide superscalar capable of fetching, decoding, and dispatching up to four instructions in every machine cycle [IBM Corp., 1994]. Instead of a single unified BTB, the PowerPC 604 employs two separate buffers to support branch prediction, namely, the branch target address cache (BTAC) and the branch history table (BHT); see Figure 5.10. The BTAC is a 64-entry fully associative cache that stores the branch target addresses, while the BHT, a 512-entry direct-mapped table, stores the history bits of branches. The reason for this separation will become clear shortly.

Both the BTAC and the BHT are accessed during the fetch stage using the current instruction fetch address in the PC. The BTAC responds in one cycle; however, the BHT requires two cycles to complete its access. If a hit occurs in the BTAC, indicating the presence of a branch instruction in the current fetch group, a predict taken occurs and the branch target address retrieved from the BTAC is used in the next fetch cycle. Since the PowerPC 604 fetches four instructions in a fetch cycle, there can be multiple branches in the fetch group. Hence, the BTAC entry indexed by the fetch address contains the branch target address of the first branch instruction in the fetch group that is predicted to be taken. In the second cycle, or during the decode stage, the history bits retrieved from the BHT are used to generate a history-based prediction on the same branch. If this prediction agrees with the taken prediction made by the BTAC, the earlier prediction is allowed to stand. On the other hand, if the BHT prediction disagrees with the BTAC prediction, the BTAC prediction is annulled and fetching from the fall-through path, corresponding to predict not taken, is initiated. In essence, the BHT prediction can overrule the BTAC prediction. As expected, in most cases the two predictions agree. In some cases, the BHT corrects the wrong prediction made by the BTAC. It is possible, however, for the BHT to erroneously change the correct prediction of the BTAC; this occurs very infrequently. When a branch is resolved, the BHT is updated; and based on its updated content the BHT in turn updates the BTAC by either leaving an entry in the BTAC if it is to be predicted taken the next time, or deleting an entry from the BTAC if that branch is to be predicted not taken the next time.

The PowerPC 604 has four entries in the reservation station that feeds the branch execution unit. Hence, it can speculate past up to four branches; i.e., there

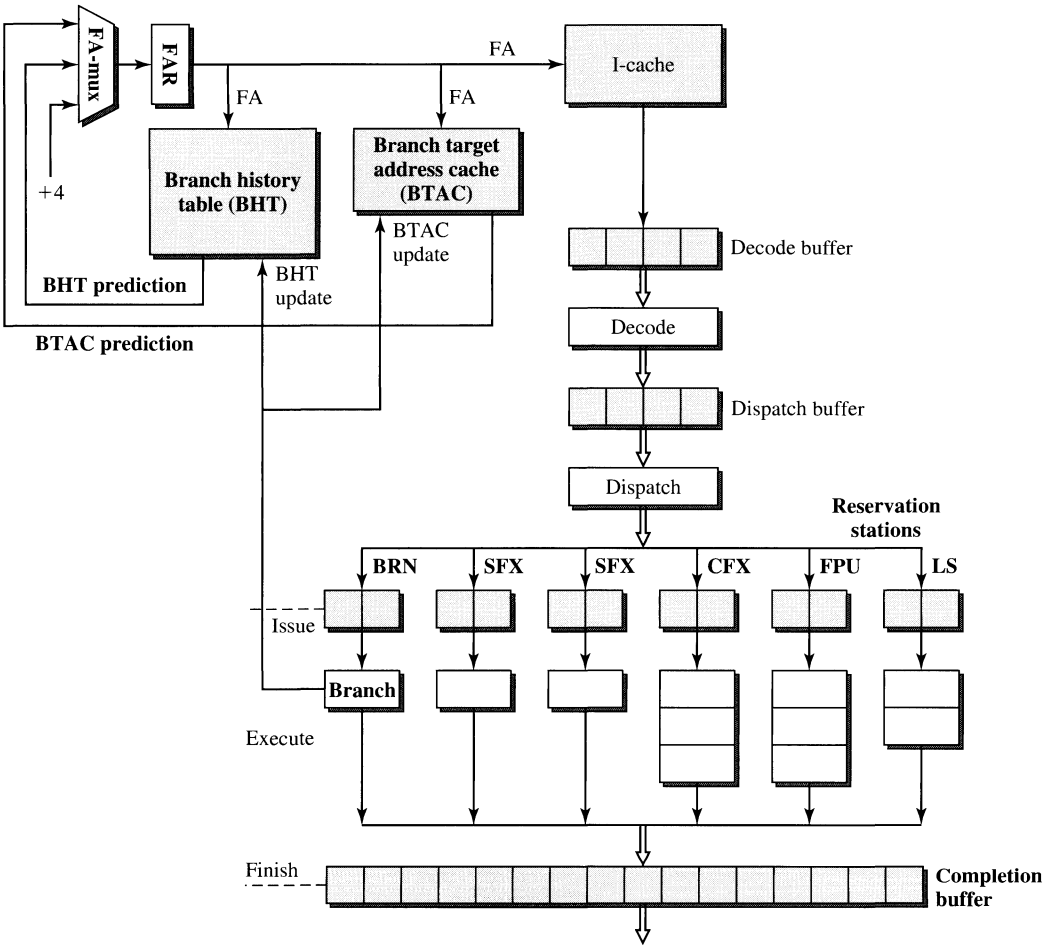


Figure 5.10
Branch Prediction in the PowerPC 604 Superscalar Microprocessor.

can be a maximum of four speculative branches present in the machine. To denote the four speculative basic blocks involved, a 2-bit tag is used to identify all speculative instructions. After a branch resolves, branch validation takes place and all speculative instructions either are made nonspeculative or are invalidated via the use of the 2-bit tag. Reorder buffer entries occupied by misspeculated instructions are deallocated. Again, this is performed using the 2-bit tag.

5.1.5 Advanced Branch Prediction Techniques

The dynamic branch prediction schemes discussed thus far have a number of limitations. Prediction for a branch is made based on the limited history of only that particular static branch instruction. The actual prediction algorithm does not take

into account the dynamic context within which the branch is being executed. For example, it does not make use of any information on the particular control flow path taken in arriving at that branch. Furthermore the same fixed algorithm is used to make the prediction regardless of the dynamic context. It has been observed experimentally that the behavior of certain branches is strongly correlated with the behavior of other branches that precede them during execution. Consequently more accurate branch prediction can be achieved with algorithms that take into account the branch history of other correlated branches and that can adapt the prediction algorithm to the dynamic branching context.

In 1991, Yeh and Patt proposed a two-level adaptive branch prediction technique that can potentially achieve better than 95% prediction accuracy by having a highly flexible prediction algorithm that can adapt to changing dynamic contexts [Yeh and Patt, 1991]. In previous schemes, a single branch history table is used and indexed by the branch address. For each branch address there is only one relevant entry in the branch history table. In the two-level adaptive scheme, a set of history tables is used. These are identified as the pattern history table (PHT); see Figure 5.11. Each branch address indexes to a set of relevant entries; one of these entries is then selected based on the dynamic branching context. The context is determined by a specific pattern of recently executed branches stored in a branch history shift register (BHSR); see Figure 5.11. The content of the BHSR is used to index into the PHT to select one of the relevant entries. The content of this entry is then used as the state for the prediction algorithm FSM to produce a prediction. When a branch is resolved, the branch result is used to update both the BHSR and the selected entry in the PHT.

The two-level adaptive branch prediction technique actually specifies a framework within which many possible designs can be implemented. There are two options

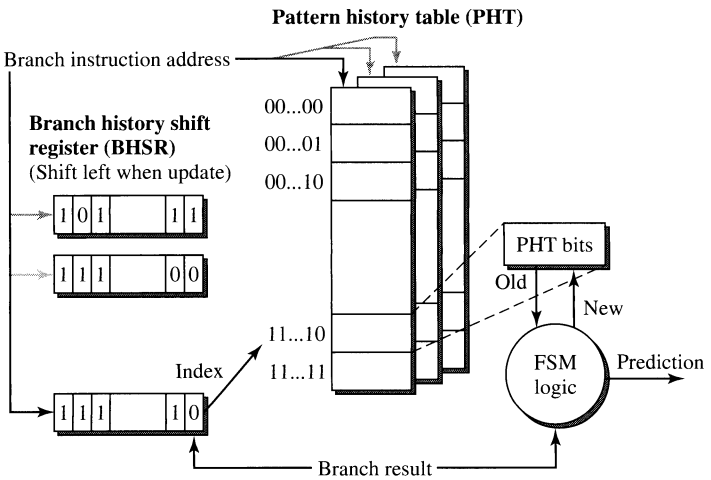


Figure 5.11

Two-Level Adaptive Branch Prediction of Yeh and Patt.

Source: Yeh and Patt, 1991.

to implementing the BHSR: *global* (G) and *individual* (P). The global implementation employs a single BHSR of k bits that tracks the branch directions of the last k dynamic branch instructions in program execution. These can involve any number (1 to k) of static branch instructions. The individual (called *per-branch* by Yeh and Patt) implementation employs a set of k-bit BHSRs as illustrated in Figure 5.11, one of which is selected based on the branch address. Essentially the global BHSR is shared by all static branches, whereas with individual BHSRs each BHSR is dedicated to each static branch or a subset of static branches if there is address aliasing when indexing into the set of BHSRs using the branch address. There are three options to implementing the PHT: *global* (g), *individual* (p), or *shared* (s). The global PHT uses a single table to support the prediction of all static branches. Alternatively, individual PHTs can be used in which each PHT is dedicated to each static branch (p) or a small subset of static branches (s) if there is address aliasing when indexing into the set of PHTs using the branch address. A third dimension to this design space involves the implementation of the actual prediction algorithm. When a history-based FSM is used to implement the prediction algorithm, Yeh and Patt identified such schemes as *adaptive* (A).

All possible implementations of the two-level adaptive branch prediction can be classified based on these three dimensions of design parameters. A given implementation can then be denoted using a three-letter notation; e.g., GAs represents a design that employs a single global BHSR, an adaptive prediction algorithm, and a set of PHTs with each being shared by a number of static branches. Yeh and Patt presented three specific implementations that are able to achieve a prediction accuracy of 97% for their given set of benchmarks:

- GAg: (1) BHSR of size 18 bits; (1) PHT of size $2^{18} \times 2$ bits.
- PAg: (512×4) BHSRs of size 12 bits; (1) PHT of size $2^{12} \times 2$ bits.
- PAs: (512×4) BHSRs of size 6 bits; (512) PHTs of size $2^{6} \times 2$ bits.

All three implementations use an adaptive (A) predictor that is a 2-bit FSM. The first implementation employs a global BHSR (G) of 18 bits and a global PHT (g) with 2^{18} entries indexed by the BHSR bits. The second implementation employs 512 sets (four-way set-associative) of 12-bit BHSRs (P) and a global PHT (g) with 2^{12} entries. The third implementation also employs 512 sets of four-way set-associative BHSRs (P), but each is only 6 bits wide. It also uses 512 PHTs (s), each having 2^{6} entries indexed by the BHSR bits. Both the 512 sets of BHSRs and the 512 PHTs are indexed using 9 bits of the branch address. Additional branch address bits are used for the set-associative access of the BHSRs. The 512 PHTs are direct-mapped, and there can be aliasing, i.e., multiple branch addresses sharing the same PHT. From experimental data, such aliasing had minimal impact on degrading the prediction accuracy. Achieving greater than 95% prediction accuracy by the two-level adaptive branch prediction schemes is quite impressive; the best traditional prediction techniques can only achieve about 90% prediction accuracy. The two-level adaptive branch prediction approach has been adopted by a number of real designs, including the Intel Pentium Pro and the AMD/NexGen Nx686.

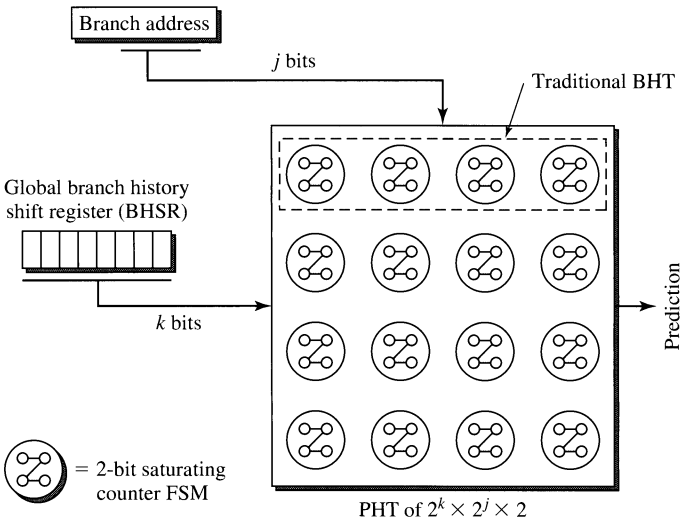


Figure 5.12
Correlated Branch Predictor with Global BHSR and Shared PHTs (GAs).

Following the original Yeh and Patt proposal, other studies by McFarling [1993], Young and Smith [1994], and Gloy et al. [1995] have gained further insights into two-level adaptive, or more recently called *correlated,* branch predictors. Figure 5.12 illustrates a correlated branch predictor with a global BHSR (G) and a shared PHT (s). The 2-bit saturating counter is used as the predictor FSM. The global BHSR tracks the directions of the last k dynamic branches and captures the dynamic control flow context. The PHT can be viewed as a single table containing a two-dimensional array, with 2^j columns and 2^k rows, of 2-bit predictors. If the branch address has n bits, a subset of j bits is used to index into the PHT to select one of the 2^j columns. Since j is less than n, some aliasing can occur where two different branch addresses can index into the same column of the PHT. Hence the designation of *shared* PHT. The k bits from the BHSR are used to select one of the 2^k entries in the selected column. The 2 history bits in the selected entry are used to make a history-based prediction. The traditional branch history table is equivalent to having only one row of the PHT that is indexed only by the j bits of the branch address, as illustrated in Figure 5.12 by the dashed rectangular block of 2-bit predictors in the first row of the PHT.

Figure 5.13 illustrates a correlated branch predictor with individual, or per-branch, BHSRs (P) and the same shared PHT (s). Similar to the GAs scheme, the PAs scheme also uses j bits of the branch address to select one of the 2^j columns of the PHT. However, i bits of the branch address, which can overlap with the j bits used to access the PHT, are used to index into a set of BHSRs. Depending on the branch address, one of the 2^i BHSRs is selected. Hence, each BHSR is associated with one particular branch address, or a set of branch addresses if there is aliasing. Essentially, instead of using a single BHSR to provide the dynamic control flow context for all static branches, multiple BHSRs are used to provide distinct dynamic control flow contexts for different subsets of static branches. This adds

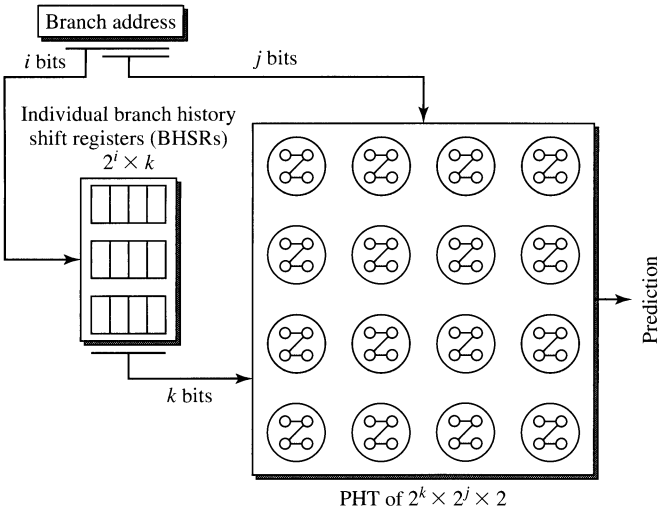


Figure 5.13
Correlated Branch Predictor with Individual BHSRs and Shared PHTs (PAs).

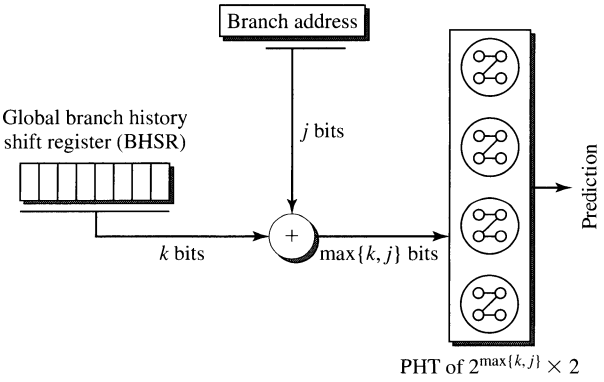


Figure 5.14
The *gshare* Correlated Branch Predictor of McFarling.
Source: McFarling, 1993.

flexibility in tracking and exploiting correlations between different branch instructions. Each BHSR tracks the directions of the last k dynamic branches belonging to the same subset of static branches. Both the GAs and the PAs schemes require a PHT of size $2^k \times 2^j \times 2$ bits. The GAs scheme has only one k-bit BHSR whereas the PAs scheme requires 2^i k-bit BHSRs.

A fairly efficient correlated branch predictor called *gshare* was proposed by Scott McFarling [1993]. In this scheme, j bits from the branch address are "hashed" (via bitwise XOR function) with the k bits from a global BHSR; see Figure 5.14. The resultant $\max\{k, j\}$ bits are used to index into a PHT of size $2^{\max\{k,j\}} \times 2$ bits to

select one of the $2^{\max\{k,j\}}$ 2-bit branch predictors. The *gshare* scheme requires only one k-bit BHSR and a much smaller PHT, yet achieves comparable prediction accuracy to other correlated branch predictors. This scheme is used in the DEC Alpha 21264 4-way superscalar microprocessor [Keller, 1996].

5.1.6 Other Instruction Flow Techniques

The primary objective for instruction flow techniques is to supply as many useful instructions as possible to the execution core of the processor in every machine cycle. The two major challenges deal with conditional branches and taken branches. For a wide superscalar processor, to provide adequate conditional branch throughput, the processor must very accurately predict the outcomes and targets of multiple conditional branches in every machine cycle. For example, in a fetch group of four instructions, it is possible that all four instructions are conditional branches. Ideally one would like to use the addresses of all four instructions to index into a four-ported BTB to retrieve the history bits and target addresses of all four branches. A complex predictor can then make an overall prediction based on all the history bits. Speculative fetching can then proceed based on this prediction. Techniques for predicting multiple branches in every cycle have been proposed by Conte et al. [1995] as well as Rotenberg et al. [1996]. It is also important to ensure high accuracy in such predictions. Global branch history can be used in conjunction with per-branch history to achieve very accurate predictions. For those branches or sequences of branches that do not exhibit strongly biased branching behavior and therefore are not predictable, *dynamic eager execution* (DEE) has been proposed by Gus Uht [Uht and Sindagi, 1995]. DEE employs multiple PCs to simultaneously fetch from multiple addresses. Essentially the fetch stage pursues down multiple control flow paths until some branches are resolved, at which time some of the wrong paths are dynamically pruned by invalidating the instructions on those paths.

Taken branches are the second major obstacle to supplying enough useful instructions to the execution core. In a wide machine the fetch unit must be able to correctly process more than one taken branch per cycle, which involves predicting each branch's direction and target, and fetching, aligning, and merging instructions from multiple branch targets. An effective approach in alleviating this problem involves the use of a *trace cache* that was initially proposed by Eric Rotenberg [Rotenberg et al., 1996]. Since then, a form of trace caching has been implemented in Intel's most recent Pentium 4 superscalar microprocessor. Trace cache is a history-based fetch mechanism that stores dynamic instruction traces in a cache indexed by the fetch address and branch outcomes. These traces are assembled dynamically based on the dynamic branching behavior and can contain multiple nonconsecutive basic blocks. Whenever the fetch address hits in the trace cache, instructions are fetched from the trace cache rather than the instruction cache. Since a dynamic sequence of instructions in the trace cache can contain multiple taken branches but is stored sequentially, there is no need to fetch from multiple targets and no need for a multiported instruction cache or complex merging and aligning logic in the fetch stage. The trace cache can be viewed as doing dynamic

basic block reordering according to the dominant execution paths taken by a program. The merging and aligning can be done at completion time, when nonconsecutive basic blocks on a dominant path are first executed, to assemble a trace, which is then stored in one line of the trace cache. The goal is that once the trace cache is warmed up, most of the fetching will come from the trace cache instead of the instruction cache. Since the reordered basic blocks in the trace cache better match the dynamic execution order, there will be fewer fetches from nonconsecutive locations in the trace cache, and there will be an effective increase in the overall throughput of taken branches.

5.2 Register Data Flow Techniques

Register data flow techniques concern the effective execution of ALU (or register-register) type instructions in the execution core of the processor. ALU instructions can be viewed as performing the "real" work specified by the program, with control flow and load/store instructions playing the supportive roles of providing the necessary instructions and the required data, respectively. In the most ideal machine, branch and load/store instructions, being "overhead" instructions, should take no time to execute and the computation latency should be strictly determined by the processing of ALU instructions. The effective processing of these instructions is foundational to achieving high performance.

Assuming a load/store architecture, ALU instructions specify operations to be performed on source operands stored in registers. Typically an ALU instruction specifies a binary operation, two source registers where operands are to be retrieved, and a destination register where the result is to be placed. $R_i \leftarrow F_n(R_j,R_k)$ specifies a typical ALU instruction, the execution of which requires the availability of (1) F_n, the functional unit; (2) R_j and R_k, the two source operand registers; and (3) R_i, the destination register. If the functional unit F_n is not available, then a structural dependence exists that can result in a structural hazard. If one or both of the source operands in R_j and R_k are not available, then a hazard due to true data dependence can occur. If the destination register R_i is not available, then a hazard due to anti- and output dependences can occur.

5.2.1 Register Reuse and False Data Dependences

The occurrence of anti- and output dependences, or false data dependences, is due to the reuse of registers. If registers are never reused to store operands, then such false data dependences will not occur. The reuse of registers is commonly referred to as *register recycling*. Register recycling occurs in two different forms, one static and the other dynamic. The static form is due to optimization performed by the compiler and is presented first. In a typical compiler, toward the back end of the compilation process two tasks are performed: *code generation* and *register allocation*. The code generation task is responsible for the actual emitting of machine instructions. Typically the code generator assumes the availability of an unlimited number of symbolic registers in which it stores all the temporary data. Each symbolic register is used to store one value and is only written once, producing what is

commonly referred to as *single-assignment* code. However, an ISA has a limited number of architected registers, and hence the register allocation tool is used to map the unlimited number of symbolic registers to the limited and fixed number of architected registers. The register allocator attempts to keep as many of the temporary values in registers as possible to avoid having to move the data out to memory locations and reloading them later on. It accomplishes this by reusing registers. A register is written with a new value when the old value stored there is no longer needed; effectively each register is recycled to hold multiple values.

Writing of a register is referred to as the *definition* of a register and the reading of a register as the *use* of a register. After each definition, there can be one or more uses of that definition. The duration between the definition and the last use of a value is referred to as the *live range* of that value. After the last use of a live range, that register can be assigned to store another value and begin another live range. Register allocation procedures attempt to map nonoverlapping live ranges into the same architected register and maximize register reuse. In single-assignment code there is a one-to-one correspondence between symbolic registers and values. After register allocation, each architected register can receive multiple assignments, and the register becomes a variable that can take on multiple values. Consequently the one-to-one correspondence between registers and values is lost.

If the instructions are executed sequentially and a redefinition is never allowed to precede the previous definition or the last use of the previous definition, then the live ranges that share the same register will never overlap during execution and the recycling of registers does not induce any problem. Effectively, the one-to-one correspondence between values and registers can be maintained implicitly if all the instructions are processed in the original program order. However, in a superscalar machine, especially with out-of-order processing of instructions, register reading and writing operations can occur in an order different from the program order. Consequently the one-to-one correspondence between values and registers can potentially be perturbed; in order to ensure semantic correctness all anti- and output dependences must be detected and enforced. Out-of-order reading (writing) of registers can be permitted as long as all the anti- (output) dependences are enforced.

The dynamic form of register recycling occurs when a loop of instructions is repeatedly executed. With an aggressive superscalar machine capable of supporting many instructions in flight and a relatively small loop body being executed, multiple iterations of the loop can be simultaneously in flight in a machine. Hence, multiple copies of a register defining instruction from the multiple iterations can be simultaneously present in the machine, inducing the dynamic form of register recycling. Consequently anti- and output dependences can be induced among these dynamic instructions from the multiple iterations of a loop and must be detected and enforced to ensure semantic correctness of program execution.

One way to enforce anti- and output dependences is to simply stall the dependent instruction until the leading instruction has finished accessing the dependent register. If an anti- [write-after-read (WAR)] dependence exists between a pair of instructions, the trailing instruction (register updating instruction) must be stalled until the leading instruction has read the dependent register. If an output [write-after-write (WAW)] dependence exists between a pair of instructions, the trailing

instruction (register updating instruction) must be stalled until the leading instruction has first updated the register. Such stalling of anti- and output dependent instructions can lead to significant performance loss and is not necessary. Recall that such false data dependences are induced by the recycling of the architected registers and are not intrinsic to the program semantics.

5.2.2 Register Renaming Techniques

A more aggressive way to deal with false data dependences is to dynamically assign different *names* to the multiple definitions of an architected register and, as a result, eliminate the presence of such false dependences. This is called *register renaming* and requires the use of hardware mechanisms at run time to undo the effects of register recycling by reproducing the one-to-one correspondence between registers and values for all the instructions that might be simultaneously in flight. By performing register renaming, single assignment is effectively recovered for the instructions that are in flight, and no anti- and output dependences can exist among these instructions. This will allow the instructions that originally had false dependences between them to be executed in parallel.

A common way to implement register renaming is to use a separate rename register file (RRF) in addition to the architected register file (ARF). A straightforward way to implement the RRF is to simply duplicate the ARF and use the RRF as a shadow version of the ARF. This will allow each architected register to be renamed once. However, this is not a very efficient way to use the registers in the RRF. Many existing designs implement an RRF with fewer entries than the ARF and allow each of the registers in the RRF to be flexibly used to rename any one of the architected registers. This facilitates the efficient use of the rename registers, but does require a mapping table to store the pointers to the entries in the RRF. The use of a separate RRF in conjunction with a mapping table to perform renaming of the ARF is illustrated in Figure 5.15.

When a separate RRF is used for register renaming, there are implementation choices in terms of where to place the RRF. One option is to implement a separate stand-alone structure similar to the ARF and perhaps adjacent to the ARF. This is shown in Figure 5.15(a). An alternative is to incorporate the RRF as part of the reorder buffer, as shown in Figure 5.15(b). In both options a busy field is added to the ARF along with a mapping table. If the busy bit of a selected entry of the ARF is set, indicating the architected register has been renamed, the corresponding entry of the map table is accessed to obtain the tag or the pointer to an RRF entry. In the former option, the tag specifies a rename register and is used to index into the RRF; whereas in the latter option, the tag specifies a reorder buffer entry and is used to index into the reorder buffer.

Based on the diagrams in Figure 5.15, the difference between the two options may seem artificial; however, there are important subtle differences. If the RRF is incorporated as part of the reorder buffer, every entry of the reorder buffer contains an additional field that functions as a rename register, and hence there is a rename register allocated for every instruction in flight. This is a design based on worst-case scenario and may be wasteful since not every instruction defines a register. For example, branch instructions do not update any architected register. On the

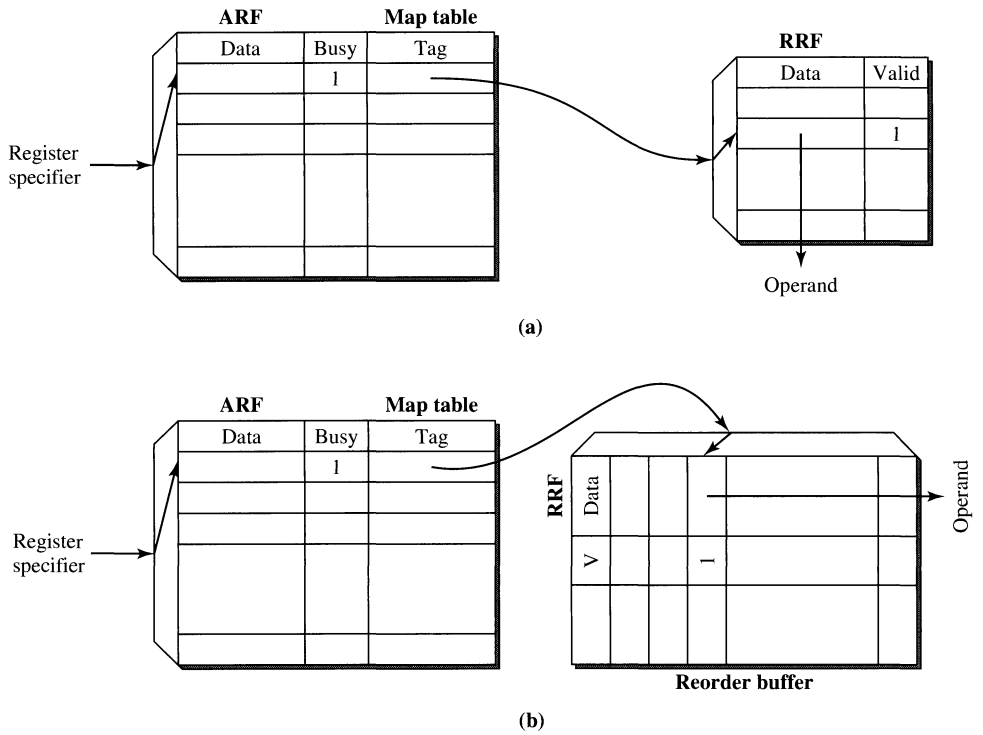


Figure 5.15
Rename Register File (RRF) Implementations: (a) Stand-Alone; (b) Attached to the Reorder Buffer.

other hand, a reorder buffer already contains ports to receive data from the functional units and to update the ARF at instruction completion time. When a separate stand-alone RRF is used, it introduces an additional structure that requires ports for receiving data from the functional units and for updating the ARF. The choice of which of the two options to implement involves design tradeoffs, and both options have been employed in real designs. We now focus on the stand-alone option to get a better feel of how register renaming actually works.

Register renaming involves three tasks: (1) *source read,* (2) *destination allocate,* and (3) *register update*. The first task of source read typically occurs during the decode (or possibly dispatch) stage and is for the purpose of fetching the register operands. When an instruction is decoded, its source register specifiers are used to index into a multiported ARF in order to fetch the register operands. Three possibilities can occur for each register operand fetch. First, if the busy bit is not set, indicating there is no pending write to the specified register and that the architected register contains the specified operand, the operand is fetched from the ARF. If the busy bit is set, indicating there is a pending write to that register and that the content of the architected register is stale, the corresponding entry of the map table is accessed to retrieve the rename tag. This rename tag specifies a

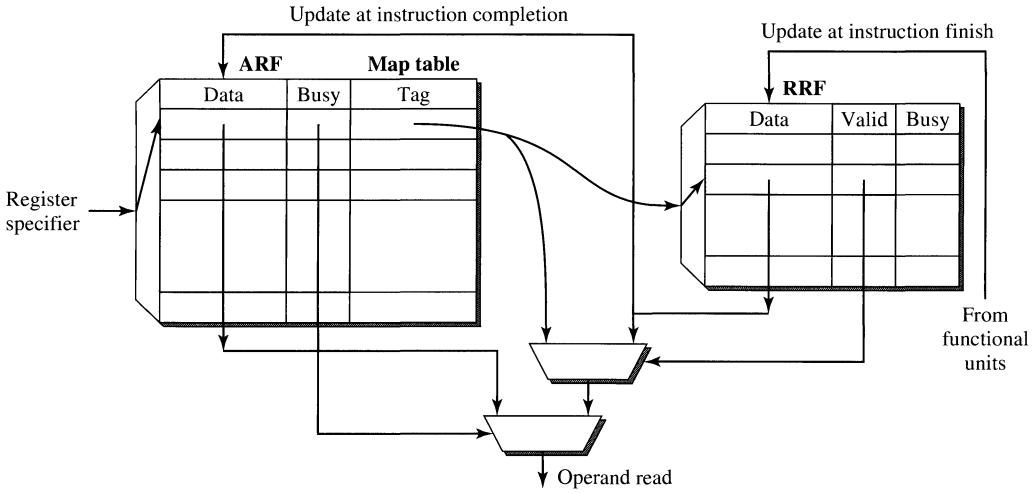


Figure 5.16
Register Renaming Tasks: Source Read, Destination Allocate, and Register Update.

rename register and is used to index into the RRF. Two possibilities can occur when indexing into the RRF. If the valid bit of the indexed entry is set, it indicates that the register-updating instruction has already finished execution, although it is still waiting to be completed. In this case, the source operand is available in the rename register and is retrieved from the indexed RRF entry. If the valid bit is not set, it indicates that the register-updating instruction still has not been executed and that the rename register has a pending update. In this case the tag, or the rename register specifier, from the map table is forwarded to the reservation station instead of to the source operand. This tag will be used later by the reservation station to obtain the operand when it becomes available. These three possibilities for source read are shown in Figure 5.16.

The task of destination allocate also occurs during the decode (or possibly dispatch) stage and has three subtasks, namely, *set busy bit, assign tag,* and *update map table*. When an instruction is decoded, its destination register specifier is used to index into the ARF. The selected architected register now has a pending write, and its busy bit must be set. The specified destination register must be mapped to a rename register. A particular unused (indicated by the busy bit) rename register must be selected. The busy bit of the selected RRF entry must be set, and the index of the selected RRF entry is used as a tag. This tag must then be written into the corresponding entry in the map table, to be used by subsequent dependent instructions for fetching their source operands.

While the task of register update takes place in the back end of the machine and is not part of the actual renaming activity of the decode/dispatch stage, it does have a direct impact on the operation of the RRF. Register update can occur in two separate steps; see Figure 5.16. When a register-updating instruction finishes execution, its result is written into the entry of the RRF indicated by the tag. Later on when this

instruction is completed, its result is then copied from the RRF into the ARF. Hence, register update involves updating first an entry in the RRF and then an entry in the ARF. These two steps can occur in back-to-back cycles if the register-updating instruction is at the head of the reorder buffer, or they can be separated by many cycles if there are other unfinished instructions in the reorder buffer ahead of this instruction. Once a rename register is copied to its corresponding architected register, its busy bit is reset and it can be used again to rename another architected register.

So far we have assumed that register renaming implementation requires the use of two separate physical register files, namely the ARF and the RRF. However, this assumption is not necessary. The architected registers and the rename registers can be pooled together and implemented as a single physical register file with its number of entries equal to the sum of the ARF and RRF entry counts. Such a pooled register file does not rigidly designate some of the registers as architected registers and others as rename registers. Each physical register can be flexibly assigned to be an architected register or a rename register. Unlike a separate ARF and RRF implementation which must physically copy a result from the RRF to the ARF at instruction completion, the pooled register file only needs to change the designation of a register from being a rename register to an architected register. This will save the data transfer interconnect between the RRF and the ARF. The key disadvantage of the pooled register file is its hardware complexity. A secondary disadvantage is that at context swap time, when the machine state must be saved, the subset of registers constituting the architected state of the machine must be explicitly identified before state saving can begin.

The pooled register file approach is used in the floating-point unit of the original IBM RS/6000 design and is illustrated in Figure 5.17 [Grohoski, 1990; Oehler and Groves, 1990]. In this design, 40 physical registers are implemented for supporting an ISA that specifies 32 architected registers. A mapping table is implemented, based on whose content any subset of 32 of the 40 physical registers can be designated as the architected registers. The mapping table contains 32 entries indexed by the 5-bit architected register specifier. Each entry when indexed returns a 6-bit specifier indicating the physical register to which the architected register has been mapped.

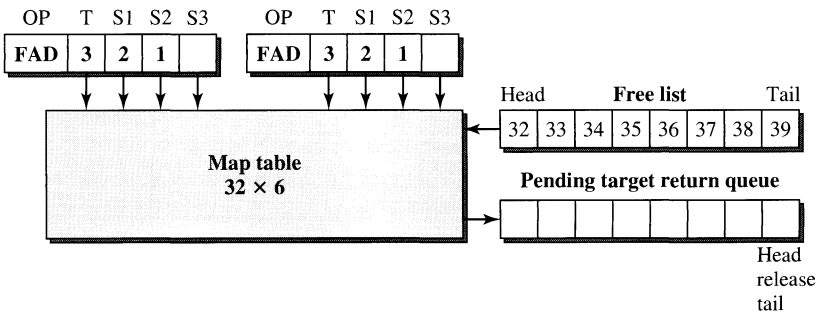


Figure 5.17
Floating-Point Unit (FPU) Register Renaming in the IBM RS/6000.

The floating-point unit (FPU) of the RS/6000 is a pipelined functional unit with the rename pipe stage preceding the decode pipe stage. The rename pipe stage contains the map table, two circular queues, and the associated control logic. The first queue is called the *free list* (FL) and contains physical registers that are available for new renaming. The second queue is called the *pending target return queue* (PTRQ) and contains those physical registers that have been used to rename architected registers that have been subsequently re-renamed in the map table. Physical registers in the PTRQ can be returned to the FL once the last use of that register has occurred. Two instructions can traverse the rename stage in every machine cycle. Because of the possibility of fused multiply-add (FMA) instructions that have three sources and one destination, each of the two instructions can contain up to four register specifiers. Hence, the map table must be eight-ported to support the simultaneous translation of the eight architected register specifiers. The map table is initialized with the identity mapping; i.e., architected register i is mapped to physical register i for $i = 0, 1, \ldots, 31$. At initialization, physical registers 32 to 39 are placed in the FL and the PTRQ is empty.

When an instruction traverses the rename stage, its architected register specifiers are used to index into the map table to obtain their translated physical register specifiers. The eight-ported map table has 32 entries, indexed by the 5-bit architected register specifier, with each entry containing 6 bits indicating the physical register to which the architected register is mapped. The content of the map table represents the latest mapping of architected registers to physical registers and specifies the subset of physical registers that currently represents the architected registers.

In the FPU of the RS/6000, by design only load instructions can trigger a new renaming. Such register renaming prevents the FPU from stalling while waiting for loads to execute in order to enforce anti- and output dependences. When a load instruction traverses the rename stage, its destination register specifier is used to index into the map table. The current content of that entry of the map table is pushed out to the PTRQ, and the next physical register in the FL is loaded into the map table. This effectively renames the redefinition of that destination register to a different physical register. All subsequent instructions that specify this architected register as a source operand will receive the new physical register specifier as the source register. Beyond the rename stage, i.e., in the decode and execute stages, the FPU uses only physical register specifiers, and all true register dependences are enforced using the physical register specifiers.

The map table approach represents the most aggressive and versatile implementation of register renaming. Every physical register can be used to represent any redefinition of any architected register. There is significant hardware complexity required to implement the multiported map table and the logic to control the two circular queues. The return of a register in the PTRQ to the FL is especially troublesome due to the difficulty in identifying the last-use instruction of a register. However, unlike approaches based on the use of separate rename registers, at instruction completion time no copying of the content of the rename registers to the architected registers is necessary. On the other hand, when interrupts occur and as part of context swap, the subset of physical registers that constitute the current architected machine state must be explicitly determined based on the map table contents.

Most contemporary superscalar microprocessors implement some form of register renaming to avoid having to stall for anti- and output register data dependences induced by the reuse of registers. Typically register renaming occurs during the instruction decoding time, and its implementation can become quite complex, especially for wide superscalar machines in which many register specifiers for multiple instructions must be simultaneously renamed. It's possible that multiple redefinitions of a register can occur within a fetch group. Implementing a register renaming mechanism for wide superscalars without seriously impacting machine cycle time is a real challenge. To achieve high performance the serialization constraints imposed by false register data dependences must be eliminated; hence, dynamic register renaming is absolutely essential.

5.2.3 True Data Dependences and the Data Flow Limit

A RAW dependence between two instructions is called a true data dependence due to the producer-consumer relationship between these two instructions. The trailing consumer instruction cannot obtain its source operand until the leading producer instruction produces its result. A true data dependence imposes a serialization constraint between the two dependent instructions; the leading instruction must finish execution before the trailing instruction can begin execution. Such true data dependences result from the semantics of the program and are usually represented by a data flow graph or data dependence graph (DDG).

Figure 5.18 illustrates a code fragment for a fast Fourier transform (FFT) implementation. Two source-level statements are compiled into 16 assembly instructions, including load and store instructions. The floating-point array variables

```
w[i+k].ip = z[i].rp + z[m+i].rp;
w[i+j].rp = e[k+1].rp * (z[i].rp - z[m+i].rp) - e[k+1].ip * (z[i].ip - z[m+i].ip);
```

(a)

```
i1:  f2 ← load,4(r2)
i2:  f0 ← load,4(r5)
i3:  f0 ← fadd,f2,f0
i4:  4(r6) ← store,f0
i5:  f14 ← laod,8(r7)
i6:  f6 ← load,0(r2)
i7:  f5 ← load,0(r3)
i8:  f5 ← fsub,f6,f5
i9:  f4 ← fmul,f14,f5
i10: f15 ← load,12(r7)
i11: f7 ← load,4(r2)
i12: f8 ← load,4(r3)
i13: f8 ← fsub,f7,f8
i14: f8 ← fmul,f15,f8
i15: f8 ← fsub,f4,f8
i16: 0(r8) ← store,f8
```

(b)

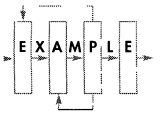


Figure 5.18
FFT Code Fragment: (a) Original Source Statements; (b) Compiled Assembly Instructions.

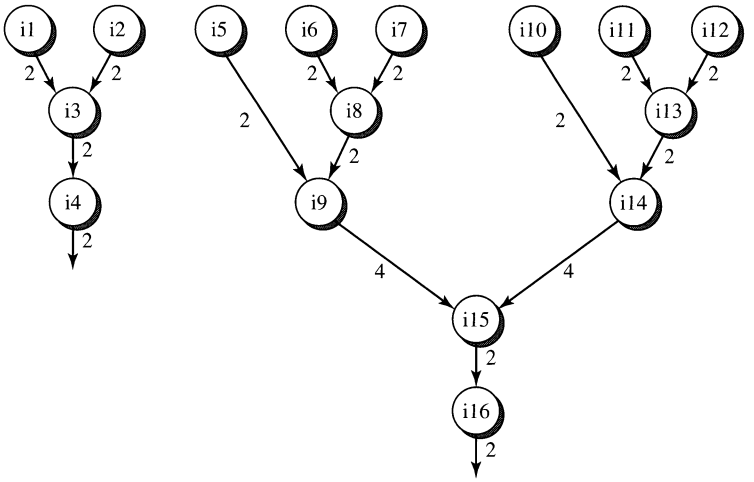


Figure 5.19
Data Flow Graph of the Code Fragment in Figure 5.18(b).

are stored in memory and must be first loaded before operations can be performed. After the computation, the results are stored back out to memory. Integer registers (*ri*) are used to hold addresses of arrays. Floating-point registers (*fj*) are used to hold temporary data. The DFG induced by the writing and reading of floating-point registers by the 16 instructions of Figure 5.18(b) is shown in Figure 5.19.

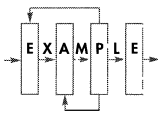
Each node in Figure 5.19 represents an instruction in Figure 5.18(b). A directed edge exists between two instructions if there exists a true data dependence between the two instructions. A dependent register can be identified for each of the dependence edges in the DFG. A latency can also be associated with each dependence edge. In Figure 5.19, each edge is labeled with the execution latency of the producer instruction. In this example, load, store, addition, and subtraction instructions are assumed to have two-cycle execution latency, while multiplication instructions require four cycles.

The latencies associated with dependence edges are cumulative. The longest dependence chain, measured in terms of total cumulative latency, is identified as the critical path of a DFG. Even assuming unlimited machine resources, a code fragment cannot be executed any faster than the length of its critical path. This is commonly referred to as the *data flow limit* to program execution and represents the best performance that can possibly be achieved. For the code fragment of Figure 5.19 the data flow limit is 12 cycles. The data flow limit is dictated by the true data dependences in the program. Traditionally, the *data flow execution model* stipulates that every instruction in a program begin execution immediately in the cycle following when all its operands become available. In effect, all existing register data flow techniques are attempts to approach the data flow limit.

5.2.4 The Classic Tomasulo Algorithm

The design of the IBM 360/91's floating-point unit, incorporating what has come to be known as *Tomasulo's algorithm,* laid the groundwork for modern superscalar processor designs [Tomasulo, 1967]. Key attributes of most contemporary register data flow techniques can be found in the classic Tomasulo algorithm, which deserves an in-depth examination. We first introduce the original design of the floating-point unit of the IBM 360, and then describe in detail the modified design of the FPU in the IBM 360/91 that incorporated Tomasulo's algorithm, and finally illustrate its operation and effectiveness in processing an example code sequence.

The original design of the IBM 360 floating-point unit is shown in Figure 5.20. The FPU contains two functional units: one floating-point add unit and one floating-point multiply/divide unit. There are three register files in the FPU: the floating-point registers (FLRs), the floating-point buffers (FLBs), and the store data buffers (SDBs). There are four FLR registers; these are the architected floating-point registers. Floating-point instructions with storage-register or storage-storage addressing modes are preprocessed. Address generation and memory



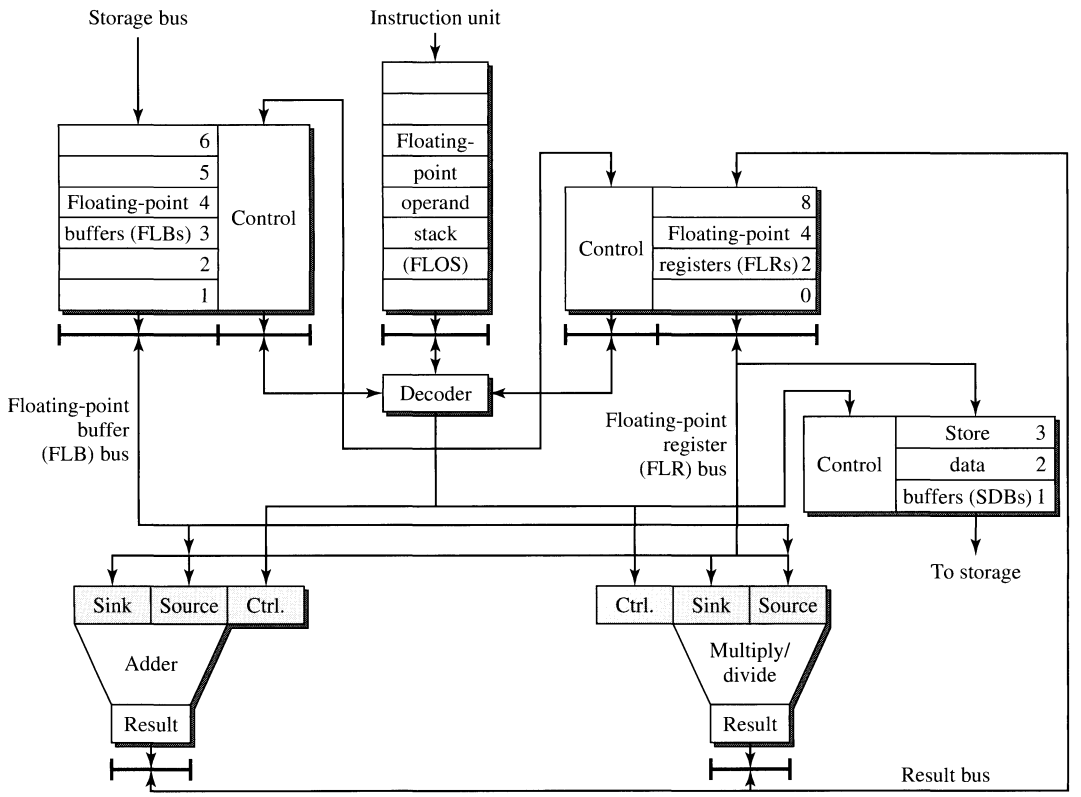


Figure 5.20
The Original Design of the IBM 360 Floating-Point Unit.

accessing are performed outside of the FPU. When the data are retrieved from the memory, they are loaded into one of the six FLB registers. Similarly if the destination of an instruction is a memory location, the result to be stored is placed in one of the three SDB registers and a separate unit accesses the SDBs to complete the storing of the result to a memory location. Using these two additional register files, the FLBs, and the SDBs, to support storage-register and storage-storage instructions, the FPU effectively functions as a register-register machine.

In the IBM 360/91, the instruction unit (IU) decodes all the instructions and passes all floating-point instructions (in order) to the floating-point operation stack (FLOS). In the FPU, floating-point instructions are then further decoded and issued in order from the FLOS to the two functional units. The two functional units are not pipelined and incur multiple-cycle latencies. The adder incurs 2 cycles for add instructions, while the multiply/divide unit incurs 3 cycles and 12 cycles for performing multiply and divide instructions, respectively.

In the mid-1960s, IBM began developing what eventually became Model 91 of the Systems 360 family. One of the goals was to achieve concurrent execution of multiple floating-point instructions and to sustain a throughput of one instruction per cycle in the instruction pipeline. This is quite aggressive considering the complex addressing modes of the 360 ISA and the multicycle latencies of the execution units. The end result is a modified FPU in the 360/91 that incorporated Tomasulo's algorithm; see Figure 5.21.

Tomasulo's algorithm consists of adding three new mechanisms to the original FPU design, namely, reservation stations, the common data bus, and register tags. In the original design, each functional unit has a single buffer on its input side to hold the instruction currently being executed. If a functional unit is busy, issuing of instructions by FLOS will stall whenever the next instruction to be issued requires the same functional unit. To alleviate this structural bottleneck, multiple buffers, called *reservation stations,* are attached to the input side of each functional unit. The adder unit has three reservation stations, while the multiply/divide unit has two. These reservation stations are viewed as virtual functional units; as long as there is a free reservation station, the FLOS can issue an instruction to that functional unit even if it is currently busy executing another instruction. Since the FLOS issues instructions in order, this will prevent unnecessary stalling due to unfortunate ordering of different floating-point instruction types.

With the availability of reservation stations, instructions can also be issued to the functional units by the FLOS even though not all their operands are yet available. These instructions can wait in the reservation station for their operands and only begin execution when they become available. The *common data bus* (CDB) connects the outputs of the two functional units to the reservation stations as well as the FLRs and SDB registers. Results produced by the functional units are broadcast into the CDB. Those instructions in the reservation stations needing the results as their operands will latch in the data from the CDB. Those registers in the FLR and SDB that are the destinations of these results also latch in the same data from the CDB. The CDB facilitates the forwarding of results directly from producer instructions to consumer instructions waiting in the reservation stations

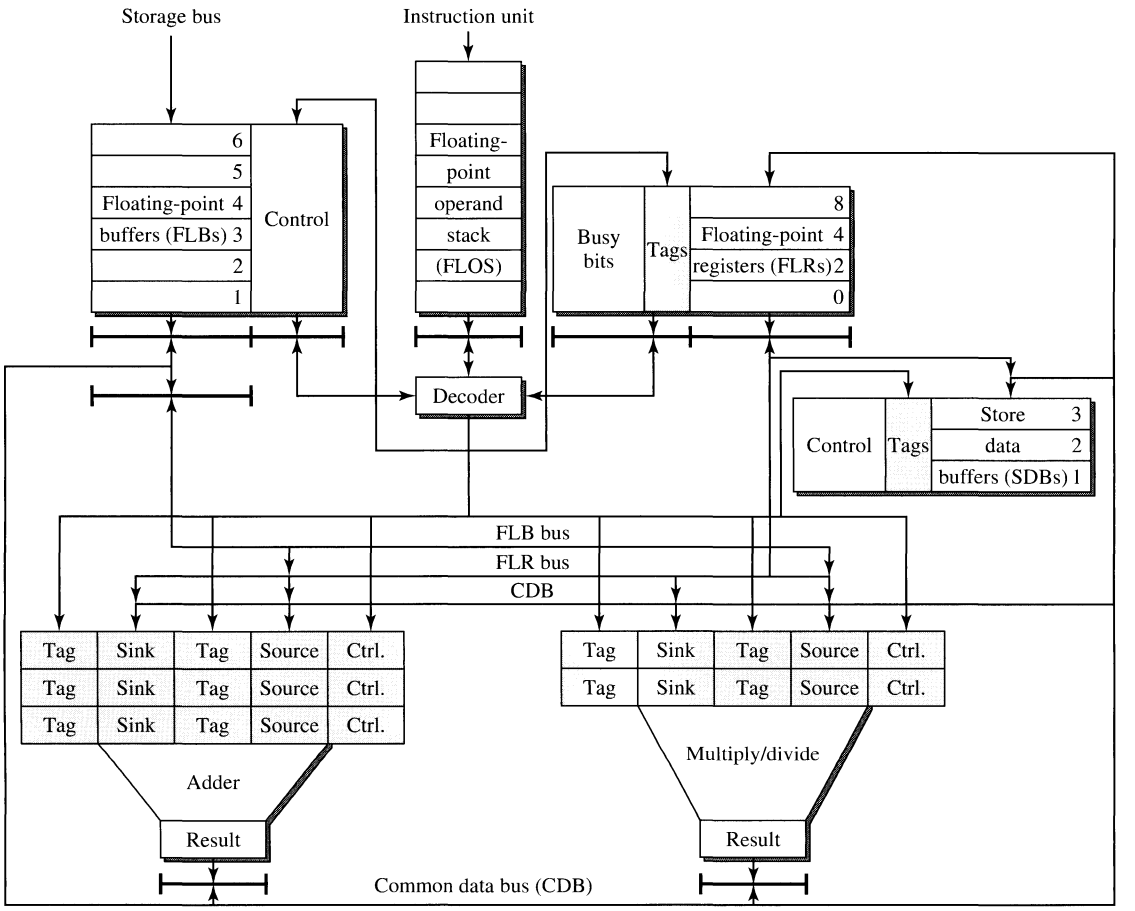
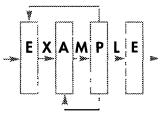


Figure 5.21
The Modified Design of the IBM 360/91 Floating-Point Unit with Tomasulo's Algorithm.



without having to go through the registers. Destination registers are updated simultaneously with the forwarding of results to dependent instructions. If an operand is coming from a memory location, it will be loaded into a FLB register once memory accessing is performed. Hence, the FLB can also output onto the CDB, allowing a waiting instruction in a reservation station to latch in its operand. Consequently, the two functional units and the FLBs can drive data onto the CDB, and the reservation station's FLRs and SDBs can latch in data from the CDB.

When the FLOS is dispatching an instruction to a functional unit, it allocates a reservation station and checks to see if the needed operands are available. If an operand is available in the FLRs, then the content of that register in the FLRs is copied to the reservation station; otherwise a tag is copied to the reservation station instead. The tag indicates where the pending operand is going to come from.

The pending operand can come from a producer instruction currently resident in one of the five reservation stations, or it can come from one of the six FLB registers. To uniquely identify one of these 11 possible sources for a pending operand, a 4-bit tag is required. If one of the two operand fields of a reservation station contains a tag instead of the actual operand, it indicates that this instruction is waiting for a pending operand. When that pending operand becomes available, the producer of that operand drives the tag along with the actual operand onto the CDB.

A waiting instruction in a reservation station uses its tag to monitor the CDB. When it detects a tag match on the CDB, it then latches in the associated operand. Essentially the producer of an operand broadcasts the tag and the operand on the CDB; all consumers of that operand monitor the CDB for that tag, and when the broadcasted tag matches their tag, they then latch in the associated operand from the CDB. Hence, all possible destinations of pending operands must carry a tag field and must monitor the CDB for a tag match. Each reservation station contains two operand fields, each of which must carry a tag field since each of the two operands can be pending. The four FLRs and the three registers in the SDB must also carry tag fields. This is a total of 17 tag fields representing 17 places that can monitor and receive operands; see Figure 5.22. The tag field at each potential consumer site is used in an associative fashion to monitor for possible matching of its content with the tag value being broadcasted on the CDB. When a tag match occurs, the consumer latches in the broadcasted operand.

The IBM 360 floating-point instructions use a two-address instruction format. Two source operands can be specified. The first operand specifier is called the *sink* because it also doubles as the destination specifier. The second operand specifier is called the *source*. Each reservation station has two operand fields, one for the sink and the other for the source. Each operand field is accompanied by a tag field. If an operand field contains real data, then its tag field is set to zero. Otherwise, its tag field identifies the source where the pending operand will be coming from, and is used to monitor the CDB for the availability of the pending operand. Whenever

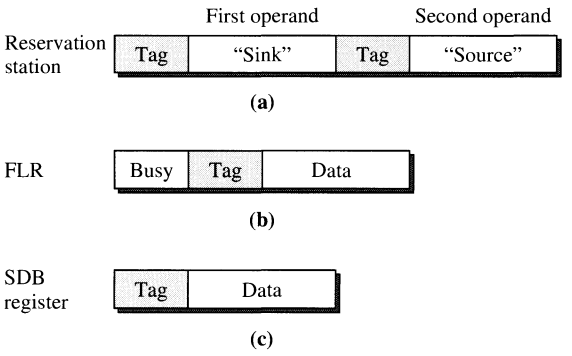


Figure 5.22

The Use of Tag Fields in (a) A Reservation Station, (b) A FLR, and (c) A SDB Register.

an instruction is dispatched by the FLOS to a reservation station, the data in the FLR corresponding to the sink operand are retrieved and copied to the reservation station. At the same time, the "busy" bit associated with this FLR is set, indicating that there is a pending update of that register, and the tag value that identifies the particular reservation station to which the instruction is being dispatched is written into the tag field of the same FLR. This clearly identifies which of the reservation stations will eventually produce the updated data for this FLR. Subsequently if a trailing instruction specifies this register as one of its source operands, when it is dispatched to a reservation station, only the tag field (called the *pseudo-operand*) will be copied to the corresponding tag field in the reservation station and not the actual data. When the busy bit is set, it indicates that the data in the FLR are stale and the tag represents the source from which the real data will come. Other than reservation stations and FLRs, SDB registers can also be destinations of pending operands and hence a tag field is required for each of the three SDB registers.

We now use an example sequence of instructions to illustrate the operation of Tomasulo's algorithm. We deviate from the actual IBM 360/91 design in several ways to help clarify the example. First, instead of the two-address format of the IBM 360 instructions, we will use three-address instructions to avoid potential confusion. The example sequence contains only register-register instructions. To reduce the number of machine cycles we have to trace, we will allow the FLOS to dispatch (in program order) up to two instructions in every cycle. We also assume that an instruction can begin execution in the same cycle that it is dispatched to a reservation station. We keep the same latencies of two and three cycles for add and multiply instructions, respectively. However, we allow an instruction to forward its result to dependent instructions during its last execution cycle, and a dependent instruction can begin execution in the next cycle. The tag values of 1, 2, and 3 are used to identify the three reservation stations of the adder functional unit, while 4 and 5 are used to identify the two reservation stations of the multiply/divide functional unit. These tag values are called the *IDs* of the reservation stations. The example sequence consists of the following four register-register instructions.

w: R4 ← R0 + R8

x: R2 ← R0 * R4

y: R4 ← R4 + R8

z: R8 ← R4 * R2

Figure 5.23 illustrates the first three cycles of execution. In cycle 1, instructions w and x are dispatched (in order) to reservation stations 1 and 4. The destination registers of instructions w and x are R4 and R2 (i.e., FLRs 4 and 2), respectively. The busy bits of these two registers are set. Since instruction w is dispatched to reservation station 1, the tag value of 1 is entered into the tag field of R4, indicating that the instruction in reservation station 1 will produce the result for updating R4. Similarly the tag value of 4 is entered into the tag field of R2. Both source operands of instruction w are available, so it begins execution immediately. Instruction x

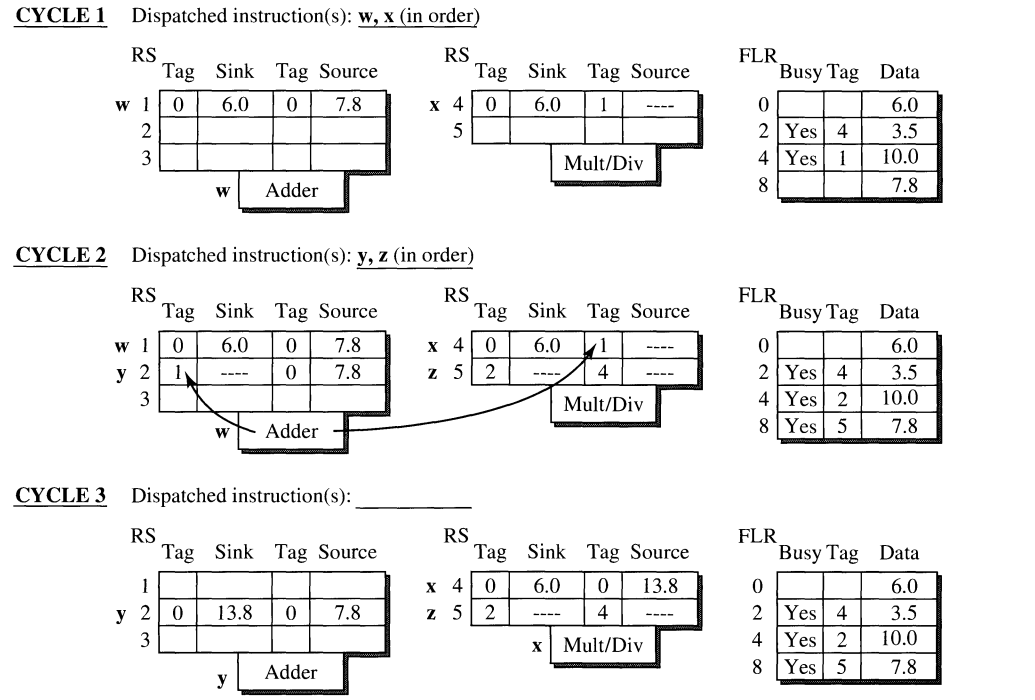
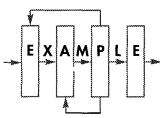


Figure 5.23
Illustration of Tomasulo's Algorithm on an Example Instruction Sequence (Part 1).



requires the result (R4) of instruction w for its second (source) operand. Hence when instruction x is dispatched to reservation station 4, the tag field of the second operand is written the tag value of 1, indicating that the instruction in reservation station 1 will produce the needed operand.

During cycle 2, instructions y and z are dispatched (in order) to reservation stations 2 and 5, respectively. Because it needs the result of instruction w for its first operand, instruction y, when it is dispatched to reservation station 2, receives the tag value of 1 in the tag field of the first operand. Similarly instruction z, dispatched to reservation station 5, receives the tag values of 2 and 4 in its two tag fields, indicating that reservation stations 2 and 4 will eventually produce the two operands it needs. Since R4 is the destination of instruction y, the tag field of R4 is updated with the new tag value of 2, indicating reservation station 2 (i.e., instruction y) is now responsible for the pending update of R4. The busy bit of R4 remains set. The busy bit of R8 is set when instruction z is dispatched to reservation station 5, and the tag field of R8 is set to 5. At the end of cycle 2, instruction w finishes execution and broadcasts its ID (reservation station 1) and its result onto the CDB. All the tag fields containing the tag value of 1 will trigger a tag match and latch in the broadcasted result. The first tag field of reservation station 2 (holding instruction y) and the second tag field of reservation station 4 (holding instruction x)

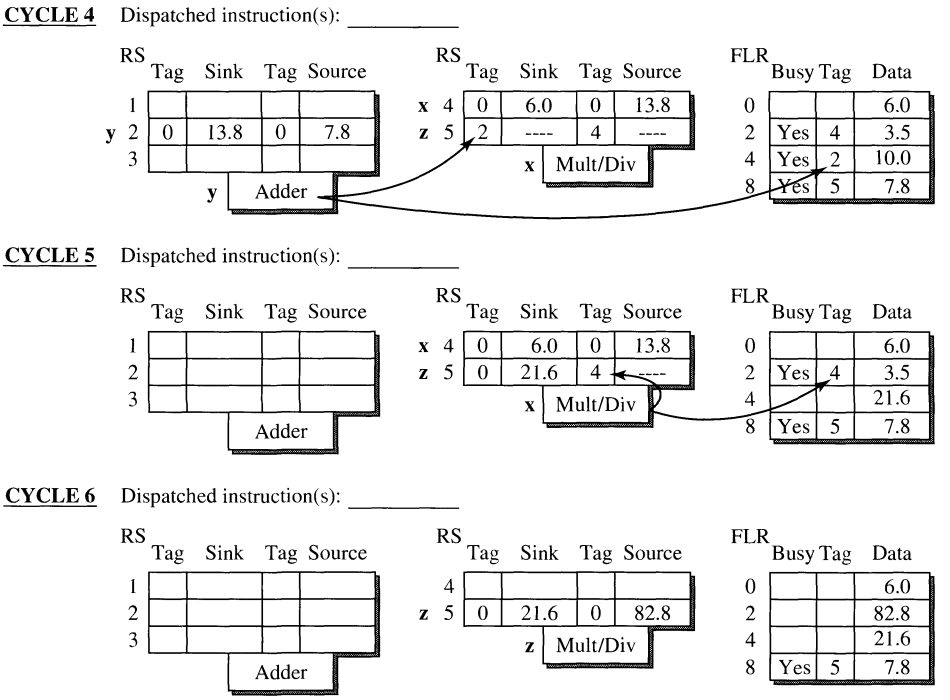
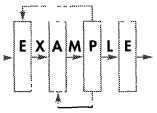


Figure 5.24

Illustration of Tomasulo's Algorithm on an Example Instruction Sequence (Part 2).



have such tag matches. Hence the result of instruction w is forwarded to dependent instructions x and y.

In cycle 3, instruction y begins execution in the adder unit, and instruction x begins execution in the multiply/divide unit. Instruction y finishes execution in cycle 4 (see Figure 5.24) and broadcasts its result on the CDB along with the tag value of 2 (its reservation station ID). The first tag field in reservation station 5 (holding instruction z) and the tag field of R4 have tag matches and pull in the result of instruction y. Instruction x finishes execution in cycle 5 and broadcasts its result on the CDB along with the tag value of 4. The second tag field in reservation station 5 (holding instruction z) and the tag field of R2 have tag matches and pull in the result of instruction x. In cycle 6, instruction z begins execution and finishes in cycle 8.

Figure 5.25(a) illustrates the data flow graph of this example sequence of four instructions. The four solid arcs represent the four true data dependences, while the other three arcs represent the anti- and output dependences. Instructions are dispatched in program order. Anti-dependences are resolved by copying an operand at dispatch time to the reservation station. Hence, it is not possible for a trailing instruction to overwrite a register before an earlier instruction has a chance to read that register. If the operand is still pending, the dispatched instruction will receive

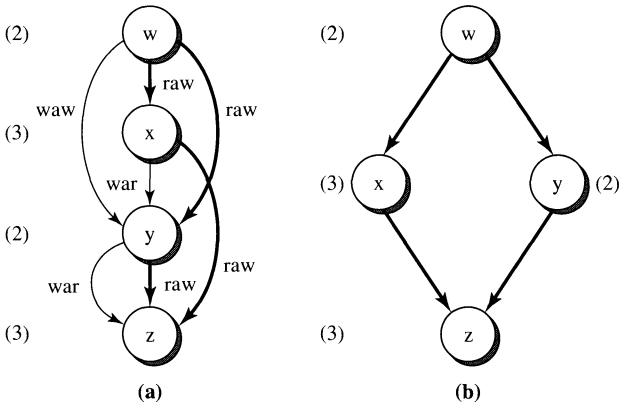
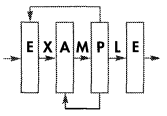


Figure 5.25
Data Flow Graphs of the Example Instruction Sequence: (a) All Data Dependences; (b) True Data Dependences.



the tag for that operand. When that operand becomes available, the instruction will receive that operand via a tag match in its reservation station.

As an instruction is dispatched, the tag field of its destination register is written with the reservation station ID of that instruction. When a subsequent instruction with the same destination register is dispatched, the same tag field will be updated with the reservation station ID of this new instruction. The tag field of a register always contains the reservation station ID of the latest updating instruction. If there are multiple instructions in flight that have the same destination register, only the latest instruction will be able to update that register. Output dependences are implicitly resolved by making it impossible for an earlier instruction to update a register after a later instruction has updated the same register. This does introduce the problem of not being able to support precise exception since the register file does not necessarily evolve through all its sequential states; i.e., a register can potentially miss an intermediate update. For example, in Figure 5.23, at the end of cycle 2, instruction w should have updated its destination register R4. However, instruction y has the same destination register, and when it was dispatched earlier in that cycle, the tag field of R4 was changed from 1 to 2 anticipating the update of R4 by instruction y. At the end of cycle 2 when instruction w broadcasts its tag value of 1, the tag field of R4 fails to trigger a tag match and does not pull in the result of instruction w. Eventually R4 will be updated by instruction y. However, if an exception is triggered by instruction x, precise exception will be impossible since the register file does not evolve through all its sequential states.

Tomasulo's algorithm resolves anti- and output dependences via a form of register renaming. Each definition of an FLR triggers the renaming of that register to a register tag. This tag is taken from the ID of the reservation station containing the instruction that redefines that register. This effectively removes false dependences from causing pipeline stalls. Hence, the data flow limit is strictly determined by the true data dependences. Figure 5.25(b) depicts the data flow graph involving only

true data dependences. As shown in Figure 5.25(a) if all four instructions were required to execute sequentially to enforce all the data dependences, including anti- and output dependences, the total latency required for executing this sequence of instructions would be 10 cycles, given the latencies of 2 and 3 cycles for addition and multiplication instructions, respectively. When only true dependences are considered, Figure 5.25(b) reveals that the critical path is only 8 cycles, i.e., the path involving instructions w, x, and z. Hence, the data flow limit for this sequence of four instructions is 8 cycles. This limit is achieved by Tomasulo's algorithm as demonstrated in Figures 5.23 and 5.24.

5.2.5 Dynamic Execution Core

Most current state-of-the-art superscalar microprocessors consist of an out-of-order execution core sandwiched between an in-order front end, which fetches and dispatches instructions in program order, and an in-order back end, which completes and retires instructions also in program order. The out-of-order execution core (also referred to as the *dynamic execution* core), resembling a refinement of Tomasulo's algorithm, can be viewed as an embedded data flow, or *micro-dataflow,* engine that attempts to approach the data flow limit in instruction execution. The operation of such a dynamic execution core can be described according to the three phases in the pipeline, namely, *instruction dispatching, instruction execution,* and *instruction completion;* see Figure 5.26.

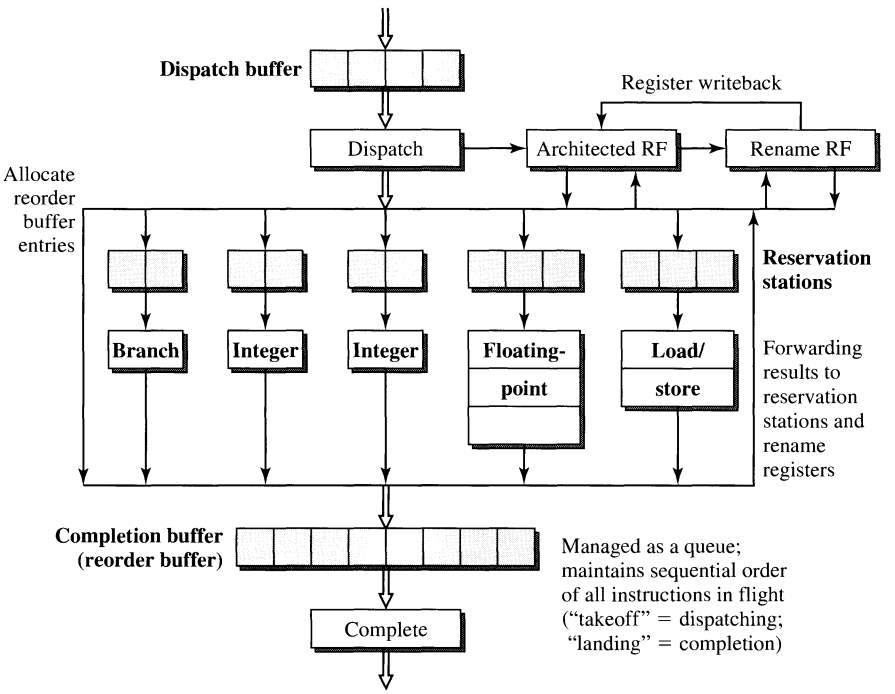


Figure 5.26
Micro-Dataflow Engine for Dynamic Execution.

The instruction dispatching phase consists of *renaming* of destination registers, *allocating* of reservation station and reorder buffer entries, and *advancing* instructions from the dispatch buffer to the reservation stations. For ease of presentation, we assume here that register renaming is performed in the dispatch stage. All redefinitions of architected registers are renamed to rename registers. Trailing uses of these redefinitions are assigned the corresponding rename register specifiers. This ensures that all producer-consumer relationships are properly identified and all false register dependences are removed.

Instructions in the dispatch buffer are then dispatched to the appropriate reservation stations based on instruction type. Here we assume the use of distributed reservation stations, and we use *reservation station* to refer to the (multientry) instruction buffer attached to each functional unit and *reservation station entry* to refer to one of the entries of this buffer. Simultaneous with the allocation of reservation station entries for the dispatched instructions is the allocation of entries in the reorder buffer for the same instructions. Reorder buffer entries are allocated according to program order.

Typically, for an instruction to be dispatched there must be the availability of a rename register, a reservation station entry, and a reorder buffer entry. If any one of these three is not available, instruction dispatching is stalled. The actual dispatching of instructions from the dispatch buffer entries to the reservation station entries is via a complex routing network. If the connectivity of this routing network is less than that of a full crossbar (this is frequently the case in real designs), then stalling can also occur due to resource contention in the routing network.

The instruction execution phase consists of *issuing* of ready instructions, *executing* the issued instructions, and *forwarding* of results. Each reservation station is responsible for identifying instructions that are ready to execute and for scheduling their execution. When an instruction is first dispatched to a reservation station, it may not have all its source operands and therefore must wait in the reservation station. Waiting instructions continually monitor the busses for tag matches. When a tag match is triggered, indicating the availability of the pending operand, the result being broadcasted is latched into the reservation station entry. When an instruction in a reservation station entry has all its operands, it becomes ready for execution and can be issued into the functional unit. In a given machine cycle if multiple instructions in a reservation station are ready, a scheduling algorithm is used (typically oldest first) to pick one of them for issuing into the functional unit to begin execution. If there is only one functional unit connected to a reservation station (as is the case for distributed reservation stations), then that reservation station can only issue one instruction per cycle.

Once issued into a functional unit, an instruction is executed. Functional units can vary in terms of their latency. Some have single-cycle latencies, others have fixed multiple-cycle latencies. Certain functional units can require a variable number of cycles, depending on the values of the operands and the operation being performed. Typically, even with function units that require multiple-cycle latencies, once an instruction begins execution in a pipelined functional unit, there is no further stalling of that instruction in the middle of the execution pipeline since all data dependences have been resolved prior to issuing and there shouldn't be any resource contention.

When an instruction finishes execution, it asserts its destination tag (i.e., the specifier of the rename register assigned for its destination) and the actual result onto a forwarding bus. All dependent instructions waiting in the reservation stations will trigger a tag match and latch in the broadcasted result. This is how an instruction forwards its result to other dependent instructions without requiring the intermediate steps of updating and then reading of the dependent register. Concurrent with result forwarding, the RRF uses the broadcasted tag as an index and loads the broadcasted result into the selected entry of the RRF.

Typically a reservation station entry is deallocated when its instruction is issued in order to allow another trailing instruction to be dispatched into it. Reservation station saturation can cause instruction dispatch to stall. Certain instructions whose execution can induce an exceptional condition may require rescheduling for execution in a future cycle. Frequently, for these instructions, their reservation station entries are not deallocated until they finish execution without triggering any exceptions. For example, a load instruction can potentially trigger a D-cache miss that may require many cycles to service. Instead of stalling the functional unit, such an excepting load can be reissued from the reservation station after the miss has been serviced.

In a dynamic execution core as described previously, a producer-consumer relationship is satisfied without having to wait for the writing and then the reading of the dependent register. The dependent operand is directly forwarded from the producer instruction to the consumer instruction to minimize the latency incurred due to the true data dependence. Assuming that an instruction can be issued in the same cycle that it receives its last pending operand via a forwarding bus, if there is no other instruction contending for the same functional unit, then this instruction should be able to begin execution in the cycle immediately following the availability of all its operands. Hence, if there are adequate resources such that no stalling due to structural dependences occurs, then the dynamic execution core should be able to approach the data flow limit.

5.2.6 Reservation Stations and Reorder Buffer

Other than the functional units, the critical components of the dynamic execution core are the *reservation stations* and the *reorder buffer*. The operations of these components dictate the function of the dynamic execution core. Here we present the issues associated with the implementation of the reservation station and the reorder buffer. We present their organization and behavior with special focus on loading and unloading of an entry of a reservation station and the reorder buffer.

There are three tasks associated with the operation of a reservation station: *dispatching, waiting,* and *issuing*. A typical reservation station is shown in Figure 5.27(b), and the various fields in an entry of a reservation station are illustrated in Figure 5.27(a). Each entry has a busy bit, indicating that the entry has been allocated, and a ready bit, indicating that the instruction in that entry has all its source operands. Dispatching involves loading an instruction from the dispatch buffer into an entry of the reservation station. Typically the dispatching of an instruction requires the following three steps: select a free, i.e., not busy, reservation

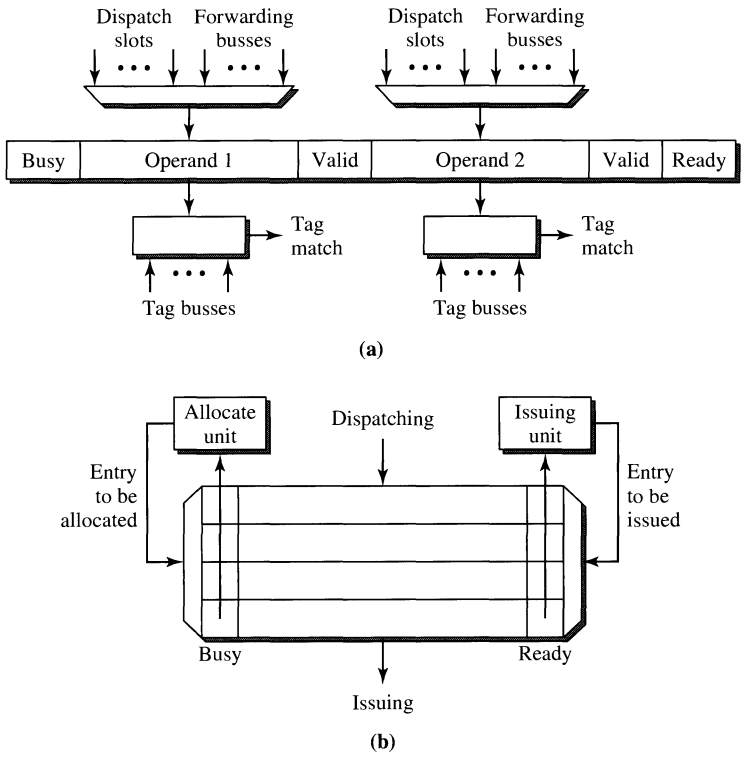


Figure 5.27

Reservation Station Mechanisms: (a) A Reservation Station Entry; (b) Dispatching into and Issuing from a Reservation Station.

station entry; load operands and/or tags into the selected entry; and set the busy bit of that entry. The selection of a free entry is based on the busy bits and is performed by the allocate unit. The allocate unit examines all the busy bits and selects one of the nonbusy entries to be the allocated entry. This can be implemented using a priority encoder. Once an entry is allocated, the operands and/or tags of the instruction are loaded into the entry. Each entry has two operand fields, each of which has an associated valid bit. If the operand field contains the actual operand, then the valid bit is set. If the field contains a tag, indicating a pending operand, then its valid bit is reset and it must wait for the operand to be forwarded. Once an entry is allocated, its busy bit must be set.

An instruction with a pending operand must wait in the reservation station. When a reservation station entry is waiting for a pending operand, it must continuously monitor the tag busses. When a tag match occurs, the operand field latches in the forwarded result and sets its valid bit. When both operand fields are valid, the ready bit is set, indicating that the instruction has all its source operands and is ready to be issued. This is usually referred to as *instruction wake up*.

The issuing step is responsible for selecting a ready instruction in the reservation station and issues it into the functional unit. This is usually referred to as *instruction select*. All the ready instructions are identified by their ready bits being set. The selecting of a ready instruction is performed by the issuing unit based on a scheduling heuristic; see Figure 5.27(b). The heuristic can be based on program order or how long each ready instruction has been waiting in the reservation station. Typically when an instruction is issued into the functional unit, its reservation station entry is deallocated by resetting the busy bit.

A large reservation station can be quite complex to implement. On its input side, it must support many possible sources, including all the dispatch slots and forwarding busses; see Figure 5.27(a). The data routing network on its input side can be quite complex. During the waiting step, all operand fields of a reservation station with pending operands must continuously compare their tags against potentially multiple tag busses. This is comparable to doing an associative search across all the reservation station entries involving multiple keys (tag busses). If the number of entries is small, this is quite feasible. However, as the number of entries increases, the increase in complexity is quite significant. This portion of the hardware is commonly referred to as the *wake-up logic*. When the entry count increases, it also complicates the issuing unit and the scheduling heuristic in selecting the best ready instruction to issue. This portion of the hardware is commonly referred to as the *select logic*. In any given machine cycle, there can be multiple ready instructions. The select logic must determine the best one to issue. For a superscalar machine, a reservation station can potentially support multiple instruction issues per cycle, in which case the select logic must pick the best subset of instructions to issue among all the ready instructions.

The reorder buffer contains all the instructions that are *in flight,* i.e., all the instructions that have been dispatched but not yet completed architecturally. These include all the instructions waiting in the reservation stations and executing in the functional units and those that have finished execution but are waiting to be completed in program order. The status of each instruction in the reorder buffer can be tracked using several bits in each entry of the reorder buffer. Each instruction can be in one of several states, i.e., waiting execution, in execution, and finished execution. These status bits are updated as an instruction traverses from one state to the next. An additional bit can also be used to indicate whether an instruction is speculative (in the predicted path) or not. If speculation can cross multiple branches, additional bits can be employed to identify which speculative basic block an instruction belongs to. When a branch is resolved, a speculative instruction can become nonspeculative (if the prediction is correct) or invalid (if the prediction is incorrect). Only finished and nonspeculative instructions can be completed. An instruction marked invalid is not architecturally completed when exiting the reorder buffer. Figure 5.28(a) illustrates the fields typically found in a reorder buffer entry; in this figure the rename register field is also included.

The reorder buffer is managed as a circular queue using a head pointer and a tail pointer; see Figure 5.28(b). The tail pointer is advanced when reorder buffer entries are allocated at instruction dispatch. The number of entries that can be

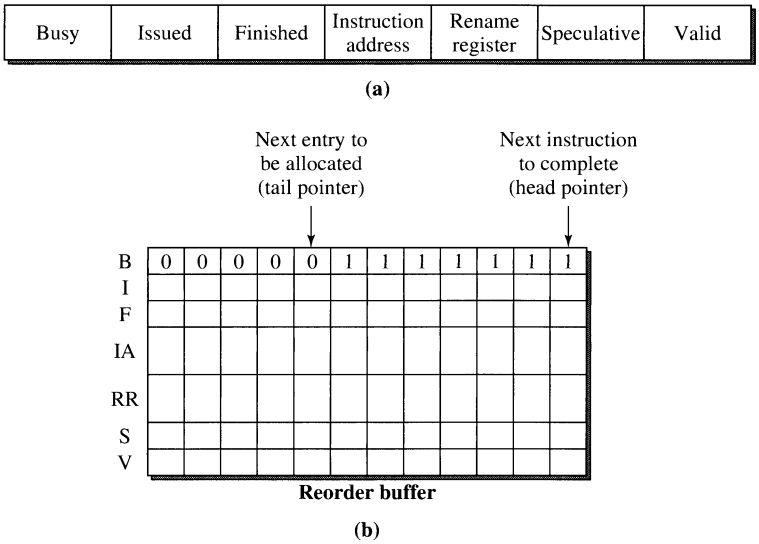


Figure 5.28
(a) Reorder Buffer Entry; (b) Reorder Buffer Organization.

allocated per cycle is limited by the dispatch bandwidth. Instructions are completed from the head of the queue. From the head of the queue as many instructions that have finished execution can be completed as the completion bandwidth allows. The completion bandwidth is determined by the capacity of another routing network and the ports available for register writeback. One of the critical issues is the number of write ports to the architected register file that are needed to support the transferring of data from the rename registers (or the reorder buffer entries if they are used as rename registers) to the architected registers. When an instruction is completed, its rename register and its reorder buffer entry are deallocated. The head pointer to the reorder buffer is also appropriately updated. In a way the reorder buffer can be viewed as the heart or the central control of the dynamic execution core because the status of all in-flight instructions is tracked by the reorder buffer.

It is possible to combine the reservation stations and the reorder buffer into one single structure, called the *instruction window,* that manages all the instructions in flight. Since at dispatch an entry in the reservation station and an entry in the reorder buffer must be allocated for each instruction, they can be combined as one entry in the instruction window. Hence, instructions are dispatched into the instruction window, entries of the instruction window monitor the tag busses for pending operands, results are forwarded into the instruction window, instructions are issued from the instruction window when ready, and instructions are completed from the instruction window. The size of the instruction window determines the maximum number of instructions that can be simultaneously in flight within the machine and consequently the degree of instruction-level parallelism that can be achieved by the machine.

5.2.7 Dynamic Instruction Scheduler

The dynamic instruction scheduler is the heart of a dynamic execution core. We use the term *dynamic instruction scheduler* to include the instruction window and its associated instruction wake-up and select logic. Currently there are two styles to the design of the dynamic instruction scheduler, namely, *with data capture* and *without data capture*.

Figure 5.29(a) illustrates a scheduler with data capture. With this style of scheduler design, when dispatching an instruction, those operands that are ready are copied from the register file (either architected or physical) into the instruction window; hence, we have the term *data captured*. For the operands that are not ready, tags are copied into the instruction window and used to latch in the operands when they are forwarded by the functional units. Results are forwarded to their waiting instructions in the instruction window. In effect, result forwarding and instruction wake up are combined, a la Tomasulo's algorithm. A separate forwarding path is needed to also update the register file so that subsequent dependent instructions can grab their source operands from the register file when they are being dispatched.

Some recent microprocessors have adopted a different style that does not employ data capture in the scheduler design; see Figure 5.29(b). In this style, register read is performed after the scheduler, as instructions are being issued to the functional units. At instruction dispatch there is no copying of operands into the instruction window; only tags (or pointers) for operands are loaded into the window. The scheduler still performs tag match to wake up ready instructions. However, results from functional units are only forwarded to the register file. All ready instructions that are issued obtain their operands directly from the register file just prior to execution. In effect, result forwarding and instruction wake up are decoupled. For instruction wake up only the tag needs to be forwarded to the scheduler. With the non-data-captured style of scheduler, the size (width) of the instruction window can be significantly reduced, and the much wider result-forwarding path to the scheduler is not needed.

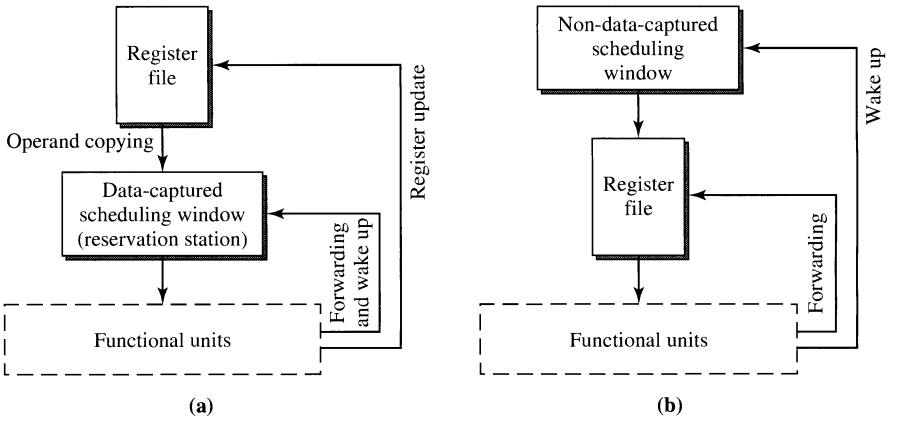


Figure 5.29

Dynamic Instruction Scheduler Design: (a) With Data Capture; (b) Without Data Capture.

There is a close relationship between register renaming and instruction scheduling. As stated earlier, one purpose for doing dynamic register renaming is to eliminate the false dependences induced by register recycling. Another purpose is to establish the producer-consumer relationship between two dependent instructions. A true data dependence is determined by the common rename register specifier in the producer and consumer instructions. The rename register specifier can function as the tag for result forwarding. In the non-data-captured scheduler of Figure 5.29(b) the register specifiers are used to access the register file for retrieving source operands; the (destination) register specifiers are used as tags for waking up dependent instructions in the scheduler. For the data-captured type of scheduler of Figure 5.29(a), the tags used for result forwarding and instruction wake up do not have to be actual register specifiers. The tags are mainly used to identify producer-consumer relationships between dependent instructions and can be assigned arbitrarily. For example, Tomasulo's algorithm uses reservation station IDs as the tags for forwarding results to dependent instructions as well as for updating architected registers. There is no explicit register renaming involving physical rename registers.

5.2.8 Other Register Data Flow Techniques

For many years the data flow limit has been assumed to be an absolute theoretical limit and the ultimate performance goal. Extensive research efforts on data flow architectures and data flow machines have been going on for over three decades. The data flow limit assumes that true data dependences are absolute and cannot possibly be overcome. Interestingly, in the late 1960s and the early 1970s a similar assumption was made concerning control dependences. It was generally thought that control dependences are absolute and that when encountering a conditional branch instruction there is no choice but to wait for that conditional branch to be executed before proceeding to the next instruction due to the uncertainty of the actual control flow. Since then, we have witnessed tremendous strides made in the area of branch prediction techniques. Conditional branches and associated control dependences are no longer absolute barriers and can frequently be overcome by speculating on the direction and the target address of the branch. What made such speculation possible is that frequently the outcome of a branch instruction is quite predictable. It wasn't until 1995 that researchers began to also question the absoluteness of true data dependences.

In 1996 several research papers appeared that proposed the concept of *value prediction*. The first paper by Lipasti, Wilkerson, and Shen focused on predicting load values based on the observation that frequently the values being loaded by a particular static load instruction are quite predictable [Lipasti et al., 1996]. Their second paper generalized the same basic idea for predicting the result of ALU instructions [Lipasti and Shen, 1996]. Experimental data based on real input data sets indicate that the results produced by many instructions are actually quite predictable. The notion of *value locality* indicates that certain instructions tend to repeatedly produce the same small set (sometimes one) of result values. By tracking the results produced by these instructions, future values can become predictable based on the historical values. Since these seminal papers, numerous papers have been published in recent years proposing various designs of value predictors [Mendelson and Gabbay, 1997; Sazeides and Smith, 1997; Calder et al., 1997; Gabbay and Mendelson, 1997;

1998a; 1998b; Calder et al., 1999]. In a recent study, it was shown that a hybrid value predictor can achieve prediction rates of up to 80% and a realistic design incorporating value prediction can achieve IPC improvements in the range of 8.6% to 23% for the SPEC benchmarks [Wang and Franklin, 1997].

When the result of an instruction is correctly predicted via value prediction, typically performed during the fetch stage, a subsequent dependent instruction can begin execution using this speculative result without having to wait for the actual decoding and execution of the leading instruction. This effectively removes the serialization constraint imposed by the true data dependence between these two instructions. In a way this particular dependence edge in the data flow graph is effectively removed when correct value prediction is performed. Hence, value prediction provides the potential to exceed the classical data flow limit. Of course, validation is still required to ensure that the prediction is correct and becomes the new limit on instruction execution throughput. Value prediction becomes effective in increasing machine performance if misprediction rarely occurs and the misprediction penalty is small (e.g., zero or one cycle) and if the validation latency is less than the average instruction execution latency. Clearly, efficient implementation of value prediction is crucial in ensuring its efficacy in improving performance.

Another recently proposed idea is called *dynamic instruction reuse* [Sodani and Sohi, 1997]. Similar to the concept of value locality, it has been observed through experiments with real programs that frequently the same sequence of instructions is repeatedly executed using the same set of input data. This results in redundant computation being performed by the machine. Dynamic instruction reuse techniques attempt to track such redundant computations, and when they are detected, the previous results are used without performing the redundant computations. These techniques are nonspeculative; hence, no validation is required. While value prediction can be viewed as the elimination of certain dependence edges in the data flow graph, dynamic instruction reuse techniques attempt to remove both nodes and edges of a subgraph from the data flow graph. A much earlier research effort had shown that such elimination of redundant computations can yield significant performance gains for programs written in functional languages [Harbison, 1980; 1982]. A more recent study also yields similar data on the presence of redundant computations in real programs [Richardson, 1992]. This is an area that is currently being actively researched, and new insightful results can be expected.

We will revisit these advanced register data flow techniques in Chapter 10 in greater detail.

5.3 Memory Data Flow Techniques

Memory instructions are responsible for moving data between the main memory and the register file, and they are essential for supporting the execution of ALU instructions. Register operands needed by ALU instructions must first be loaded from memory. With a limited number of registers, during the execution of a program not all the operands can be kept in the register file. The compiler generates *spill code* to temporarily place certain operands out to the main memory and to

reload them when they are needed. Such spill code is implemented using store and load instructions. Typically, the compiler only allocates scalar variables into registers. Complex data structures, such as arrays and linked lists, that far exceed the size of the register file are usually kept in the main memory. To perform operations on such data structures, load and store instructions are required. The effective processing of load/store instructions can minimize the overhead of moving data between the main memory and the register file.

The processing of load/store instructions and the resultant memory data flow can become a bottleneck to overall machine performance due to the potential long latency for executing memory instructions. The long latency of load/store instructions results from the need to compute a memory address and the need to access a memory location. To support virtual memory, the computed memory address (called the *virtual address*) also needs to be translated into a physical address before the physical memory can be accessed. Cache memories are very effective in reducing the effective latency for accessing the main memory. Furthermore, various techniques have been developed to reduce the overall latency and increase the overall throughput for processing load/store instructions.

5.3.1 Memory Accessing Instructions

The execution of memory data flow instructions occurs in three steps: memory address generation, memory address translation, and data memory accessing. We first state the basis for these three steps and then describe the processing of load/store instructions in a superscalar pipeline.

The register file and the main memory are defined by the instruction set architecture for data storage. The main memory as defined in an instruction set architecture is a collection of 2^n memory locations with random access capability; i.e., every memory location is identified by an n-bit address and can be directly accessed with the same latency. Just like the architected register file, the main memory is an architected entity and is visible to the software instructions. However, unlike the register file, the address that identifies a particular memory location is usually not explicitly stored as part of the instruction format. Instead, a memory address is usually generated based on a register and an offset specified in the instruction. Hence, address generation is required and involves the accessing of the specified register and the adding of the offset value.

In addition to address generation, address translation is required when virtual memory is implemented in a system. The architected main memory constitutes the virtual address space of the program and is viewed by each program as its private address space. The physical memory that is implemented in a machine constitutes the physical address space, which may be smaller than the virtual address space and may even be shared by multiple programs. Virtual memory is a mechanism that maps the virtual address space of a program to the physical address space of the machine. With such address mapping, virtual memory is able to support the execution of a program with a virtual address space that is larger than the physical address space, and the multiprogramming paradigm by mapping multiple virtual address spaces to the same physical address space. This mapping mechanism involves the translation of the

computed effective address, i.e., the virtual address, into a physical address that can be used to access the physical memory. This mechanism is usually implemented using a mapping table, and address translation is performed via a table lookup.

The third step in processing a load/store instruction is memory accessing. For load instructions data are read from a memory location and stored into a register, while for store instructions a register value is stored into a memory location. While the first two steps of address generation and address translation are performed in identical fashion for both loads and stores, the third step is performed differently for loads and stores by a superscalar pipeline.

In Figure 5.30, we illustrate these three steps as occurring in three pipeline stages. The first pipe stage performs effective address generation. We assume the typical addressing mode of register indirect with an offset for both load and store instructions. For a load instruction, as soon as the address register operand is available, it is issued into the pipelined functional unit and the effective address is generated by the first pipe stage. A store instruction must wait for the availability of both the address register and the data register operands before it is issued.

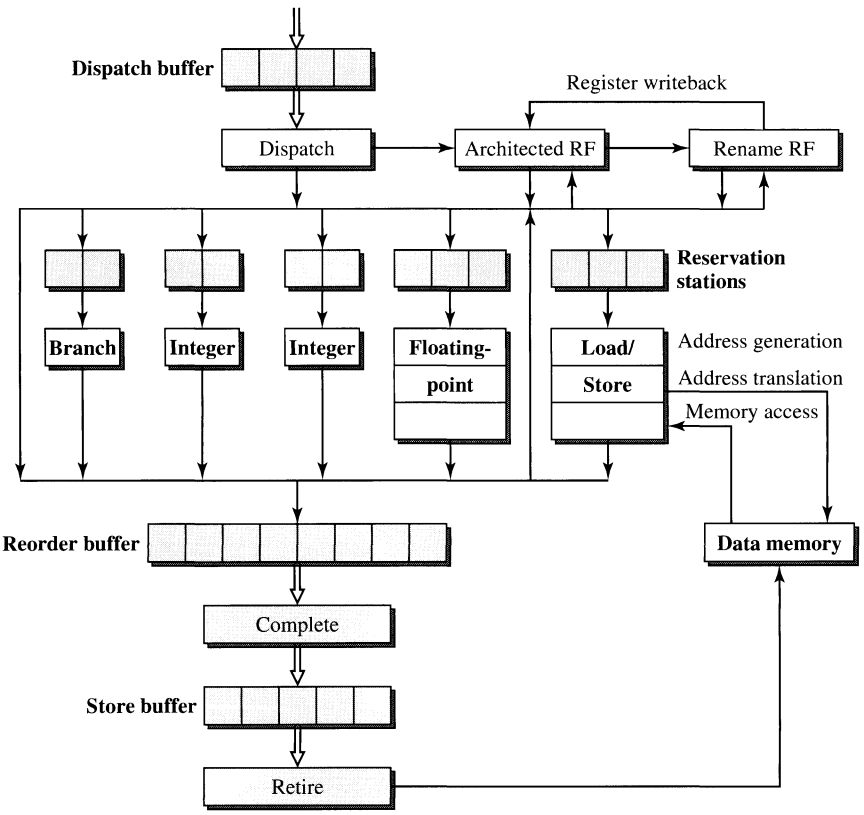


Figure 5.30
Processing of Load/Store Instructions.

After the first pipe stage generates the effective address, the second pipe stage translates this virtual address into a physical address. Typically, this is done by accessing the translation lookaside buffer (TLB), which is a hardware-controlled table containing the mapping of virtual to physical addresses. The TLB is essentially a cache of the page table that is stored in the main memory. (Section 3.6 provides more background material on the page table and the TLB.) It is possible that the virtual address being translated belongs to a page whose mapping is not currently resident in the TLB. This is called a *TLB miss*. If the particular mapping is present in the page table, then it can be retrieved by accessing the page table in the main memory. Once the missing mapping is retrieved and loaded into the TLB, the translation can be completed. It is also possible that the mapping is not resident even in the page table, meaning that the particular page being referenced has not been mapped and is not resident in the main memory. This will induce a *page fault* and require accessing disk storage to retrieve the missing page. This constitutes a program exception and will necessitate the suspension of the execution of the current program.

After successful address translation in the second pipe stage, a load instruction accesses the data memory during the third pipe stage. At the end of this machine cycle, the data are retrieved from the data memory and written into either the rename register or the reorder buffer. At this point the load instruction finishes execution. The updating of the architected register is not performed until this load instruction is completed from the reorder buffer. Here we assume that data memory access can be done in one machine cycle in the third pipe stage. This is possible if a data cache is employed. (Section 3.6 provides more background material on caches.) With a data cache, it is possible that the data being loaded are not resident in the data cache. This will result in a data *cache miss* and require the filling of the data cache from the main memory. Such cache misses can necessitate the stalling of the load/store pipeline.

Store instructions are processed somewhat differently than load instructions. Unlike a load instruction, a store instruction is considered as having *finished* execution at the end of the second pipe stage when there is a successful translation of the address. The register data to be stored to memory are kept in the reorder buffer. At the time when the store is being completed, these data are then written out to memory. The reason for this delayed access to memory is to prevent the premature and potentially erroneous update of the memory in case the store instruction may have to be flushed due to the occurrence of an exception or a branch misprediction. Since load instructions only read the memory, their flushing will not result in unwanted side effects to the memory state.

For a store instruction, instead of updating the memory at completion, it is possible to move the data to the store buffer at completion. The store buffer is a FIFO buffer that buffers architecturally completed store instructions. Each of these store instructions is then retired, i.e., updates the memory, when the memory bus is available. The purpose of the store buffer is to allow stores to be retired when the memory bus is not busy, thus giving priority to loads that need to access the memory bus. We use the term *completion* to refer to the updating of the CPU state and the term *retiring* to refer to the updating of the memory state. With the store buffer, a store instruction can be architecturally complete but not yet retired to memory.

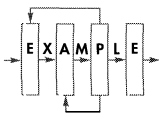
When a program exception occurs, the instructions that follow the excepting instruction and that may have finished out of order, must be flushed from the reorder buffer; however, the store buffer must be drained, i.e., the store instructions in the store buffer must be retired, before the excepting program can be suspended.

We have assumed here that both address translation and memory accessing can be done in one machine cycle. Of course, this is only the case when both the TLB and the first level of the memory hierarchy return hits. An in-depth treatment of memory hierarchies that maximize the occurrence of cache hits by exploiting temporal and spatial locality and using various forms of caching is provided in Chapter 3. In Section 5.3.2, we will focus specifically on the additional complications that result from out-of-order execution of memory references and on some of the mechanisms used by modern problems to address these complications.

5.3.2 Ordering of Memory Accesses

A memory data dependence exists between two load/store instructions if they both reference the same memory location, i.e., there exists an *aliasing*, or collision, of the two memory addresses. A load instruction performs a read from a memory location, while a store instruction performs a write to a memory location. Similar to register data dependences, read-after-write (RAW), write-after-read (WAR), and write-after-write (WAW) dependences can exist between load and store instructions. A store (load) instruction followed by a load (store) instruction involving the same memory location will induce a RAW (WAR) memory data dependence. Two stores to the same memory location will induce a WAW dependence. These memory data dependences must be enforced in order to preserve the correct semantics of the program.

One way to enforce memory data dependences is to execute all load/store instructions in program order. Such total ordering of memory instructions is sufficient for enforcing memory data dependences but not necessary. It is conservative and can impose an unnecessary limitation on the performance of a program. We use the example in Figure 5.31 to illustrate this point. DAXPY is the name of a piece




```
Y(i) = A * X(i) + Y(i)

    F0 ← LD,a
    R4 ← ADDI,Rx,#512      ;last address

Loop:
    F2 ← LD,0(Rx)          ;load X(i)
    F2 ← MULTD,F0,F2       ;A*X(i)
    F4 ← LD,0(Ry)          ;load Y(i)
    F4 ← ADDD,F2,F4        ;A*X(i)+Y(i)
    0(Ry) ← SD,F4          ;store into Y(i)
    Rx ← ADDI,Rx,#8        ;inc. index to X
    Ry ← ADDI,Ry,#8        ;inc. index to Y
    R20 ← SUB,R4,Rx        ;compute bound
    BNZ,R20,Loop           ;check if done
```

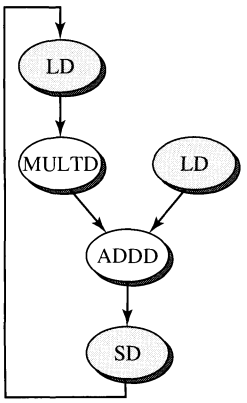


Figure 5.31
The DAXPY Example.

of code that multiplies an array by a coefficient and then adds the resultant array to another array. DAXPY (derived from "double precision A times X plus Y") is a kernel in the LINPAC routines and is commonly found in many numerical programs. Notice that all the iterations of this loop are data-independent and can be executed in parallel. However, if we impose the constraint that all load/store instructions be executed in total order, then the first load instruction of the second iteration cannot begin until the store instruction of the first iteration is performed. This constraint will effectively serialize the execution of all the iterations of this loop.

By allowing load/store instructions to execute out of order, without violating memory data dependences, performance gain can be achieved. Take the example of the DAXPY loop. The graph in Figure 5.31 represents the true data dependences involving the core instructions of the loop body. These dependences exist among the instructions of the same iteration of the loop. There are no data dependences between multiple iterations of the loop. The loop closing branch instruction is highly predictable; hence, the fetching of subsequent iterations can be done very quickly. The same architected registers specified by instructions from subsequent iterations are dynamically renamed by register renaming mechanisms; hence, there are no register dependences between the iterations due to the dynamic reuse of the same architected registers. Consequently if load/store instructions are allowed to execute out of order, the load instructions from a trailing iteration can begin before the execution of the store instruction from an earlier iteration. By overlapping the execution of multiple iterations of the loop, performance gain is achieved for the execution of this loop.

Memory models impose certain limitations on the out-of-order execution of load/store instructions by a processor. First, to facilitate recovery from exceptions, the sequential state of the memory must be preserved. In other words, the memory state must evolve according to the sequential execution of load/store instructions. Second, many shared-memory multiprocessor systems assume the sequential consistency memory model, which requires that the accessing of the shared memory by each processor be done according to program order [Lamport, 1979, Adve and Gharachorloo, 1996].[1] Both of these reasons effectively require that store instructions be executed in program order, or at least the memory must be updated as if stores are performed in program order. If stores are required to execute in program order, WAW and WAR memory data dependences are implicitly enforced and are not an issue. Hence, only RAW memory data dependences must be enforced.

5.3.3 Load Bypassing and Load Forwarding

Out-of-order execution of load instructions is the primary source for potential performance gain. As can be seen in the DAXPY example, load instructions are frequently at the beginning of dependence chains, and their early execution can facilitate the early execution of other dependent instructions. While relative to memory data dependences, load instructions are viewed as performing read operations on the memory locations, they are actually performing write operations to their destination registers. With loads being register-defining (DEF) instructions, they

[1]Consistency models are discussed in greater detail in Chapter 11.

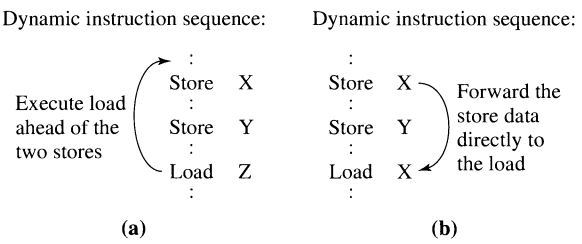


Figure 5.32
Early Execution of Load Instructions: (a) Load Bypassing; (b) Load Forwarding.

are typically followed immediately by other dependent register use (USE) instructions. The goal is to allow load instructions to begin execution as early as possible, possibly jumping ahead of other preceding store instructions, as long as RAW memory data dependences are not violated and that memory is updated according to the sequential memory consistency model.

Two specific techniques for early out-of-order execution of loads are *load bypassing* and *load forwarding*. As shown in Figure 5.32(a), load bypassing allows a trailing load to be executed earlier than preceding stores if the load address does not alias with the preceding stores; i.e., there is no memory data dependence between the stores and the load. On the other hand, if a trailing load aliases with a preceding store, i.e., there is a RAW dependence from the store to the load, load forwarding allows the load to receive its data directly from the store without having to access the data memory; see Figure 5.32(b). In both of these cases, earlier execution of a load instruction is achieved.

Before we discuss the issues that must be addressed in order to implement load bypassing and load forwarding, first we present the organization of the portion of the execution core responsible for processing load/store instructions. This organization, shown in Figure 5.33, is used as the vehicle for our discussion on load bypassing and load forwarding. There is one store unit (two pipe stages) and one load unit (three pipe stages); both are fed by a common reservation station. For now we assume that load and store instructions are issued from this shared reservation station in program order. The store unit is supported by a store buffer. The load unit and the store buffer can access the data cache.

Given the organization of Figure 5.33, a store instruction can be in one of several states while it is in flight. When a store instruction is dispatched to the reservation station, an entry in the reorder buffer is allocated for it. It remains in the reservation station until all its source operands become available and it is issued into the pipelined execution unit. Once the memory address is generated and successfully translated, it is considered to have finished execution and is placed into the finished portion of the store buffer (the reorder buffer is also updated). The store buffer operates as a queue and has two portions, *finished* and *completed*. The finished portion contains those stores that have finished execution but are not yet architecturally

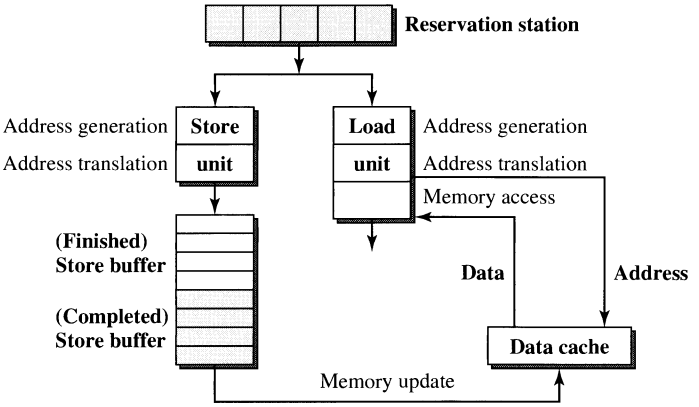


Figure 5.33

Mechanisms for Load/Store Processing: Separate Load and Store Units with In-Order Issuing from a Common Reservation Station.

completed. The completed portion of the store buffer contains those stores that are completed architecturally but waiting to update the memory. The identification of the two portions of the store buffer can be done via a pointer to the store buffer or a status bit in the store buffer entries. A store in the finished portion of the store buffer can potentially be speculative, and when a misspeculation is detected, it will need to be flushed from the store buffer. When a finished store is completed by the reorder buffer, it changes from the finished state to the completed state. This can be done by updating the store buffer pointer or flipping the status bit. When a completed store finally exits the store buffer and updates the memory, it is considered retired. Viewed from the perspective of the memory state, a store does not really finish its execution until it is retired. When an exception occurs, the stores in the completed portion of the store buffer must be drained in order to appropriately update the memory. So between being dispatched and retired, a store instruction can be in one of three states: *issued* (in the execution unit), *finished* (in the finished portion of the store buffer), or *completed* (in the completed portion of the store buffer).

One key issue in implementing load bypassing is the need to check for possible aliasing with preceding stores, i.e., those stores being bypassed. A load is considered to bypass a preceding store if the load reads from the memory before the store writes to the memory. Hence, before such a load is allowed to execute or read from the memory, it must be determined that it does not alias with all the preceding stores that are still in flight, i.e., those that have been issued but not retired. Assuming in-order issuing of load/store instructions from the load/store reservation station, all such stores should be sitting in the store buffer, including both the finished and the completed portions. The alias checking for possible dependence between the load and the preceding store can be done using the store buffer. A tag field containing the memory address of the store is incorporated with each entry of the store buffer. Once

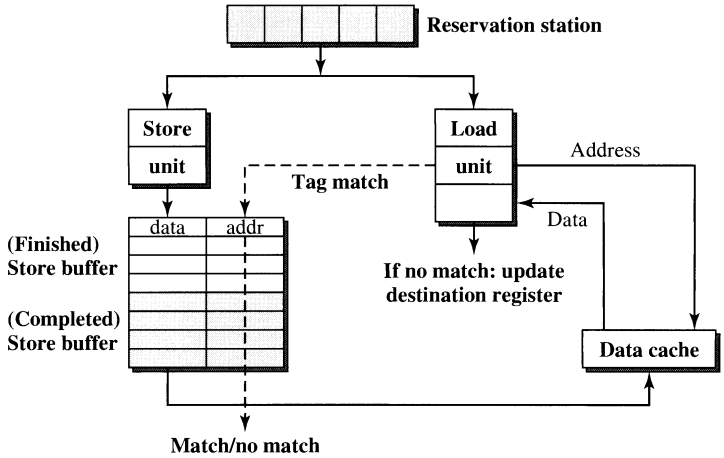


Figure 5.34
Illustration of Load Bypassing.

the memory address of the load is available, this address can be used to perform an associative search on the tag field of the store buffer entries. If a match occurs, then aliasing exists and the load is not allowed to execute out of order. Otherwise, the load is independent of the preceding stores in the store buffer and can be executed ahead of them. This associative search can be performed in the third pipe stage of the load unit concurrent with the accessing of the data cache; see Figure 5.34. If no aliasing is detected, the load is allowed to finish and the corresponding renamed destination register is updated with the data returned from the data cache. If aliasing is detected, the data returned by the data cache are discarded and the load is held back in the reservation station for future reissue.

Most of the complexity in implementing load bypassing lies in the store buffer and the associated associative search mechanism. To reduce the complexity, the tag field used for associative search can be reduced to contain only a subset of the address bits. Using only a subset of the address bits can reduce the width of the comparators needed for associative search. However, the result can be pessimistic. Potentially, an alias can be indicated by the narrower comparator when it really doesn't exist if the full-length address bits were used. Some of the load bypassing opportunities can be lost due to this compromise in the implementation. In general, the degradation of performance is minimal if enough address bits are used.

The load forwarding technique further enhances and complements the load bypassing technique. When a load is allowed to jump ahead of preceding stores, if it is determined that the load does alias with a preceding store, there is the potential to satisfy that load by forwarding the data directly from the aliased store. Essentially a memory RAW dependence exists between the leading store and the trailing load. The same associative search of the store buffer is needed. When aliasing is detected, instead of holding the load back for future reissue, the data from the aliased entry of the store buffer are forwarded to the renamed destination register

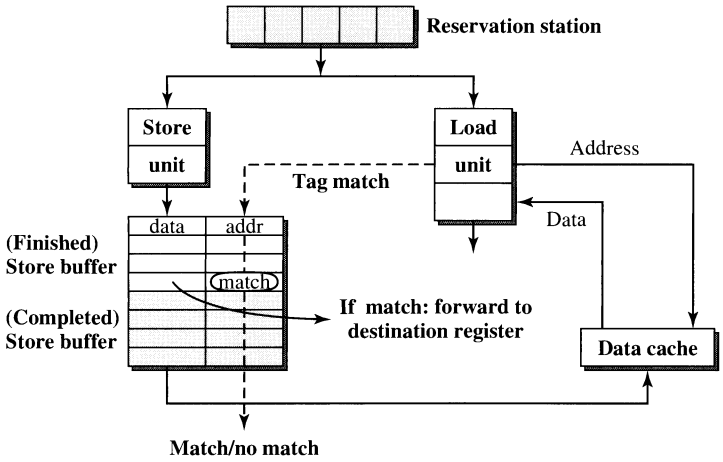
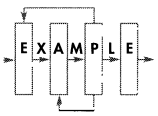


Figure 5.35
Illustration of Load Forwarding.



of the load instruction. This technique not only allows the load to be executed early, but also eliminates the need for the load to access the data cache. This can reduce the bandwidth pressure on the bus to the data cache.

To support load forwarding, added complexity to the store buffer is required; see Figure 5.35. First, the full-length address bits must be used for performing the associative search. When a subset of the address bits is used for supporting load bypassing, the only negative consequence is lost opportunity. For load forwarding the alias detection must be exact before forwarding of data can be performed; otherwise, it will lead to semantic incorrectness. Second, there can be multiple preceding stores in the store buffer that alias with the load. When such multiple matches occur during the associative search, there must be logic added to determine which of the aliased stores is the most recent. This will require additional priority encoding logic to identify the latest store on which the load is dependent before forwarding is performed. Third, an additional read port may be required for the store buffer. Prior to the incorporation of load forwarding, the store buffer has one write port that interfaces with the store unit and one read port that interfaces with the data cache. A new read port is now required that interfaces with the load unit; otherwise, port contention can occur between load forwarding and data cache update.

Significant performance improvement can be obtained with load bypassing and load forwarding. According to Mike Johnson, typically load bypassing can yield 11% to 19% performance gain, and load forwarding can yield another 1% to 4% of additional performance improvement [Johnson, 1991].

So far we have assumed that loads and stores share a common reservation station with instructions being issued from the reservation station to the store and the load units in program order. This in-order issuing assumption ensures that all the preceding stores to a load will be in the store buffer when the load is executed. This simplifies memory dependence checking; only an associative search of the

store buffer is necessary. However, this in-order issuing assumption introduces an unnecessary limitation on the out-of-order execution of loads. A load instruction can be ready to be issued; however, a preceding store can hold up the issuing of the load even though the two memory instructions do not alias. Hence, allowing out-of-order issuing of loads and stores from the load/store reservation station can permit a greater degree of out-of-order and early execution of loads. This is especially beneficial if these loads are at the beginnings of critical dependence chains and their early execution can remove critical performance bottlenecks.

If out-of-order issuing from the load/store reservation station is allowed, a new problem must be solved. If a load is allowed to be issued out of order, then it is possible for some of the stores that precede it to still be in the reservation station or in the execution pipe stages, and not yet in the store buffer. Hence, simply performing an associative search on the entries of the store buffer is not adequate for checking for potential aliasing between the load and all its preceding stores. Worse yet, the memory addresses for these preceding stores that are still in the reservation station or in the execution pipe stages may not be available yet.

One approach is to allow the load to proceed, assuming no aliasing with the preceding stores that are not yet in the store buffer, and then validate this assumption later. With this approach, a load is allowed to issue out of order and be executed speculatively. If it does not alias with any of the stores in the store buffer, the load is allowed to finish execution. However, this load must be put into a new buffer called the finished load buffer; see Figure 5.36. The finished load buffer is managed in a similar fashion as the finished store buffer. A load is only resident in the finished load buffer after it finishes execution and before it is completed.

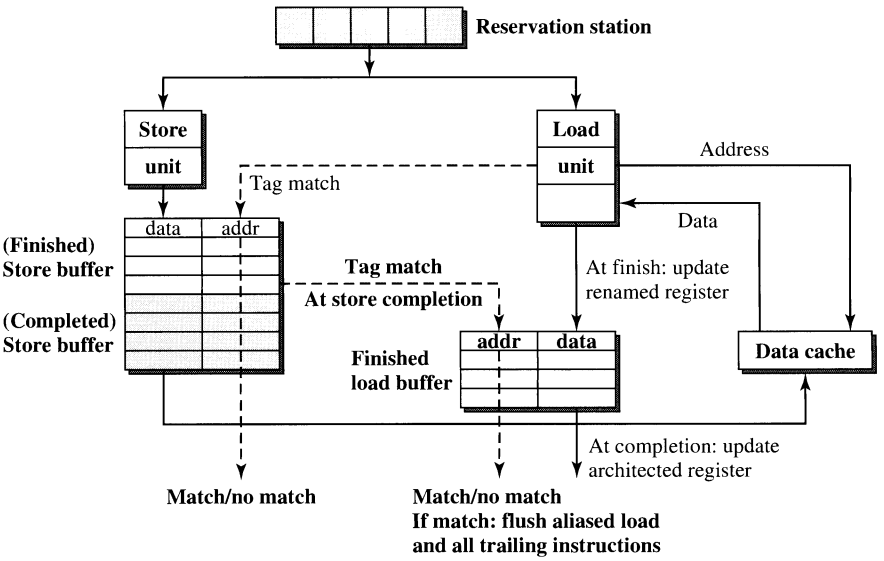


Figure 5.36
Fully Out-of-Order Issuing and Execution of Load and Store Instructions.

Whenever a store instruction is being completed, it must perform alias checking against the loads in the finished load buffer. If no aliasing is detected, the store is allowed to complete. If aliasing is detected, then it means that there is a trailing load that is dependent on the store, and that load has already finished execution. This implies that the speculative execution of that load must be invalidated and corrected by reissuing, or even refetching, that load and all subsequent instructions. This can require significant hardware complexity and performance penalty.

Aggressive early issuing of load instructions can lead to significant performance benefits. The ability to speculatively issue loads ahead of stores can lead to early execution of many dependent instructions, some of which can be other loads. This is important especially when there are cache misses. Early issuing of loads can lead to early triggering of cache misses which can in turn mask some or all of the cache miss penalty cycles. The downside with speculative issuing of loads is the potential overhead of having to recover from misspeculation. One way to reduce this overhead is to do alias or *dependence prediction*. In typical programs, the dependence relationship between a load and its preceding stores is quite predictable. A memory dependence predictor can be implemented to predict whether a load is likely to alias with its preceding stores. Such a predictor can be used to determine whether to speculatively issue a load or not. To obtain actual performance gain, aggressive speculative issuing of loads must be done very judiciously.

Moshovos [1998] proposed a memory dependence predictor that was used to directly bypass store data to dependent loads via memory cloaking. In subsequent work, Chrysos and Emer [1998] described a similar predictor called the store-set predictor.

5.3.4 Other Memory Data Flow Techniques

Other than load bypassing and load forwarding, there are other memory data flow techniques. These techniques all have the objectives of increasing the memory bandwidth and/or reducing the memory latency. As superscalar processors get wider, greater memory bandwidth capable of supporting multiple load/store instructions per cycle will be needed. As the disparity between processor speed and memory speed continues to increase, the latency of accessing memory, especially when cache misses occur, will become a serious bottleneck to machine performance. Sohi and Franklin [1991] studied high-bandwidth data memory systems.

One way to increase memory bandwidth is to employ multiple load/store units in the execution core, supported by a multiported data cache. In Section 5.3.3 we have assumed the presence of one store unit and one load unit supported by a single-ported data cache. The load unit has priority in accessing the data cache. Store instructions are queued in the store buffer and are retired from the store buffer to the data cache whenever the memory bus is not busy and the store buffer can gain access to the data cache. The overall data memory bandwidth is limited to one load/store instruction per cycle. This is a serious limitation, especially when there are bursts of load instructions. One way to alleviate this bottleneck is to provide two load units, as shown in Figure 5.37, and a *dual-ported data cache*. A dual-ported data cache is able to support two simultaneous cache accesses in

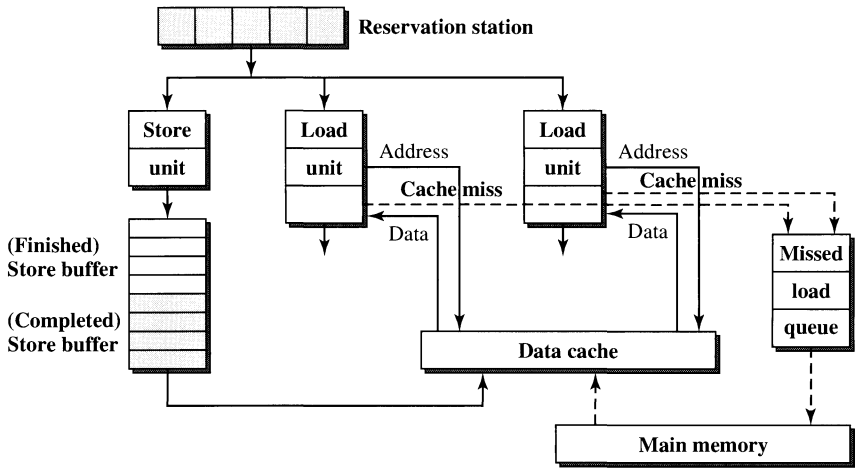


Figure 5.37
Dual-Ported and Nonblocking Data Cache.

every cycle. This will double the potential memory bandwidth. However, it comes with the cost of hardware complexity; a dual-ported cache can require doubling of the cache hardware. One way to alleviate this hardware cost is to implement interleaved data cache banks. With the data cache being implemented as multiple banks of memory, two simultaneous accesses to different banks can be supported in one cycle. If two accesses need to access the same bank, a bank conflict occurs and the two accesses must be serialized. From practical experience, a cache with eight banks can keep the frequency of bank conflicts down to acceptable levels.

The most common way to reduce memory latency is through the use of a cache. Caches are now widely employed. As the gap between processor speed and memory speed widens, multiple levels of caches are required. Most high-performance superscalar processors incorporate at least two levels of caches. The first level (L1) cache can usually keep up with the processor speed with access latency of one or very few cycles. Typically there are separate L1 caches for storing instructions and data. The second level (L2) cache typically supports the storing of both instructions and data, can be either on-chip or off-chip, and can be accessed in series (in case of a miss in the L1) or in parallel with the L1 cache. In some of the emerging designs a third level (L3) cache is used. It is likely that in future high-end designs a very large on-chip L3 cache will become commonplace. Other than the use of a cache or a hierarchy of caches, there are two other techniques for reducing the effective memory latency, namely, *nonblocking cache* and *prefetching cache*.

A nonblocking cache, first proposed by Kroft [1981], can reduce the effective memory latency by reducing the performance penalty due to cache misses. Traditionally, when a load instruction encounters a cache miss, it will stall the load unit pipeline and any further issuing of load instructions until the cache miss is serviced.

Such stalling is overly conservative and prevents subsequent and independent loads that may hit in the data cache from being issued. A nonblocking data cache alleviates this unnecessary penalty by putting aside a load that has missed in the cache into a *missed load queue* and allowing subsequent load instructions to issue; see Figure 5.37. A missed load sits in the missed load queue while the cache miss is serviced. When the missing block is fetched from the main memory, the missed load exits the missed load queue and finishes execution.

Essentially the cache miss penalty cycles are overlapped with, and masked by, the processing of subsequent independent instructions. Of course, if a subsequent instruction depends on the missed load, the issuing of that instruction is stalled. The number of penalty cycles that can be masked depends on the number of independent instructions following the missed load. A missed load queue can contain multiple entries, allowing multiple missed loads to be serviced concurrently. Potentially the cache penalty cycles of multiple missed loads can be overlapped to result in fewer total penalty cycles.

A number of issues must be considered when implementing nonblocking caches. Load misses can occur in bursts. The ability to support multiple misses and overlap their servicing is important. The interface to main memory, or a lower-level cache, must be able to support the overlapping or pipelining of multiple accesses. The filling of the cache triggered by the missed load may need to contend with the store buffer for the write port to the cache. There is one complication that can emerge with nonblocking caches. If the missed load is on a speculative path, i.e., the predicted path, there is the possibility that the speculation, i.e., branch prediction, will turn out to be incorrect. If a missed load is on a mispredicted path, the question is whether the cache miss should be serviced. In a machine with very aggressive branch prediction, the number of loads on the mispredicted path can be significant; servicing their misses speculatively can require significant memory bandwidth. Studies have shown that a nonblocking cache can reduce the amount of load miss penalty by about 15%.

Another way to reduce or mask the cache miss penalty is through the use of a prefetching cache. A prefetching cache anticipates future misses and triggers these misses early so as to overlap the miss penalty with the processing of instructions preceding the missing load. Figure 5.38 illustrates a prefetching data cache. Two structures are needed to implement a prefetching cache, namely, a *memory reference prediction table* and a *prefetch queue*. The memory reference prediction table stores information about previously executed loads in three different fields. The first field contains the instruction address of the load and is used as a tag field for selecting an entry of the table. The second field contains the previous data address of the load, while the third field contains a stride value that indicates the difference between the previous two data addresses used by that load. The memory reference prediction table is accessed via associative search using the fetch address produced by the branch predictor and the first field of the table. When there is a tag match, indicating a hit in the memory reference prediction table, the previous address is added to the stride value to produce a predicted memory address. This predicted address is then loaded into the prefetch queue. Entries in the prefetch queue are

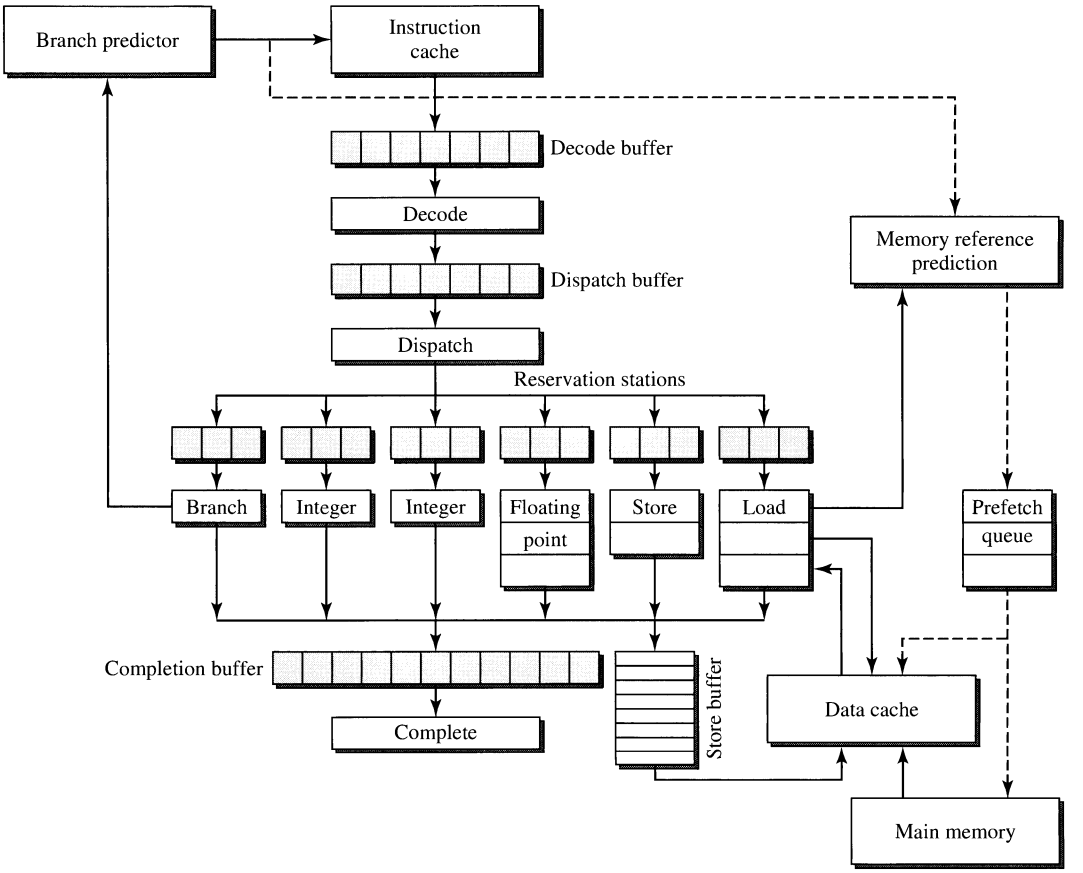


Figure 5.38
Prefetching Data Cache.

retrieved to speculatively access the data cache, and if a cache miss is triggered, the main memory or the next-level cache is accessed. The access to the data cache is in reality a cache touch operation; i.e., access to the cache is attempted in order to trigger a potential cache miss and not necessarily to actually retrieve the data from the cache. Early work on prefetching was done by Baer and Chen [1991] and Jouppi [1990].

The goal of a prefetching cache is to try to anticipate forthcoming cache misses and to trigger those misses early so as to hide the cache miss penalty by overlapping cache refill time with the processing of instructions preceding the missing load. When the anticipated missing load is executed, the data will be resident in the cache; hence, no cache miss is triggered, and no cache miss penalty is incurred. The actual effectiveness of prefetching depends on a number of factors. The prefetching distance, i.e., how far in advance the prefetching is being triggered, must be large enough to fully mask the miss penalty. This is the reason that

the predicted instruction fetch address is used to access the memory reference prediction table, with the hope that the data prefetch will occur far enough in advance of the load. However, this makes prefetching effectiveness subject to the effectiveness of branch prediction. Furthermore, there is the potential of polluting the data cache with prefetches that are on the mispredicted path. Status or confidence bits can be added to each entry of the memory reference prediction table to modulate the aggressiveness of prefetching. Another problem can occur when the prefetching is performed too early so as to evict a useful block from the cache and induce an unnecessary miss. One more factor is the actual memory reference prediction algorithm used. Load address prediction based on stride is quite effective for loads that are stepping through an array. For other loads that are traversing linked list data structures, the stride prediction will not work very well. Prefetching for such memory references will require much more sophisticated prediction algorithms.

To enhance memory data flow, load instructions must be executed early and fast. Store instructions are less important because experimental data indicate that they occur less frequently than load instructions and they usually are not on the performance critical path. To speed up the execution of loads we must reduce the latency for processing load instructions. The overall latency for processing a load instruction includes four components: (1) pipeline front-end latency for fetching, decoding, and dispatching the load instruction; (2) reservation station latency of waiting for register data dependence to resolve; (3) execution pipeline latency for address generation and translation; and (4) the cache/memory access latency for retrieving the data from memory. Both nonblocking and prefetching caches address only the fourth component, which is a crucial component due to the slow memory speed. To achieve higher clocking rates, superscalar pipelines are quickly becoming deeper and deeper. Consequently the latencies, in terms of number of machine cycles, of the first three components are also becoming quite significant. A number of speculative techniques have been proposed to address the reduction of these latencies; they include load address prediction, load value prediction, and memory dependence prediction.

Recently *load address prediction* techniques have been proposed to address the latencies associated with the first three components [Austin and Sohi, 1995]. To deal with the latency associated with the first component, a load prediction table, similar to the memory reference prediction table, is proposed. This table is indexed with the predicted instruction fetch address, and a hit in this table indicates the presence of a load instruction in the upcoming fetch group. Hence, the prediction of the presence of a load instruction in the upcoming fetch group is performed during the fetch stage and without requiring the decode and dispatch stages. Each entry of this table contains the predicted effective address which is retrieved during the fetch stage, in effect eliminating the need for waiting in the reservation station for the availability of the base register value and the address generation stage of the execution pipeline. Consequently, data cache access can begin in the next cycle, and potentially data can be retrieved from the cache at the end of the decode stage. Such a form of load address prediction can effectively collapse the latencies of the first three components down to just two cycles, i.e.,

fetch and decode stages, if the address prediction is correct and there is a hit in the data cache.

While the hardware structures needed to support load address prediction are quite similar to those needed for memory prefetching, the two mechanisms have significant differences. Load address prediction is actually executing, though speculatively, the load instruction early, whereas memory prefetching is mainly trying to prefetch the needed data into the cache without actually executing the load instruction. With load address prediction, instructions that depend on the load can also execute early because their dependent data are available early. Since load address prediction is a speculative technique, it must be validated, and if misprediction is detected, recovery must be performed. The validation is performed by allowing the actual load instruction to be fetched from the instruction cache and executed in a normal fashion. The result from the speculative version is compared with that of the normal version. If the results concur, then the speculative result becomes nonspeculative and all the dependent instructions that were executed speculatively are also declared as nonspeculative. If the results do not agree, then the nonspeculative result is used and all dependent instructions must be reexecuted. If the load address prediction mechanism is quite accurate, mispredictions occur only infrequently, the misprediction penalty is minimal, and overall net performance gain can be achieved.

Even more aggressive than load address prediction is the technique of *load value prediction* [Lipasti et al., 1996]. Unlike load address prediction which attempts to predict the effective address of a load instruction, load value prediction actually attempts to predict the value of the data to be retrieved from memory. This is accomplished by extending the load prediction table to contain not just the predicted address, but also the predicted value for the destination register. Experimental studies have shown that many load instructions' destination values are quite predictable. For example, many loads actually load the same value as last time. Hence, by storing the last value loaded by a static load instruction in the load prediction table, this value can be used as the predicted value when the same static load is encountered again. As a result, the load prediction table can be accessed during the fetch stage, and at the end of that cycle, the actual destination value of a predicted load instruction can be available and used in the next cycle by a dependent instruction. This significantly reduces the latency required for processing a load instruction if the load value prediction is correct. Again, validation is required, and at times a misprediction penalty must be paid.

Other than load address prediction and load value prediction, a third speculative technique has been proposed called *memory dependence prediction* [Moshovos, 1998]. Recall from Section 5.3.3 that to perform load bypassing and load forwarding, memory dependence checking is required. For load bypassing, it must be determined that the load does not alias with any of the stores being bypassed. For load forwarding, the most recent aliased store must be identified. Memory dependence checking can become quite complex if a larger number of load/store instructions are involved and can potentially require an entire pipe stage. It would be nice to eliminate this latency. Experimental data have shown

that the memory dependence relationship is quite predictable. It is possible to track the memory dependence relationship when load/store instructions are executed and use this information to make memory dependence prediction when the same load/store instructions are encountered again. Such memory dependence prediction can facilitate earlier execution of load bypassing and load forwarding. As with all speculative techniques, validation is needed and a recovery mechanism for misprediction must be provided.

5.4 Summary

In this chapter we attempt to cover all the fundamental microarchitecture techniques used in modern superscalar microprocessor design in a systematic and easy-to-digest way. We intentionally avoid inundating readers with lots of quantitative data and bar charts. We also focus on generic techniques instead of features of specific commercial products. In Chapters 6 and 7 we present detailed case studies of two actual products. Chapter 6 introduces the IBM/Motorola PowerPC 620 in great detail along with quantitative performance data. While not a successful commercial product, the PowerPC 620 represents one of the earlier and most aggressive out-of-order designs. Chapter 7 presents the Intel P6 microarchitecture, the first out-of-order implementation of the IA32 architecture. The Intel P6 is likely the most commercially successful microarchitecture. The fact that the P6 microarchitecture core provided the foundation for multiple generations of products, including the Pentium Pro, the Pentium II, the Pentium III, and the Pentium M, is a clear testimony to the effectiveness and elegance of its original design.

Superscalar microprocessor design is a rapidly evolving art which has been simultaneously harnessing the creative ideas of researchers and the insights and skills of architects and designers. Chapter 8 is a historical chronicle of the practice of this art form. Interesting and valuable lessons can be gleaned from this historical chronicle. The body of knowledge on superscalar microarchitecture techniques is constantly expanding. New innovative ideas from the research community as well as ideas from the traditional "macroarchitecture" domain are likely to find their way into future superscalar microprocessor designs. Chapters 9, 10, and 11 document some of these ideas.

REFERENCES

Adve, S. V., and K. Gharachorloo: "Shared memory consistency models: A tutorial," *IEEE Computer,* 29, 12, 1996, pp. 66–76.

Austin, T. M., and G. S. Sohi: "Zero-cycle loads: Microarchitecture support for reducing load latency," *Proc. 28th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1995, pp. 82–92.

Baer, J., and T. Chen: "An effective on-chip preloading scheme to reduce data access penalty," *Proc. Supercomputing '91,* 1991, pp. 176–186.

Calder, B., P. Feller, and A. Eustace: "Value profiling," *Proc. 30th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1997, pp. 259–269.

Calder, B., G. Reinman, and D. Tullsen: "Selective value prediction," *Proc. 26th Annual Int. Symposium on Computer Architecture (ISCA'99),* vol. 27, 2 of *Computer Architecture News,* New York, N.Y.: ACM Press, 1999, pp. 64–75.

Chrysos, G., and J. Emer: "Memory dependence prediction using store sets," *Proc. 25th Int. Symposium on Computer Architecture,* 1998, pp. 142–153.

Conte, T., K. Menezes, P. Mills, and B. Patel: "Optimization of instruction fetch mechanisms for high issue rates," *Proc. 22nd Annual Int. Symposium on Computer Architecture,* 1995, pp. 333–344.

Diefendorf, K., and M. Allen: "Organization of the Motorola 88110 superscalar RISC microprocessor," *IEEE MICRO,* 12, 2, 1992, pp. 40–63.

Gabbay, F., and A. Mendelson: "Using value prediction to increase the power of speculative execution hardware," *ACM Transactions on Computer Systems,* 16, 3, 1988b, pp. 234–270.

Gabbay, F., and A. Mendelson: "Can program profiling support value prediction," *Proc. 30th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1997, pp. 270–280.

Gabbay, F., and A. Mendelson: "The effect of instruction fetch bandwidth on value prediction," *Proc. 25th Annual Int. Symposium on Computer Architecture,* Barcelona, Spain, 1998a, pp. 272–281.

Gloy, N. C., M. D. Smith, and C. Young: "Performance issues in correlated branch prediction schemes," *Proc. 27th Int. Symposium on Microarchitecture,* 1995, pp. 3–14.

Grohoski, G.: "Machine organization of the IBM RISC System/6000 processor," *IBM Journal of Research and Development,* 34, 1, 1990, pp. 37–58.

Harbison, S. P.: *A Computer Architecture for the Dynamic Optimization of High-Level Language Programs.* PhD thesis, Carnegie Mellon University, 1980.

Harbison, S. P.: "An architectural alternative to optimizing compilers," *Proc. Int. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS),* 1982, pp. 57–65.

IBM Corp.: *PowerPC 604 RISC Microprocessor User's Manual.* Essex Junction, VT: IBM Microelectronics Division, 1994.

Johnson, M.: *Superscalar Microprocessor Design.* Englewood Cliffs, NJ: Prentice Hall, 1991.

Jouppi, N. P.: "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," *Proc. of 17th Annual Int. Symposium on Computer Architecture,* 1990, pp. 364–373.

Kessler, R.: "The Alpha 21264 microprocessor," *IEEE MICRO,* 19, 2, 1999, pp. 24–36.

Kroft, D.: "Lockup-free instruction fetch/prefetch cache organization," *Proc. 8th Annual Symposium on Computer Architecture,* 1981, pp. 81–88.

Lamport, L.: "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Trans. on Computers,* C-28, 9, 1979, pp. 690–691.

Lee, J., and A. Smith: "Branch prediction strategies and branch target buffer design," *IEEE Computer,* 21, 7, 1984, pp. 6–22.

Lipasti, M. H., and J. P. Shen: "Exceeding the dataflow limit via value prediction," *Proc. 29th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1996, pp. 226–237.

Lipasti, M. H., C. B. Wilkerson, and J. P. Shen: "Value locality and load value prediction," *Proc. Seventh Int. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-VII),* 1996, pp. 138–147.

McFarling, S.: "Combining branch predictors," Technical Report TN-36, Digital Equipment Corp. (**http://research.compaq.com/wrl/techreports/abstracts/TN-36.html**), 1993.

Mendelson, A., and F. Gabbay: "Speculative execution based on value prediction," Technical report, Technion (**http://www-ee.technion.ac.il/%7efredg**), 1997.

Moshovos, A.: "Memory Dependence Prediction," PhD thesis, University of Wisconsin, 1998.

Nair, R.: "Branch behavior on the IBM RS/6000," Technical report, IBM Computer Science, 1992.

Oehler, R. R., and R. D. Groves: "IBM RISC System/6000 processor architecture," *IBM Journal of Research and Development*, 34, 1, 1990, pp. 23–36.

Richardson, S. E.: "Caching function results: Faster arithmetic by avoiding unnecessary computation," Technical report, Sun Microsystems Laboratories, 1992.

Rotenberg, E., S. Bennett, and J. Smith: "Trace cache: a low latency approach to high bandwidth instruction fetching," *Proc. 29th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1996, pp. 24–35.

Sazeides, Y., and J. E. Smith: "The predictability of data values," *Proc. 30th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1997, pp. 248–258.

Smith, J. E.: "A study of branch prediction techniques," *Proc. 8th Annual Symposium on Computer Architecture,* 1981, pp. 135–147.

Sodani, A., and G. S. Sohi: "Dynamic instruction reuse," *Proc. 24th Annual Int. Symposium on Computer Architecture,* 1997, pp 194–205.

Sohi, G., and M. Franklin: "High-bandwidth data memory systems for superscalar processors," *Proc. 4th Int. Conference on Architectural Support for Programming Languages and Operating Systems,* 1991, pp. 53–62.

Tomasulo, R.: "An efficient algorithm for exploiting multiple arithmetic units," *IBM Journal of Research and Development,* 11, 1967, pp. 25–33.

Uht, A. K., and V. Sindagi: "Disjoint eager execution: An optimal form of speculative execution," *Proc. 28th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1995, pp. 313–325.

Wang, K., and M. Franklin: "Highly accurate data value prediction using hybrid predictors," *Proc. 30th Annual ACM/IEEE Int. Symposium on Microarchitecture,* 1997, pp. 281–290.

Yeh, T. Y., and Y. N. Patt: "Two-level adaptive training branch prediction," *Proc. 24th Annual Int. Symposium on Microarchitecture,* 1991, pp. 51–61.

Young, C., and M. D. Smith: "Improving the accuracy of static branch prediction using branch correlation," *Proc. 6th Int. Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-VI),* 1994, pp. 232–241.

HOMEWORK PROBLEMS

Problems 5.1 through 5.6

The displayed code that follows steps through the elements of two arrays (A[] and B[]) concurrently, and for each element, it puts the larger of the two values into the corresponding element of a third array (C[]). The three arrays are of length *N*.

The instruction set used for Problems 5.1 through 5.6 is as follows:

```
add     rd, rs, rt          rd ← rs + rt
addi    rd, rs, imm         rd ← rs + imm
lw      rd, offset(base)    rd ← MEM[offset+base]
                            (offset = imm, base = reg)
sw      rs, offset(base)    MEM[offset+base] ← rs
                            (offset = imm, base = reg)
bge     rs, rt, address     if (rs >= rt) PC ← address
blt     rs, rt, address     if (rs < rt) PC ← address
b       address             PC ← address
```

Note: r0 is hardwired to 0.

The benchmark code is as follows:

```
Static Inst#    Label       Assembly_Instruction
                main:
1                           addi  r2,   r0,   A
2                           addi  r3,   r0,   B
3                           addi  r4,   r0,   C
4                           addi  r5,   r0,   N
5                           add   r10,  r0,   r0
6                           bge   r10,  r5,   end
                loop:
7                           lw    r20,  0(r2)
8                           lw    r21,  0(r3)
9                           bge   r20,  r21,  T1
10                          sw    r21,  0(r4)
11                          b     T2
                T1:
12                          sw    r20,  0(r4)
                T2:
13                          addi  r10,  r10,  1
14                          addi  r2,   r2,   4
15                          addi  r3,   r3,   4
16                          addi  r4,   r4,   4
17                          blt   r10,  r5,   loop
                end:
```

P5.1 Identify the basic blocks of this benchmark code by listing the static instructions belonging to each basic block in the following table. Number the basic blocks based on the lexical ordering of the code. *Note:* There may be more boxes than there are basic blocks.

	Basic Block No.								
	1	2	3	4	5	6	7	8	9
Instr. nos.									

P5.2 Draw the control flow graph for this benchmark.

P5.3 Now generate the instruction execution trace (i.e., the sequence of basic blocks executed). Use the following arrays as input to the program, and trace the code execution by recording the number of each basic block that is executed.

```
N = 5;
A[] = {8, 3, 2, 5, 9};
B[] = {4, 9,  8, 5, 1};
```

P5.4 Fill in Tables 5.1 and 5.2 based on the data you generated in Problem 5.3.

Table 5.1
Instruction mix

	Static		Dynamic	
Instr. Class	**Number**	**%**	**Number**	**%**
ALU				
Load/store				
Branch				

Table 5.2
Basic block/branch data*

	Static	Dynamic
Average basic block size (no. of instr.)		
Number of taken branches		
Number of not-taken branches		

*Count unconditional branches as taken branches.

P5.5 Given the branch profile information you collected in Problem 5.4, rearrange the basic blocks and reverse the sense of the branches in the program snippet to minimize the number of taken branches and to pack

the code so that the frequently executed paths are placed together. Show the new program.

P5.6 Given the new program you wrote in Problem 5.5, recompute the branch statistics in the last two rows of Table 5.2.

Problems 5.7 through 5.13

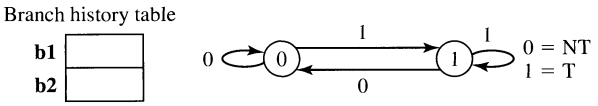
Consider the following code segment within a loop body for Problems 5.7 through 5.13:

```
if (x is even) then                          ←(branch b1)
      increment a                            ←(b1 taken)
if (x is a multiple of 10) then              ←(branch b2)
      increment b                            ←(b2 taken)
```

Assume that the following list of nine values of x is to be processed by nine iterations of this loop.

8, 9, 10, 11, 12, 20, 29, 30, 31

Note: Assume that predictor entries are updated by each dynamic branch before the next dynamic branch accesses the predictor (i.e., there is no update delay).



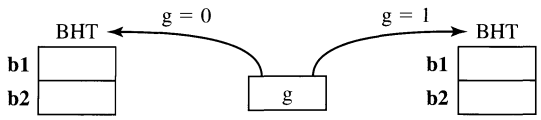
P5.7 Assume that a 1-bit (history bit) state machine (see above) is used as the prediction algorithm for predicting the execution of the two branches in this loop. Indicate the predicted and actual branch directions of the b1 and b2 branch instructions for each iteration of this loop. Assume an initial state of 0, i.e., NT, for the predictor.

	8	9	10	11	12	20	29	30	31
b1 predicted									
b1 actual									
b2 predicted									
b2 actual									

P5.8 What are the prediction accuracies for b1 and b2?

P5.9 What is the overall prediction accuracy?

P5.10 Assume a two-level branch prediction scheme is used. In addition to the 1-bit predictor, a 1-bit global register (g) is used. Register g stores the direction of the last branch executed (which may not be the same branch as the branch currently being predicted) and is used to index into two separate 1-bit branch history tables (BHTs) as shown in the following figure.



Depending on the value of g, one of the two BHTs is selected and used to do the normal 1-bit prediction. Again, fill in the predicted and actual branch directions of b1 and b2 for nine iterations of the loop. Assume the initial value of g = 0, i.e., NT. For each prediction, depending on the current value of g, only one of the two BHTs is accessed and updated. Hence, some of the entries in the following table should be empty.

Note: Assume that predictor entries are updated by each dynamic branch before the next dynamic branch accesses the predictor (i.e., there is no update delay).

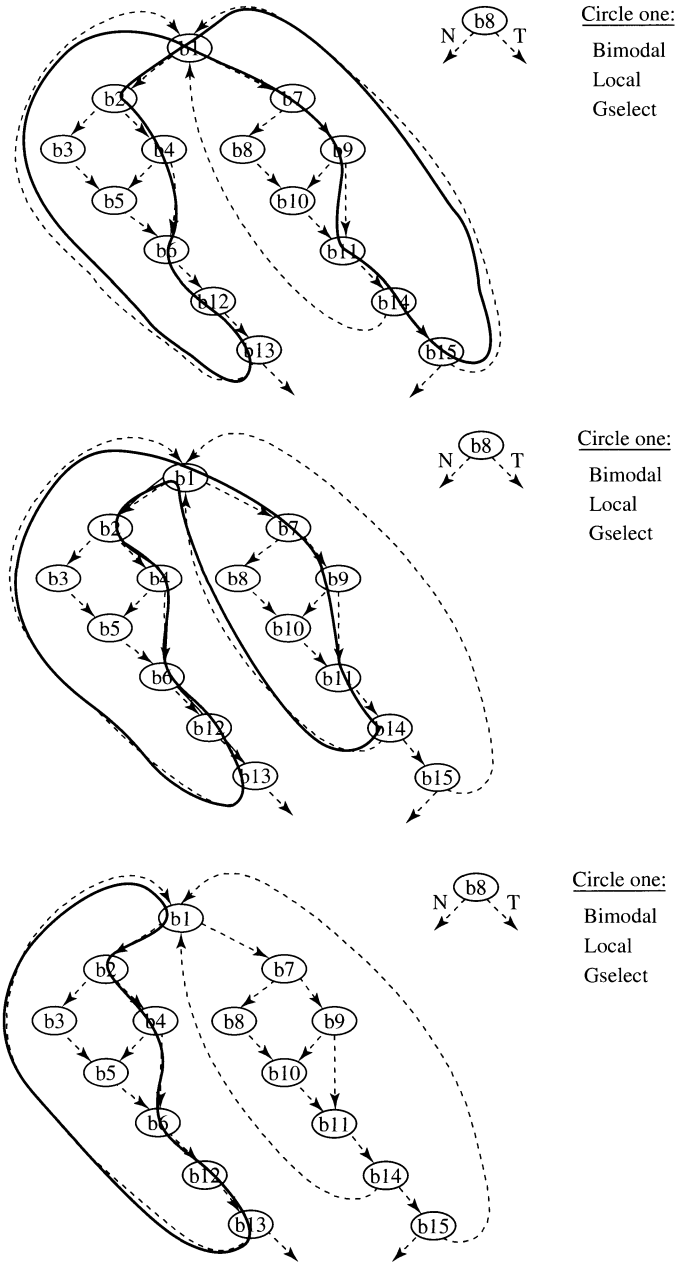
	8	9	10	11	12	20	29	30	31
For g = 0:									
b1 predicted									
b1 actual									
b2 predicted									
b2 actual									
For g = 1:									
b1 predicted									
b1 actual									
b2 predicted									
b2 actual									

P5.11 What are the prediction accuracies for b1 and b2?

P5.12 What is the overall prediction accuracy?

P5.13 What is the prediction accuracy of b2 when g = 0? Explain why.

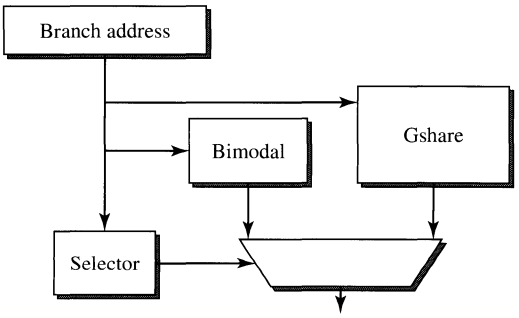
P5.14 The figure shows the control flow graph of a simple program. The CFG is annotated with three different execution trace paths. For each execution trace, circle which branch predictor (bimodal, local, or gselect) will *best* predict the branching behavior of the given trace. More



than one predictor may perform equally well on a particular trace. However, you are to use each of the three predictors *exactly once* in choosing the best predictors for the three traces. *Circle* your choice for each of the three traces and add. (Assume each trace is executed many times and every node in the CFG is a conditional branch. The branch history register for the local, global, and gselect predictors is limited to 4 bits.)

Problems 5.15 and 5.16: Combining Branch Prediction

Given a combining branch predictor with a two-entry direct-mapped bimodal branch direction predictor, a gshare predictor with a 1-bit BHR and two PHT entries, and a two-entry selector table, simulate a sequence of taken and not-taken branches as shown in the rows of the table in Problem 5.15, record the prediction made by the predictor before the branch is resolved as well as any change to the predictor entries after the branch resolves.



Use the following assumptions:

- Instructions are a fixed 4 bytes long; hence, the two low-order bits of the branch address should be shifted out when indexing into the predictor. Use the next lowest-order bit to index into the predictor.
- Each predictor and selector entry is a saturating up-down 2-bit Smith counter with the initial states shown.
- A taken branch (T) increments the predictor entry; a not-taken branch (N) decrements the predictor entry.
- A predictor entry less than 2 (0 or 1) results in a not-taken (N) prediction.
- A predictor entry greater than or equal to 2 (2 or 3) results in a taken (T) prediction.

- A selector value of 0 or 1 selects the bimodal predictor, while a selector value of 2 or 3 selects the gshare predictor.
- None of the predictors are tagged with the branch address.
- Avoid destructive interference by not updating the "wrong" predictor whenever the other predictor is right.

P5.15 Fill in the following table with the prediction outcomes, and the predictor state following resolution of each branch.

					Predictor State after Branch Is Resolved						
					Selector		Bimodal Predictor		Gshare Predictor		
Branch Address	Branch Outcome (TNT)	Predicted Outcome (T/N)									
		Bimodal	Gshare	Combined	PHT0	PHT1	PHT0	PHT1	BHR	PHT0	PHT1
Initial	N/A	N/A	N/A	N/A	2	0	0	2	0	2	1
0x654	N										
0x780	T										
0x78C	T										
0x990	T										
0xA04	N										
0x78C	N										

P5.16 Compute the overall branch prediction rates (number of correctly predicted branches / total number of predicted branches) for the bimodal, gshare, and final (combined) predictors.

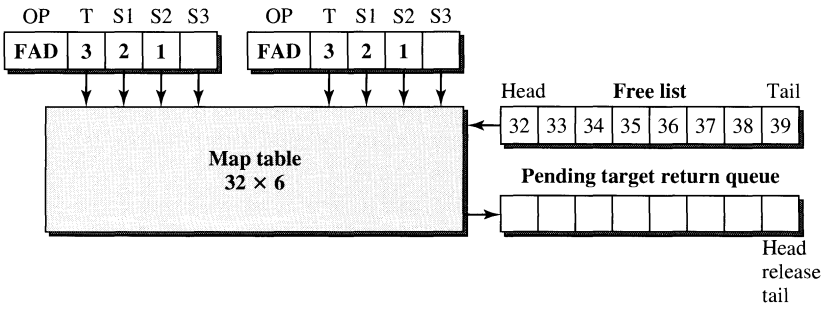
P5.17 Branch predictions are resolved when a branch instruction executes. Early out-of-order processors like the PowerPC 604 simplified branch resolution by forcing branches to execute strictly in program order. Discuss why this simplifies branch redirect logic, and explain in detail how the microarchitecture must change to accommodate out-of-order branch resolution.

P5.18 Describe a scenario in which out-of-order branch resolution would be important for performance. State your hypothesis and describe a set of experiments to validate your hypothesis. Optionally, modify a timing simulator and conduct these experiments.

Problems 5.19 and 5.20: Register Renaming

[redacted]

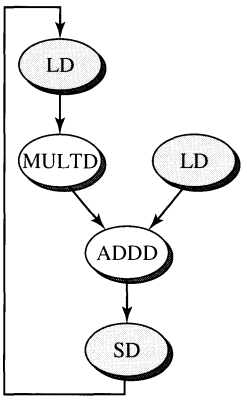
- Assume the initial state shown in the table for Problem 5.19.
- Note the table only contains columns for the registers that are referenced in the DAXPY loop.
- As in the RS/6000 implementation discussed, assume only a single load instruction is renamed per cycle and that only a single floating-point instruction can complete per cycle.
- Only floating-point load, multiply, and add instructions are shown in the table, since only these are relevant to the renaming scheme.
- Remember that only load destination registers are renamed.
- The first load from the loop prologue is filled in for you.
- Registers are freed when the next FP ALU op retires.




```
Y(i) = A * X(i) + Y(i)

    F0 ← LD,a
    R4 ← ADDI,Rx,#512      ;last address

Loop:
    F2 ← LD,0(Rx)          ;load X(i)
    F2 ← MULTD,F0,F2       ;A*X(i)
    F4 ← LD,0(Ry)          ;load Y(i)
    F4 ← ADDD,F2,F4        ;A*X(i)+Y(i)
    0(Ry) ← SD,F4          ;store into Y(i)
    Rx ← ADDI,Rx,#8        ;inc. index to X
    Ry ← ADDI,Ry,#8        ;inc. index to Y
    R20 ← SUB,R4,Rx        ;compute bound
    BNZ,R20,Loop           ;check if done
```



P5.19 Fill in the remaining rows in the following table with the map table state and pending target return queue state after the instruction is renamed, and the free list state after the instruction completes.

Floating-Point Instruction	Pending Target Return Queue after Instruction Renamed	Free List after Instruction Completes	Map Table Subset		
			F0	F2	F4
Initial state		32,33,34,35,36,37,38,39	0	2	4
F0 ⇐ LD, a	0	33,34,35,36,37,38,39	32	2	4
F2 ⇐ LD, 0(Rx)					
F2 ⇐ MULTD, F0, F2					
F4 ⇐ LD, 0(Ry)					
F4 ⇐ ADDD, F2, F4					
F2 ⇐ LD, 0(Rx)					
F2 ⇐ MULTD, F0, F2					
F4 ⇐ LD, 0(Ry)					
F4 ⇐ ADDD, F2, F4					

P5.20 Given that the RS/6000 can rename a floating-point load in parallel with a floating-point arithmetic instruction (mult/add), and assuming the map table is a write-before-read structure, is any internal bypassing needed within the map table? Explain why or why not.

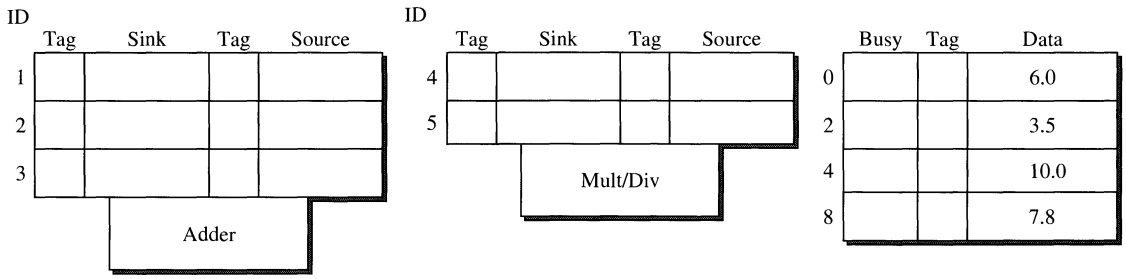
P5.21 Simulate the execution of the following code snippet using Tomasulo's algorithm. Show the contents of the reservation station entries, register file busy, tag (the tag is the RS ID number), and data fields for each cycle (make a copy of the table shown on the next page for each cycle that you simulate). Indicate which instruction is executing in each functional unit in each cycle. Also indicate any result forwarding across a common data bus by circling the producer and consumer and connecting them with an arrow.

i: R4 ← R0 + R8
j: R2 ← R0 * R4
k: R4 ← R4 + R8
l: R8 ← R4 * R2

Assume dual dispatch and a dual common data bus (CDB). Add latency is two cycles, and multiply latency is three cycles. An instruction can begin execution in the same cycle that it is dispatched, assuming all dependences are satisfied.

P5.22 Determine whether or not the code executes at the data-flow limit for Problem 5.21. Explain why or why not. Show your work.

CYCLE #: _____



DISPATCHED INSTRUCTION(S): _____

P5.23 As presented in this chapter, load bypassing is a technique for enhancing memory data flow. With load bypassing, load instructions are allowed to jump ahead of earlier store instructions. Once address generation is done, a store instruction can be completed architecturally and can then enter the store buffer to await an available bus cycle for writing to memory. Trailing loads are allowed to bypass these stores in the store buffer if there is no address aliasing.

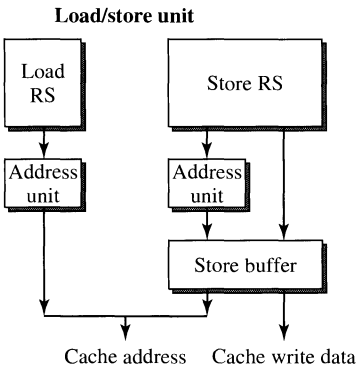
In this problem you are to simulate such load bypassing (there is no load forwarding). You are given a sequence of load/store instructions and their addresses (symbolic). The number to the left of each instruction indicates the cycle in which that instruction is dispatched to the reservation station; it can begin execution in that same cycle. Each store instruction will have an additional number to its right, indicating the cycle in which it is ready to retire, i.e., exit the store buffer and write to the memory.

Use the following assumptions:

- All operands needed for address calculation are available at dispatch.
- One load and one store can have their addresses calculated per cycle.
- One load or one store can be executed, i.e., allowed to access the cache, per cycle.
- The reservation station entry is deallocated the cycle after address calculation and issue.
- The store buffer entry is deallocated when the cache is accessed.
- A store instruction can access the cache the cycle after it is ready to retire.
- Instructions are issued in order from the reservation stations.
- Assume 100% cache hits.

Example:

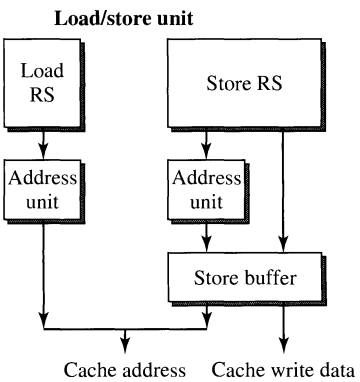
Dispatch cycle	Instruction	Retire cycle
1	Load A	
1	Load B	
1	Store C	5



Cycle	Load Reservation Station	Store Reservation Station	Store Buffer	Cache Address	Cache Write Data
1	Ld A Ld B	St C			
2	Ld B		St C	Ld A	
3			St C	Ld B	
4			St C		
5			St C		
6				St C	data

Code:

Dispatch cycle	Instruction	Retire cycle
1	Store A	6
2	Load B	
3	Load A	
4	Store D	10
4	Load E	
4	Load A	
5	Load D	



Cycle	Load Reservation Station	Store Reservation Station	Store Buffer	Cache Address	Cache Write Data
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

P5.24 In one or two sentences compare and contrast load forwarding with load bypassing.

P5.25 Would load forwarding improve the performance of the code sequence from Problem 5.23? Why or why not?

Problems 5.26 through 5.28

The goal of lockup-free cache designs is to increase the amount of concurrency or parallelism in the processor by overlapping cache miss handling with other processing or other cache misses. For this problem, assume the following simple workload running on a processor with a primary cache with 64-byte lines and a 16-byte memory bus. Assume that t_{miss} is 5 cycles, and $t_{transfer}$ for each 16-byte subblock is 1 cycle. Hence, for a simple blocking cache, a miss will take $t_{miss} + (64/16) \times t_{transfer} = 5 + 4 \times 1 = 9$ cycles. Here is the workload:

```
for(i=1;i<10000;++i)
          a += A[i] + B[i];
```

In RISC assembly language, assuming r3 points to A[0] and r4 points to B[0]:

```
        li    r2,9999   # load iteration count into r2
loop:   lfdu  r5,8(r3)  # load A[i], incr. pointer in r3
        lfdu  r6,8(r4)  # load B[i], incr. pointer in r4
```

```
add   r7,r7,r5  # add A[i] to a
add   r7,r7,r6  # add B[i] to a
bdnz  r2,loop   # decrement r2, branch if not zero
```

Here is a timing diagram for the loop body assuming neither array hits in the cache and the cache is a simple blocking design (m = miss latency, t = transfer latency, A = load from A, B = load from B, a = add, and b = branch):

	Cycle										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A		m	m	m	m	m	t	t	t	t																				
Array B												m	m	m	m	m	t	t	t	t										
Execution	A										B										a	a	b	A	B	a	a	b	A	B

For Problems 5.26 through 5.28 assume that each instruction takes a single cycle to execute and that all subsequent instructions are stalled on a cache miss until the requested data are returned by the cache (as shown in the timing diagram). Furthermore, assume that no portion of either array (A or B) is in the cache initially, but must be fetched on demand misses. Also, assume there are enough loop iterations to fill all the table entries provided.

P5.26 Assume a blocking cache design with critical word forwarding (i.e., the requested word is forwarded as soon as it has been transferred), but support for only a single outstanding miss. Fill in the timing diagram and explain your work.

	Cycle										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A		m	m	m	m	m																								
Array B																														
Execution	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A																														
Array B																														
Execution																														

P5.27 Now fill in the timing diagram for a lockup-free cache that supports multiple outstanding misses, and explain your work.

	Cycle										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A		m	m	m	m	m																								
Array B																														
Execution	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A																														
Array B																														
Execution																														

P5.28 Instead of a 64-byte cache line, assume a 32-byte line size for the cache, and fill in the timing diagram as in Problem 5.27. Explain your work

	Cycle										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A		m	m	m	m	m																								
Array B																														
Execution	A																													

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A																														
Array B																														
Execution																														

Problems 5.29 through 5.30

The goal of prefetching and nonblocking cache designs is to increase the amount of concurrency or parallelism in the processor by overlapping cache miss handling with other processing or other cache misses. For this problem, assume the same simple workload from Problem 5.26 running on a processor with a primary cache with 16-byte lines and an 4-byte memory bus. Assume that t_{miss} is 2 cycles and

$t_{transfer}$ for each 4-byte subblock is 1 cycle. Hence, a miss will take $t_{miss} + (16/4) \times t_{transfer} = 2 + 4 \times 1 = 6$ cycles.

For Problems 5.29 and 5.30, assume that each instruction takes a single cycle to execute and that all subsequent instructions are stalled on a cache miss until the requested data are returned by the cache (as shown in the table in Problem 5.29). Furthermore, assume that no portion of either array (A or B) is in the cache initially, but must be fetched on demand misses. Also, assume there are enough loop iterations to fill all the table entries provided.

Further assume a stride-based hardware prefetch mechanism that can track up to two independent strided address streams, and issues a stride prefetch the cycle after it has observed the same stride twice, from observing three strided misses (e.g., misses to A, A + 32, A + 64 triggers a prefetch for A + 96). The prefetcher will issue its next strided prefetch in the cycle following a demand reference to its previous prefetch, but no sooner (to avoid overwhelming the memory subsystem). Assume that a demand reference will always get priority over a prefetch for any shared resource.

P5.29 Fill in the following table to indicate miss (m) and transfer (t) cycles for both demand misses and prefetch requests for the assumed workload. Annotate each prefetch with a symbolic address (e.g., A + 64). The first 30 cycles are filled in for you.

	Cycle										1										2									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A		m	m	t	t	t	t												m		m	t	t	t	t					
Array B							m	m	t	t	t	t													m	m	t	t	t	t
Prefetch 1																														
Prefetch 2																														
Execution	A					B					a	a	b	A	B	a	a	b	A					B					a	a

	3										4										5									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A																														
Array B																														
Prefetch 1																														
Prefetch 2																														
Execution																														

	6										7										8									
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
Array A																														
Array B																														
Prefetch 1																														
Prefetch 2																														
Execution																														

P5.30 Report the overall miss rate for the portion of execution shown in Problem 5.29, as well as the coverage of the prefetches generated (coverage is defined as: (number of misses eliminated by prefetching)/(number of misses without prefetching).

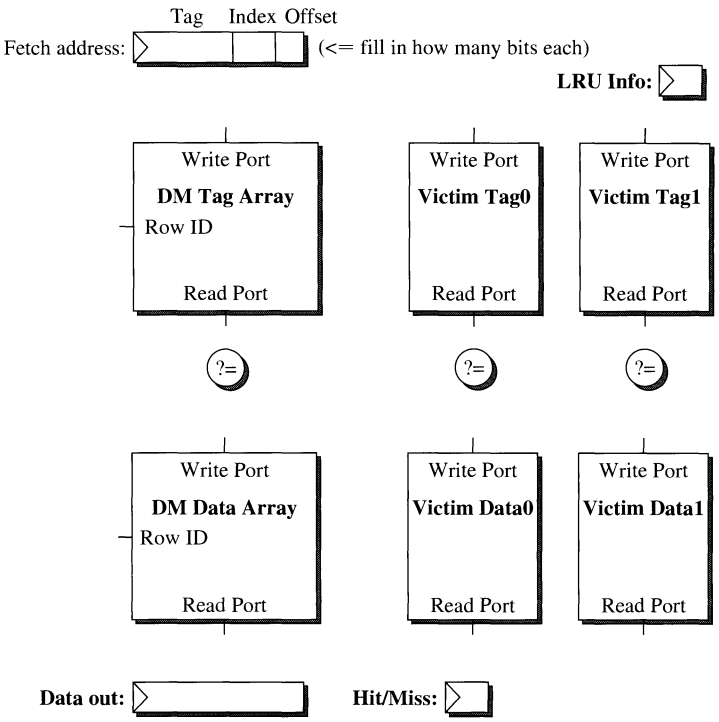
P5.31 A victim cache is used to augment a direct-mapped cache to reduce conflict misses. For additional background on this problem, read Jouppi's paper on victim caches [Jouppi, 1990]. Please fill in the following table to reflect the state of each cache line in a four-entry direct-mapped cache and a two-entry fully associative victim cache following each memory reference shown. Also, record whether the reference was a cache hit or a cache miss. The reference addresses are shown in hexadecimal format. Assume the direct-mapped cache is indexed with the low-order bits above the 16-byte line offset (e.g., address 40 maps to set 0, address 50 maps to set 1). Use a dash (—) to indicate an invalid line and the address of the line to indicate a valid line. Assume LRU policy for the victim cache and mark the LRU line as such in the table.

		Direct-Mapped Cache				Victim Cache	
Reference Address	**Hit/Miss**	**Line 0**	**Line 1**	**Line 2**	**Line 3**	**Line 0**	**Line 1**
[init state]		—	110	—	FF0	1F0	210/LRU
80							
A0							
200							
80							
B0							
E0							
200							
80							
200							

P5.32 Given your results from Problem 5.31, and excluding the data provided to the processor to supply the requested instruction words, compute the total number of bytes that were transferred into and out of the direct-mapped cache array. Assume this is a read-only instruction cache; hence, there are no writebacks of dirty lines. Show your work.

P5.33 Fill in the details for the block diagram of the read-only victim I-cache design shown in the following figure (based on the victim cache outlined in Problem 5.31). For additional background on this problem, read Jouppi's paper on victim caches [Jouppi, 1990]. Show data and address paths, identify which address bits are used for indexing and tag comparison, and show the logic for generating a hit/miss signal as well as control logic for any multiplexers that are included in your design. Don't forget the data paths for "swapping" cache lines between the

victim cache and the DM cache. Assume that the arrays are read in the first half of each clock cycle and written in the second half of each clock cycle. Do not include LRU update or control or data paths for handling misses.



Problems 5.34 through 5.36: Simplescalar Simulation

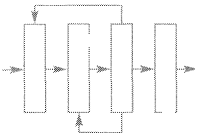
Problems 5.34 through 5.36 require you to use and modify the Simplescalar 3.0 simulator suite, available from **http://www.simplescalar.com.** Use the Simplescalar simulators to execute the four benchmarks in the instructional benchmark suite from **http://www.simplescalar.com.**

P5.34 First use the sim-outorder simulator with the default machine parameters to simulate an out-of-order processor that performs both right-path and wrong-path speculative references against the instruction cache, data cache, and unified level 2 cache. Report the instruction cache, data cache, and l2 cache miss rates (misses per reference) as well as the total number of references and the total number of misses for each of the caches.

P5.35 Now simulate the exact same memory hierarchy as in Problem 5.34 but using the sim-cache simulator. Note that you will have to determine the

parameters to use when you invoke sim-cache. Report the same statistics as in Problem 5.34, and compute the increase (or decrease) in each statistic.

P5.36 Now modify sim-outorder.c to inhibit cache misses caused by wrong-path references. This is very easy to do for instruction fetches in sim-outorder, since the global variable "spec_mode" is set whenever the processor begins to execute instructions from an incorrect branch path. You can use this global flag to inhibit instruction cache misses from wrong path instruction fetches. For data cache misses, you can check the "spec_mode" flag within each RUU entry and inhibit data cache misses for any such instruction. "Inhibiting misses" means don't even bother to check the cache hierarchy; simply treat these references as if they hit the cache. You can find the places where the cache is accessed by searching for calls to the function "cache_access" within sim-outorder.c. Now recollect the statistics from Problem 5.34 in your modified simulator and compare your results to Problem 5.35. If your results still differ from Problem 5.35, explain why that might be the case.



CHAPTER

6

Trung A. Diep

The PowerPC 620

CHAPTER OUTLINE

The PowerPC family of microprocessors includes the 64-bit PowerPC 620 microprocessor. The 620 was the first 64-bit superscalar processor to employ true out-of-order execution, aggressive branch prediction, distributed multientry reservation stations, dynamic renaming for all register files, six pipelined execution units, and a completion buffer to ensure precise exceptions. Most of these features had not been previously implemented in a single-chip microprocessor. Their actual effectiveness is of great interest to both academic researchers as well as industry designers. This chapter presents an instruction-level, or machine-cycle level, performance evaluation of the 620 microarchitecture using a VMW-generated performance simulator of the 620 (VMW is the Visualization-based Microarchitecture Workbench from Carnegie Mellon University) [Levitan et al., 1995; Diep et al., 1995].

We also describe the IBM POWER3 and POWER4 designs, and we highlight how they differ from the predecessor PowerPC 620. While they are fundamentally

similar in that they aggressively extract instruction-level parallelism from sequential code, the differences between the 620, the POWER3, and the POWER4 designs help to highlight recent trends in processor implementation: increased memory bandwidth through aggressive cache hierarchies, better branch prediction, more execution resources, and deeper pipelining.

6.1 Introduction

The PowerPC Architecture is the result of the PowerPC alliance among IBM, Motorola, and Apple [May et al., 1994]. It is based on the Performance Optimized with Enhanced RISC (POWER) Architecture, designed to facilitate parallel instruction execution and to scale well with advancing technology. The PowerPC alliance has released and announced a number of chips. The first, which provided a transition from the POWER Architecture to the PowerPC Architecture, was the PowerPC 601 microprocessor [IBM Corp., 1993]. The second, a low-power chip, was the PowerPC 603 microprocessor [Motorola, Inc., 2002]. Subsequently, a more advanced chip for desktop systems, the PowerPC 604 microprocessor, has been shipped [IBM Corp., 1994]. The fourth chip was the 64-bit 620 [Levitan et al., 1995; Diep et al., 1995].

More recently, Motorola and IBM have pursued independent development of general-purpose PowerPC-compatible parts. Motorola has focused on 32-bit desktop chips for Apple, while IBM has concentrated on server parts for its Unix (AIX) and business (OS/400) systems. Recent 32-bit Motorola designs, not detailed here, are the PowerPC G3 and G4 designs [Motorola, Inc., 2001; 2003]. These are 32-bit parts derived from the PowerPC 603, with short pipelines, limited execution resources, but very low cost. IBM's server parts have included the in-order multithreaded Star series (Northstar, Pulsar, S-Star [Storino et al., 1998]), as well as the out-of-order POWER3 [O'Connell and White, 2000] and POWER4 [Tendler et al., 2001]. In addition, both Motorola and IBM have developed various PowerPC cores for the embedded marketplace. Our focus in this chapter is on the PowerPC 620 and its heirs at the high-performance end of the marketplace, the POWER3 and the POWER4.

The PowerPC Architecture has 32 general-purpose registers (GPRs) and 32 floating-point registers (FPRs). It also has a condition register which can be addressed as one 32-bit register (CR), as a register file of 8 four-bit fields (CRFs), or as 32 single-bit fields. The architecture has a count register (CTR) and a link register (LR), both primarily used for branch instructions, and an integer exception register (XER) and a floating-point status and control register (FPSCR), which are used to record the exception status of the appropriate instruction types. The PowerPC instructions are typical RISC instructions, with the addition of floating-point fused multiply-add (FMA) instructions, load/store instructions with addressing modes that update the effective address, and instructions to set, manipulate, and branch off of the condition register bits.

The 620 is a four-wide superscalar machine. It uses aggressive branch prediction to fetch instructions as early as possible and a dispatch policy to distribute those

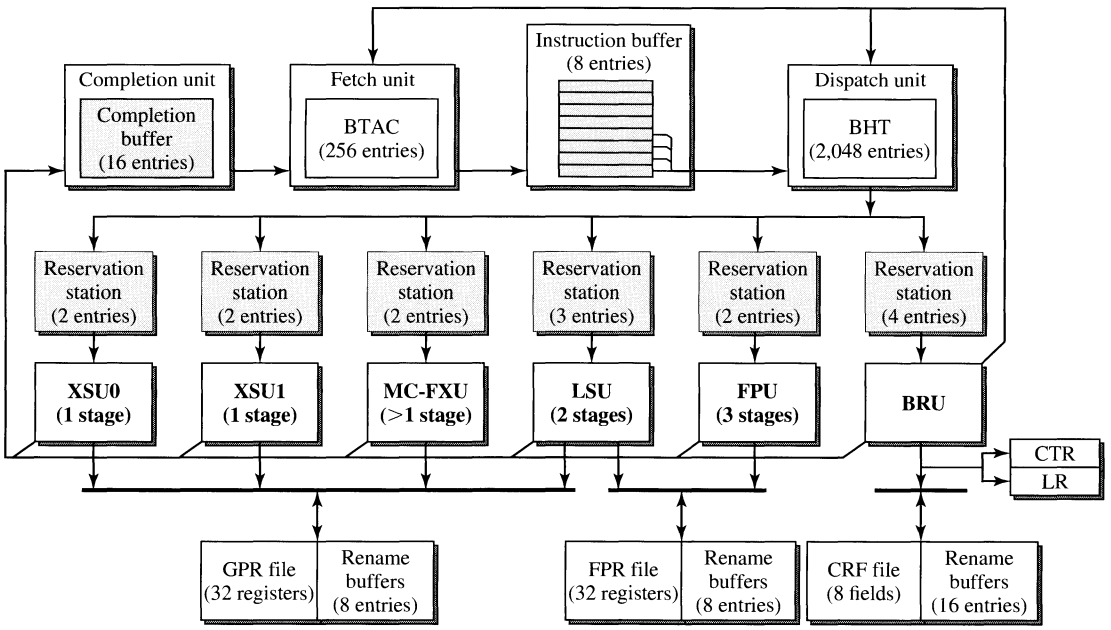


Figure 6.1
Block Diagram of the PowerPC 620 Microprocessor.

instructions to the execution units. The 620 uses six parallel execution units: two simple (single-cycle) integer units, one complex (multicycle) integer unit, one floating-point unit (three stages), one load/store unit (two stages), and a branch unit. The 620 uses distributed reservation stations and register renaming to implement out-of-order execution. The block diagram of the 620 is shown in Figure 6.1.

The 620 processes instructions in five major stages, namely the fetch, dispatch, execute, complete, and writeback stages. Some of these stages are separated by buffers to take up slack in the dynamic variation of available parallelism. These buffers are the instruction buffer, the reservation stations, and the completion buffer. The pipeline stages and their buffers are shown in Figure 6.2. Some of the units in the execute stage are actually multistage pipelines.

Fetch Stage. The fetch unit accesses the instruction cache to fetch up to four instructions per cycle into the instruction buffer. The end of a cache line or a taken branch can prevent the fetch unit from fetching four useful instructions in a cycle. A mispredicted branch can waste cycles while fetching from the wrong path. During the fetch stage, a preliminary branch prediction is made using the branch target address cache (BTAC) to obtain the target address for fetching in the next cycle.

Instruction Buffer. The instruction buffer holds instructions between the fetch and dispatch stages. If the dispatch unit cannot keep up with the fetch unit, instructions are buffered until the dispatch unit can process them. A maximum of eight

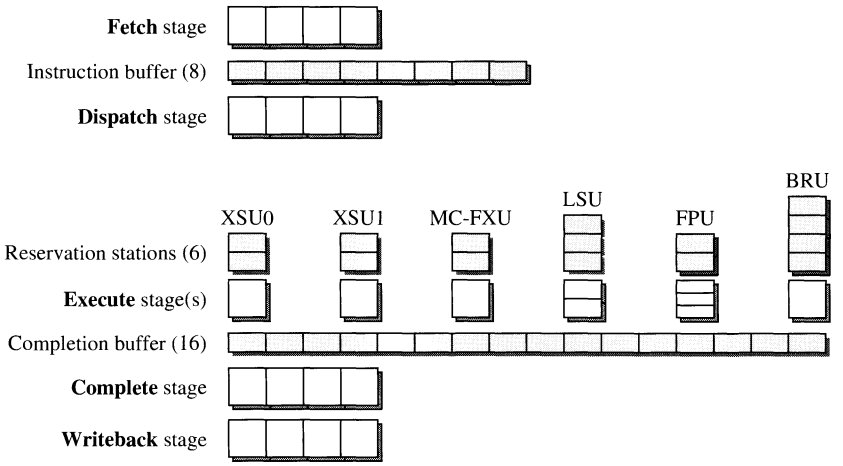


Figure 6.2
Instruction Pipeline of the PowerPC 620 Microprocessor.

instructions can be buffered at a time. Instructions are buffered and shifted in groups of two to simplify the logic.

Dispatch Stage. The dispatch unit decodes instructions in the instruction buffer and checks whether they can be dispatched to the reservation stations. If all dispatch conditions are fulfilled for an instruction, the dispatch stage will allocate a reservation station entry, a completion buffer entry, and an entry in the rename buffer for the destination, if needed. Each of the six execution units can accept at most one instruction per cycle. Certain infrequent serialization constraints can also stall instruction dispatch. Up to four instructions can be dispatched in program order per cycle.

There are eight integer register rename buffers, eight floating-point register rename buffers, and 16 condition register field rename buffers. The count register and the link register have one shadow register each, which is used for renaming. During dispatch, the appropriate buffers are allocated. Any source operands which have been renamed by previous instructions are marked with the tags of the associated rename buffers. If the source operand is not available when the instruction is dispatched, the appropriate result busses for forwarding results are watched to obtain the operand data. Source operands which have not been renamed by previous instructions are read from the architected register files.

If a branch is being dispatched, resolution of the branch is attempted immediately. If resolution is still pending, that is, the branch depends on an operand that is not yet available, it is predicted using the branch history table (BHT). If the prediction made by the BHT disagrees with the prediction made earlier by the BTAC in the fetch stage, the BTAC-based prediction is discarded and fetching proceeds along the direction predicted by the BHT.

Reservation Stations. Each execution unit in the execute stage has an associated reservation station. Each execution unit's reservation station holds those

instructions waiting to execute there. A reservation station can hold two to four instruction entries, depending on the execution unit. Each dispatched instruction waits in a reservation station until all its source operands have been read or forwarded and the execution unit is available. Instructions can leave reservation stations and be issued into the execution units out of order [except for FPU and branch unit (BRU)].

Execute Stage. This major stage can require multiple cycles to produce its results, depending on the type of instruction being executed. The load/store unit is a two-stage pipeline, and the floating-point unit is a three-stage pipeline. At the end of execution, the instruction results are sent to the destination rename buffers and forwarded to any waiting instructions.

Completion Buffer. The 16-entry completion buffer records the state of the in-flight instructions until they are architecturally complete. An entry is allocated for each instruction during the dispatch stage. The execute stage then marks an instruction as finished when the unit is done executing the instruction. Once an instruction is finished, it is eligible for completion.

Complete Stage. During the completion stage, finished instructions are removed from the completion buffer in order, up to four at a time, and passed to the write-back stage. Fewer instructions will complete in a cycle if there are an insufficient number of write ports to the architected register files. By holding instructions in the completion buffer until writeback, the 620 guarantees that the architected registers hold the correct state up to the most recently completed instruction. Hence, precise exception is maintained even with aggressive out-of-order execution.

Writeback Stage. During this stage, the writeback logic retires those instructions completed in the previous cycle by committing their results from the rename buffers to the architected register files.

6.2 Experimental Framework

The performance simulator for the 620 was implemented using the VMW framework developed at Carnegie Mellon University. The five machine specification files for the 620 were generated based on design documents provided and periodically updated by the 620 design team. Correct interpretation of the design documents was checked by a member of the design team through a series of refinement cycles as the 620 design was finalized.

Instruction and data traces are generated on an existing PowerPC 601 microprocessor via software instrumentation. Traces for several SPEC 92 benchmarks, four integer and three floating-point, are generated. The benchmarks and their dynamic instruction mixes are shown in Table 6.1. Most integer benchmarks have similar instruction mixes; *li* contains more multicycle instructions than the rest. Most of these instructions move values to and from special-purpose registers. There is greater diversity among the floating-point benchmarks. *Hydro2d* uses more nonpipelined floating-point instructions. These instructions are all floating-point divides, which require 18 cycles on the 620.

Table 6.1
Dynamic instruction mix of the benchmark set*

	Integer Benchmarks (SPECInt92)				Floating-Point Benchmarks (SPECfp92)		
Instruction Mix	***compress***	***eqntott***	***espresso***	***li***	***alvinn***	***hydro2d***	***tomcatv***
Integer							
Arithmetic (single cycle)	42.73	48.79	48.30	29.54	37.50	26.25	19.93
Arithmetic (multicycle)	0.89	1.26	1.25	5.14	0.29	1.19	0.05
Load	25.39	23.21	24.34	28.48	0.25	0.46	0.31
Store	16.49	6.26	8.29	18.60	0.20	0.19	0.29
Floating-point							
Arithmetic (pipelined)	0.00	0.00	0.00	0.00	12.27	26.99	37.82
Arithmetic (nonpipelined)	0.00	0.00	0.00	0.00	0.08	1.87	0.70
Load	0.00	0.00	0.00	0.01	26.85	22.53	27.84
Store	0.00	0.00	0.00	0.01	12.02	7.74	9.09
Branch							
Unconditional	1.90	1.87	1.52	3.26	0.15	0.10	0.01
Conditional	12.15	17.43	15.26	12.01	10.37	12.50	3.92
Conditional to count register	0.00	0.44	0.10	0.39	0.00	0.16	0.05
Conditional to link register	4.44	0.74	0.94	2.55	0.03	0.01	0.00

*Values given are percentages.

Trace-driven performance simulation is used. With trace-driven simulation, instructions with variable latency such as integer multiply/divide and floating-point divide cannot be simulated accurately. For these instructions, we assume the minimum latency. The frequency of these operations and the amount of variance in the latencies are both quite low. Furthermore, the traces only contain those instructions that are actually executed. No speculative instructions that are later discarded due to misprediction are included in the simulation runs. Both I-cache and D-cache activities are included in the simulation. The caches are 32K bytes and 8-way set-associative. The D-cache is two-way interleaved. Cache miss latency of eight cycles and a perfect unified L2 cache are also assumed.

Table 6.2
Summary of benchmark performance

Benchmarks	Dynamic Instructions	Execution Cycles	IPC
compress	6,884,247	6,062,494	1.14
eqntott	3,147,233	2,188,331	1.44
espresso	4,615,085	3,412,653	1.35
li	3,376,415	3,399,293	0.99
alvinn	4,861,138	2,744,098	1.77
hydro2d	4,114,602	4,293,230	0.96
tomcatv	6,858,619	6,494,912	1.06

Table 6.2 presents the total number of instructions simulated for each benchmark and the total number of 620 machine cycles required. The sustained average number of instructions per cycle (IPC) achieved by the 620 for each benchmark is also shown. The IPC rating reflects the overall degree of instruction-level parallelism achieved by the 620 microarchitecture, the detailed analysis of which is presented in Sections 6.3 to 6.6.

6.3 Instruction Fetching

Provided that the instruction buffer is not saturated, the 620's fetch unit is capable of fetching four instructions in every cycle. If the fetch unit were to wait for branch resolution before continuing to fetch nonspeculatively, or if it were to bias naively for branch-not-taken, machine execution would be drastically slowed by the bottleneck in fetching down taken branches. Hence, accurate branch prediction is crucial in keeping a wide superscalar processor busy.

6.3.1 Branch Prediction

Branch prediction in the 620 takes place in two phases. The first prediction, done in the fetch stage, uses the BTAC to provide a preliminary guess of the target address when a branch is encountered during instruction fetch. The second, and more accurate, prediction is done in the dispatch stage using the BHT, which contains branch history and makes predictions based on the two history bits.

During the dispatch stage, the 620 attempts to resolve immediately a branch based on available information. If the branch is unconditional, or if the condition register has the appropriate bits ready, then no branch prediction is necessary. The branch is executed immediately. On the other hand, if the source condition register bits are unavailable because the instruction generating them is not finished, then branch prediction is made using the BHT. The BHT contains two history bits per entry that are accessed during the dispatch stage to predict whether the branch will be taken or not taken. Upon resolution of the predicted branch, the actual direction of the branch is updated to the BHT. The 2048-entry BHT is a direct-mapped table, unlike the BTAC, which is an associative cache. There is no concept of a

hit or a miss. If two branches that update the BHT are an exact multiple of 2048 instructions apart, i.e., aliased, they will affect each other's predictions.

The 620 can resolve or predict a branch at the dispatch stage, but even that can incur one cycle delay until the new target of the branch can be fetched. For this reason, the 620 makes a preliminary prediction during the fetch stage, based solely on the address of the instruction that it is currently fetching. If one of these addresses hits in the BTAC, the target address stored in the BTAC is used as the fetch address in the next cycle. The BTAC, which is smaller than the BHT, has 256 entries and is two-way set-associative. It holds only the targets of those branches that are predicted taken. Branches that are predicted not taken (fall through) are not stored in the BTAC. Only unconditional and PC-relative conditional branches use the BTAC. Branches to the count register or the link register have unpredictable target addresses and are never stored in the BTAC. Effectively, these branches are always predicted not taken by the BTAC in the fetch stage. A link register stack, which stores the addresses of subroutine returns, is used for predicting conditional return instructions. The link register stack is not modeled in the simulator.

There are four possible cases in the BTAC prediction: a BTAC miss for which the branch is not taken (correct prediction), a BTAC miss for which the branch is taken (incorrect prediction), a BTAC hit for a taken branch (correct prediction), and a BTAC hit for a not-taken branch (incorrect prediction). The BTAC can never hit on a taken branch and get the wrong target address; only PC-relative branches can hit in the BTAC and therefore must always use the same target address. Two predictions are made for each branch, once by the BTAC in the fetch stage, and another by the BHT in the dispatch stage. If the BHT prediction disagrees with the BTAC prediction, the BHT prediction is used, while the BTAC prediction is discarded. If the predictions agree and are correct, all instructions that are speculatively fetched are used and no penalty is incurred.

In combining the possible predictions and resolutions of the BHT and BTAC, there are six possible outcomes. In general, the predictions made by the BTAC and BHT are strongly correlated. There is a small fraction of the time that the wrong prediction made by the BTAC is corrected by the right prediction of the BHT. There is the unusual possibility of the correct prediction made by the BTAC being undone by the incorrect prediction of the BHT. However, such cases are quite rare; see Table 6.3. The BTAC makes an early prediction without using branch history. A hit in the BTAC effectively implies that the branch is predicted taken. A miss in the BTAC implicitly means a not-taken prediction. The BHT prediction is based on branch history and is more accurate but can potentially incur a one-cycle penalty if its prediction differs from that made by the BTAC. The BHT tracks the branch history and updates the entries in the BTAC. This is the reason for the strong correlation between the two predictions.

Table 6.3 summarizes the branch prediction statistics for the benchmarks. The BTAC prediction accuracy for the integer benchmarks ranges from 75% to 84%. For the floating-point benchmarks it ranges from 88% to 94%. For these correct predictions by the BTAC, no branch penalty is incurred if they are likewise predicted

Table 6.3
Branch prediction data*

Branch Processing	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
Branch resolution							
Not taken	40.35	31.84	40.05	33.09	6.38	17.51	6.12
Taken	59.65	68.16	59.95	66.91	93.62	82.49	93.88
BTAC prediction							
Correct	84.10	82.64	81.99	74.70	94.49	88.31	93.31
Incorrect	15.90	17.36	18.01	25.30	5.51	11.69	6.69
BHT prediction							
Resolved	19.71	18.30	17.09	28.83	17.49	26.18	45.39
Correct	68.86	72.16	72.27	62.45	81.58	68.00	52.56
Incorrect	11.43	9.54	10.64	8.72	0.92	5.82	2.05
BTAC incorrect and BHT correct	0.01	0.79	1.13	7.78	0.07	0.19	0.00
BTAC correct and BHT incorrect	0.00	0.12	0.37	0.26	0.00	0.08	0.00
Overall branch prediction accuracy	88.57	90.46	89.36	91.28	99.07	94.18	97.95

*Values given are percentages.

correctly by the BHT. The overall branch prediction accuracy is determined by the BHT. For the integer benchmarks, about 17% to 29% of the branches are resolved by the time they reach the dispatch stage. For the floating-point benchmarks, this range is 17% to 45%. The overall misprediction rate for the integer benchmarks ranges from 8.7% to 11.4%; whereas for the floating-point benchmarks it ranges from 0.9% to 5.8%. The existing branch prediction mechanisms work quite well for the floating-point benchmarks. There is still room for improvement in the integer benchmarks.

6.3.2 Fetching and Speculation

The main purpose for branch prediction is to sustain a high instruction fetch bandwidth, which in turn keeps the rest of the superscalar machine busy. Misprediction translates into wasted fetch cycles and reduces the effective instruction fetch bandwidth. Another source of fetch bandwidth loss is due to I-cache misses. The effects of these two impediments on fetch bandwidth for the benchmarks are shown in Table 6.4. Again, for the integer benchmarks, significant percentages (6.7% to 11.8%) of the fetch cycles are lost due to misprediction. For all the benchmarks, the I-cache misses resulted in the loss of less than 1% of the fetch cycles.

Table 6.4
Zero bandwidth fetch cycles*

Benchmarks	Misprediction	I-Cache Miss
compress	6.65	0.01
eqntott	11.78	0.08
espresso	10.84	0.52
li	8.92	0.09
alvinn	0.39	0.02
hydro2d	5.24	0.12
tomcatv	0.68	0.01

*Values given are percentages.

Table 6.5
Distribution and average number of branches bypassed*

	Number of Bypassed Branches					
Benchmarks	**0**	**1**	**2**	**3**	**4**	**Average**
compress	66.42	27.38	5.40	0.78	0.02	0.41
eqntott	48.96	28.27	20.93	1.82	0.02	0.76
espresso	53.39	29.98	11.97	4.63	0.03	0.68
li	63.48	25.67	7.75	2.66	0.45	0.51
alvinn	83.92	15.95	0.13	0.00	0.00	0.16
hydro2d	68.79	16.90	10.32	3.32	0.67	0.50
tomcatv	92.07	2.30	3.68	1.95	0.00	0.16

*Columns 0–4 show percentage of cycles.

Branch prediction is a form of speculation. When speculation is done effectively, it can increase the performance of the machine by alleviating the constraints imposed by control dependences. The 620 can speculate past up to four predicted branches before stalling the fifth branch at the dispatch stage. Speculative instructions are allowed to move down the pipeline stages until the branches are resolved, at which time if the speculation proves to be incorrect, the speculated instructions are canceled. Speculative instructions can potentially finish execution and reach the completion stage prior to branch resolution. However, they are not allowed to complete until the resolution of the branch.

Table 6.5 displays the frequency of bypassing specific numbers of branches, which reflects the degree of speculation sustained. The average number of branches bypassed is determined by obtaining the number of correctly predicted branches that

are bypassed in each cycle. Once a branch is determined to be mispredicted, speculation of instructions beyond that branch is not simulated. For the integer benchmarks, in 34% to 51% of the cycles, the 620 is speculatively executing beyond one or more branches. For floating-point benchmarks, the degree of speculation is lower. The frequency of misprediction, shown in Table 6.4, is related to the combination of the average number of branches bypassed, provided in Table 6.5, and the prediction accuracy, provided in Table 6.3.

6.4 Instruction Dispatching

The primary objective of the dispatch stage is to advance instructions from the instruction buffer to the reservation stations. The 620 uses an in-order dispatch policy.

6.4.1 Instruction Buffer

The eight-entry instruction buffer sits between the fetch stage and the dispatch stage. The fetch stage is responsible for filling the instruction buffer. The dispatch stage examines the first four entries of the instruction buffer and attempts to dispatch them to the reservation stations. As instructions are dispatched, the remaining instructions in the instruction buffer are shifted in groups of two to fill the vacated entries.

Figure 6.3(a) shows the utilization of the instruction buffer by profiling the frequencies of having specific numbers of instructions in the instruction buffer. The instruction buffer decouples the fetch stage and the dispatch stage and moderates the temporal variations of and differences between the fetching and dispatching parallelisms. The frequency of having zero instructions in the instruction buffer is significantly lower in the floating-point benchmarks than in the integer benchmarks. This frequency is directly related to the misprediction frequency shown in Table 6.4. At the other end of the spectrum, instruction buffer saturation can cause fetch stalls.

6.4.2 Dispatch Stalls

The 620 dispatches instructions by checking in parallel for all conditions that can cause dispatch to stall. This list of conditions is described in the following in greater detail. During simulation, the conditions in the list are checked one at a time and in the order listed. Once a condition that causes the dispatch of an instruction to stall is identified, checking of the rest of the conditions is aborted, and only that condition is identified as the source of the stall.

Serialization Constraints. Certain instructions cause *single-instruction serialization*. All previously dispatched instructions must complete before the serializing instruction can begin execution, and all subsequent instructions must wait until the serializing instruction is finished before they can dispatch. This condition, though extremely disruptive to performance, is quite rare.

Branch Wait for mtspr. Some forms of branch instructions access the count register during the dispatch stage. A move to special-purpose register (mtspr) instruction that writes to the count register will cause subsequent dependent branch instructions to delay dispatching until it is finished. This condition is also rare.

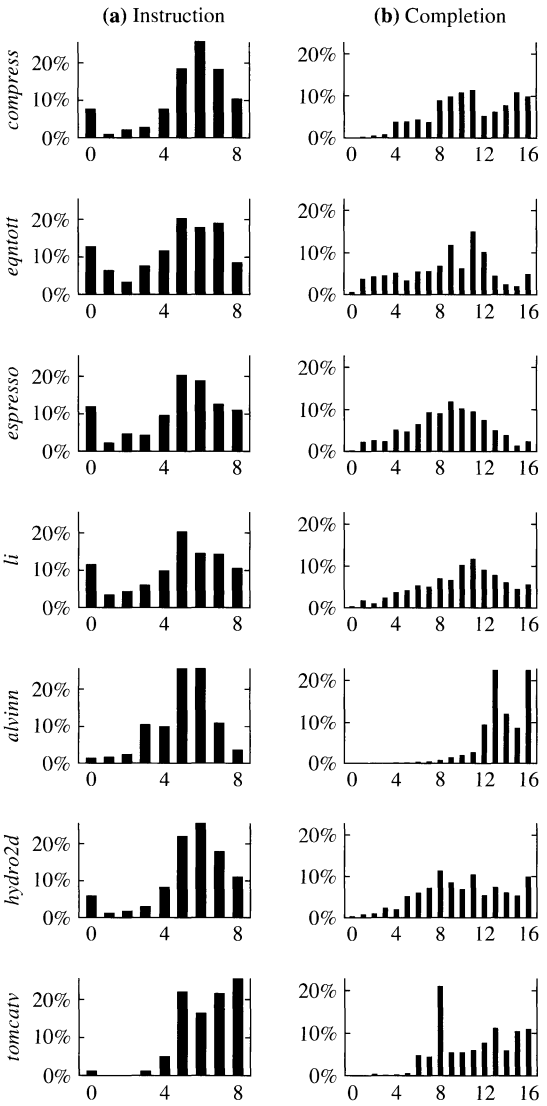


Figure 6.3
Profiles of the (a) Instruction Buffer and (b) Completion Buffer Utilizations.

Register Read Port Saturation. There are seven read ports for the general-purpose register file and four read ports for the floating-point register file. Occasionally, saturation of the read ports occurs when a read port is needed but none is available. There are enough condition register field read ports (three) that saturation cannot occur.

Reservation Station Saturation. As an instruction is dispatched, the instruction is placed into the reservation station of the instruction's associated execution unit. The instruction remains in the reservation station until it is issued. There is one reservation station per execution unit, and each reservation station has multiple entries, depending on the execution unit. Reservation station saturation occurs when an instruction can be dispatched to a reservation station but that reservation station has no more empty entries.

Rename Buffer Saturation. As each instruction is dispatched, its destination register is renamed into the appropriate rename buffer files. There are three rename buffer files, for general-purpose registers, floating-point registers, and condition register fields. Both the general-purpose register file and the floating-point register file have eight rename buffers. The condition register field file has 16 rename buffers.

Completion Buffer Saturation. Completion buffer entries are also allocated during the dispatch stage. They are kept until the instruction has completed. The 620 has 16 completion buffer entries; no more than 16 instructions can be in flight at the same time. Attempted dispatch beyond 16 in-flight instructions will cause a stall. Figure 6.3(b) illustrates the utilization profiles of the completion buffer for the benchmarks.

Another Dispatched to Same Unit. Although a reservation station has multiple entries, each reservation station can receive at most one instruction per cycle even when there are multiple available entries in a reservation station. Essentially, this constraint is due to the fact that each of the reservation stations has only one write port.

6.4.3 Dispatch Effectiveness

The average utilization of all the buffers is provided in Table 6.6. Utilization of the load/store unit's three reservation station entries averages 1.36 to 1.73 entries for integer benchmarks and 0.98 to 2.26 entries for floating-point benchmarks. Unlike the other execution units, the load/store unit does not deallocate a reservation station entry as soon as an instruction is issued. The reservation station entry is held until the instruction is finished, usually two cycles after the instruction is issued. This is due to the potential miss in the D-cache or the TLB. The reservation station entries in the floating-point unit are more utilized than those in the integer units. The in-order issue constraint of the floating-point unit and the nonpipelining of some floating-point instructions prevent some ready instructions from issuing. The average utilization of the completion buffer ranges from 9 to 14 for the benchmarks and corresponds with the average number of instructions that are in flight.

Sources of dispatch stalls are summarized in Table 6.7 for all benchmarks. The data in the table are percentages of all the cycles executed by each of the benchmarks. For example, in 24.35% of the *compress* execution cycles, no dispatch stalls occurred; i.e., all instructions in the dispatch buffer (first four entries of the instruction buffer) are dispatched. A common and significant source of bottleneck

Table 6.6
Summary of average number of buffers used

Buffer Usage	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
Instruction buffers (8)	5.41	4.43	4.72	4.65	5.13	5.44	6.42
Dispatch buffers (4)	3.21	2.75	2.89	2.85	3.40	3.10	3.53
XSU0 RS entries (2)	0.37	0.66	0.68	0.36	0.48	0.23	0.11
XSU1 RS entries (2)	0.42	0.51	0.65	0.32	0.24	0.17	0.10
MC-FXU RS entries (2)	0.04	0.07	0.09	0.28	0.01	0.10	0.00
FPU RS entries (2)	0.00	0.00	0.00	0.00	0.70	1.04	0.89
LSU RS entries (3)	1.69	1.36	1.60	1.73	2.26	0.98	1.23
BRU RS entries (4)	0.45	0.84	0.75	0.59	0.19	0.54	0.17
GPR rename buffers (8)	2.73	3.70	3.25	2.77	3.79	1.83	1.97
FPR rename buffers (8)	0.00	0.00	0.00	0.00	5.03	2.85	3.23
CR rename buffers (16)	1.25	1.32	1.19	0.98	1.27	1.20	0.42
Completion buffers (16)	10.75	8.83	8.75	9.87	13.91	10.10	11.16

Table 6.7
Frequency of dispatch stall cycles*

Sources of Dispatch Stalls	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
Serialization	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Move to special register constraint	0.00	4.49	0.94	3.44	0.00	0.95	0.08
Read port saturation	0.26	0.00	0.02	0.00	0.32	2.23	6.73
Reservation station saturation	36.07	22.36	31.50	34.40	22.81	42.70	36.51
Rename buffer saturation	24.06	7.60	13.93	17.26	1.36	16.98	34.13
Completion buffer saturation	5.54	3.64	2.02	4.27	21.12	7.80	9.03
Another to same unit	9.72	20.51	18.31	10.57	24.30	12.01	7.17
No dispatch stalls	24.35	41.40	33.28	30.06	30.09	17.33	6.35

*Values given are percentages.

for all the benchmarks is the saturation of reservation stations, especially in the load/store unit. For the other sources of dispatch stalls, the degrees of various bottlenecks vary among the different benchmarks. Saturation of the rename buffers is significant for *compress* and *tomcatv,* even though on average their rename

buffers are less than one-half utilized. Completion buffer saturation is highest in *alvinn*, which has the highest frequency of having all 16 entries utilized; see Figure 6.3(b). Contention for the single write port to each reservation station is also a serious bottleneck for many benchmarks.

Figure 6.4(a) displays the distribution of dispatching parallelism (the number of instructions dispatched per cycle). The number of instructions dispatched in each cycle can range from 0 to 4. The distribution indicates the frequency (averaged across the entire trace) of dispatching n instructions in a cycle, where $n = 0, 1, 2, 3, 4$. In all benchmarks, at least one instruction is dispatched per cycle for over one-half of the execution cycles.

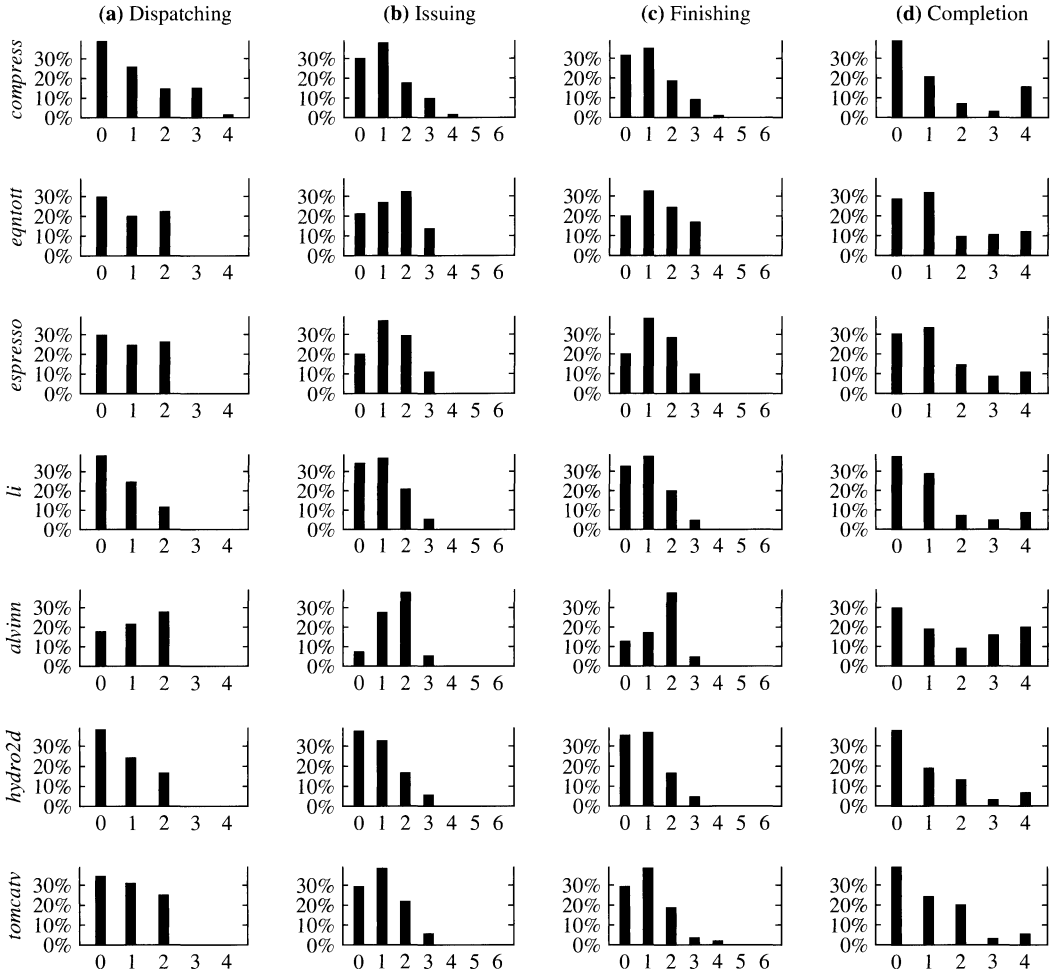


Figure 6.4

Distribution of the Instruction (a) Dispatching, (b) Issuing, (c) Finishing, and (d) Completion Parallelisms.

6.5 Instruction Execution

The 620 widens in the execute stage. While the fetch, dispatch, complete, and writeback stages all are four wide, i.e., can advance up to four instructions per cycle, the execute stage contains six execution units and can issue and finish up to six instructions per cycle. Furthermore, unlike the other stages, the execute stage processes instructions out of order to achieve maximum throughput.

6.5.1 Issue Stalls

Once instructions have been dispatched to reservation stations, they must wait for their source operands to become available, and then begin execution. There are a few other constraints, however. The full list of issuing hazards is described here.

Out of Order Disallowed. Although out-of-order execution is usually allowed from reservation stations, it is sometimes the case that certain instructions may not proceed past a prior instruction in the reservation station. This is the case in the branch unit and the floating-point unit, where instructions must be issued in order.

Serialization Constraints. Instructions which read or write non-renamed registers (such as XER), which read or write renamed registers in a non-renamed fashion (such as load/store multiple instructions), or which change or synchronize machine state (such as the *eieio* instruction, which enforces in-order execution of I/O) must wait for all prior instructions to complete before executing. These instructions stall in the reservation stations until their serialization constraints are satisfied.

Waiting for Source Operand. The primary purpose of reservation stations is to hold instructions until all their source operands are ready. If an instruction requires a source that is not available, it must stall here until the operand is forwarded to it.

Waiting for Execution Unit. Occasionally, two or more instructions will be ready to begin execution in the same cycle. In this case, the first will be issued, but the second must wait. This condition also applies when an instruction is executing in the MC-FXU (a nonpipelined unit) or when a floating-point divide instruction puts the FPU into nonpipelined mode.

The frequency of occurrence for each of the four issue stall types is summarized in Table 6.8. The data are tabulated for all execution units except the branch unit. Thus, the in-order issuing restriction only concerns the floating-point unit. The number of issue serialization stalls is roughly proportional to the number of multicycle integer instructions in the benchmark's instruction mix. Most of these multicycle instructions access the special-purpose registers or the entire condition register as a non-renamed unit, which requires serialization. Most issue stalls due to waiting for an execution unit occur in the load/store unit. More load/store instructions are ready to execute than the load/store execution unit can accommodate. Across all benchmarks, in significant percentages of the cycles, no issuing stalls are encountered.

Table 6.8
Frequency of issue stall cycles*

Sources of Issue Stalls	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
Out of order disallowed	0.00	0.00	0.00	0.00	0.72	11.03	1.53
Serialization	1.69	1.81	3.21	10.81	0.03	4.47	0.01
Waiting for source	21.97	29.30	37.79	32.03	17.74	22.71	3.52
Waiting for execution unit	13.67	3.28	7.06	11.01	2.81	1.50	1.30
No issue stalls	62.67	65.61	51.94	46.15	78.70	60.29	93.64

*Values given are percentages.

6.5.2 Execution Parallelism

Here we examine the propagation of instructions through issuing, execution, and finish. Figure 6.4(b) shows the distribution of issuing parallelism (the number of instructions issued per cycle). The maximum number of instructions that can be issued in each cycle is six, the number of execution units. Although the issuing parallelism and the dispatch parallelism distributions must have the same average value, issuing is less centralized and has fewer constraints and can therefore achieve a more consistent rate of issuing. In most cycles, the number of issued instructions is close to the overall sustained IPC, while the dispatch parallelism has more extremes in its distribution.

We expected the distribution of finishing parallelism, shown in Figure 6.4(c), to look like the distribution of issuing parallelism because an instruction after it is issued must finish a certain number of cycles later. Yet this is not always the case as can be seen in the issuing and finishing parallelism distributions of the *eqntott, alvinn,* and *hydro2d* benchmarks. The difference comes from the high frequency of load/store and floating-point instructions. Since these instructions do not take the same amount of time to finish after issuing as the integer instructions, they tend to shift the issuing parallelism distribution. The integer benchmarks, with their more consistent instruction execution latencies, generally have more similarity between their issuing and finishing parallelism distributions.

6.5.3 Execution Latency

It is of interest to examine the average latency encountered by an instruction from dispatch until finish. If all the issuing constraints are satisfied and the execution unit is available, an instruction can be dispatched from the dispatch buffer to a reservation station, and then issued into the execution unit in the next cycle. This is the best case. Frequently, instructions must wait in the reservation stations. Hence, the overall execution latency includes the waiting time in the reservation station and the actual latency of the execution units. Table 6.9 shows the average overall execution latency encountered by the benchmarks in each of the six execution units.

Table 6.9
Average execution latency (in cycles) in each of the six execution units for the benchmarks

Execution Units	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
XSU0 (1 stage)	1.53	1.62	1.89	2.28	1.05	1.48	1.01
XSU1 (1 stage)	1.72	1.73	2.23	2.39	1.13	1.78	1.03
MC-FXU (>1 stage)	4.35	4.82	6.18	5.64	3.48	9.61	1.64
FPU (3 stages)	—*	—*	—*	—*	5.29	6.74	4.45
LSU (2 stages)	3.56	2.35	2.87	3.22	2.39	2.92	2.75
BRU (≥1 stage)	2.71	2.86	3.11	3.28	1.04	4.42	4.14

*Very few instructions are executed in this unit for these benchmarks.

6.6 Instruction Completion

Once instructions finish execution, they enter the completion buffer for in-order completion and writeback. The completion buffer functions as a reorder buffer to reorder the out-of-order execution in the execute stage back to the sequential order for in-order retiring of instructions.

6.6.1 Completion Parallelism

The distributions of completion parallelism for all the benchmarks are shown in Figure 6.4(d). Again, similar to dispatching, up to four instructions can be completed per cycle. An average value can be computed for each of the parallelism distributions. In fact, for each benchmark, the average completion parallelism should be exactly equal to the average dispatching, issuing, and finishing parallelisms, which are all equal to the sustained IPC for the benchmark. In the case of instruction completion, while instructions are allowed to finish out of order, they can only complete in order. This means that occasionally the completion buffer will have to wait for one slow instruction to finish, but then will be able to retire its maximum of four instructions at once. On some occasions, the completion buffer can saturate and cause the stalling at the dispatch stage; see Figure 6.3(b).

The integer benchmarks with their more consistent execution latencies usually have one instruction completed per cycle. *Hydro2d* completes zero instructions in a large percentage of cycles because it must wait for floating-point divide instructions to finish. Usually, instructions cannot complete because they, or instructions preceding them, are not finished yet. However, occasionally there are other reasons. The 620 has four integer and two floating-point writeback ports. It is rare to run out of integer register file write ports. However, floating-point write port saturation occurs occasionally.

6.6.2 Cache Effects

The D-cache behavior has a direct impact on the CPU performance. Cache misses can cause additional stall cycles in the execute and complete stages. The D-cache in the 620 is interleaved in two banks, each with an address port. A load or store

instruction can use either port. The cache can service at most one load and one store at the same time. A load instruction and a store instruction can access the cache in the same cycle if the accesses are made to different banks. The cache is nonblocking, only for the port with a load access. When a load cache miss is encountered and while a cache line is being filled, a subsequent load instruction can proceed to access the cache. If this access results in a cache hit, the instruction can proceed without being blocked by the earlier miss. Otherwise, the instruction is returned to the reservation station. The multiple entries in the load/store reservation station and the out-of-order issuing of instructions allow the servicing of a load with a cache hit past up to three outstanding load cache misses.

The sequential consistency model for main memory imposes the constraint that all memory instructions must appear to execute in order. However, if all memory instructions are to execute in sequential order, a significant amount of performance can be lost. The 620 executes all store instructions, which access the cache after the complete stage (using the physical address), in order; however, it allows load instructions, which access the cache in the execute stage (using the virtual address), to bypass store instructions. Such relaxation is possible due to the weak memory consistency model specified by the PowerPC ISA [May et al., 1994]. When a store is being completed, aliasing of its address with that of loads that have bypassed and finished is checked. If aliasing is detected, the machine is flushed when the next load instruction is examined for completion, and refetching of that load instruction is carried out. No forwarding of data is made from a pending store instruction to a dependent load instruction. The weak ordering of memory accesses can eliminate some unnecessary stall cycles. Most load instructions are at the beginning of dependence chains, and their earliest possible execution can make available other instructions for earlier execution.

Table 6.10 summarizes the nonblocking cache effect and the weak ordering of load/store instructions. The first line in the table gives the D-cache hit rate for all the benchmarks. The hit rate ranges from 94.2% to 99.9%. Because of the nonblocking feature of the cache, a load can bypass another load if the trailing load is a cache hit at the time that the leading load is being serviced for a cache miss. The percentage (as percentage of all load instructions) of all such trailing loads that actually bypass a missed load is given in the second line of Table 6.10. When a store is completed, it enters the complete store queue, waits there until the store writes to the cache, and then exits the queue. During the time that a pending store is in the queue, a load can potentially access the cache and bypass the store. The third line of Table 6.10 gives the percentage of all loads that, at the time of the load cache access, bypass at least one pending store. Some of these loads have addresses that alias with the addresses of the pending stores. The percentage of all loads that bypass a pending store and alias with any of the pending store addresses is given in the fourth line of the table. Most of the benchmarks have an insignificant number of aliasing occurrences. The fifth line of the table gives the average number of pending stores, or the number of stores in the store complete queue, in each cycle.

Table 6.10
Cache effect data*

Cache Effects	*compress*	*eqntott*	*espresso*	*li*	*alvinn*	*hydro2d*	*tomcatv*
Loads/stores with cache hit	94.17	99.57	99.92	99.74	99.99	94.58	96.24
Loads that bypass a missed load	8.45	0.53	0.11	0.14	0.01	4.82	5.45
Loads that bypass a pending store	58.85	21.05	27.17	48.49	98.33	58.26	43.23
Load that aliased with a pending store	0.00	0.31	0.77	2.59	0.27	0.21	0.29
Average number of pending stores per cycle	1.96	0.83	0.97	2.11	1.30	1.01	1.38

*Values given are percentages, except for the average number of pending stores per cycle.

6.7 Conclusions and Observations

The most interesting parts of the 620 microarchitecture are the branch prediction mechanisms, the out-of-order execution engine, and the weak ordering of memory accesses. The 620 does reasonably well on branch prediction. For the floating-point benchmarks, about 94% to 99% of the branches are resolved or correctly predicted, incurring little or no penalty cycles. Integer benchmarks yield another story. The range drops down to 89% to 91%. More sophisticated prediction algorithms, for example, those using more history information, can increase prediction accuracy. It is also clear that floating-point and integer benchmarks exhibit significantly different branching behaviors. Perhaps separate and different branch prediction schemes can be employed for dealing with the two types of benchmarks.

Even with having to support precise exceptions, the out-of-order execution engine in the 620 is still able to achieve a reasonable degree of instruction-level parallelism, with sustained IPC ranging from 0.99 to 1.44 for integer benchmarks and from 0.96 to 1.77 for floating-point benchmarks. One hot spot is the load/store unit. The number of load/store reservation station entries and/or the number of load/store units needs to be increased. Although the difficulties of designing a system with multiple load/store units are myriad, the load/store bottleneck in the 620 is evident. Having only one floating-point unit for three integer units is also a source of bottleneck. The integer benchmarks rarely stall on the integer units, but the floating-point benchmarks do stall while waiting for floating-point resources. The single dispatch to each reservation station in a cycle is also a source of dispatch stalls, which can reduce the number of instructions available for out-of-order execution. One interesting tradeoff involves the choice of implementing distributed reservation stations, as in the 620, versus one centralized reservation station, as in the Intel P6. The former approach permits simpler hardware since there are only

single-ported reservation stations. However, the latter can share the multiple ports and the reservation station entries among different instruction types.

Allowing weak-ordering memory accesses is essential in achieving high performance in modern wide superscalar processors. The 620 allows loads to bypass stores and other loads; however, it does not provide forwarding from pending stores to dependent loads. The 620 allows loads to bypass stores but does not check for aliasing until completing the store. This store-centric approach makes forwarding difficult and requires that the machine be flushed from the point of the dependent load when aliasing occurs. The 620 does implement the D-cache as two interleaved banks, and permits the concurrent processing of one load and one store in the same cycle if there is no bank conflict. Using the standard rule of thumb for dynamic instruction mix, there is a clear imbalance with the processing of load/store instructions in current superscalar processors. Increasing the throughput of load/store instructions is currently the most critical challenge. As future superscalar processors get wider and their clock speeds increase, the memory bottleneck problem will be further exacerbated. Furthermore, commercial applications such as transaction processing (not characterized by the SPEC benchmarks) put even greater pressure on the memory and chip I/O bottleneck.

It is interesting to examine superscalar introductions contemporaneous to the 620 by different companies, and how different microprocessor families have evolved; see Figure 6.5. The PowerPC microprocessors and the Alpha AXP microprocessors represent two different approaches to achieving high performance in superscalar machines. The two approaches have been respectively dubbed "brainiacs vs. speed demons." The PowerPC microprocessors attempt to achieve

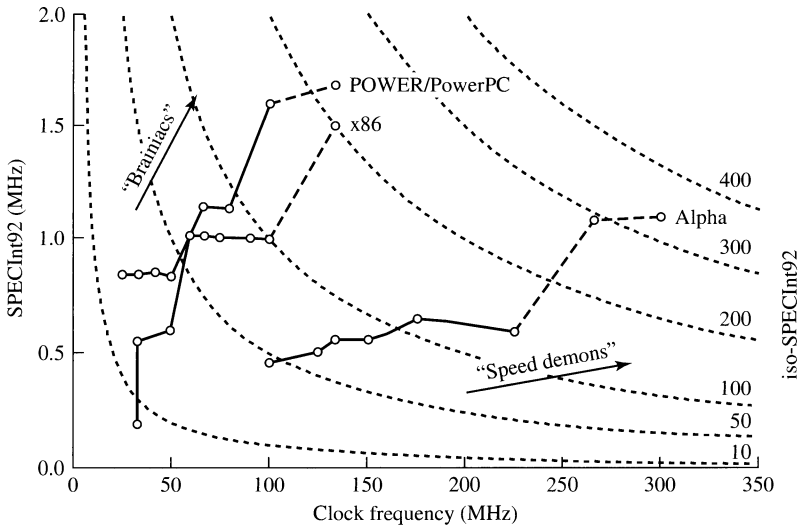


Figure 6.5
Evolution of Superscalar Families.

the highest level of IPC possible without overly compromising the clock speed. On the other hand, the Alpha AXP microprocessors go for the highest possible clock speed while achieving a reasonable level of IPC. Of course, future versions of PowerPC chips will get faster and future Alpha AXP chips will achieve higher IPC. The key issue is, which should take precedence, IPC or MHz? Which approach will yield an easier path to get to the next level of performance? Although these versions of the PowerPC 620 microprocessor and the Alpha AXP 21164 microprocessor seem to indicate that the speed demons are winning, there is no strong consensus on the answer for the future. In an interesting way, the two rivaling approaches resemble, and perhaps are a reincarnation of, the CISC vs. RISC debate of a decade earlier [Colwell et al., 1985].

The announcement of the P6 from Intel presents another interesting case. The P6 is comparable to the 620 in terms of its microarchitectural aggressiveness in achieving high IPC. On the other hand the P6 is somewhat similar to the 21164 in that they both are more "superpipelined" than the 620. The P6 represents yet a third, and perhaps hybrid, approach to achieving high performance. Figure 6.5 reflects the landscape that existed circa the mid-1990s. Since then the landscape has shifted significantly to the right. Today we are no longer using SPECInt92 benchmarks but SPECInt2000, and we are dealing with frequencies in the multiple-gigahertz range.

6.8 Bridging to the IBM POWER3 and POWER4

The PowerPC 620 was intended as the initial high-end 64-bit implementation of the PowerPC architecture that would satisfy the needs of the server and high-performance workstation market. However, because of numerous difficulties in finishing the design in a timely fashion, the part was delayed by several years and ended up only being used in a few server systems developed by Groupe Bull. In the meantime, IBM was able to satisfy its need in the server product line with the Star series and POWER2 microprocessors, which were developed by independent design teams and differed substantially from the PowerPC 620.

However, the IBM POWER3 processor, released in 1998, was heavily influenced by the PowerPC 620 design and reused its overall pipeline structure and many of its functional blocks [O'Connell and White, 2000]. Table 6.11 summarizes some of the key differences between the 620 and the POWER3 processors. Design optimization combined with several years of advances in semiconductor technology resulted in nearly tripling the processor frequency, even with a similar pipeline structure, resulting in noticeable performance improvement.

The POWER3 addressed some of the shortcomings of the PowerPC 620 design by substantially improving both instruction execution bandwidth as well as memory bandwidth. Although the front and back ends of the pipeline remained the same width, the POWER3 increased the peak issue rate to eight instructions per cycle by providing two load/store units and two fully pipelined floating-point units. The effective window size was also doubled by increasing the completion buffer to 32 entries and by doubling the number of integer rename registers and tripling the number of floating-point rename registers. Memory bandwidth was further enhanced with a novel 128-way set-associative cache design that embeds

Table 6.11
PowerPC 620 versus IBM POWER3 and POWER4

Attribute	620	POWER3	POWER4
Frequency	172 MHz	450 MHz	1.3 GHz
Pipeline length	5+	5+	15+
Branch prediction	Bimodal BHT/BTAC	Same as 620	3 × 16K combining
Fetch/issue/completion width	4/6/4	4/8/4	4/8/5
Rename/physical registers	8 Int, 8 FP	16 Int, 24 FP	80 Int, 72 FP
In-flight instructions	16	32	Up to 200
Floating-point units	1	2	2
Load/store units	1	2	2
Instruction cache	32K 8-way SA	32K 128-way SA	64K DM
Data cache	32K 8-way SA	64K 128-way SA	32K 2-way SA
L2/L3 size	4 Mbytes	16 Mbytes	~1.5 Mbytes/32 Mbytes
L2 bandwidth	1 Gbytes/s	6.4 Gbytes/s	100+ Gbytes/s
Store queue entries	6 × 8 bytes	16 × 8 bytes	12 × 64 bytes
MSHRs	I:1/D:1	I:2/D:4	I:2/D:8
Hardware prefetch	None	4 streams	8 streams

SA—set-associative DM—direct mapped

tag match hardware directly into the tag arrays of both L1 caches, significantly reducing the miss rates, and by doubling the overall size of the data cache. The L2 cache size also increased substantially, as did available bandwidth to the off-chip L2 cache. Memory latency was also effectively decreased by incorporating an aggressive hardware prefetch engine that can detect up to four independent reference streams and prefetch them from memory. This prefetching scheme works extremely well for floating-point workloads with regular, predictable access patterns. Finally, support for multiple outstanding cache misses was added by providing two miss-status handling registers (MSHRs) for the instruction cache and four MSHRs for the data cache.

The next new high-performance processor in the PowerPC family was the POWER4 processor, introduced in 2001 [Tendler et al., 2001]. Key attributes of this entirely new core design are summarized in Table 6.11. IBM achieved yet another tripling of processor frequency, this time by employing a substantially deeper pipeline in conjunction with major advances in process technology (i.e., reduced feature sizes, copper interconnects, and silicon-on-insulator technology). The POWER4 pipeline is illustrated in Figure 6.6 and extends to 15 stages for the best case of single-cycle integer ALU instructions. To keep this pipeline fed with useful instructions, the POWER4 employs an advanced combining branch predictor that uses a 16K entry selector table to choose between a 16K entry *bimodal* predictor and a 16K entry *gshare* predictor. Each entry in each of the three tables is only 1 bit,

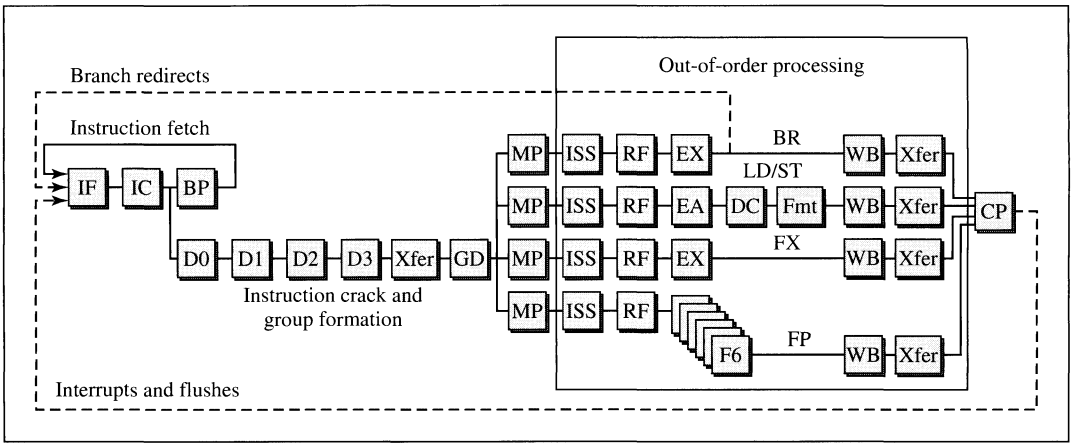


Figure 6.6
POWER4 Pipeline Structure.
Source: Tendler et al., 2001.

rather than a 2-bit up-down counter, since studies showed that 16K 1-bit entries performed better than 8K 2-bit entries. This indicates that for the server workloads the POWER4 is optimized for, branch predictor capacity misses are more important than the hysteresis provided by 2-bit counters.

The POWER4 matches the POWER3 in execution bandwidth, but provides substantially more rename registers (now in the form of a single physical register file) and supports up to 200 in-flight instructions in its pipeline. As in the POWER3, memory bandwidth and latency were important considerations, and multiple load/store units, support for up to eight outstanding cache misses, a very-high-bandwidth interface to the on-chip L2, and support for a massive off-chip L3 cache, all play an integral role in improving overall performance. The POWER4 also packs two complete processor cores sharing an L2 cache on a single chip in a *chip multiprocessor* configuration. More details on this arrangement are discussed in Chapter 11.

6.9 Summary

The PowerPC 620 is an interesting first-generation out-of-order superscalar processor design that exemplifies the short pipelines, aggressive support for extracting instruction-level parallelism, and support for weak ordering of memory references that are typical for other processors of a similar vintage (for example, the HP PA-8000 [Gwennap, 1994] and the MIPS R10000 [Yeager, 1996]). Its evolution into the IBM POWER3 part illustrates the natural extension of execution resources to extract even greater parallelism while also tackling the memory bandwidth and latency bottlenecks. Finally, the recent POWER4 design highlights the seemingly heroic efforts of microprocessors today to tolerate memory bandwidth and latency with aggressive on- and off-chip cache hierarchies, stream-based hardware

prefetching, and very large instruction windows. At the same time, the POWER4 illustrates the trend toward higher and higher clock frequency through extremely deep pipelining, which can only be sustained as a result of increasingly accurate branch predictors that keep such pipelines filled with useful instructions.

REFERENCES

Colwell, R., C. Hitchcock, E. Jensen, H. B. Sprunt, and C. Kollar: "Instructions sets and beyond: Computers, complexity, and controversy," *IEEE Computer,* 18, 9, 1985, pp. 8–19.

Diep, T. A., C. Nelson, and J. P. Shen: "Performance evaluation of the PowerPC 620 microarchitecture," *Proc. 22nd Int. Symposium on Computer Architecture,* Santa Margherita Ligure, Italy, 1995.

Gwennap, L.: "PA-8000 combines complexity and speed," *Microprocessor Report,* 8, 15, 1994, pp. 6–9.

IBM Corp.: *PowerPC 601 RISC Microprocessor User's Manual.* IBM Microelectronics Division, 1993.

IBM Corp.: *PowerPC 604 RISC Microprocessor User's Manual.* IBM Microelectronics Division, 1994.

Levitan, D., T. Thomas, and P. Tu: "The PowerPC 620 microprocessor: A high performance superscalar RISC processor," *Proc. of COMPCON 95,* 1995, pp. 285–291.

May, C., E. Silha, R. Simpson, and H. Warren: *The PowerPC Architecture: A Specification for a New Family of RISC Processors,* 2nd ed. San Francisco, CA, Morgan Kauffman, 1994.

Motorola, Inc.: *MPC750 RISC Microprocessor Family User's Manual.* Motorola, Inc., 2001.

Motorola, Inc.: *MPC603e RISC Microprocessor User's Manual.* Motorola, Inc., 2002.

Motorola, Inc.: *MPC7450 RISC Microprocessor Family User's Manual.* Motorola, Inc., 2003.

O'Connell, F., and S. White: "POWER3: the next generation of PowerPC processors," *IBM Journal of Research and Development,* 44, 6, 2000, pp. 873–884.

Storino, S., A. Aipperspach, J. Borkenhagen, R. Eickemeyer, S. Kunkel, S. Levenstein, and G. Uhlmann: "A commercial multi-threaded RISC processor," *Int. Solid-State Circuits Conference,* 1998.

Tendler, J. M., S. Dodson, S. Fields, and B. Sinharoy: "IBM eserver POWER4 system microarchitecture," IBM Whitepaper, 2001.

Yeager, K.: "The MIPS R10000 superscalar microprocessor," *IEEE Micro,* 16, 2, 1996, pp. 28–40.

HOMEWORK PROBLEMS

P6.1 Assume the IBM instruction mix from Chapter 2, and consider whether or not the PowerPC 620 completion buffer versus integer rename buffer design is reasonably balanced. Assume that load and ALU instructions need an integer rename buffer, while other instructions do not. If the 620 design is not balanced, how many rename buffers should there be?

P6.2 Assuming instruction mixes in Table 6.2, which benchmarks are likely to be rename buffer constrained? That is, which ones run out of rename buffers before they run out of completion buffers and vice versa?

P6.3 Given the dispatch and retirement bandwidth specified, how many integer architected register file (ARF) read and write ports are needed to sustain peak throughput? Given instruction mixes in Table 6.2, also compute average ports needed for each benchmark. Explain why you would not just build for the average case. Given the actual number of read and write ports specified, how likely is it that dispatch will be port-limited? How likely is it that retirement will be port-limited?

P6.4 Given the dispatch and retirement bandwidth specified, how many integer rename buffer read and write ports are needed? Given instruction mixes in Table 6.2, also compute average ports needed for each benchmark. Explain why you would not just build for the average case.

P6.5 Compare the PowerPC 620 BTAC design to the next-line/next-set predictor in the Alpha 21264 as described in the *Alpha 21264 Microprocessor Hardware Reference Manual* (available from **www.compaq.com**). What are the key differences and similarities between the two techniques?

P6.6 How would you expect the results in Table 6.5 to change for a more recent design with a deeper pipeline (e.g., 20 stages, like the Pentium 4)?

P6.7 Judging from Table 6.7, the PowerPC 620 appears reservation station–starved. If you were to double the number of reservation stations, how much performance improvement would you expect for each of the benchmarks? Justify your answer.

P6.8 One of the most obvious bottlenecks of the 620 design is the single load/store unit. The IBM POWER3, a subsequent design based heavily on the 620 microarchitecture, added a second load/store unit along with a second floating-point multiply/add unit. Compare the SPECInt2000 and SPECFP2000 score of the IBM POWER3 (as reported on **www.spec.org**) with another modern processor, the Alpha AXP 21264. Normalized to frequency, which processor scores higher? Why is it not fair to normalize to frequency?

P6.9 The data in Table 6.10 seems to support the 620 designers' decision to not implement load/store forwarding in the 620 processor. Discuss how this tradeoff changes as pipeline depth increases and relative memory latency (as measured in processor cycles) increases.

P6.10 Given the data in Table 6.9, present a hypothesis for why XSU1 appears to have consistently longer execution latency than XSU0. Describe an experiment you might conduct to verify your hypothesis.

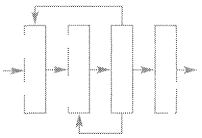
P6.11 The IBM POWER3 can detect up to four regular access streams and issue prefetches for future references. Construct an address reference trace that will utilize all four streams.

P6.12 The IBM POWER4 can detect up to eight regular access streams and issue prefetches for future references. Construct an address reference trace that will utilize all eight streams.

P6.13 The stream prefetching of the POWER3 and POWER4 processors is done outside the processor core, using the physical addresses of cache lines that miss the L1 cache. Explain why large virtual memory page sizes can improve the efficiency of such a prefetch scheme.

P6.14 Assume that a program is streaming sequentially through a 1-Gbyte array by reading each aligned 8-byte floating-point double in the 1-Gbyte array. Further assume that the prefetch engine will start prefetching after it has seen three consecutive cache line references that miss the L1 cache (i.e., the fourth cache line in a sequential stream will be prefetched). Assuming 4K page sizes and given the cache line sizes for the POWER3 and POWER4, compute the overall miss rate for this program for each of the following three assumptions: no prefetching, physical-address prefetching, and virtual-address prefetching. Report the miss rate per L1 D-cache reference, assuming that there is a single reference to every 8-byte word.

P6.15 Download and install the sim-outorder simulator from the Simplescalar simulator suite (available from **www.simplescalar.com**). Configure the simulator to match (as closely as possible) the microarchitecture of the PowerPC 620. Now collect branch prediction and cache hit data using the instructional benchmarks available from the Simplescalar website. Compare your results to Tables 6.3 and 6.10 and provide some reasons for differences you might observe.



CHAPTER 7

Robert P. Colwell
Dave B. Papworth
Glenn J. Hinton
Mike A. Fetterman
Andy F. Glew

Intel's P6 Microarchitecture

CHAPTER OUTLINE

In 1990, Intel began development of a new 32-bit Intel Architecture (IA32) microarchitecture core known as the P6. Introduced as a product in 1995 [Colwell and Steck, 1995], it was named the Pentium Pro processor and became very popular in workstation and server systems. A desktop proliferation of the P6 core, the Pentium II processor, was launched in May 1997, which added the MMX instructions to the basic P6 engine. The P6-based Pentium III processor followed in 1998, which included MMX and SSE instructions. This chapter refers to the core as the P6 and to the products by their respective product names.

The P6 microarchitecture is a 32-bit Intel Architecture-compatible, high-performance, superpipelined dynamic execution engine. It is order-3 superscalar and uses out-of-order and speculative execution techniques around a micro-dataflow execution core. P6 includes nonblocking caches and a transactions-based snooping bus. This chapter describes the various components of the design and how they combine to deliver extraordinary performance on an economical die size.

7.1 Introduction

The basic block diagram of the P6 microarchitecture is shown in Figure 7.1. There are three basic sections to the microarchitecture: the in-order front end, an out-of-order middle, and an in-order back-end "retirement" process. To be Intel Architecture–compatible, the machine must obey certain conventions on execution of its program code. But to achieve high performance, it must relax other conventions, such as execution of the program's operators strictly in the order implied by the program itself. True data dependences must be observed, but beyond that, only certain memory ordering constraints and the precise faulting semantics of the IA32 architecture must be guaranteed.

To maintain precise faulting semantics, the processor must ensure that asynchronous events such as interrupts and synchronous but awkward events such as faults and traps will be handled in exactly the same way as they would have in an i486 system.[1] This implies an in-order retirement process that reimposes the original program ordering to the commitment of instruction results to permanent architectural

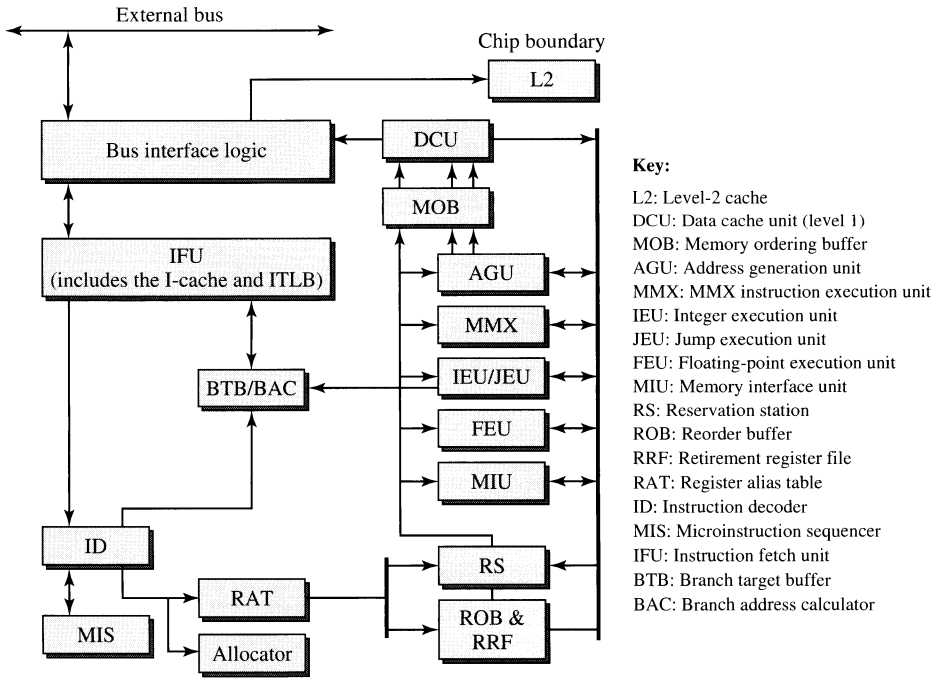


Figure 7.1
P6 Microarchitecture Block Diagram.

[1] Branches and faults use the same mechanism to recover state. However, for performance reasons, branches clear and restart the front end as early as possible. Page faults are handled speculatively, but floating-point faults are handled only when the machine is sure the faulted instructions were on the path of certain execution.

machine state. With these in-order mechanisms at both ends of the execution pipeline, the actual execution of the instructions can proceed unconstrained by any artifacts other than true data dependences and machine resources. We will explore the details of all three sections of the machine in the remainder of this chapter.

There are many novel aspects to this microarchitecture. For instance, it is almost universal that processors have a central controller unit somewhere that monitors and controls the overall pipeline. This controller "understands" the state of the instructions flowing through the pipeline, and it governs and coordinates the changes of state that constitute the computation process. The P6 microarchitecture purposely avoids having such a centralized resource. To simplify the hardware in the rest of the machine, this microarchitecture translates the Intel Architecture instructions into simple, stylized atomic units of computation called *micro-operations* (micro-ops or μops). All that the microarchitecture knows about the state of a program's execution, and the only way it can change its machine state, is through the manipulation of these μops.

The P6 microarchitecture is very deeply pipelined, relative to competitive designs of its era. This deep pipeline is implemented as several short pipe segments connected by queues. This approach affords a much higher clock rate, and the negative effects of deep pipelining are ameliorated by an advanced branch predictor, very fast high-bandwidth access to the L2 cache, and a much higher clock rate (for a given semiconductor process technology).

This microarchitecture is a speculative, out-of-order engine. Any engine that speculates can also misspeculate and must provide means to detect and recover from that condition. To ensure that this fundamental microarchitecture feature would be implemented as error-free as humanly possible, we designed the recovery mechanism to be extremely simple. Taking advantage of that simplicity, and the fact that this mechanism would be very heavily validated, we mapped the machine's event-handling events (faults, traps, interrupts, breakpoints) onto the same set of protocols and mechanisms.

The front-side bus is a change from Intel's Pentium processor family. It is transaction-oriented and designed for high-performance multiprocessing systems. Figure 7.2 shows how up to four Pentium Pro processors can be connected to a single shared bus. The chipset provides for main memory, through the data

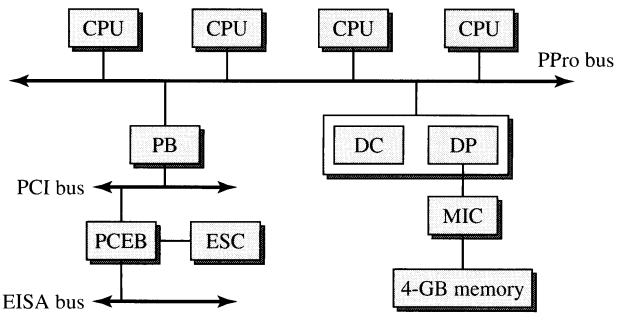


Figure 7.2
P6 Pentium Pro System Block Diagram.

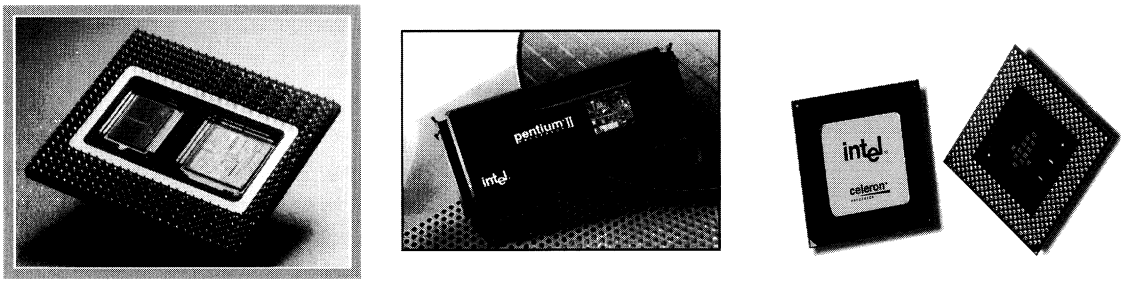


Figure 7.3
P6 Product Packaging.

path (DP) and data controller (DC) parts, and then through the memory interface controller (MIC) chip. I/O is provided via the industry standard PCI bus, with a bridge to the older Extended Industry Standard Architecture (EISA) standard bus.

Various proliferations of the P6 had chipsets and platform designs that were optimized for multiple market segments including the (1) high-volume, (2) workstation, (3) server, and (4) mobile market segments. These differed mainly in the amount and type of memory that could be accommodated, the number of CPUs that could be supported in a single platform, and L2 cache design and placement. The Pentium II processor does not use the two-die-in-a-package approach of the original Pentium Pro; that approach yielded a very fast system product but was expensive to manufacture due to the special ceramic dual-cavity package and the unusual manufacturing steps required. P6-based CPUs were packaged in several formats (see Figure 7.3):

- Slot 1 brought the P6 microarchitecture to volume price points, by combining one P6 CPU with two commodity cache RAMs and a tag chip on one FR4 fiberglass substrate cartridge. This substrate has all its electrical contacts contained in one edge connector, and a heat sink attached to one side of the packaged substrate. Slot 1's L2 cache runs at one-half the clock frequency of the CPU.
- Slot 2 is physically larger than Slot 1, to allow up to four custom SRAMs to form the very large caches required by the high-performance workstation and server markets. Slot 2 cartridges are carefully designed so that, despite the higher number of loads on the L2 bus, they can access the large L2 cache at the full clock frequency of the CPU.
- With improved silicon process technology, in 1998 Intel returned to pin-grid-array packaging on the Celeron processor, with the L2 caches contained on the CPU die itself. This obviated the need for the Pentium Pro's two-die-in-a-package or the Slot 1/Slot 2 cartridges.

7.1.1 Basics of the P6 Microarchitecture

In subsequent sections, the operation of the various components of the microarchitecture will be examined. But first, it may be helpful to consider the overall machine organization at a higher level.

A useful way to view the P6 microarchitecture is as a dataflow engine, fed by an aggressive front end, constrained by implementation and code compatibility. It is not difficult to design microarchitectures that are capable of expressing instruction-level parallelism; adding multiple execution units is trivial. Keeping those execution units gainfully employed is what is hard.

The P6 solution to this problem is to

- Extract useful work via deep speculation on the front end (instruction cache, decoder, and register renaming).
- Provide enough temporary storage that a lot of work can be "kept in the air."
- Allow instructions that are ready to execute to pass others that are not (in the out-of-order middle section).
- Include enough memory bandwidth to keep up with all the work in progress.

When speculation is proceeding down the right path, the μops generated in the front end flow smoothly into the reservation station (RS), execute when all their data operands have become available (often in an order other than that implied by the source program), take their place in the retirement line in the reorder buffer (ROB), and retire when it is their turn.

Micro-ops carry along with them all the information required for their scheduling, dispatch, execution, and retirement. Micro-ops have two source references, one destination, and an operation-type field. These logical references are renamed in the register alias table (RAT) to physical registers residing in the ROB.

When the inevitable misprediction occurs,[2] a very simple protocol is exercised within the out-of-order core. This protocol ensures that the out-of-order core flushes the speculative state that is now known to be bogus, while keeping any other work that is not yet known to be good or bogus. This same protocol directs the front end to drop what it was doing and start over at the mispredicted target's correct address.

Memory operations are a special category of μop. Because the IA32[3] instruction set architecture has so few registers, IA32 programs must access memory frequently. This means that the dependency chains that characterize a program generally start with a memory load, and this in turn means that it is important that loads be speculative. (If they were not, all the rest of the speculative engine would be starved while waiting for loads to go in order.) But not all loads can be speculative; consider a load in a memory-mapped I/O system where the load has a nonrecoverable side effect. Section 7.6 will cover some of these special cases. Stores are never speculative, there being no way to "put back the old data" if a misspeculated store were later found to have been in error. However, for performance reasons, store data can be forwarded from the store buffer (SB), before the data have actually

[2]For instance, on the first encounter with a branch, the branch predictor does not "know" there is a branch at all, much less which way the branch might go.

[3]This chapter uses IA32 to refer to the standard 32-bit Intel Architecture, as embodied in processors such as the Intel 486, or the Pentium, Pentium II, Pentium III, and Pentium 4 processors.

appeared in any caches or memory, to allow dependent loads (and their progeny) to proceed. This is closely analogous to a writeback cache, where data can be loaded from a cache that has not yet written its data to main memory.

Because of IA32 coding semantics, it is important to carefully control the transfer of information from the out-of-order, speculative engine to the permanent machine state that is saved and restored in program context switches. We call this "retirement." Essentially, all the machine's activity up until this point can be undone. Retirement is the act of irrevocably committing changes to a program's state. The P6 microarchitecture can retire up to three μops per clock cycle, and therefore can retire as many as three IA32 instructions' worth of changes to the permanent state. (If more than three ops are needed to express a given IA32 instruction, the retirement process makes sure the necessary all-or-none atomicity is obeyed.)

7.2 Pipelining

We will examine the individual elements of the P6 micro-architecture in this chapter, but before we look at the pieces, it may help to see how they all fit together. The pipeline diagram of Figure 7.4 may help put the pieces into perspective. The first thing to note about the figure is that it appears to show many separate pipelines, rather than one single pipeline. This is intentional; it reflects both the philosophy and the design of the microarchitecture.

The pipeline segments are in three clusters. First is the in-order front end, second is the out-of-order core, and third is the retirement. For reasons that should become clear, it is essential that these pipeline segments be separable operationally. For example, when recovering from a mispredicted branch, the front end of the machine will immediately flush the bogus information it had been processing from the mispredicted target address and will refetch the corrected stream from the corrected branch target, and all the while the out-of-order core continues working on previously fetched instructions (up until the mispredicted branch).

It is also important to separate the overall pipeline into independent segments so that when the various queues happen to fill up, and thus require their suppliers to stall until the tables have drained, only as little of the overall machine as necessary stalls, not the entire machine.

7.2.1 In-Order Front-End Pipeline

The first stage (pipe stage 11[4]) of the in-order front-end pipeline is used by the branch target buffer (BTB) to generate a pointer for the instruction cache (I-cache) to use, in accessing what we hope will be the right set of instruction bytes. Remember that the machine is always speculating, and this guess can be wrong; if it is, the error will be recognized at one of several places in the machine and a misprediction recovery sequence will be initiated at that time.

The second pipe stage in the in-order pipe (stage 12) initiates an I-cache fetch at the address that the BTB generated in pipe stage 11. The third pipe stage (stage 13)

[4]Why is the first stage numbered 11 instead of 1? We don't remember. We think it was arbitrary.

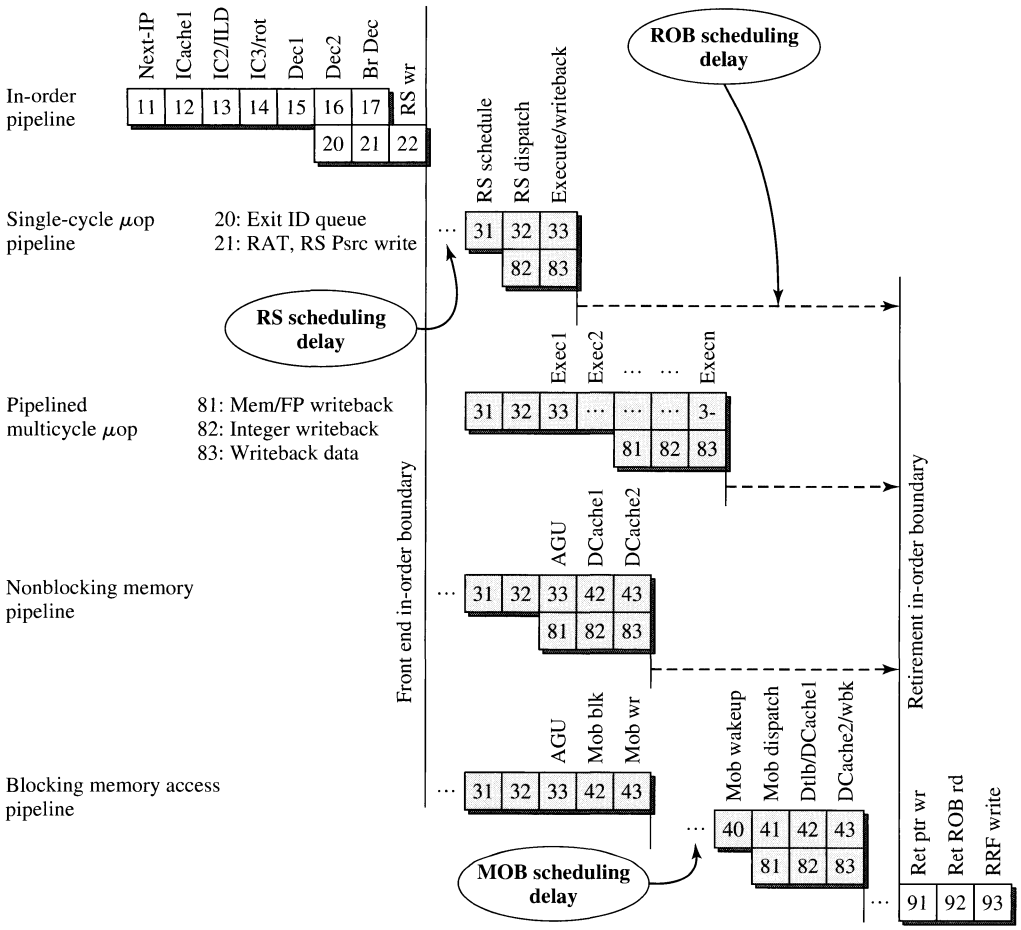


Figure 7.4
P6 Pipelining.

continues the I-cache access. The fourth pipe stage (stage 14) completes the I-cache fetch and transfers the newly fetched cache line to the instruction decoder (ID) so it can commence decoding.

Pipe stages 15 and 16 are used by the ID to align the instruction bytes, identify the ends of up to three IA32 instructions, and break these instructions down into sequences of their constituent μops.

Pipe stage 17 is the stage where part of the ID can detect branches in the instructions it has just decoded. Under certain conditions (e.g., an unpredicted but unconditional branch), the ID can notice that a branch went unpredicted by the BTB (probably because the BTB had never seen that particular branch before) and can flush the in-order pipe and refetch from the branch target, without having to wait until the branch actually tries to retire many cycles in the future.

Pipe stage 17 is synonymous with pipe stage 21,[5] which is the rename stage. Here the register alias table (RAT) renames μop destination/source linkages to a large set of physical registers in the reorder buffer. In pipe stage 22, the RAT transfers the μops (three at a time, since the P6 microarchitecture is an order-3 superscalar) to the out-of-order core. Pipe stage 22 marks the transition from the in-order section to the out-of-order section of the machine. The μops making this transition are written into both the reservation station (where they will wait until they can execute in the appropriate execution unit) and the reorder buffer (where they "take a place in line," so that eventually they can commit their changes to the permanent machine state in the order implied by the original user program).

7.2.2 Out-of-Order Core Pipeline

Once the in-order front end has written a new set of (up to) three μops into the reservation station, these μops become possible candidates for execution. The RS takes several factors into account when deciding which μops are ready for execution: The μop must have all its operands available; the execution unit (EU) needed by that μop must be available; a writeback bus must be ready in the cycle in which the EU will complete that μop's execution; and the RS must not have other μops that it thinks (for whatever reason) are more important to overall performance than the μop under discussion. Remember that this is the out-of-order core of the machine. The RS does not know, and does not care about, the original program order. It only observes data dependences and tries to maximize overall performance while doing so.

One implication of this is that any given μop can wait from zero to dozens or even hundreds of clock cycles after having been written into the RS. That is the point of the RS scheduling delay label on Figure 7.4. The scheduling delay can be as low as zero, if the machine is recovering from a mispredicted branch, and these are the first "known good" μops from the new instruction stream. (There would be no point to writing the μops into the RS, only to have the RS "discover" that they are data-ready two cycles later. The first μop to issue from the in-order section is guaranteed to be dependent on no speculative state, because there is no more speculative state at that point!)

Normally, however, the μops do get written into the RS, and there they stay until the RS notices that they are data-ready (and the other constraints previously listed are satisfied). It takes the RS two cycles to notice, and then *dispatch,* μops to the execution units. These are pipe stages 31 and 32.

Simple, single-cycle-execution μops such as logical operators or simple arithmetic operations execute in pipe stage 33. More complex operations, such as integer multiply, or floating-point operations, take as many cycles as needed.

One-cycle operators provide their results to the writeback bus at the end of pipe stage 33. The writeback busses are a shared resource, managed by the RS, so the RS must ensure that there will be a writeback bus available in some future cycle for a given μop at the time the μop is dispatched. Writeback bus scheduling occurs in pipe stage 82, with the writeback itself in the execution cycle, 83 (which is synonymous with pipe stage 33).

[5]Why do some pipe stages have more than one name? Because the pipe segments are independent. Sometimes part of one pipe segment lines up with one stage of a different pipe segment, and sometimes with another.

Memory operations are a bit more complicated. All memory operations must first generate the effective address, per the usual IA32 methods of combining segment base, offset, base, and index. The μop that generates a memory address executes in the address generation unit (AGU) in pipe stage 33. The data cache (DCache) is accessed in the next two cycles, pipe stages 42 and 43. If the access is a cache hit, the accessed data return to the RS and become available as a source to other μops.

If the DCache reference was a miss, the machine tries the L2 cache. If that misses, the load μop is suspended (no sense trying it again any time soon; we must refill the miss from main memory, which is a slow operation). The memory ordering buffer (MOB) maintains the list of active memory operations and will keep this load μop suspended until its cache line refill has arrived. This conserves cache access bandwidth for other μop sequences that may be independent of the suspended μop; these other μops can go around the suspended load and continue making forward progress. Pipe stage 40 is used by the MOB to identify and "wake up" the suspended load. Pipe stage 41 re-dispatches the load to the DCache, and (as earlier) pipe stages 42 and 43 are used by the DCache in accessing the line. This MOB scheduling delay is labeled in Figure 7.4.

7.2.3 Retirement Pipeline

Retirement is the act of transferring the speculative state into the permanent, irrevocable architectural machine state. For instance, the speculative out-of-order core may have a μop that wrote 0xFA as its instruction result into the appropriate field of ROB entry 14. Eventually, if no mispredicted branch is found in the interim, it will become that μop's turn to retire next, when it has become the oldest μop in the machine. At that point, the μop's original intention (to write 0xFA into, e.g., the EAX register) is realized by transferring the 0xFA data in ROB Slot 14 to the retirement register file's (RRF) EAX register.

There are several complicating factors to this simple idea. First, what the ROB is actually retiring should not be viewed as just a sequence of μops, but rather a series of IA32 instructions. Since it is architecturally illegal to retire only part of an IA32 instruction, then either all μops comprising an IA32 instruction retire, or none do. This atomicity requirement generally demands that the partially modified architectural state never be visible to the world outside the processor. So part of what the ROB must do is to detect the beginning and end of a given IA32 instruction and to make sure the atomicity rule is strictly obeyed. The ROB does this by observing some marks left on the μops by the instruction decoder (ID): some μops are marked as the first μop in an IA32 instruction, and others are marked as last. (Obviously, others are not marked at all, implying they are somewhere in the middle of an IA32 μop sequence.)

While retiring a sequence of μops that comprise an IA32 instruction, no external events can be handled. Those simply have to wait, just as they do in previous generations of the Intel Architecture (i486 and Pentium processors, for instance). But between two IA32 instructions, the machine must be capable of taking interrupts, breakpoints, traps, handling faults, and so on. The reorder buffer makes sure that these events are only possible at the right times and that multiple pending events are serviced in the priority order implied by the Intel instruction set architecture.

The processor must have the capability to stop a microcode flow partway through, switch to a microcode assist routine, perform some number of μops, and then resume the flow at the point of interruption, however. In that sense, an instruction may be considered to be partially executed at the time the trap is taken. The first part of the instruction cannot be discarded and restarted, because this would prevent forward progress. This kind of behavior occurs for TLB updates, some kinds of floating-point assists, and more.

The reorder buffer is implemented as a circular list of μops, with one retirement pointer and one new-entry pointer. The reorder buffer writes the results of a just-executed μop into its array in pipe stage 22. The μop results from the ROB are read in pipe stage 82 and committed to the permanent machine state in the RRF in pipe stage 93.

7.3 The In-Order Front End

The primary responsibility of the front end is to keep the execution engine full of useful work to do. On every clock cycle, the front end makes a new guess as to the best I-cache address from which to fetch a new line, and it sends the cache line guessed from the last clock to the decoders so they can get started. This guess can, of course, be discovered to have been incorrect, whereupon the front end will later be redirected to where the fetch really should have been from. A substantial performance penalty occurs when a mispredicted branch is discovered, and a key challenge for a microarchitecture such as this one is to ensure that branches are predicted correctly as often as possible, and to minimize the recovery time when they are found to have been mispredicted. This will be discussed in greater detail shortly.

The decoders convert up to three IA instructions into their corresponding μops (or μop flows, if the IA instructions are complex enough) and push these into a queue. A register renamer assigns new physical register designators to the source and destination references of these μops, and from there the μops issue to the out-of-order core of the machine.

As implied by the pipelining diagram in Figure 7.4, the in-order front end of the P6 microarchitecture runs independently from the rest of the machine. When a mispredicted branch is detected in the out-of-order core [in the jump execution unit (JEU)], the out-of-order core continues to retire μops older than the mispredicted branch μop, but flushes everything younger. Refer to Figure 7.5. Meanwhile, the front end is immediately flushed and begins to refetch and decode instructions starting at the correct branch target (supplied by the JEU). To simplify the handoff between the in-order front end and the out-of-order core, the new μops from the corrected branch target are strictly quarantined from whatever μops remain in the out-of-order core, until the out-of-order section has drained. Statistically, the out-of-order core will usually have drained by the time the new FE2 μops get through the in-order front end.

7.3.1 Instruction Cache and ITLB

The on-chip instruction cache (I-cache) performs the usual function of serving as a repository of recently used instructions. Figure 7.6 shows the four pipe stages of the instruction fetch unit (IFU). In its first pipe stage (pipe stage 11), the IFU

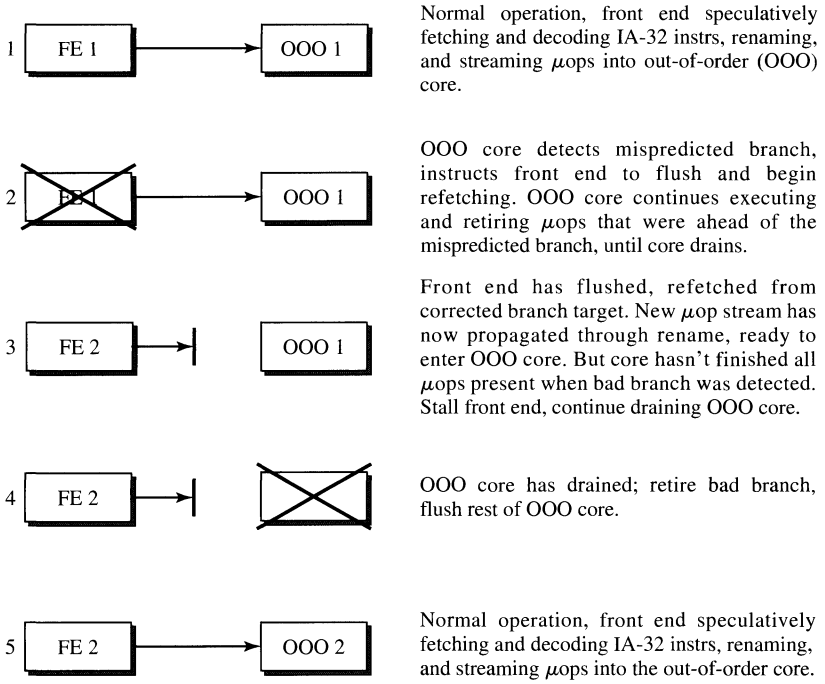
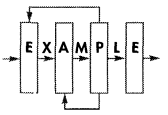


Figure 7.5
Branch Misspeculation Recovery.



selects the address of the next cache access. This address is selected from a number of competing fetch requests that arrive at the IFU from (among others) the BTB and branch address calculator (BAC). The IFU picks the request with the highest priority and schedules it for service by the second pipe stage (pipe stage 12). In the second pipe stage, the IFU accesses its many caches and buffers using the fetch address selected by the previous stage. Among the caches and buffers accessed are the instruction cache and the instruction streaming buffer. If there is a hit in any of these caches or buffers, instructions are read out and forwarded to the third pipe stage. If there is a miss in all these buffers, an external fetch is initiated by sending a request to the external bus logic (EBL).

Two other caches are also accessed in pipe stage 12 using the same fetch address: the ITLB in the IFU and the branch target buffer (BTB). The ITLB access obtains the physical address and memory type of the fetch, and the BTB access obtains a branch prediction. The BTB takes two cycles to complete one access. In the third pipe stage (13), the IFU marks the instructions received from the previous stage (12). Marking is the process of determining instruction boundaries. Additional marks for predicted branches are delivered by the BTB by the end of pipe stage 13. Finally, in the fourth pipe stage (14), the instructions and their marks are written into the instruction buffer and optionally steered to the ID, if the instruction buffer is empty.

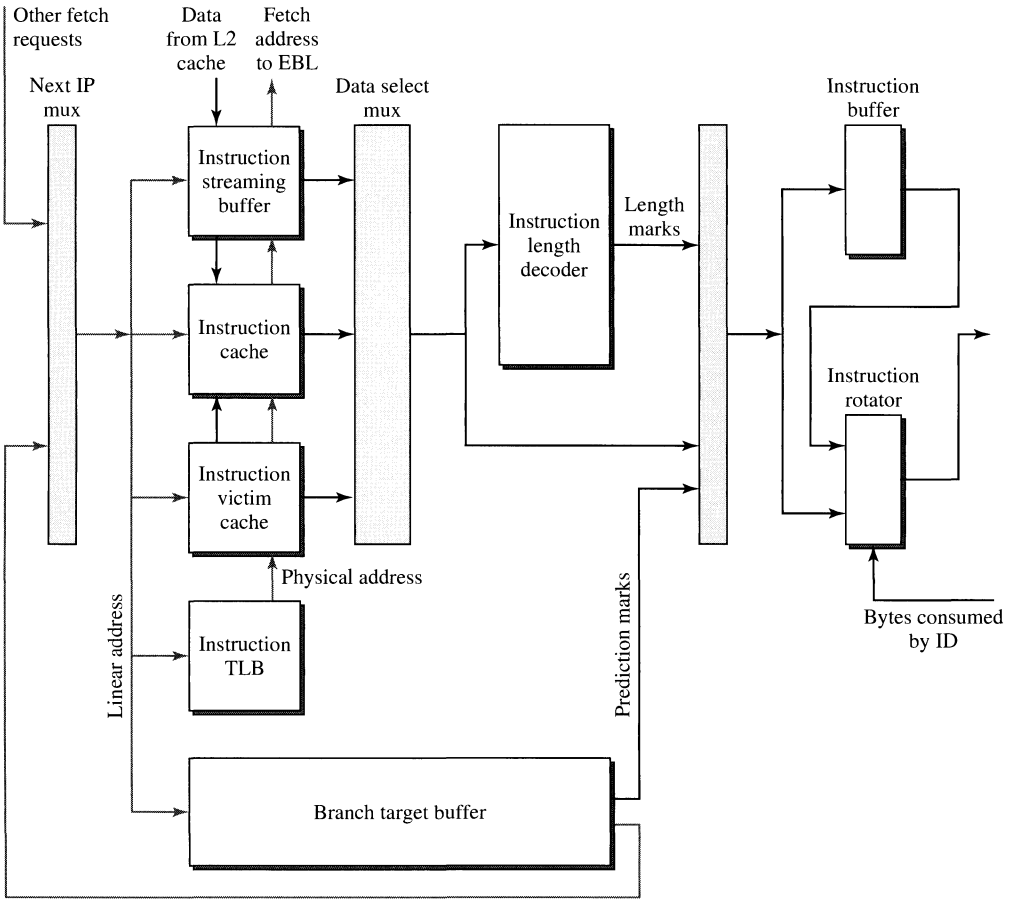


Figure 7.6
Front-End Pipe Staging.

The fetch address selected by pipe stage 11 for service in pipe stage 12 is a *linear* address, not a *virtual* or *physical* address. In fact, the IFU is oblivious to virtual addresses and indeed all segmentation. This allows the IFU to ignore segment boundaries while delaying the checking of segmentation-related violations to units downstream from the IFU in the P6 pipeline. The IFU does, however, deal with paging. When paging is turned off, the linear fetch address selected by pipe stage 11 is identical to the physical address and is directly used to search all caches and buffers in pipe stage 12. However, when paging is turned on, the linear address must be translated by the ITLB into a physical address. The virtual to linear to physical sequence is shown in Figure 7.7.

The IFU caches and buffers that require a physical address (actually, untranslated bits, with a match on physical address) for access are the instruction cache, the instruction streaming buffer, and the instruction victim cache. The branch target

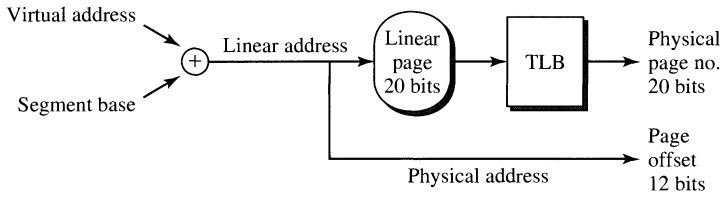


Figure 7.7
Virtual to Linear to Physical Addresses.

buffer is accessed using the linear fetch address. A block diagram of the front end, and the BTB's place in it, is shown in Figure 7.6.

7.3.2 Branch Prediction

The branch target buffer has two major functions: to predict branch direction and to predict branch targets. The BTB must operate early in the instruction pipeline to prevent the machine from executing down a wrong program stream. (In a speculative engine such as the P6, executing down the wrong stream is, of course, a performance issue, not a correctness issue. The machine will always execute correctly, the only question is how quickly.)

The branch *decision* (taken or not taken) is known when the jump execution unit (JEU) resolves the branch (pipe stage 33). Cycles would be wasted were the machine to wait until the branch is resolved to start fetching the instructions after the branch. To avoid this delay, the BTB predicts the decision of the branch as the IFU fetches it (pipe stage 12). This prediction can be wrong. The machine is able to detect this case and recover. All predictions made by the BTB are verified downstream by either the branch address calculator (pipe stage 17) or the JEU (pipe stage 33).

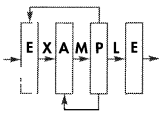
The BTB takes the starting linear address of the instructions being fetched and produces the prediction and target address of the branch instructions being fetched. This information (prediction and target address) is sent to the IFU, and the next cache line fetch will be redirected if a branch is predicted taken. A branch's entry in the BTB is updated or allocated in the BTB cache only when the JEU resolves it. A branch update is sometimes too late to help the next instance of the branch in the instruction stream. To overcome this delayed update problem, branches are also speculatively updated (in a separately maintained BTB state) when the BTB makes a prediction (pipe stage 13).

7.3.2.1 Branch Prediction Algorithm. Dynamic branch prediction in the P6 BTB is related to the two-level adaptive training algorithm proposed by Yeh and Patt [1991]. This algorithm uses two levels of branch history information to make predictions. The first level is the history of the branches. The second level is the branch behavior for a specific pattern of branch history. For each branch, the BTB keeps N bits of "real" branch history (i.e., the branch decision for the last N dynamic occurrences). This history is called the branch history register (BHR).

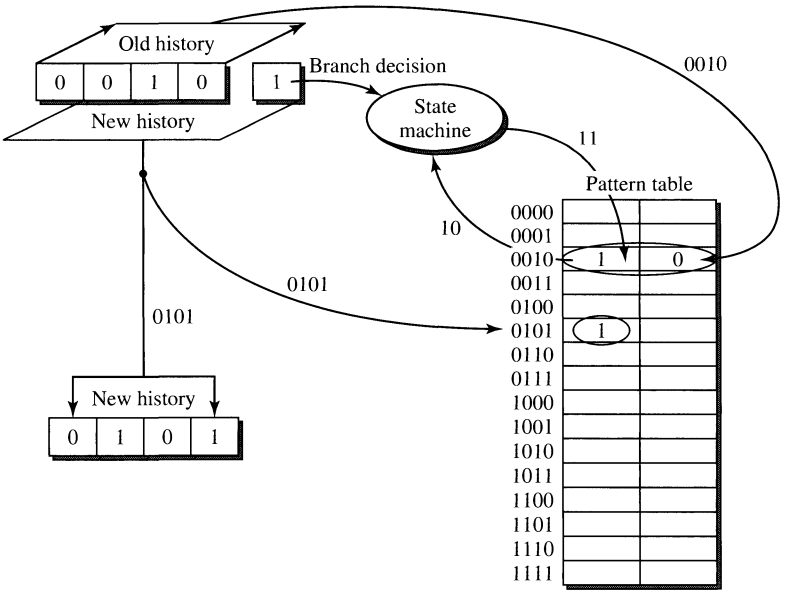
The pattern in the BHR indexes into a 2^N entry table of states, the pattern table (PT). The state for a given pattern is used to predict how the branch will act the next time it is seen. The states in the pattern table are updated using Lee and Smith's [1984] saturating up-down counter.

The BTB uses a 4-bit semilocal pattern table per set. This means 4 bits of history are kept for each entry, and all entries in a set use the same pattern table (the four branches in a set share the same pattern table). This has equivalent performance to a 10-bit global table, with less hardware complexity and a smaller die area. A speculative copy of the BHR is updated in pipe stage 13, and the real one is updated upon branch resolution in pipe stage 83. But the pattern table is updated only for conditional branches, as they are computed in the jump execution unit.

To obtain the prediction of a branch, the *decision* of the branch (taken or not taken) is shifted into the old history pattern of the branch, and this field is used to index the pattern table. The most significant bit of the state in the pattern table indicates the prediction used the next time it is seen. The old state indexed by the old history pattern is updated using the Lee and Smith state machine.



An example of how the algorithm works is shown in Figure 7.8. The history of the entry to be updated is *0010,* and the branch *decision* was taken. The new



Two processes occur in parallel:

1. The new history is used to access the pattern table to get the new prediction bit. This prediction bit is written into the BTB in the next phase.

2. The old history is used to access the pattern table to get the state that has to be updated. The updated state is then written back to the pattern table.

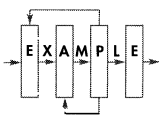


Figure 7.8
Yeh's Algorithm.

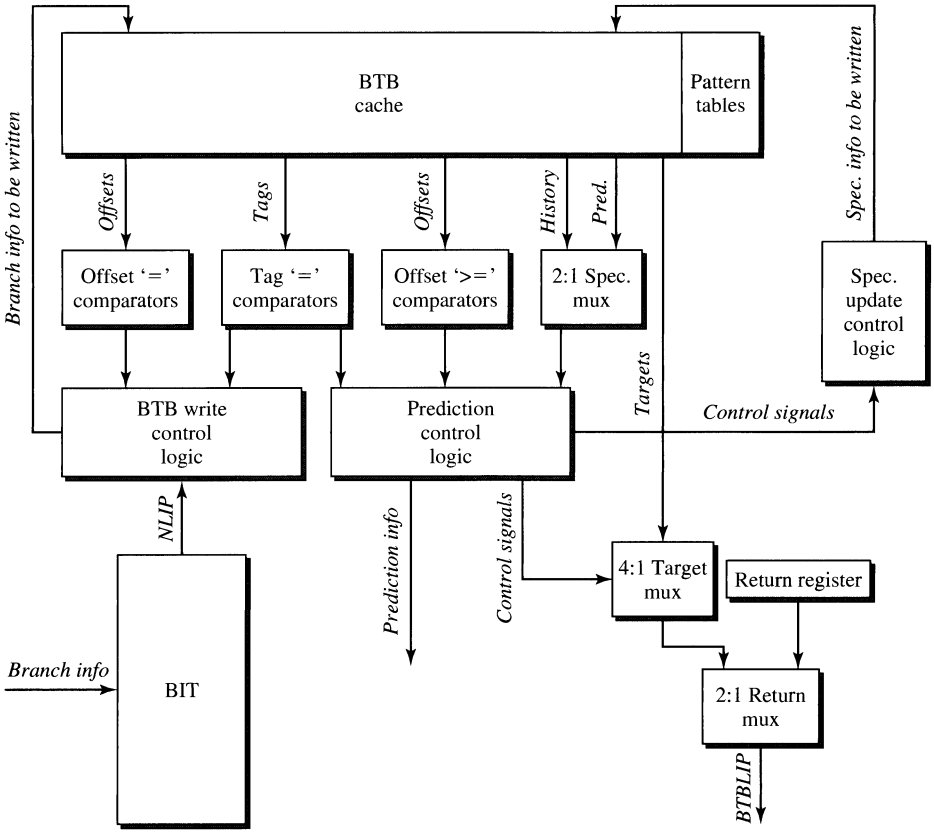


Figure 7.9
Simplified BTB Block Diagram.

history *0101* is used to index into the pattern table and the new prediction *1* for the branch (the most significant bit of the state) is obtained. The old history *0010* is used to index the pattern table to get the old state *10*. The old state *10* is sent to the state machine along with the branch *decision,* and the new state *11* is written back into the pattern table.

The BTB also maintains a 16-deep return stack buffer to help predict returns. For circuit speed reasons, BTB accesses require two clocks. This causes predicted-taken branches to insert a one-clock fetch "bubble" into the front end. The double-buffered fetch lines into the instruction decoder and the ID's output queue help eliminate most of these bubbles in normal execution. A block diagram of the BTB is shown in Figure 7.9.

7.3.3 Instruction Decoder

The first stage of the ID is known as the instruction steering block (ISB) and is responsible for latching instruction bytes from the IFU, picking off individual instructions in order, and steering them to each of the three decoders. The ISB

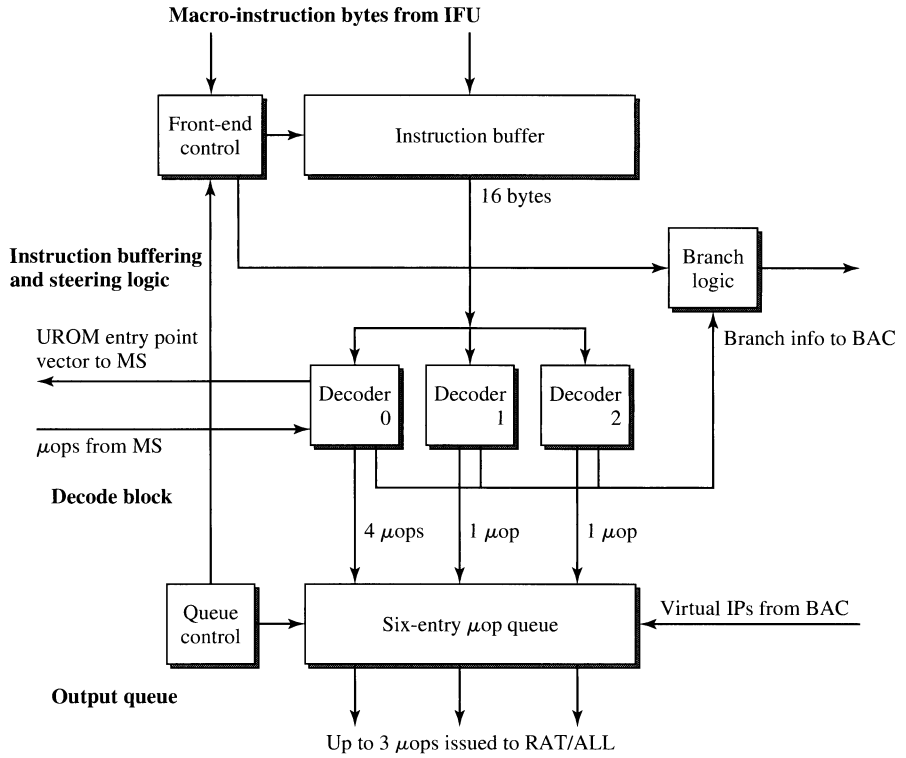


Figure 7.10
ID Block Diagram.

quickly detects how many instructions are decoded each clock to make a fast determination of whether or not the instruction buffer is empty. If empty, it enables the latch to receive more instruction bytes from the IFU (refer to Figure 7.10).

There are other miscellaneous functions performed by this logic as the "front end" of the ID. It detects and generates the correct sequencing of predicted branches. In addition, the ID front end generates the valid bits for the μops produced by the decode PLAs and detects stall conditions in the ID.

Next, the instruction buffer loads 16 bytes at a time from the IFU. These data are aligned such that the first byte in the buffer is guaranteed to be the first byte of a complete instruction. The average instruction length is 2.7 to 3.1 bytes. This means that on average five to six complete instructions will be loaded into the buffer. Loading a new batch of instruction bytes is enabled under any of the following conditions:

- A processor front-end reset occurs due to branch misprediction.
- All complete instructions currently in the buffer are successfully decoded.
- A BTB predicted-taken branch is successfully decoded in any of the three decoders.

Steering three properly aligned macro-instructions to three decoders in one clock is complicated due to the variable length of IA32 instructions. Even determining the length of one instruction itself is not straightforward, as the first bytes of an instruction must be decoded in order to interpret the bytes that follow. Since the process of steering three variable-length instructions is inherently serial, it is helpful to know beforehand the location of each macro-instruction's boundaries. The instruction length decoder (ILD), which resides in the IFU, performs this pre-decode function. It scans the bytes of the macro-instruction stream locating instruction boundaries and marking the first opcode and end-bytes of each. In addition, the IFU marks the bytes to indicate BTB branch predictions and code breakpoints.

There may be from 1 to 16 instructions loaded into the instruction buffer during each load. Each of the first three instructions is steered to one of three decoders. If the instruction buffer does not contain three complete instructions, then as many as possible are steered to the decoders. The steering logic uses the *first opcode markers* to align and steer the instructions in parallel.

Since there may be up to 16 instructions in the instruction buffer, it may take several clocks to decode all of them. The starting byte location of the three instructions steered in a given clock may lie anywhere in the buffer. Hardware aligns three instructions and steers them to the three decoders.

Even though three instructions may be steered to the decoders in one cycle, all three may not get successfully decoded. When an instruction is not successfully decoded, then that specific decoder is flushed and all μops resulting from that decode attempt will be invalidated. It can take multiple cycles to consume (decode) all the instructions in the buffer. The following situations result in the invalidation of μops and the resteering of their corresponding macro-instructions to another decoder during a subsequent cycle:

- If a complex macro-instruction is detected on decoder 0, requiring assistance from the microcode sequencer (MS) microcode read-only memory (UROM), then the μops from all subsequent decoders are invalidated. When the MS has completed sequencing the rest of the *flow,*[6] subsequent macro-instructions are decoded.
- If a macro-instruction is steered to a limited-functionality decoder (which is not able to decode it), then the macro-instructions and all subsequent macro-instructions are resteered to other decoders in the next cycle. All μops produced by this and subsequent decoders are invalidated.
- If a branch is encountered, then all μops produced by subsequent decoders are invalidated. Only one branch can be decoded per cycle.

Note that the number of macro-instructions that can be decoded simultaneously does not directly relate to the number of μops that the ID can issue because the decoder queue can store μops and issue them later.

[6] *Flow* refers to a sequence of μops emitted by the microcode ROM. Such sequences are commonly used by the microcode to express IA32 instructions, or microarchitectural housekeeping.

7.3.3.1 Complex Instructions. *Complex instructions* are those requiring the MS to sequence μops from the UROM. Only decoder 0 can handle these instructions. There are two ways in which the MS microcode will be invoked:

- Long flows where decoder 0 generates up to the first four μops of the flow and the MS sequences the remaining μops.
- Low-performance instructions where decoder 0 issues no μops but transfers control to the MS to sequence from the UROM.

Decoders 1 and 2 cannot decode complex instructions, a design tradeoff that reflects both the silicon expense of implementation as well as the statistics of dynamic IA32 code execution. Complex instructions will be resteered to the next-lower available decoder during subsequent clocks until they reach decoder 0. The MS receives a UROM entry point vector from decoder 0 and begins sequencing μops until the end of the microcode flow is encountered.

7.3.3.2 Decoder Branch Prediction. When the macro-instruction buffer is loaded from the IFU, the ID looks at the prediction byte marks to see if there are any predicted-taken branches (predicted dynamically by the BTB) in the set of complete instructions in the buffer. A proper prediction will be found on the byte corresponding to the last byte of a branch instruction. If a predicted-taken branch is found anywhere in the buffer, the ID indicates to the IFU that the ID has "grabbed" the predicted branch. The IFU can now let the 16-byte block, fetched at the target address of the branch, enter the buffer at the input of its rotator. The rotator then aligns the instruction at the branch target so that it will be the next instruction loaded into the ID's instruction buffer. The ID may decode the predicted branch immediately, or it may take several cycles (due to decoding all the instructions ahead of it). After the branch is finally decoded, the ID will latch the instructions at the branch target in the next clock cycle.

Static branch prediction (prediction made without reference to run-time history) is made by the branch address calculator (BAC). If the BAC decides to take a branch, it gives the IFU a target IP where the IFU should start fetching instructions. The ID must not, of course, issue any μops of instructions after the branch, until it decodes the branch target instruction. The BAC will make a static branch prediction under two conditions: It sees an absolute branch that the BTB did not make a prediction on, or it sees a conditional branch with a target address whose direction is "backward" (which suggests it is the return edge of a loop).

7.3.4 Register Alias Table

The register alias table (RAT) provides register renaming of integer and floating-point registers and flags to make available a larger register set than is explicitly provided in the Intel Architecture. As μops are presented to the RAT, their logical sources and destination are mapped to the corresponding physical ROB addresses where the data are found. The mapping arrays are then updated with new physical destination addresses granted by the allocator for each new μop.

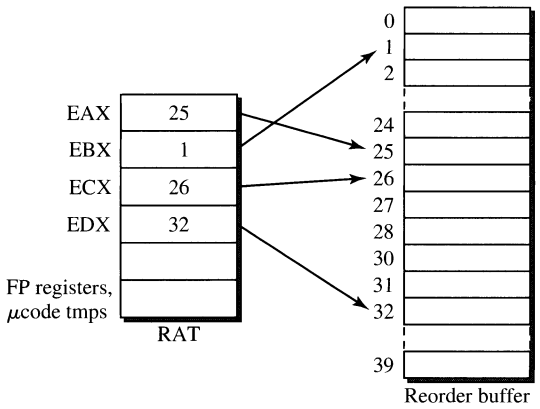


Figure 7.11
Basic RAT Register Renaming.

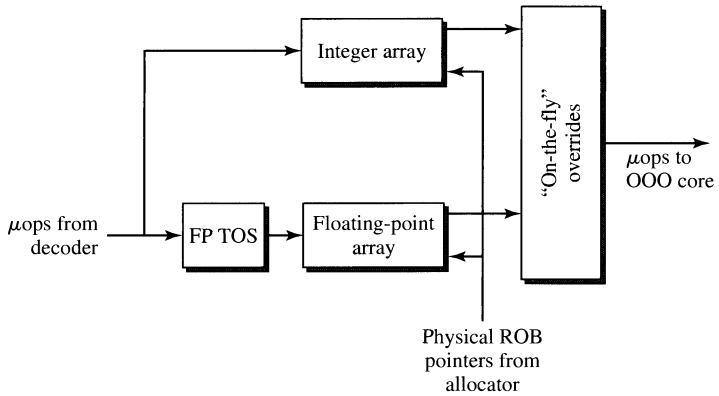
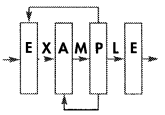


Figure 7.12
RAT Block Diagram.

Refer to Figures 7.11 and 7.12. In each clock cycle, the RAT must look up the physical ROB locations corresponding to the logical source references of each μop. These physical designators become part of the μop's overall state and travel with the μop from this point on. Any machine state that will be modified by the μop (its "destination" reference) is also renamed, via information provided by the allocator. This physical destination reference becomes part of the μop's overall state and is written into the RAT for use by subsequent μops whose sources refer to the same logical destination. Because the physical destination value is unique to each μop, it is used as an identifier for the μop throughout the out-of-order section. All checks and references to a μop are performed by using this physical destination (PDst) as its name.



Since the P6 is a superscalar design, multiple μops must be renamed in a given clock cycle. If there is a true dependency chain through these three μops, say,

```
μop0: ADD EAX, EBX; src 1 = EBX, src 2 = EAX, dst = EAX
μop1: ADD EAX, ECX;
μop2: ADD EAX, EDX;
```

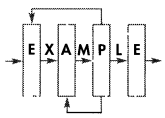
then the RAT must supply the renamed source locations "on the fly," via logic, rather than just looking up the destination, as it does for dependences tracked across clock cycles. Bypass logic will directly supply μop1's source register, src 2, EAX, to avoid having to wait for μop0's EAX destination to be written into the RAT and then read as μop1's src.

The state in the RAT is speculative, because the RAT is constantly updating its array entries per the μop destinations flowing by. When the inevitable branch misprediction occurs, the RAT must flush the bogus state it has collected and revert to logical-to-physical mappings that will work with the next set of μops. The P6's branch misprediction recovery scheme guarantees that the RAT will have to do no new renamings until the out-of-order core has flushed all its bogus misspeculated state. That is useful, because it means that register references will now reside in the retirement register file until new speculative μops begin to appear. Therefore, to recover from a branch misprediction, all the RAT needs to do is to revert all its integer pointers to point directly to their counterparts in the RRF.

7.3.4.1 RAT Implementation Details. The IA32 architecture allows partial-width reads and writes to the general-purpose integer registers (i.e., EAX, AX, AH, AL), which presents a problem for register renaming. The problem occurs when a partial-width write is followed by a larger-width read. In this case, the data required by the larger-width read must be an assimilation of multiple previous writes to different pieces of the register.

The P6 solution to the problem requires that the RAT remember the width of each integer array entry. This is done by maintaining a 2-bit *size* field for each entry in the integer low and high banks. The 2-bit encoding will distinguish between the three register write sizes of 32, 16, and 8 bits. The RAT uses the register size information to determine if a larger register value is needed than has previously been written. In this case, the RAT must generate a partial-write stall.

Another case, common in 16-bit code, is the *independent* use of the 8-bit registers. If only one alias were maintained for all three sizes of an integer register access, then independent use of the 8-bit subsets of the registers would cause a tremendous number of false dependences. Take, for example, the following series of μops:




```
μop0: MOV AL,#DATA1
μop1: MOV AH,#DATA2
μop2: ADD AL,#DATA3
μop3: ADD AH,#DATA4
```

Micro-ops 0 and 1 move independent data into AL and AH. Micro-ops 3 and 4 source AL and AH for the addition. If only one alias were available for the "A" register, then μop1's pointer to AH would overwrite μop0's pointer to AL. Then when μop2 tried to read AL, the RAT would not know the correct pointer and would have to stall until μop1 retired. Then μop3's AH source would again be lost due to μop2's write to AL. The CPU would essentially be serialized, and performance would be diminished.

To prevent this, two integer register banks are maintained in the RAT. For 32-bit and 16-bit RAT accesses, data are *read* only from the low bank, but data are *written* into both banks simultaneously. For 8-bit RAT accesses, however, only the appropriate high or low bank is read or written, according to whether it was a high byte or low byte access. Thus, the high and low byte registers use different rename entries, and both can be renamed independently. Note that the high bank only has four array entries because four of the integer registers (namely, EBP, ESP, EDI, ESI) cannot have 8-bit accesses, per the Intel Architecture specification.

The RAT physical source (PSrc) designators point to locations in the ROB array where data may currently be found. Data do not actually appear in the ROB until after the μop generating the data has executed and written back on one of the write-back busses. Until execution writeback of a PSrc, the ROB entry contains junk.

Each RAT entry has an RRF bit to select one of two address spaces, the RRF or the ROB. If the RRF bit is set, then the data are found in the real register file; the physical address bits are set to the appropriate entry of the RRF. If the RRF bit is clear, then the data are found in the ROB, and the physical address points to the correct position in the ROB. The 6-bit physical address field can access any of the ROB entries. If the RRF bit is set, the entry points to the real register file; its physical address field contains the pointer to the appropriate RRF register. The busses are arranged such that the RRF can source data in the same way that the ROB can.

7.3.4.2 Basic RAT Operation. To rename logical sources (LSrc's), the six sources from the three ID-issued μops are used as the indices into the RAT's integer array. Each entry in the array has six read ports to allow all six LSrc's to each read any logical entry in the array.

After the read phase has been completed, the array must be updated with new physical destinations (PDst's) from the allocator associated with the destinations of the current μops being processed. Because of possible intracycle destination dependences, a priority write scheme is employed to guarantee that the correct PDst is written to each array destination.

The priority write mechanism gives priority in the following manner:

Highest:	Current μop2's physical destination
	Current μop1's physical destination
	Current μop0's physical destination
Lowest:	Any of the retiring μops physical destinations

Retirement is the act of removing a completed μop from the ROB and committing its state to the appropriate permanent architectural state in the machine. The ROB informs the RAT that the retiring μop's destination can no longer be found in the

reorder buffer but must (from now on) be taken from the real register file (RRF). If the retiring PDst is found in the array, the matching entry (or entries) is reset to point to the RRF.

The retirement mechanism requires the RAT to do three associative matches of each array PSrc against all three retirement pointers that are valid in the current cycle. For all matches found, the corresponding array entries are reset to point to the RRF. Retirement has lowest priority in the priority writeback mechanism; logically, retirement should happen before any new μops write back. Therefore, if any μops want to write back concurrently with a retirement reset, then the PDst writeback would happen last.

Resetting the floating-point register rename apparatus is more complicated, due to the Intel Architecture FP register stack organization. Special hardware is provided to remove the top-of-stack (TOS) offset from FP register references. In addition, a retirement FP RAT (RfRAT) table is maintained, which contains nonspeculative alias information for the floating-point stack registers. It is updated only upon μop retirement. Each RfRAT entry is 4 bits wide: a 1-bit retired stack valid and a 3-bit RRF pointer. In addition, the RfRAT maintains its own nonspeculative TOS pointer. The reason for the RfRAT's existence is to be able to recover from mispredicted branches and other events in the presence of the FXCH instruction.

The FXCH macro-op swaps the floating-point TOS register entry with any stack entry (including itself, oddly enough). FXCH could have been implemented as three MOV μops, using a temporary register. But the Pentium processor-optimized floating-point code uses FXCH extensively to arrange data for its dual execution units. Using three μops for the FXCH would be a heavy performance hit for the P6 processors on Pentium processor-optimized FP code, hence the motivation to implement FXCH as a single μop.

P6 processors handle the FXCH operation by having the FP part of the RAT (fRAT) merely swap its array pointers for the two source registers. This requires extra write ports in the fRAT but obviates having to swap 80+ bits of data between any two stack registers in the RRF. In addition, since the pointer swap operation would not require the resources of an execution unit, the FXCH is marked as "completed" in the ROB as soon as the ROB receives it from the RAT. So the FXCH effectively takes no RS resources and executes in zero cycles.

Because of any number of previous FXCH operations, the fRAT may speculatively swap any number of its entries before a mispredicted branch occurs. At this point, all instructions issued down this branch are stopped. Sometime later, a signal will be asserted by the ROB indicating that all μops up to and including the branching μop have retired. This means that all arrays in the CPU have been reset, and macroarchitectural state must be restored to the machine state existing at the time of the mispredicted branch. The trick is to be able to correctly undo the effects of the speculative FXCHs. The fRAT entries cannot simply be reset to constant RRF values, as integer rename references are, because any number of retired FXCHs may have occurred, and the fRAT must forevermore remember the retired FXCH mappings. This is the purpose of the retirement fRAT: to "know" what to reset the FP entries to when the front end must be flushed.

7.3.4.3 Integer Retirement Overrides. When a retiring μop's PDst is still being referenced in the RAT, then at retirement that RAT entry reverts to pointing into the retirement register file. This implies that the retirement of μops must take precedence over the table read. This operation is performed as a bypass *after* the table read in hardware. This way, the data read from the table will be overridden by the most current μop retirement information.

The integer retirement override mechanism requires doing an associative match of the integer arrays' PSrc entries against all retirement pointers that are valid in the current cycle. For all matches found, the corresponding array entries are reset to point to the RRF.

Retirement overrides must occur, because retiring PSrc's read from the RAT will no longer point to the correct data. The ROB array entries that are retiring during the current cycle cannot be referenced by any current μop (because the data will now be found in the RRF).

7.3.4.4 New PDst Overrides. Micro-op logical source references are used as indices into the RAT's multiported integer array, and physical sources are output by the array. These sources are then subject to retirement overrides. At this time, the RAT also receives newly allocated physical destinations (PDst's) from the allocator. Priority comparisons of logical sources and destinations from the ID are used to gate out either PSrc's from the integer array or PDst's from the allocator as the actual renamed μop physical sources. Notice that source 0 is never overridden because it has no previous μop in the cycle on which to be dependent. A block diagram of the RAT's override hardware is shown in Figure 7.13.

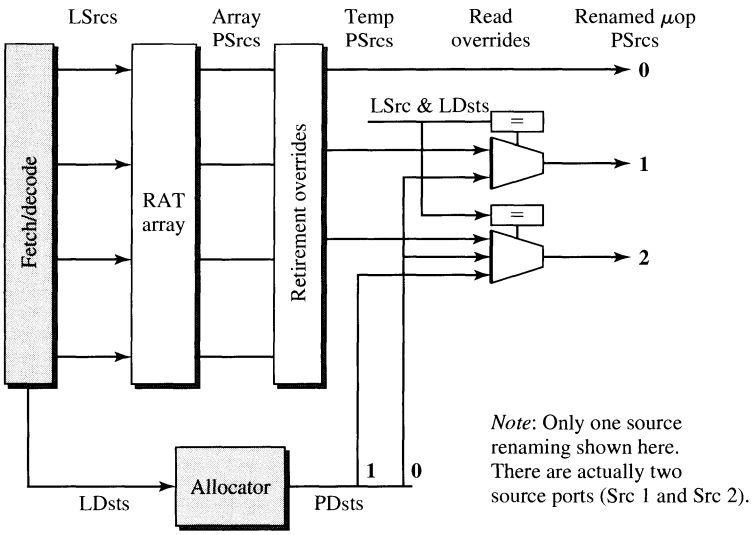


Figure 7.13
RAT New PDst Overrides.

Suppose that the following μops are being processed:

μop0: r1 + r3 → r3

μop1: r3 + r2 → r3

μop2: r3 + r4 → r5

Notice that a μop1 source relies on the destination reference of μop0. This means that the data required by μop1 are not found in the register pointed to by the RAT, but rather are found at the new location provided by the allocator. The PSrc information in the RAT is made stale by the allocator PDst of μop0 and must be overridden before the renamed μop physical sources are output to the RS and to the ROB. Also notice that a μop2 source uses the same register as was written by both μop0 and μop1. The *new PDst override* control must indicate that the PDst of μop1 (not μop0) is the appropriate pointer to use as the override for μop2's source.

Note that the μop groups can be a mixture of both integer and floating-point operations. Although there are two separate control blocks to perform integer and FP overrides, comparison of the logical register names sufficiently isolates the two classes of μops. It is naturally the case that only like types of sources and destinations can override each other. (For example, an FP destination cannot override an integer source.) Therefore, differences in the floating-point overrides can be handled independently of the integer mechanism.

The need for floating-point overrides is the same as for the integer overrides. Retirement and concurrent issue of μops prevent the array from being updated with the newest information before those concurrent μops read the array. Therefore, PSrc information read from the RAT arrays must be overridden by both retirement overrides and new PDst overrides.

Floating-point retirement overrides are identical to integer retirement overrides except that the value to which a PSrc is overridden is not determined by the logical register source name as in the integer case. Rather, the retiring logical register destination reads the RfRAT for the reset value. Depending on which retirement μop content addressable memory (CAM) matched with this array read, the retirement override control must choose between one of the three RfRAT reset values. These reset values must have been modified by any concurrent retiring FXCHs as well.

7.3.4.5 RAT Stalls. The RAT can stall in two ways, internally and externally. The RAT generates an internal stall if it is unable to completely process the current set of μops, due to a partial register write, a flag mismatch, or other microarchitectural conditions. The allocator may also be unable to process all μops due to an RS or ROB table overflow; this is an external stall to the RAT.

Partial Write Stalls. When a partial-width write (e.g., AX, AL, AH) is followed by a larger-width read (e.g., EAX), the RAT must stall until the last partial-width write of the desired register has retired. At this point, all portions of the register have been reassembled in the RRF, and a single PSrc can be specified for the required data.

The RAT performs this function by maintaining the size information (8, 16, or 32 bits) for each register alias. To handle the independent use of 8-bit registers, two entries and aliases (H and L) are maintained in the integer array for each of the registers EAX, EBX, ECX, and EDX. (The other macroregisters cannot be partially written, as per the Intel Architecture specification.) When 16- or 32-bit writes occur, both entries are updated. When 8-bit writes occur, only the corresponding entry (H or L, not both) is updated.

Thus when an entry is targeted by a logical source, the size information read from the array is compared to the requested size information specified by the μop. If the size needed is greater than the size available (read from array), then the RAT stalls both the instruction decoder and the allocator. In addition, the RAT clears the "valid bits" on the μop causing the stall (and any μops younger than it is) until the partial write retires; this is the in-order pipe, and subsequent μops cannot be allowed to pass the stalling μop here.

Mismatch Stalls. Since reading and writing the flags are common occurrences and are therefore performance-critical, they are renamed just as the registers are. There are two alias entries for flags, one for arithmetic flags and one for floating-point condition code flags, that are maintained in much the same fashion as the other integer array entries. When a μop is known to write the flags, the PDst granted for the μop is written into the corresponding flag entry (as well as the destination register entry). When subsequent μops use the flags as a source, the appropriate flag entry is read to find the PDst where the flags live.

In addition to the general renaming scheme, each μop emitted by the ID has associated flag information, in the form of masks, that tell the RAT which flags the μop touches and which flags the μop needs as input. In the event a previous but not yet retired μop did not touch all the flags that a current μop needs as input, the RAT stalls the in-order machine. This informs the ID and allocator that no new μops can be driven to the RAT because one or more of the current μops cannot be issued until a previous flag write retires.

7.3.5 Allocator

For each clock cycle, the allocator assumes that it will have to allocate three reorder buffer, reservation station, and load buffer entries and two store buffer entries. The allocator generates pointers to these entries and decodes the μops coming from the ID unit to determine how many entries of each resource are really needed and which RS dispatch port they will be dispatched on.

Based on the μop decoding and valid bits, the allocator will determine whether or not resource needs have been met. If not, then a stall is asserted and μop issue is frozen until sufficient resources become available through retirement of previous μops.

The first step in allocation is the decoding of the μops that are delivered by the ID. Some μops need an LB or SB entry; all μops need an ROB entry.

7.3.5.1 ROB Allocation. The ROB entry addresses are the physical destinations or PDst's which were assigned by the allocator. The PDst's are used to directly

address the ROB. This means if the ROB is full, the allocator must assert a stall signal early enough to prevent overwriting valid ROB data.

The ROB buffer is treated as a circular buffer by the allocator. In other words, entry addresses are assigned sequentially from 0 until the highest address, and then wraparound back to 0. A three-or-none allocation policy is used: every cycle, at least three ROB entries must be available or the allocator will stall. This means ROB allocation is independent of the type of μop and does not even depend on the μop's validity. The three-or-none policy simplifies allocation.

At the end of the cycle, the address of the last ROB entry allocated is preserved and becomes the new allocation starting point. Note that this does depend on the real number of valid μops. The ROB also uses the number of valid μops to determine where to stop retiring.

7.3.5.2 MOB Allocation. All μops have a load buffer ID and a store buffer ID (together known as a MOB ID, or MBID) stored with them. Load μops will have a newly allocated LB address and the last SB address that was allocated. Nonload μops (store or any other μop) have MBID with LBID = 0 and the SBID (or store color) of the last store allocated.

The LB and SB are treated as circular buffers, as is the ROB. However, the allocation policy is slightly different. Since every μop does not need an LB or SB entry, it would be a big performance hit to use a three-or-none policy (or two-or-none for SB) and stall whenever the LB or SB has less than three free entries. Instead we use an all-or-none policy. This means that stalling will occur only when not all the valid MOB μops can be allocated.

Another important part of MOB allocation is the handling of entries containing *senior* stores. These are stores that have been committed or retired by the CPU but are still actually awaiting completion of execution to memory. These store buffer entries cannot be deallocated until the store is actually performed to memory.

7.3.5.3 RS Allocation. The allocator also generates write enable bits which are used by the RS directly for its entry enables. If the RS is full, a stall indication must be given early in order to prevent the overwrite of valid RS data. In fact if the RS is full, the enable bits will all be cleared and thus no entry will be enabled for writing. If the RS is not full but a stall occurs due to some other resource conflict, the RS invalidates data written to any RS entry in that cycle (i.e., data get written but are marked as invalid).

The RS allocation works differently from the ROB or MOB circular buffer model. Since the RS dispatches μops out of order (as they become data-ready), its free entries are typically interspersed with used or allocated entries, and so a circular buffer model does not work. Instead, a bitmap scheme is used where each RS entry maps to a bit of the RS allocation pool. In this way, entries may be drawn or replaced from the pool in any order. The RS searches for free entries by scanning from location 0 until the first three free entries are found.

Some μops can dispatch to more than one port, and the act of committing a given μop to a given port is called binding. The binding of μops to the RS functional unit interfaces is done at allocation. The allocator has a load-balancing algorithm

that knows how many μops in the RS are waiting to be executed on a given interface. This algorithm is only used for μops that can execute on more than one EU. This is referred to as a *static binding with load balancing* of ready μops to an execution interface.

7.4 The Out-of-Order Core

7.4.1 Reservation Station

The reservation station (RS) is basically a place for μops to wait until their operands have all become ready and the appropriate execution unit has become available. In each cycle, the RS determines execution unit availability and source data validity, performs out-of-order scheduling, dispatches μops to execution units, and controls data bypassing to RS array and execution units. All entries of the RS are identical and can hold any kind of μops.

The RS has 20 entries. The control portion of an entry (μop, entry valid, etc.) can be written from one of three ports (there are three ports because the P6 microarchitecture is of superscalar order 3.). This information comes from the allocator and RAT. The data portion of an entry can be written from one of six ports (three ROB and three execution unit writebacks). CAMs control the snarfing of valid writeback data into μop Src fields and data bypassing at the execution unit (EU) interfaces. The CAMs, EU arbitration, and control information are used to determine data validity and EU availability for each entry (ready bit generation). The scheduler logic uses this ready information to schedule up to five μops. The entries that have been scheduled for dispatch are then read out of the array and driven to the execution unit.

During pipe stage 31, the RS determines which entries are, or will be, ready for dispatch in stage 32. To do this, it is necessary to know the availability of data and execution resources (EU/AGU units). This ready information is sent to the scheduler.

7.4.1.1 Scheduling. The basic function of the scheduler is to enable the dispatching of up to five μops per clock from the RS. The RS has five schedulers, one for each execution unit interface. Figure 7.14 shows the mapping of the functional units to their RS ports.

The RS uses a priority pointer to specify where the scheduler should begin its scan of the 20 entries. The priority pointer will change according to a pseudo-FIFO algorithm. This is used to reduce stale entry effects and increase performance in the RS.

7.4.1.2 Dispatch. The RS can dispatch up to five μops per clock. There are two EU and two AGU interfaces and one store data (STD) interface. Figure 7.14 shows the connections of the execution units to the RS ports. Before instruction dispatch time, the RS determines whether or not all the resources needed for a particular μop to execute are available, and then the ready entries are scheduled. The RS then dispatches all the necessary μop information to the scheduled functional unit. Once a μop has been dispatched to a functional unit and no cancellation has

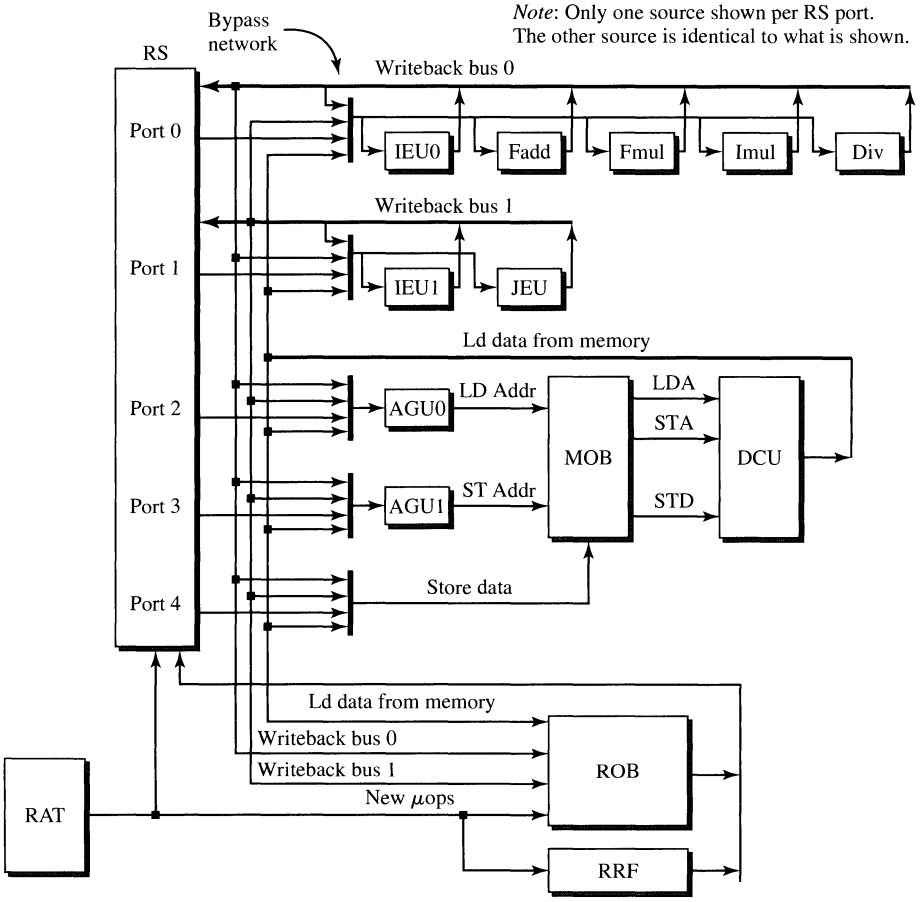


Figure 7.14
Execution Unit Data Paths.

occurred due to a cache miss, the entry can be deallocated for use by a new μop. Every cycle, deallocation pointers are used to signal the allocator about the availability of all 20 entries in the RS.

7.4.1.3 Data Writeback. It is possible that source data will not be valid at the time the RS entry is initially written. The μop must then remain in the RS until all its sources are valid. The content addressable memories (CAMs) are used to compare the writeback physical destination (PDst) with the stored physical sources (PSrc). When a match occurs, the corresponding write enables are asserted to snarf the needed writeback data into the appropriate source in the array.

7.4.1.4 Cancellation. Cancellation is the inhibiting of a μop from being scheduled, dispatched, or executed due to a cache miss or possible future resource conflict.

All canceled μops will be rescheduled at a later time unless the out-of-order machine is reset.

There are times when writeback data are invalid, e.g., when the memory unit detects a cache miss. In this case, dispatching μops that are dependent on the writeback data need to be canceled and rescheduled at a later time. This can happen because the RS pipeline assumes cache accesses will be hits, and schedules dependent μops based on that assumption.

7.5 Retirement

7.5.1 The Reorder Buffer

The reorder buffer (ROB) participates in three fundamental aspects of the P6 microarchitecture: speculative execution, register renaming, and out-of-order execution. In some ways, the ROB is similar to the register file in an in-order machine, but with additional functionality to support retirement of speculative operations and register renaming.

The ROB supports speculative execution by buffering the results of the execution units (EUs) before committing them to architecturally visible state. This allows most of the microengine to fetch and execute instructions at a maximum rate by assuming that branches are properly predicted and that no exceptions occur. If a branch is mispredicted or if an exception occurs in executing an instruction, the microengine can *recover* simply by discarding the speculative results stored in the ROB. The microengine can also *restart* at the proper instruction by examining the committed architectural state in the ROB. A key function of the ROB is to control retirement or completion of μops.

The buffer storage for EU results is also used to support register renaming. The EUs write result data *only* into the renamed register in the ROB. The retirement logic in the ROB updates the architectural registers based upon the contents of each renamed instance of the architectural registers. Micro-ops which source an architectural register obtain either the contents of the actual architectural register or the contents of the renamed register. Since the P6 microarchitecture is superscalar, different μops in the same clock which use the same architectural register may in fact access different physical registers.

The ROB supports out-of-order execution by allowing EUs to complete their μops and write back the results without regard to other μops which are executing simultaneously. Therefore, as far as the execution units are concerned, μops complete out of order. The ROB retirement logic *reorders* the completed μops into the original sequence issued by the instruction decoder as it updates the architectural state during retirement.

The ROB is active in three separate parts of the processor pipeline (refer to Figure 7.4): the rename and register read stages, the execute/writeback stage, and the retirement stages.

The placement of the ROB relative to other units in the P6 is shown in the block diagram in Figure 7.1. The ROB is closely tied to the allocator (ALL) and register

alias table (RAT) units. The allocator manages ROB physical registers to support speculative operations and register renaming. The actual renaming of architectural registers in the ROB is managed by the RAT. Both the allocator and the RAT function within the in-order part of the P6 pipeline. Thus, the rename and register read (or ROB read) functions are performed in the same sequence as in the program flow.

The ROB interface with the reservation station (RS) and the EUs in the out-of-order part of the machine is loosely coupled in nature. The data read from the ROB during the register read pipe stage consist of operand sources for the μop. These operands are stored in the RS until the μop is dispatched to an execution unit. The EUs write back μop results to the ROB through the five writeback ports (three full writeback ports, two partial writebacks for STD and STA). The result writeback is out of order with respect to μops issued by the instruction decoder. Because the results from the EUs are speculative, any exceptions that were detected by the EUs may or may not be "real." Such exceptions are written into a special field of the μop. If it turns out that the μop was misspeculated, then the exception was not "real" and will be flushed along with the rest of the μop. Otherwise, the ROB will notice the exceptional condition during retirement of the μop and will cause the appropriate exception-handling action to be invoked then, before making the decision to commit that μop's result to architectural state.

The ROB retirement logic has important interfaces to the micro-instruction sequencer (MS) and the memory ordering buffer (MOB). The ROB/MS interface allows the ROB to signal an exception to the MS, forcing the micro-instruction sequencer to jump to a particular exception handler microcode routine. Again, the ROB must force the control flow change because the EUs report events out of order with respect to program flow. The ROB/MOB interface allows the MOB to commit memory state from stores when the store μop is committed to the machine state.

7.5.1.1 ROB Stages in the Pipeline. The ROB is active in both the in-order and out-of-order sections of the P6 pipeline. The ROB is used in the in-order pipe in pipe stages 21 and 22. Entries in the reorder buffer which will hold the results of the speculative μops are allocated in pipe stage 21. The reorder buffer is managed by the allocator and the retirement logic as a circular buffer. If there are unused entries in the reorder buffer, the allocator will use them for the μops being issued in the clock. The entries used are signaled to the RAT, allowing it to update its renaming or alias tables. The addresses of the entries used (PDst's) are also written into the RS for each μop. The PDst is the key token used by the out-of-order section of the machine to identify μops in execution; it is the actual slot number in the ROB. As the entries in the ROB are allocated, certain fields in them are written with data from fields in the μops. This information can be written either at allocation time or with the results written back by the EUs. To reduce the width of the RS entries as well as to reduce the amount of information which must be circulated to the EUs or memory subsystem, any μop information required to retire a μop which is determined strictly at decode time is written into the ROB at allocation time.

In pipe stage 22, immediately following entry allocation, the sources for the μops are read from the ROB. The physical source addresses, PSrc's, are delivered

by the RAT based upon the alias table update performed in pipe stage 21. A source may reside in one of three places: in the committed architectural state (retirement register file), in the reorder buffer, or from a writeback bus. (The RRF contains both architectural state and microcode visible state. Subsequent references to RRF state will call them macrocode and microcode visible state). Source operands read from the RRF are always valid, ready for execution unit use. Sources read from the ROB may or may not be valid, depending on the timing of the source read with respect to writebacks of previous μops which updated the entries read. If the source operand delivered by the ROB is invalid, the RS will wait until an EU writes back to a PDst which matches the physical source address for a source operand in order to capture (or bypass at the EU) the valid source operand for a given μop.

An EU writes back destination data into the entry allocated for the μop, along with any event information, in pipe stage 83. (*Event* refers to exceptions, interrupts, microcode assists, and so on.) The writeback pipe stage is decoupled from the rename and register read pipe stages because the μops are issued out of order from the RS. Arbitration for use of writeback busses is determined by the EUs along with the RS. The ROB is simply the terminus for each of the writeback busses and stores whatever data are on the busses into the writeback PDst's signaled by the EUs.

The ROB retirement logic commits macrocode and microcode visible state in pipe stages 92 and 93. The retirement pipe stages are decoupled from the writeback pipe stage because the writebacks are out of order with respect to the program or microcode order. Retirement effectively *reorders* the out-of-order completion of μops by the EUs into an in-order completion of μops by the machine as a whole. Retirement is a two-clock operation, but the retirement stages are pipelined. If there are allocated entries in the reorder buffer, the retirement logic will attempt to deallocate or retire them. Retirement treats the reorder buffer as FIFO in deallocating the entries, since the μops were originally allocated in a sequential FIFO order earlier in the pipeline. This ensures that retirement follows the original program source order, in terms of allowing the architectural state to be modified.

The ROB contains all the P6 macrocode and microcode state which may be modified without serialization of the machine. (Serialization limits to one the number of μops which may flow through the out-of-order section of the machine, effectively making them execute in order.) Much of this state is updated directly from the speculative state in the reorder buffer. The extended instruction pointer (EIP) is the one architectural register which is an exception to this norm. The EIP requires a significant amount of hardware in the ROB for each update. The reason is the number of μops which may retire in a clock varies from zero to three.

The ROB is implemented as a multiported register file with separate ports for allocation time writes of μop fields needed at retirement, EU writebacks, ROB reads of sources for the RS, and retirement logic reads of speculative result data.

The ROB has 40 entries. Each entry is 157 bits wide. The allocator and retirement logic manage the register file as FIFO. Both source read and destination writeback functions treat the reorder buffer as a register file.

Table 7.1
Registers in the RRF

Qty.	Register Name(s)	Size (bits)	Description
8	i486 general registers	32	EAX, ECX, EDX, EBX, EBP, ESP, ESI, EDI
8	i486 FP stack registers	86	FST(0-7)
12	General microcode temp. registers	86	For storing both integer and FP values
4	Integer microcode temp. registers	32	For storing integer values
1	EFLAGS	32	The i486 system flags register
1	ArithFlags	8	The i486 flags which are renamed
2	FCC	4	The FP condition codes
1	EIP	32	The architectural instruction pointer
1	FIP	32	The architectural FP instruction pointer
1	EventUIP	12	The micro-instruction reporting an event
2	FSW	16	The FP status word

The RRF contains both the macrocode and microcode visible state. Not all such processor state is located in the RRF, but any state which may be renamed is there. Table 7.1 gives a listing of the registers in the RRF.

Retirement logic generates the addresses for the retirement reads performed in each clock. The retirement logic also computes the retirement valid signals indicating which entries with valid writeback data may be retired.

The IP calculation block produces the architectural instruction pointer as well as several other macro- and micro-instruction pointers. The macro-instruction pointer is generated based on the lengths of all the macro-instructions which may retire, as well as any branch target addresses which may be delivered by the jump execution unit.

When the ROB has determined that the processor has started to execute operations down the wrong path of a branch, any operations in that path must not be allowed to retire. The ROB accomplishes this by asserting a "clear" signal at the point just before the first of these operations would have retired. All speculative operations are then flushed from the machine. When the ROB retires an operation that faults, it clears both the in-order and out-of-order sections of the machine in pipe stages 93 and 94.

7.5.1.2 Event Detection. Events include faults, traps, assists, and interrupts. Every entry in the reorder buffer has an event information field. The execution

units write back into this field. During retirement the retirement logic looks at this field for the three entries that are candidates for retirement. The event information field tells the retirement logic whether there is an exception and whether it is a fault or a trap or an assist. Interrupts are signaled directly by the interrupt unit. The jump unit marks the event information field in case of taken or mispredicted branches.

If an event is detected, the ROB clears the machine of all μops and forces the MS to jump to a microcode event handler. Event records are saved to allow the microcode handler to properly repair the result or invoke the correct macrocode handler. Macro- and micro-instruction pointers are also saved to allow program resumption upon termination of the event handler.

7.6 Memory Subsystem

The memory ordering buffer (MOB) is a part of the memory subsystem of the P6. The MOB interfaces the processor's out-of-order engine to the memory subsystem. The MOB contains two main buffers, the load buffer (LB) and the store address buffer (SAB). Both of these buffers are circular queues with each entry within the buffer representing either a load or a store micro-operation, respectively. The SAB works in unison with the memory interface unit's (MIU) store data buffer (SDB) and the DCache's physical address buffer (PAB) to effectively manage a processor store operation. The SAB, SDB, and PAB can be viewed as one buffer, the store buffer (SB).

The LB contains 16 buffer entries, holding up to 16 loads. The LB queues up load operations that were unable to complete when originally dispatched by the reservation station (RS). The queued operations are redispatched when the conflict has been removed. The LB maintains processor ordering for loads by snooping external writes against completed loads. A second processor's write to a speculatively read memory location forces the out-of-order engine to clear and restart the load operation (as well as any younger μops).

The SB contains 12 entries, holding up to 12 store operations. The SB is used to queue up all store operations before they dispatch to memory. These stores are then dispatched in original program order, when the OOO engine signals that their state is no longer speculative. The SAB also checks all loads for store address conflicts. This checking keeps loads consistent with previously executed stores still in the SB.

The MOB resources are allocated by the allocator when a load or store operation is issued into the reservation station. A load operation decodes into one μop and a store operation is decoded into two μops: store data (STD) and store address (STA). At allocation time, the operation is tagged with its eventual location in the LB or SB, collectively referred to as the MOB ID (MBID). Splitting stores into two distinct μops allows any possible concurrency between generation of the address and data to be stored to be expressed.

The MOB receives speculative LD and STA operations from the reservation station. The RS provides the opcode, while the address generation unit (AGU)

calculates and provides the linear address for the access. The DCache either executes these operations immediately, or they are dispatched later by the MOB. In either case they are written into one of the MOB arrays. During memory operations, the data translation lookaside buffer (DTLB) converts the linear address to a physical address or signals a page miss to the page miss handler (PMH). The MOB will also perform numerous checks on the linear address and data size to determine if the operation can continue or if it must block.

In the case of a load, the data cache unit is expected to return the data to the core. In parallel, the MOB writes address and status bits into the LB, to signal the operation's completion. In the case of a STA, the MOB completes the operation by writing a valid bit (AddressDone) into the SAB array and to the reorder buffer. This indicates that the address portion of the store has completed. The data portion of the store is executed by the SDB. The SDB will signal the ROB and SAB when the data have been received and written into the buffer. The MOB will retain the store information until the ROB indicates that the store operation is retired and committed to the processor state. It will then dispatch from the MOB to the data cache unit to commit the store to the system state. Once completed, the MOB signals deallocation of SAB resources for reuse by the allocator. Stores are executed by the memory subsystem in program order.

7.6.1 Memory Access Ordering

Micro-op register operand dependences are tracked explicitly, based on the register references in the original program instructions. Unfortunately, memory operations have implicit dependences, with load operations having a dependency on any previous store that has address overlap with the load. These operations are often speculative inside the MOB, both the stores and loads, so that system memory access may return stale data and produce incorrect results.

To maintain self-consistency between loads and stores, the P6 employs a concept termed *store coloring*. Each load operation is tagged with the store buffer ID (SBID) of the store previous to it. This ID represents the relative location of the load compared to all stores in the execution sequence. When the load executes in the memory subsystem, the MOB will use this SBID as a beginning point for analyzing the load against all older stores in the buffer, while also allowing the MOB to ignore younger stores.

Store coloring is used to maintain ordering consistency between loads and stores of the same processor. A similar problem occurs between processors of a multiprocessing system. If loads execute out of order, they can effectively make another processor's store operations appear out of order. This results from a younger load passing an older load that has not been performed yet. This younger load reads old data, while the older load, once performed, has the chance of reading new data written by another processor. If allowed to commit to state, these loads would violate *processor ordering*. To prevent this violation, the LB watches (snoops) all data writes on the bus. If another processor writes a location that was speculatively read, the speculatively completed load and subsequent operations will be cleared and re-executed to get the correct data.

7.6.2 Load Memory Operations

Load operations issue to the RS from the allocator and register allocation table (RAT). The allocator assigns a new load buffer ID (LBID) to each load that issues into the RS. The allocator also assigns a store color to the load, which is the SBID of the last store previously allocated. The load waits in the RS for its data operands to become available. Once available, the RS dispatches the load on port 2 to the AGU and LB. Assuming no other dispatches are waiting for this port, the LB bypasses this operation for immediate execution by the memory subsystem. The AGU generates the linear address to be used by the DTLB, MOB, and DCU. As the DTLB does its translation to the physical address, the DCU does an initial data lookup using the lower-order 12 bits. Likewise, the SAB uses the lower-order 12 bits along with the store color SBID to check potential conflicting addresses of previous stores (previous in program order, not time order). Assuming a DTLB page hit and no SAB conflicts, the DCU uses the physical address to do a final tag match and return the correct data (assuming no miss or block). This completes the load operation, and the RS, ROB, and MOB write their completion status.

If the SAB noticed an address match, the SAB would cause the SDB to forward SDB data, ignoring the DCU data. If a SAB conflict existed but the addresses did not match (a false conflict detection), then the load would be blocked and written into the LB. The load will wait until the conflicting store has left the store buffer.

7.6.3 Basic Store Memory Operations

Store operations are split into two micro-ops, store data (STD) followed by a store address (STA). Since a store is represented by the combination of these operations, the allocator allocates a store buffer entry only when the STD is issued into the RS. The allocation of a store buffer entry reserves the same location in the SAB, the SDB, and the PAB. When the store's source data become available, the RS dispatches the STD on port 4 to the MOB for writing into the SDB. As the STA address source data become available, the RS dispatches the STA on port 3 to the AGU and SAB. The AGU generates the linear address for translation by the DTLB and for writing into the SAB. Assuming a DTLB page hit, the physical address is written into the PAB. This completes the STA operation, and the MOB and ROB update their completion status.

Assuming no faults or mispredicted branches, the ROB retires both the STD and STA. Monitoring this retirement, the SAB marks the store (STD/STA pair) as the committed, or *senior,* processor state. Once senior, the MOB dispatches these operations by sending the opcode, SBID, and lower 12 address bits to the DCU. The DCU and MIU use the SBID to access the physical address in the PAB and store data in the SDB, respectively, to complete the final store operation.

7.6.4 Deferring Memory Operations

In general, most memory operations are expected to complete three cycles after dispatch from the RS (which is only two clocks longer than an ALU operation). However, memory operations are not totally predictable as to their translation and availability from the L1 cache. In cases such as these, the operations require other

resources, e.g., DCU fill buffers on a pending cache miss, that may not be available. Thus, the operations must be deferred until the resource becomes available.

The MOB load buffer employs a general mechanism of blocking load memory operation until a later wakeup is received. The blocking information associated with each entry of the load buffer contains two fields: a blocking code or type and a blocking identifier. The block code identifies the source of the block (e.g., address block, PMH resource block). The block identifier refers to a specific ID of a resource associated with the block code. When a wakeup signal is received, all deferred memory operations that match the blocking code and identifier are marked "ready for dispatch." The load buffer then schedules and dispatches one of these ready operations in a manner that is very similar to RS dispatching.

The MOB store buffer uses a restricted mechanism for blocking STA memory operations. The operations remain blocked until the ROB retirement pointers indicate that STA μop is the oldest nonretired operation in the machine. This operation will then dispatch at retirement with the write to the DCU occurring simultaneously with the dispatch of the STA. This simplified mechanism for stores was used because STAs are rarely blocked.

7.6.5 Page Faults

The DTLB translates the linear addresses to physical addresses for all memory load and store address μops. The DTLB does the address translation by performing a lookup in a cache array for the physical address of the page being accessed. The DTLB also caches page attributes with the physical address. The DTLB uses this information to check for page protection faults and other paging-related exceptions.

The DTLB stores physical addresses for only a subset of all possible memory pages. If an address lookup fails, the DTLB signals a miss to the PMH. The PMH executes a *page walk* to fetch the physical address from the page tables located in physical memory. The PMH then looks up the effective memory type for the physical address from its on-chip memory type range registers and supplies both the physical address and the effective memory type to the DTLB to store in its cache array. (These memory type range registers are usually configured at processor boot time.)

Finally, the DTLB performs the fault detection and writeback for various types of faults including page faults, assists, and machine check architecture errors for the DCU. This is true for data and instruction pages. The DTLB also checks for I/O and data breakpoint traps, and either writes back (for store address μops) or passes (for loads and I/O μops) the results to the DCU which is responsible for supplying the data for the ROB writeback.

7.7 Summary

The design described in this chapter began as the brainchild of the authors of this chapter, but also reflects the myriad contributions of hundreds of designers, microcoders, validators, and performance analysts. Subject only to the economics that rule Intel's approach to business, we tried at all times to obey the prime directive: Make choices that maximize delivered performance, and quantify those choices

wherever possible. The out-of-order, speculative execution, superpipelined, superscalar, micro-dataflow, register-renaming, glueless multiprocessing design that we described here was the result. Intel has shipped approximately one billion P6-based microprocessors as of 2002, and many of the fundamental ideas described in this chapter have been reused for the Pentium 4 processor generation.

Further details on the P6 microarchitecture can be found in Colwell and Steck [1995] and Papworth [1996].

7.8 Acknowledgments

The design of the P6 microarchitecture was a collaborative effort among a large group of architects, designers, validators, and others. The microarchitecture described here benefited enormously from contributions from these extraordinarily talented people. They also contributed some of the text descriptions found in this chapter. Thank you, one and all.

We would also like to thank Darrell Boggs for his careful proofreading of a draft of this chapter.

REFERENCES

Colwell, Robert P., and Randy Steck: "A 0.6 μm BiCMOS microprocessor with dynamic execution," *Proc. Int. Solid State Circuits Conference,* San Francisco, CA 1995, pp. 176–177.

Lee, J., and A. J. Smith: "Branch predictions and branch target buffer design," *IEEE Computer,* January 1984, 21, 7, pp. 6–22.

Papworth, David B.: "Tuning the Pentium Pro microarchitecture," *IEEE Micro,* August 1996, pp. 8–15.

Yeh, T.-Y., and Y. N. Patt: "Two-level adaptive branch prediction," *The 24th ACM/IEEE Int. Symposium and Workshop on Microarchitecture,* November 1991, pp. 51–61.

HOMEWORK PROBLEMS

P7.1 The PowerPC 620 does not implement load/store forwarding, while the Pentium Pro does. Explain why both design teams are likely to have made the right design tradeoff.

P7.2 The P6 recovers from branch mispredictions in a somewhat coarse-grained manner, as illustrated in Figure 7.5. Explain how this simplifies the misprediction recovery logic that manages the reorder buffer (ROB) as well as the register alias table (RAT).

P7.3 AMD's Athlon (K7) processor takes a somewhat different approach to dynamic translation from IA32 macro-instructions to the machine instructions it actually executes. For example, an ALU instruction with one memory operand (e.g., add eax,[eax]) would translate into two μops in the Pentium Pro: a load that writes a temporary register followed by a register-to-register add instruction. In contrast, the Athlon

would simply dispatch the original instruction into the issue queue as a macro-op, but it would issue from the queue twice: once as a load and again as an ALU operation. Identify and discuss at least two microarchitectural benefits that accrue from this "macro-op" approach to instruction-set translation.

P7.4 The P6 two-level branch predictor has a speculative and nonspeculative branch history register stored at each entry. Describe when and how each branch history register is updated, and provide some reasoning that justifies this design decision.

P7.5 The P6 dynamic branch predictor is backed up by a static predictor that is able to predict branch instructions that for some reason were not predicted by the dynamic predictor. The static prediction occurs in pipe stage 17 (refer to Figure 7.4). One scenario in which the static prediction is used occurs when the BTB reports a tag mismatch, reflecting the fact that it has no branch history information for this particular branch. Assume the static branch prediction turns out to be correct. One possible optimization would be to avoid installing such a branch (one that is statically predictable) in the BTB, since it might displace another branch that needs dynamic prediction. Discuss at least one reason why this might be a bad idea.

P7.6 Early in the P6 development, the design had two different ROBs, one for integers and another for floating-point. To save die size, these were combined into one during the Pentium Pro development. Explain the advantages and disadvantages of the separate integer ROB and floating-point ROB versus the unified ROB.

P7.7 From the timing diagrams you can see that the P6 retirement process takes three clock cycles. Suppose you knew a way to implement the ROB so that retirement only took two clock cycles. Would you expect a substantial performance boost? Explain.

P7.8 Section 7.3.4.5 describes a mismatch stall that occurs when condition flags are only partially written by an in-flight μop. Suggest a solution that would prevent the mismatch stall from occurring in the renaming process.

P7.9 Section 7.3.5.3 describes the RS allocation policy of the Pentium Pro. Based on this description, would you call the P6 a centralized RS design or a distributed RS design? Justify your answer.

P7.10 If the P6 microarchitecture had to support an instruction set that included predication, what effect would that have on the register renaming process?

P7.11 As described in the text, the P6 microarchitecture splits store operations into a STA and STD pair for handling address generation and data movement. Explain why this makes sense from a microarchitectural implementation perspective.

P7.12 Following up on Problem 7.11, would there be a performance benefit (measured in instructions per cycle) if stores were not split? Explain why or why not?

P7.13 What changes would one have to make to the P6 microarchitecture to accommodate stores that are not split into separate STA and STD operations? What would be the likely effect on cycle time?

P7.14 AMD has recently announced the x86-64 extensions to the Intel IA32 architecture that add support for 64-bit registers and addressing. Investigate these extensions (more information is available from **www.amd.com**) and outline the changes you would need to make to the P6 architecture to accommodate these additional instructions.

R. M. Tomasulo

An Efficient Algorithm for Exploiting Multiple Arithmetic Units

Abstract: This paper describes the methods employed in the floating-point area of the System/360 Model 91 to exploit the existence of multiple execution units. Basic to these techniques is a simple common data busing and register tagging scheme which permits simultaneous execution of independent instructions while preserving the essential precedences inherent in the instruction stream. The common data bus improves performance by efficiently utilizing the execution units without requiring specially optimized code. Instead, the hardware, by 'looking ahead' about eight instructions, automatically optimizes the program execution on a local basis.

The application of these techniques is not limited to floating-point arithmetic or System/360 architecture. It may be used in almost any computer having multiple execution units and one or more 'accumulators.' Both of the execution units, as well as the associated storage buffers, multiple accumulators and input/output buses, are extensively checked.

Introduction

After storage access time has been satisfactorily reduced through the use of buffering and overlap techniques, even after the instruction unit has been pipelined to operate at a rate approaching one instruction per cycle,[1] there remains the need to optimize the actual performance of arithmetic operations, especially floating-point. Two familar problems confront the designer in his attempt to balance execution with issuing. First, individual operations are not fast enough* to allow simple serial execution. Second, it is difficult to achieve the fastest execution times in a universal execution unit. In other words, circuitry designed to do both multiply and add will do neither as fast as two units each limited to one kind of instruction.

The first step toward surmounting these obstacles has been presented,[2] i.e., the division of the execution function into two independent parts, a fixed-point execution area and a floating-point execution area. While this relieves the physical constraint and makes concurrent execution possible, there is another consideration. In order to secure a performance increase the program must contain an intimate mixture of fixed-point and floating-point instructions. Obviously, it is not always feasible for the programmer to arrange this and, indeed, many of the programs of greatest interest to the user consist almost wholly of floating-point instructions. The subject of this paper, then, is the method used to achieve concurrent execution of floating-point instructions in the IBM System/360 Model 91. Obviously, one begins with multiple execution units, in this case an adder and a multiplier/divider.[1]

It might appear that achieving the concurrent operation of these two units does not differ substantially from the attainment of fixed-floating overlap. However, in the latter case the architecture limits each of the instruction classes to its own set of accumulators and this guarantees independence.* In the former case there is only one set of accumulators, which implies program-specified sequences of dependent operations. Now it is no longer simply a matter of classifying each instruction as fixed-point or floating-point, a classification which is independent of previous instructions. Rather, it is a question of determining each instruction's relationship with all previous, incompleted instructions. Simply stated, the objective must be to preserve essential precedences while allowing the greatest possible overlap of independent operations.

This objective is achieved in the Model 91 through a scheme called the common data bus (CDB). It makes possible maximum concurrency with minimal effort (usually none) by the programmer or, more importantly, by the compiler. At the same time, the hardware required is small and logically simple. The CDB can function with any number of accumulators and any number of execution units. In short, it provides a hardware algorithm for the automatic, efficient exploitation of multiple execution units.

* During the planning phase, floating-point multiply was taken to be six cycles, divide as eighteen cycles and add as two cycles. A subsequent paper[2] explains how times of 3, 12, and 2 were actually achieved. This permitted the use of only one, instead of two, multipliers and one adder, pipelined to start an add cycle.

* Such dependencies as exist are handled by the store-fetch sequencing of the storage bus and the condition code control described in the following paper.[2]

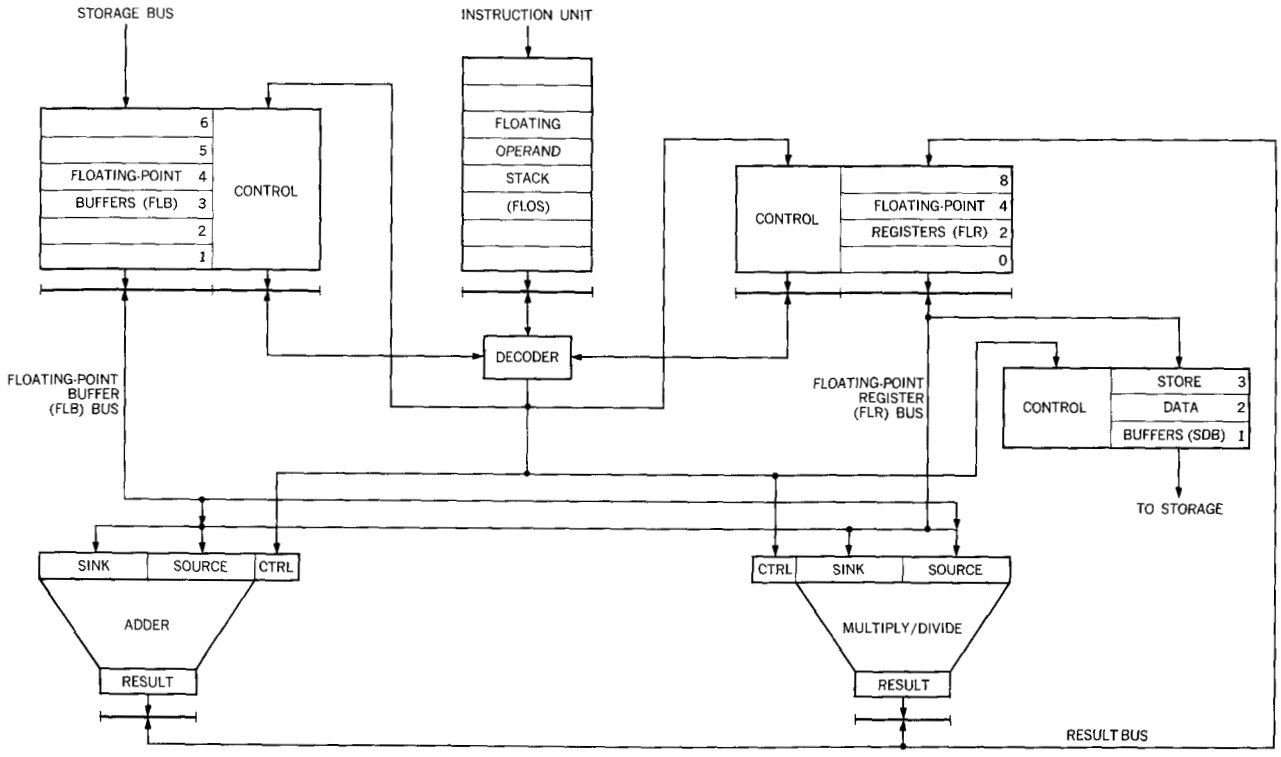


Figure 1 Data registers and transfer paths without CDB.

The next section of this paper will discuss the physical framework of registers, data paths and execution circuitry which is implied by the architecture and the overall CPU structure presented in a previous paper.[1] Within this framework one can subsequently discuss the problem of precedence, some possible solutions, and the selected solution, the CDB. In conclusion will be a summary of the results obtained.

Definitions and data paths

While the reader is assumed to be familiar with System/360 architecture and mnemonics, the terminology as modified by the context of the Model 91 organization will be reviewed here. The instruction unit, in preparing instructions for the floating-point operation stack (FLOS), maps both storage-to-register and register-to-register instructions into a pseudo-register-to-register format. In this format R1 is always one of the four floating-point registers (FLR) defined by the architecture. It is usually the *sink* of the instruction, i.e., it is the FLR whose contents are set equal to the result of the operation. Store operations are the sole exception* wherein R1 specifies the *source* of the operand to be placed in storage. A word in storage is really the sink of a store. (R1 and R2 refer to fields as defined by System/360 architecture.)

In the pseudo-register-to-register format "seen" by the FLOS the R2 field can have three different meanings. It can be an FLR as in a normal register-to-register instruction. If the program contains a storage-to-register instruction, the R2 field designates the floating-point buffer (FLB) assigned by the instruction unit to receive the storage operand. Finally, R2 can designate a store data buffer (SDB) assigned by the instruction unit to store instructions. In the first two cases R2 is the *source* of an operand; in the last case it is a sink. Thus, the instruction unit maps all of storage into the 6 floating-point buffers and 3 store data buffers so that the FLOS sees only pseudo-register-to-register operations.

The distinction between source and sink will become quite important during the discussion of precedence and should be fixed firmly in mind. All of the instructions (except store and compare) have the following form:

R1	op	R2 ⟶ R1
Register		Register Register
		or
		buffer
source		source sink

* Compares do not, of course, alter the contents of R1.

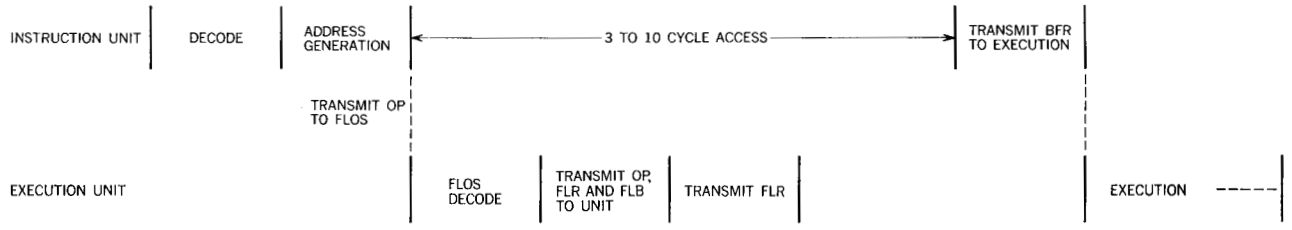


Figure 2 Timing relationship between instruction unit and FLOS decode for the processing of one instruction.

For example, the instruction AD0, 2 means "place the double-precision sum of registers 0 and 2 in register 0," i.e., R0 + R2 → R0. Note that R1 is really both a source and a sink.* Nevertheless, it will be called the sink and R2 the source in all subsequent discussion.

This definition of operations and the machine organization taken together imply a set of data registers with transfer paths among them. These are shown in Fig. 1. The major sets of registers (FLR's, FLB's, FLOS and SDB's) have already been discussed, both above and in a preceding paper.[1] Two additional registers, one sink and one source, are shown feeding each execution circuit. Initially these registers were considered to be the internal working registers required by the execution circuits and put to multiple use in a way to be described below. Later, their function was generalized under the reservation station concept and they were dissociated from their "working" function.

In actually designing a machine the data paths evolve as the design progresses. Here, however, a complete, first-pass data path will be shown to facilitate discussion. To illustrate the operation let us consider, in turn, four kinds of instructions—load of a register from storage, storage-to-register arithmetic, register-to-register arithmetic, and store. Let us first see how each can be accomplished *in vacuo*; then what difficulties arise when each is embedded in the context of a program. For simplicity double-precision (64-bit operands) will be used throughout.

Figure 2 shows the timing relationship between the instruction unit's handling of an instruction and its processing by the FLOS decode. When the FLOS decodes a load, the buffer which will receive the operand has not yet been loaded from storage.† Rather than holding the decode until the operand arrives, the FLOS sets control bits associated with the buffer which cause its content to be transmitted to the adder when it "goes full." The adder receives control information which causes it to send data to floating-point register R1, when its source register is set full by the buffer.

If the instruction is a storage-to-register arithmetic function, the storage operand is handled as in load (control bits cause it to be forwarded to the proper unit) but the floating-point register, along with the operation, is sent by the decoder to the appropriate unit. After receiving the buffer the unit will execute the operation and send the result to register R1.

In register-to-register arithmetic instructions two floating point registers are transmitted on successive cycles to the appropriate execution unit.

Stores are handled like storage-to-register arithmetic functions, except that the content of the floating-point register is sent to a store data buffer rather than to an execution unit.

Thus far, the handling of one instruction at a time has proven rather straightforward. Now consider the following "program":

Example 1

LD F0 FLB1 LOAD register F0 from buffer 1

MD F0 FLB2 MULTIPLY register F0 by buffer 2

The load can be handled as before, but what about the multiply? Certainly F0 and FLB2 cannot be sent to the multiplier as in the case of the isolated multiply, since FLB1 has not yet been set into F0.* This sequence illustrates the cardinal precedence principle: No floating-point register may participate in an operation if it is the sink of another, incompleted instruction. That is, a register cannot be used until its contents reflect the result of the most recent operation to use that register as its sink.

The design presented thus far has not incorporated any mechanism for dealing with this situation. Three functions must be required of any such mechanism:

(1) It must recognize the existence of a dependency.

* This economy of specification compounds the difficulties of achieving concurrency while preserving precedence, as will be seen later.

† A FULL/EMPTY control bit indicates this. The bit is set FULL by the Main Storage Control Element and EMPTY when the buffer is used. LOAD uses the adder in order to minimize the buffer outgates and the FLR ingates.

* Note that the program calls for the product of FLB1 and FLB2 to be placed in F0. This hints at the CDB concept.

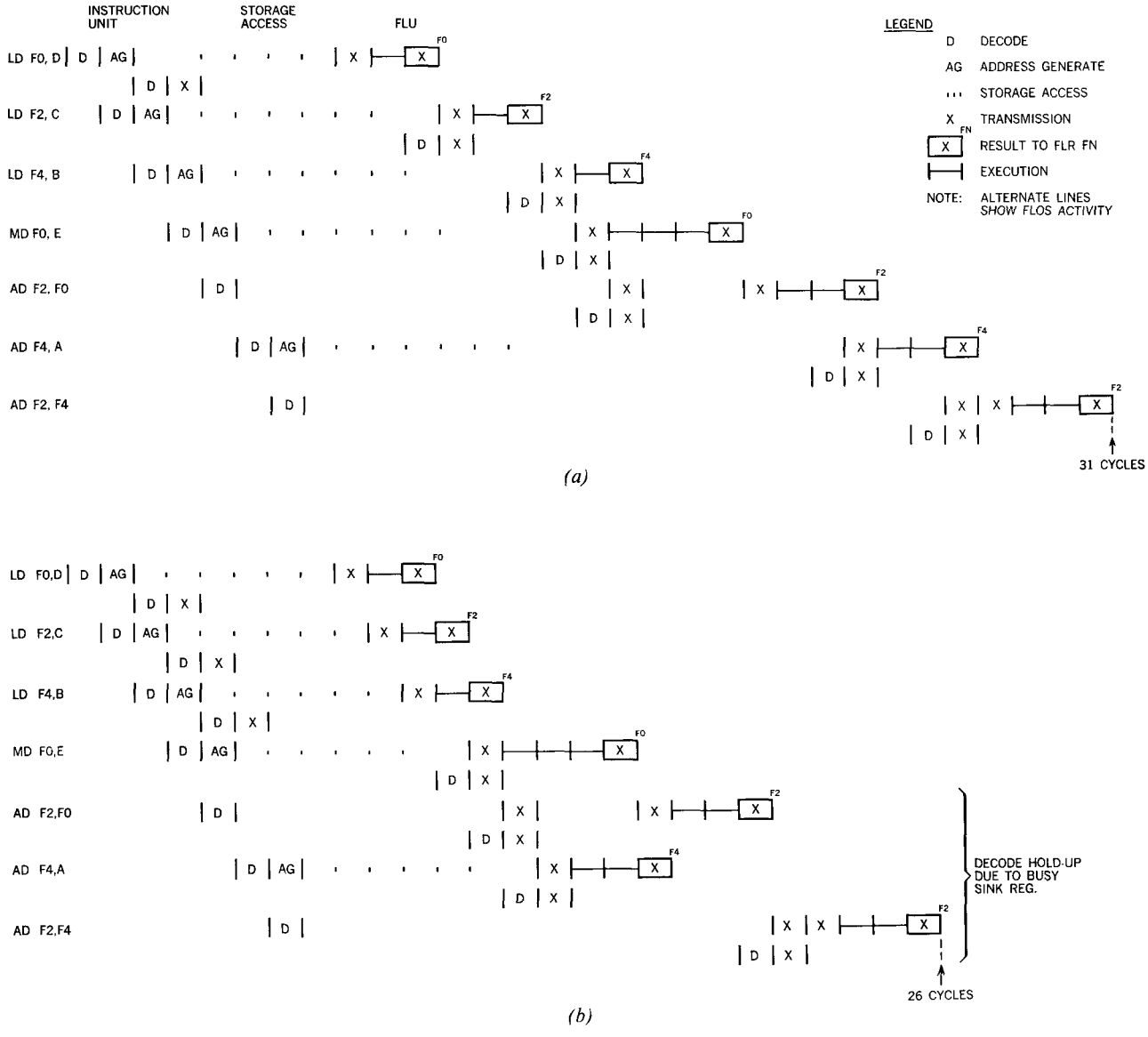


Figure 3 Timing for the instruction sequence required to perform the function A + B + C + D * E : (a) without reservation stations, (b) with reservation stations included in the register set.

(2) It must cause the correct sequencing of the dependent instructions.

(3) It must distinguish between the given sequence and such sequences as

LD F0, FLB1
MD F2, FLB2

Here it must allow the independent MD to proceed regardless of the disposition of the LD.

The first two requirements are necessary to preserve the logical integrity of the program; the third is necessary to meet the performance goal. The next section will present several alternatives for accomplishing these objectives.

Preservation of precedence

Perhaps the simplest scheme for preserving precedence is as follows. A "busy" bit is associated with each of the four floating-point registers. This bit is set when the FLOS decode issues an instruction designating the register as a sink; it is reset when the executing unit returns the result to the register. No instruction can be issued by the FLOS if the busy bit of its sink is on. If the source of a register-to-register instruction has its busy bit on, the FLOS sets

control bits associated with the source register. When a result is entered into the register, these control bits cause the register to be sent via the FLR bus to the unit waiting for it as a source.

This scheme easily meets the first two requirements. The third is met with the help of the programmer; he must use different registers to achieve overlap. For example, the expression A + B + C + D * E can be programmed as follows:

Example 2

```
LD   F0,  D      F0 = D
LD   F2,  C      F2 = C
LD   F4,  B      F4 = B
MD   F0,  E      F0 = D * E
AD   F2,  F0     F2 = C + D * E
AD   F4,  A      F4 = A + B
AD   F2,  F4     F2 = A + B + C + D * E
```

The busy bit scheme should allow the second add and the multiply to be executed simultaneously (really, in any order) since they use different sinks. Unfortunately, the timing chart of Fig. 3a shows not only that the expected overlap does not occur but also that many cycles are lost to transmission time. The overlap fails to materialize because the first add uses the result of the multiply, and the adder must wait for that result. Cycles are lost to control because so many of the instructions use the adder. The FLOS cannot decode an instruction unless a unit is available to execute it. When an assigned unit finishes execution, it takes one cycle to transmit the fact to the FLOS so that it can decode a waiting instruction. Similarly, when the FLOS is held up because of a busy sink register, it cannot begin to decode until the result has been entered into the register.

One solution that could be considered is the addition of one or more adders. If this were done and some programs timed, however, it would become apparent that the execution circuitry would be in use only a small part of the time. Most of the lost time would occur while the adder waited for operands which are the result of previous instructions. What is required is a device to collect operands (and control information) and then engage the execution circuitry when all conditions are satisfied. But this is precisely the function of the sink and source registers in Fig. 1. Therefore, the better solution is to associate more than one set of registers (control, sink, source) with each execution unit. Each such set is called a *reservation station.** Now instruction issuing depends on the availability of the appropriate kind of reservation station. In the Model 91 there are three add and two multiply/divide reservation stations. For simplicity they are treated as if they were actual units. Thus, in the future, we will speak of Adder 1 (A1), Adder 2 (A2), etc., and M/D 1 and M/D 2.

Figure 3b shows the effect of the addition of reservation stations on the problem running time: five cycles have been eliminated. Note that the second AD now overlaps the MD and actually executes before the first AD. While the speed increase is gratifying and the busy bit method easy to implement, there remains a dependence on the programmer. Note that the expression could have been coded this way:

Example 3a

```
LD   F0,  E
MD   F0,  D
AD   F0,  C
AD   F0,  B
AD   F0,  A
```

Now overlap is impossible and the program will run six cycles longer despite having two fewer instructions. Suppose however, that this program is part of a loop, as below:

Example 3b

```
LOOP 1   LD   F0,  Ei
         MD   F0,  Di
         AD   F0,  Ci
         AD   F0,  Bi
         AD   F0,  Ai
         STD  F0,  Fi
         BXH i, -1, 0, LOOP 1 (decrease i by 1,
         branch if i > 0)
LOOP 2   LD   F0,  Ei
         LD   F2,  Ei + 1
         MD   F0,  Di
         MD   F2,  Di + 1
         AD   F0,  Ci
         AD   F2,  Ci + 1
         AD   F0,  Bi
         AD   F2,  Bi + 1
         AD   F0,  Ai
         AD   F2,  Ai + 1
         STD  F0,  Fi
         STD  F2,  Fi + 1
         BXH i, -2, 0, LOOP 2
```

Iteration $n + 1$ of LOOP 1 will appear to the FLOS to depend on iteration n, since the instructions in both iterations have the same sink. But it is clear that the two iterations are, in fact, independent. This example illustrates a second way in which two instruction sequences can be independent. The first way, of course, is for the two strings to have different sink registers. The second way is for the second string to begin with a load. By its definition a

* The fetch and store buffers can be considered as specialized, one-operand reservation stations. Previous systems, such as the IBM 7030, have in effect employed one "reservation station" ahead of each execution unit. The extension to several reservation stations adds to the effectiveness of the execution hardware.

load launches a new, independent string because it instructs the computer to destroy the previous contents of the specified register. Unfortunately, the busy bit scheme does not recognize this possibility. If overlap is to be achieved with this scheme, the programmer must write LOOP 2. (This technique is called *doubling* or *unravelling*. It requires twice as much storage but it runs faster by enabling two iterations to be executed simultaneously.)

Attempts were made to improve the busy bit scheme so as to handle this case. The most tempting approach is the expansion of the bit into a counter. This would appear to allow more than one instruction with a given sink to be issued. As each is issued, the FLOS increments the counter; as each is executed the counter is decremented. However, major difficulty is caused by the fact that storage operands do not return in sequence. This can cause the result of instruction $n + 1$ to be placed in a register before that of n. When n completes, it erroneously destroys the register contents.

Some of the other proposals considered would, if implemented, have been of such logical complexity as to jeopardize the achievement of a fast cycle.

The Common Data Bus

The preceding sections were intended to portray the difficulties of achieving concurrency among floating-point instructions and to show some of the steps in the evolution of a design to overcome them. It is clear, in retrospect, that the previous algorithms failed for lack of a way to uniquely identify each instruction and to use this information to sequence execution and set results into the floating-point registers. As far as action by the FLOS is concerned, the only thing unique to a particular instruction is the unit which will execute it. This, then, must form the basis of the common data bus (CDB).

Figure 4 shows the data paths required for operation of the CDB.* When Fig. 4 is compared with Fig. 1 the following changes, in addition to the reservation stations, are evident: Another output port has been added to the buffers. This port has been combined with the results from the adder and multiplier/divider; the combination is the CDB. The CDB now goes not only to the registers but also to the sink and source registers of all reservation stations, including the store data buffers but excluding the floating-point buffers. This data path will enable loads to be executed without the adder and will make the result of any operation available to all units without first going through a floating-point register.

Note that the CDB is fed by all units that can alter a register and that it feeds all units which can have a register as an operand. The control part of the CDB enumerates the units which feed the CDB. Thus the floating-point buffers 1 through 6 are assigned the numbers 1 through 6; the three adders (actually reservation stations) are numbered 10 through 12; the two multiplier/dividers are 8 and 9. Since there are eleven contributors to the CDB, a four-bit binary number suffices to enumerate them. This number is called a *tag*. A tag is associated with each of the four floating-point registers (in addition to the busy bit*), with both the source and sink registers of each of the five reservation stations and with each of the three Store Data Buffers. Thus a total of 17 four-bit tag registers has been added, as shown in Fig. 4.

Tags also appear in another context. A tag is generated by the CDB priority controls to identify the unit whose result will next appear on the CDB. Its use will be made clear shortly.

Operation of this complex is as follows. In decoding each instruction the FLOS checks the busy bit of each of the specified floating-point registers. If that bit is zero, the content of the register(s) may be sent to the selected unit via the FLR bus, just as before. Upon issuing the instruction, which requires only that a unit be available to execute it, the FLOS not only sets the busy bit of the sink register but also sets its tag to the designation of the selected unit. The source register control bits remain unchanged. As an example, take the instruction, AD F0, FLB1. After issuing this instruction to Adder 1 the control bits of F0 would be:

BB	TAG
1	1010 (A1)

So far the only change from previous methods is the setting of the tag. The significant difference occurs when the FLOS finds the busy bit on at decode time. Previously, this caused a suspension of decoding until the bit went off. Now the FLOS will issue the instruction and update the tag. In so doing it will not transmit the register contents to the selected unit but it will transmit the "old" tag. For example, suppose the previous AD was followed by a second AD. At the end of the decode of this second AD, F0's control bits would be:

BB	TAG
1	1011 (A2)

One cycle later the sink tag of the A2 reservation station would be 1010, i.e., the same as A1, the unit whose result will be required by A2.

Let us look ahead temporarily to the execution of the first AD. Some time after the start of execution but before the end,† A1 will request the CDB. Since the CDB is fed by many sources, its time-sharing is controlled by a central

* The FLB and FLR busses are retained for performance reasons. Everything could be done by a slight extension of the CDB but time would be lost due to conflicts over the common facility.

* The busy bit is no longer necessary since its function can be performed by use of an unassigned tag number. However, it is convenient to retain it.

† Since the required lead time is two cycles, the request is made at the start of execution for an add-type instruction.

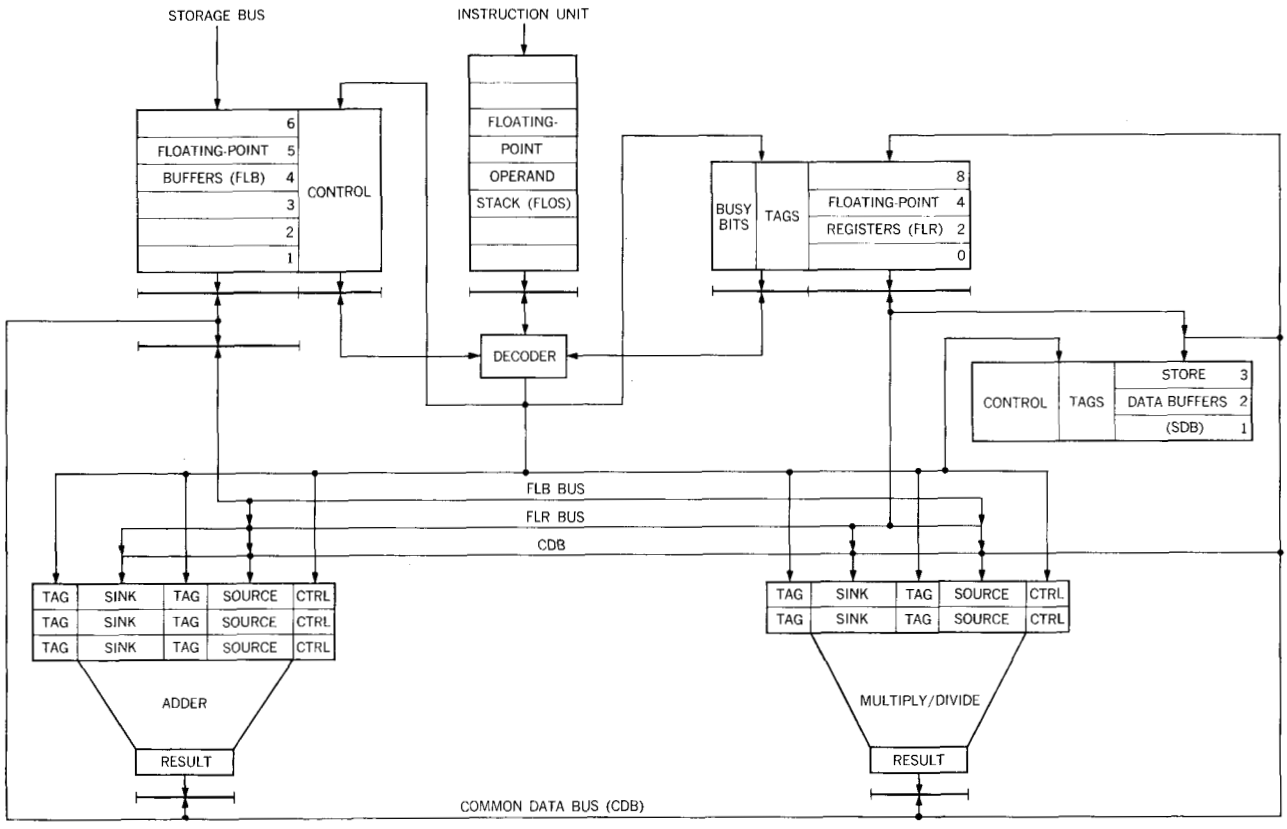


Figure 4 Data registers and transfer paths, including CDB and reservation stations.

priority circuit. If the CDB is free, the priority control signals the requesting adder, A1, to outgate its result *and* it broadcasts the tag of the requestor (1010 in this case) to all reservation stations. Each active reservation station (selected but awaiting a register operand) compares its sink and source tags to the CDB tag. If they match, the reservation station ingates the data from the CDB. In a similar manner, the CDB tag is compared with the tag of each *busy* floating-point register. All busy registers with matching tags ingate from the CDB and reset their busy bits.

Two steps toward the goal of preserving precedence have been accomplished by the foregoing. First, the second AD cannot start until the first AD finishes because it cannot receive both its operands until the result of the first AD appears on the CDB. Secondly, the result of the first AD cannot change register F0 once the second AD is issued, since the tag in F0 will not match A1. These are precisely the desired effects.

Before proceeding with more detailed considerations let us recapitulate the essence of the method. The floating-point register tag identifies the last unit whose result is destined for the register. When an instruction is issued that requires a busy register the tag is sent to the selected unit in place of the register contents. The unit continuously compares this tag with that generated by the CDB priority control. When a match is detected, the unit ingates from the CDB. The unit begins executing as soon as it has both operands. It may receive one or both operands from either the CDB or the FLR bus; the source operand for storage-to-register instructions is transmitted via the FLB bus.

As each instruction is issued the existing tag(s) is (are) transmitted to the selected unit and then the sink tag is updated. By passing tags around in this fashion, all operations having the same sink are correctly sequenced while other operations are allowed to proceed independently. Finally, the floating-point register tag controls the changing of the register itself, thereby ensuring that only the most recent instruction will change the register. This has the interesting consequence that a loop of the following kind:

Example 5

```
LOOP  LD    F0,  Ai
      AD    F0,  Bi
      STD   F0,  Ci   STORE
      BXH    i,  −1,  0,  LOOP
```

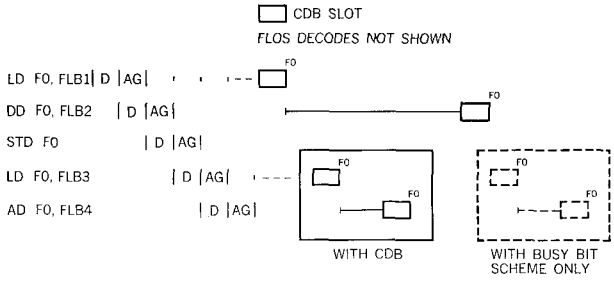


Figure 5 Timing sequence for Example 6, showing effect of CDB.

may execute indefinitely without any change in the contents of F0. Under normal conditions only the final iteration will place its result in F0.

As mentioned previously, there are two ways of starting an independent instruction string. The first is to specify a different sink register and the second is to load a register. The CDB handles the former in essentially the same way as the busy bit scheme. The load, which had been a difficult problem previously, is now very simple. Regardless of the register tag or busy bit, a load turns the busy bit on and sets the tag equal to the floating-point buffer which the instruction unit had assigned to the load. This causes subsequent instructions to sequence on the buffer rather than on whatever unit may have identified the register as its sink prior to the load. The buffer controls are set to request the CDB when the storage operand arrives. The following example and Fig. 5 show this clearly.

Example 6

```
LD      F0,  FLB1
DD      F0,  FLB2      DIVIDE
STD     F0,  A
LD      F0,  FLB3
AD      F0,  FLB4
```

Note that the add finishes before the divide. The dashed line portion of Fig. 5 shows what would happen if the busy bit scheme alone were used. Figure 6 displays the sequences followed under the two schemes. This figure graphically illustrates the bottleneck caused by using a single sink register with a busy bit scheme. Because all data must pass through this register, the program is reduced to strictly sequential execution, steps 1 through 7. With the CDB, on the other hand, the sink register hardly appears and the program is broken into two independent, concurrent sequences. This facility of the CDB obviates the need for loop doubling.

The CDB makes it possible to execute some instructions in, effectively, no time at all. In the above example the store took place during the CDB cycle following the divide. In a similar fashion a register-to-register load of a busy register is accomplished by moving the tag of the source floating-point register to the tag of the sink floating-point register. For example, in the sequence

```
AD      F0,  FLB1
LDR     F2,  F0      move F0 to F2
```

the tag of F0 will be 1010 (A1) at the time the LDR is decoded. The decoder simply sets F2's tag to 1010. Now, when the result of the AD appears on the CDB both F0 and F2 will ingate since the CDB tag of 1010 will match the tag of each register. Thus, no unit or extra time was required for the execution of the LDR.

A number of details have been omitted from this discussion in order to clarify the concept, but really only two are of operational significance. First, every unit must request the CDB two cycles before it finishes execution. (These two cycles are required for propagation of the request to the CDB controls, the establishment of priority among competing units, and propagation of a "select" signal to the chosen unit.) This limits the execution time of any instruction to a two-cycle minimum. (Of course, the faster the execution the less the need for, or gain from, concurrency.) It also adds one* cycle to the access time for loads. Because of buffering and overlap, this does not usually cause an increase in problem running time.

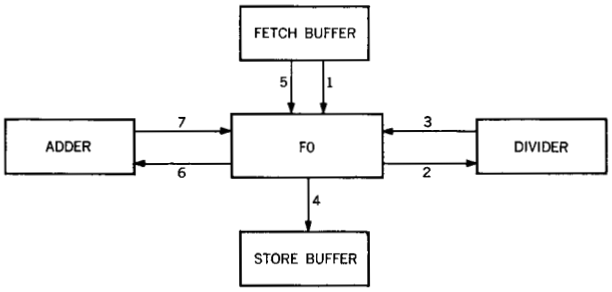
The second point is concerned with mixed precision. Because the architectural definition causes the low-order part of an FLR to be preserved during single-precision operation, an error can occur in the following kind of program:

```
LD      F0,      FLB1
AD      F0,      FLB2
AE      F0,      FLB3
```

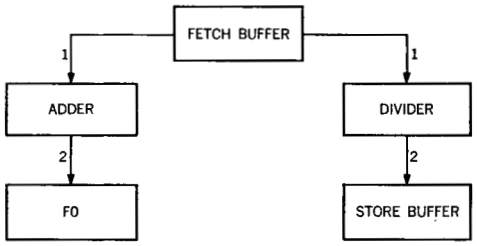
Since only the last instruction, which is single-precision, will change F0, the low order result of the double-precision AD will be lost. This is handled by associating a bit with each register to indicate whether a particular register is the sink of an outstanding single- or double-precision instruction. If this bit does not match the "length" of the instruction being decoded, the decode is suspended until the busy bit goes off. While this stratagem† solves the logic problem, it does so at the expense of performance. Unfortunately, no way has been found to avoid this. Note, however, that all-single- or all-double-precision programs run at the maximum possible speed. It is only the interface between single- and double-precision to the *same* sink register that suffers delay.

* It does not add two cycles since storage gives one cycle prenotification of the arrival of data.

† Further complications arise from the fact that single-precision multiply produces a double-precision product. This is handled separately but with the same time penalty as above.



(a)



(b)

Figure 6 Functional sequence for Example 6 (a) with busy bit controls only, (b) with CDB.

Conclusions

Two concepts of some significance to the design of high-performance computers have been presented. The first, reservation stations, is simply an expeditious method of buffering, in an environment where the transmission time between units is of consequence. Because of the disparity between storage access and circuit speeds and because of dependencies between successive operations, it is observed (given multiple execution units) that each unit spends much of its time waiting for operands. In effect, the reservation stations do the waiting for operands while the execution circuitry is free to be engaged by whichever reservation station fills first.

The second, and more important, innovation, the CDB, utilizes the reservation stations and a simple tagging scheme to preserve precedence while encouraging concurrency. In conjunction with the various kinds of buffering in the CPU, the CDB helps render the Model 91 less sensitive to programming. It should be evident, however, that the programmer still exercises substantial control over how much concurrency will occur. The two different programs for doing A + B + C + D * E illustrate this clearly.

It might appear that the CDB adds one cycle to the execution time of each operation, but in fact it does not. In practice only 30 nsec of the 60-nsec CDB interval are required to perform all of the CDB functions. The remaining time could, in this case, be used by the execution unit to achieve a shorter effective cycle. For example, if an add requires 120 nsec, then add plus the CDB time required is 150 nsec. Therefore, as far as the add is concerned, the machine cycle could be 50 nsec. Besides, even without the CDB, a similar amount of time would be required to transmit results both to the floating-point registers and back as an input to the unit generating the result.

The following program, a typical partial differential equation inner loop, illustrates the possible performance increase.

```
LOOP    MD      F0,  Ai
        AD      F0,  Bi
        LD      F2,  Ci
        SDR     F2,  F0
        MDR     F2,  F6
        AD2     F2,  Ci
        STD     F2,  Ci
        BXH     i,  −1,  0,  LOOP
```

Without the CDB one iteration of the loop would use 17 cycles, allowing 4 per MD, 3 per AD and nothing for LD or STD. With the CDB one iteration requires 11 cycles. For this kind of code the CDB improves performance by about one-third.

Acknowledgments

The author wishes to acknowledge the contributions of Messrs. D. W. Anderson and D. M. Powers, who extended the original concept, and Mr. W. D. Silkman, who implemented all of the central control logic discussed in the paper.

References

1. D. W. Anderson, F. J. Sparacio and R. M. Tomasulo, "The System/360 Model 91: Machine Philosophy and Instruction Handling," *IBM Journal* 11, 8 (1967) (this issue).
2. S. F. Anderson, J. Earle, R. E. Goldschmidt and D. M. Powers, "The System/360 Model 91 Floating-Point Execution Unit," *IBM Journal* 11, 34 (1967) (this issue).

Received September 16, 1965.